

Machine Learning

Appunti del corso (654AA)

Versione 1.1

Fabio Piscitelli

Professore: Alessio Micheli

Università di Pisa — Laurea Magistrale in Informatica, a.a. 2026/27

24 settembre 2026

Indice

1	Introduzione al Machine Learning	1
1.1	Che cos'è il Machine Learning	1
1.1.1	L'apprendimento come principio universale	1
1.1.2	Perché far apprendere le macchine: lusso o necessità?	1
1.1.3	Quando serve il ML	2
1.1.4	Aree applicative	2
1.1.5	Alcuni successi recenti	2
1.1.6	Lo scopo ultimo	3
1.2	Perché studiare il Machine Learning	3
1.2.1	Il ML moderno	4
1.2.2	Tre punti di vista sul ML	4
1.3	Informazioni sul corso	4
1.3.1	Prerequisiti e obiettivi	4
1.3.2	Struttura del corso	5
1.3.3	Organizzazione pratica	6
1.3.4	Esame	6
1.3.5	Materiale e bibliografia	8
1.4	Richiami matematici	8
1.4.1	Prodotto scalare e norme	9
1.4.2	Derivate parziali e gradiente	9
1.4.3	Densità di probabilità: la gaussiana	11
1.5	Sintesi e prossimi passi	11
2	Il ML nei curricula della magistrale	12
2.1	I curricula della magistrale	12
2.2	Il ruolo del Machine Learning	12
2.3	Sinergia con Computational Mathematics	13
2.4	Note per categorie particolari di studenti	13
3	Concetti fondamentali del ML	14
3.1	Il Machine Learning in ambito computazionale	14
3.1.1	Quando usare il ML	14
3.2	L'apprendimento come approssimazione di funzioni	15
3.2.1	L'esempio guida: riconoscimento di cifre	15
3.2.2	Esempi di coppie $x \rightarrow f(x)$	15
3.3	Gli ingredienti di un sistema di ML	16
3.3.1	Dati	16

3.3.2	Task	17
3.3.3	Modello	20
3.3.4	Algoritmo di apprendimento	22
3.4	Il bias induttivo	22
3.4.1	Imparare funzioni booleane: un problema mal posto	23
3.4.2	Restringere lo spazio: le regole congiuntive	23
3.4.3	Unbiased learner	24
3.4.4	Sistemi induttivi e deduttivi equivalenti	24
3.4.5	Language bias o search bias?	25
3.5	Task e funzioni di loss	26
3.5.1	Regressione: errore quadratico	26
3.5.2	Classificazione: loss 0/1	27
3.5.3	Clustering e vector quantization (anticipazione)	27
3.5.4	Stima di densità (anticipazione)	27
3.6	Generalizzazione e validazione	27
4	Generalizzazione e validazione (introduzione)	29
4.1	Il ML in breve	29
4.2	I problemi fondamentali del ML	29
4.2.1	Un problema mal posto	29
4.2.2	Generalizzazione e overfitting	30
4.3	Un caso di studio: il fitting polinomiale	30
4.3.1	Effetto del grado del polinomio	31
4.3.2	Errore di training e di test al variare di M	32
4.3.3	Effetto della quantità di dati	32
4.4	Verso la Statistical Learning Theory	33
4.4.1	Il setting formale (semplificato)	33
4.4.2	VC-dimension e bound sul rischio	33
4.4.3	Perché la SLT è importante	35
4.5	Validazione	36
4.5.1	I due obiettivi della validazione	36
4.5.2	Hold-out	36
4.5.3	K-fold cross-validation (anticipazione)	36
4.6	Accuratezza nella classificazione	37
4.6.1	Matrice di confusione	37
4.6.2	Curva ROC	38
4.7	Il ciclo di progettazione	38
4.8	Errori di interpretazione	39
5	Modelli lineari e K-nearest neighbors	40
5.1	Notazione sui dati	40
5.2	Modelli lineari	40
5.2.1	Regressione lineare univariata	40

5.2.2	Notazione per input multidimensionali	42
5.3	Classificazione con modelli lineari	42
5.3.1	Il problema	42
5.3.2	Iperpiani e regioni di decisione	42
5.3.3	Usare il modello: due esempi	43
5.3.4	Proprietà utili dell'iperpiano	44
5.4	Il problema di apprendimento	45
5.4.1	Formulazione	45
5.4.2	Sostituire la loss con una funzione liscia	45
5.4.3	Approccio diretto: le equazioni normali	46
5.4.4	Approccio iterativo: la discesa del gradiente	46
5.4.5	Riepilogo per la classificazione	50
5.4.6	Torniamo al problema: è una buona soluzione?	50
5.4.7	Bias induttivo del modello lineare	51
5.5	Limiti dei modelli lineari	51
5.5.1	Regressione di problemi non lineari	51
5.5.2	Classificazione: separabilità lineare	52
5.6	Estendere il modello lineare	52
5.6.1	“Lineare” nei parametri, non negli input	52
5.6.2	Linear Basis Expansion (LBE)	53
5.7	Regolarizzazione: il controllo della complessità	54
5.7.1	Ridge regression (regolarizzazione di Tikhonov)	54
5.7.2	Altre tecniche di regolarizzazione	56
5.7.3	Altri miglioramenti (opzionali)	56
5.7.4	Classificazione multi-classe (opzionale)	56
5.7.5	Uno sguardo avanti	57
5.8	K-nearest neighbors	57
5.8.1	Apprendimento eager e lazy	57
5.8.2	1-nearest neighbor	58
5.8.3	K-nearest neighbors	58
5.8.4	Varianti	59
5.8.5	Un estremo del ML	60
5.8.6	K-NN contro modello lineare	60
5.8.7	Il classificatore ottimo di Bayes	61
5.8.8	Bias induttivo del K-NN	61
5.8.9	Limiti del K-NN	61
5.8.10	Scelte di progetto del K-NN	63
5.8.11	E adesso?	63
6	Reti neurali (parte 1): dal neurone al MLP	65
6.1	Le reti neurali come strumento di ML	65
6.2	Ispirazione biologica	65

6.3	Il neurone artificiale	66
6.4	Il Perceptron	67
6.4.1	Le reti di McCulloch e Pitts (1943)	68
6.4.2	Lo strato nascosto come rappresentazione	69
6.5	Apprendimento per una singola unità	70
6.5.1	L'algoritmo di apprendimento del Perceptron	70
6.5.2	Delta rule: due letture	71
6.5.3	Rappresentare e apprendere	71
6.5.4	Il teorema di convergenza del Perceptron	72
6.6	Perceptron contro LMS	74
6.7	Funzioni di attivazione sigmoidali	75
6.7.1	Altre funzioni di attivazione	76
6.7.2	Derivate	76
6.7.3	LMS con un'unità sigmoidale	76
6.8	Le reti neurali: il Multi-Layer Perceptron	77
6.8.1	Vista A: una rete di unità	78
6.8.2	Vista B: una funzione flessibile	78
6.8.3	Componenti di una rete neurale	78
6.8.4	Notazione uniforme per le unità	79
6.8.5	Architettura e processing feedforward	79
6.9	Perché le reti neurali sono flessibili	80
6.9.1	Quali task	80
6.9.2	La rete come espansione in basi adattive	80
6.9.3	L'approssimazione universale	81
6.9.4	Potere espressivo e VC-dimension	81
6.9.5	Quanti strati? (anticipazione)	82
6.9.6	Il bias induttivo delle reti neurali	82
6.10	Verso l'algoritmo di apprendimento	82
6.10.1	Il problema del credit assignment	82
6.10.2	Il loading problem	82
6.10.3	L'idea chiave: estendere la discesa del gradiente	83
6.10.4	Questioni pratiche nell'addestramento	83
7	Note sulla backpropagation	85
7.1	Strumenti di calcolo differenziale	85
7.2	Il problema	85
7.3	L'algoritmo iterativo	86
7.4	La derivazione	87
7.4.1	Il gradiente per un peso generico	87
7.4.2	Caso 1: unità di uscita ($t = k$)	87
7.4.3	Caso 2: unità nascosta ($t = j$)	88
7.5	Sintesi delle formule	88

7.5.1	Generalizzazione a m strati nascosti	89
7.6	Il ciclo di addestramento completo	89
7.7	Proprietà e interpretazione	90
8	Reti neurali (parte 2): addestramento in pratica	91
8.1	Filosofia	91
8.2	Valori iniziali dei pesi	92
8.3	Minimi multipli	92
8.3.1	Serve davvero il minimo globale?	93
8.4	On-line, batch e mini-batch	93
8.4.1	Mini-batch	94
8.5	Il learning rate η	94
8.5.1	LMS: dividere per l	95
8.5.2	Migliorare la discesa	95
8.5.3	Oltre lo SGD di base	97
8.6	Criteri di arresto	97
8.7	Overfitting e regolarizzazione	97
8.7.1	Fermarsi al minimo?	97
8.7.2	Come nasce l'overfitting in una rete	98
8.7.3	Regolarizzazione (!)	98
8.7.4	Mettere tutto insieme: momentum e weight decay	101
8.8	Numero di unità nascoste	101
8.8.1	Cascade Correlation	101
8.9	Input e output	103
8.10	Pregi e difetti delle reti neurali	103
8.11	Verso il progetto: il benchmark MONK	104
9	Validazione (parte 1): model selection e model assessment	107
9.1	Premessa: bias e varianza	107
9.2	Motivazioni	108
9.2.1	Come stimare	108
9.3	I due obiettivi della validazione	108
9.3.1	Hold-out	108
9.3.2	Test o model selection?	109
9.4	Un controesempio istruttivo	109
9.5	Grid search	110
9.5.1	Alternative alla grid search	110
9.6	K-fold cross-validation	111
9.6.1	Quanti dati servono?	111
9.6.2	Richiamo	112
9.6.3	Un esempio completo di selezione e valutazione	112
9.7	Casi particolari	112
9.7.1	Campionamento fortunato o sfortunato	112

9.7.2 Pochissimi dati	112
9.8 Misure d'errore per la valutazione	113
10 Validazione (parte 2): schemi formali	114
10.1 Notazione	114
10.2 Metodologia: identificare l'obiettivo	115
10.3 Model selection	115
10.3.1 Hold-out	115
10.3.2 K-fold cross-validation	115
10.4 Stima del rischio (model assessment)	116
10.4.1 Hold-out	116
10.4.2 K-fold cross-validation	116
10.5 Combinare model selection e model assessment	117
10.5.1 Hold-out training-validation-test	117
10.5.2 K-fold CV per la selezione + hold-out per il test	117
10.5.3 K-fold CV per il test con hold-out interno per la selezione	118
10.5.4 Double (nested) K-fold CV	118
11 Validazione (parte 3): errori tipici e FAQ	121
11.1 Arresto dell'addestramento	121
11.1.1 Non fissare un numero arbitrario di epoche	121
11.1.2 Early stopping	121
11.2 Inizializzazione casuale e model selection	122
11.3 Selezione sequenziale degli iperparametri	122
11.4 Cosa fare dopo selezione e valutazione?	123
11.5 La cross-validation può andare in overfitting	123
11.6 Quale cross-validation?	123
11.7 Esempi di model selection	124
11.7.1 Esempio 1: modelli diversi	124
11.7.2 Esempio 2: numero di unità nascoste	124
11.7.3 Esempio 3: K-NN	125
11.8 Conclusioni pratiche	125
12 Statistical Learning Theory e VC-dimension	126
12.1 La VC-dimension	126
12.1.1 Shattering	126
12.1.2 Definizione di VC-dimension	127
12.1.3 VC-dimension degli iperpiani nel piano	128
12.1.4 Altri esempi	128
12.1.5 VC-dimension e numero di parametri	129
12.2 Il bound analitico sul rischio	129
12.3 Structural Risk Minimization	130
12.3.1 SRM per la model selection	130

12.3.2 A cosa serve il bound	130
12.3.3 Due approcci pratici alla SRM	131
13 Support Vector Machines	133
13.1 Parte I — SVM per la classificazione binaria	133
13.1.1 Iperpiano separatore	134
13.1.2 Il margine di separazione	134
13.1.3 Rappresentazione canonica e support vector	135
13.1.4 Calcolo del margine	135
13.1.5 Il problema di ottimizzazione quadratica (hard margin)	136
13.1.6 Come si usa	138
13.1.7 Perché migliora la generalizzazione?	138
13.1.8 Perché è un approccio elegante	138
13.2 SVM soft margin: ammettere errori	139
13.2.1 Variabili slack	139
13.2.2 Il problema primale soft margin	140
13.2.3 Il duale soft margin*	140
13.3 Kernel: SVM per problemi non lineari	140
13.3.1 Mappare in uno spazio ad alta dimensione	141
13.3.2 Spazi di feature ad alta dimensione	142
13.3.3 Il kernel trick	142
13.3.4 La matrice kernel	142
13.3.5 SVM con kernel	143
13.3.6 Vista “architetturale” di una SVM	143
13.3.7 Esempi di kernel	143
13.4 Parte II — SVM per la regressione non lineare	144
13.4.1 Il problema	144
13.4.2 La loss ϵ -insensitive	144
13.4.3 Formulazione	144
14 SVM e kernel: aspetti pratici e visione critica	147
14.1 Caratteristiche principali	147
14.2 Risultati	147
14.2.1 La prima applicazione famosa	147
14.2.2 L'esempio della miscela di gaussiane	148
14.3 Le SVM nella pratica	149
14.3.1 Il ruolo critico degli iperparametri	149
14.3.2 Luoghi comuni sulle SVM	150
14.4 Metodi kernel	150
14.4.1 Kernel come misura di similarità	150
14.4.2 Argomenti avanzati	151

15 Bias-varianza ed ensemble	153
15.1 L'idea	153
15.2 Analisi nella regressione	153
15.2.1 L'errore di predizione atteso	153
15.2.2 Richiamo di statistica	154
15.2.3 La decomposizione	155
15.2.4 Interpretazione delle tre componenti	155
15.3 Bias, varianza e regolarizzazione	157
15.4 Ensemble learning	158
15.4.1 Bagging (bootstrap aggregating)	159
15.4.2 Boosting (es. AdaBoost)	159
15.5 Feature selection (cenni)	159
16 Reti neurali convoluzionali (CNN)	161
16.1 Il problema: riconoscere caratteri	161
16.2 La convoluzione	162
16.2.1 Convoluzione 1D: una unità su uno stream	162
16.2.2 Convoluzione 2D	162
16.2.3 Stride maggiore di 1	163
16.3 Una rete di "filtri"	163
16.4 Pooling	164
16.5 La CNN nel complesso	164
16.6 Come si usano	166
16.7 Risultati storici	166
16.7.1 Le architetture di LeCun sul dataset ZIP code	166
16.7.2 MNIST	166
16.7.3 Riconoscimento facciale e ImageNet	168
16.8 CNN moderne e GPU	168
17 Deep learning	170
17.1 Che cos'è il deep learning	170
17.1.1 Un po' di storia	170
17.1.2 Il quadro generale	170
17.1.3 Gli ingredienti concettuali: astrazione gerarchica	171
17.1.4 Combinare per generalizzare	172
17.2 Servono davvero molti strati?	173
17.2.1 Un esempio dai circuiti logici	173
17.2.2 Il caso delle reti neurali	173
17.2.3 Perché: la composizionalità	174
17.2.4 Il bias induttivo	175
17.2.5 Maledizione della dimensionalità e bias	175
17.2.6 Questioni pratiche	176
17.3 Representation learning	176

17.3.1	Idee di base	176
17.3.2	Pre-training	177
17.3.3	Transfer learning	178
17.4	Rappresentazioni distribuite	179
17.4.1	Contare la differenza	180
17.4.2	Attributi condivisi: separare i concetti	180
17.4.3	Rappresentazioni distribuite profonde	181
17.5	Altri spunti e risultati recenti	182
17.6	Tecniche	183
17.6.1	Problemi del gradiente	183
17.6.2	ReLU	184
17.6.3	Batch normalization	185
17.6.4	Dropout	185
17.6.5	Regolarizzazione L1	186
17.6.6	Apprendimento avversario	187
17.7	Applicazioni, sfide e conclusioni	187
18	Reti neurali randomizzate	188
18.1	La casualità nel Machine Learning	188
18.1.1	Modelli intrinsecamente casuali	188
18.2	Reti neurali a pesi casuali	189
18.2.1	Struttura generale	189
18.2.2	Reti feedforward randomizzate	190
18.2.3	Pro e contro	191
18.2.4	Dove sono utili	191
19	Apprendimento non supervisionato: K-means e SOM	193
19.1	Apprendimento non supervisionato	193
19.1.1	Clustering	193
19.2	Quantizzazione vettoriale	194
19.2.1	L'errore di quantizzazione	195
19.3	K-means	195
19.3.1	Versione on-line	195
19.3.2	Versione batch	196
19.3.3	Pregi e difetti	197
19.3.4	Soft-max	197
19.4	Self-Organizing Map	197
19.4.1	Obiettivo	197
19.4.2	Apprendimento competitivo e ispirazione biologica	198
19.4.3	L'algoritmo	198
19.4.4	L'emergere dell'ordine topologico	199
19.4.5	Visualizzazione	200
19.5	Valutare un clustering (cenni)	201

20 Reti neurali ricorrenti (RNN)	203
20.1 Perché i dati sequenziali	203
20.2 Dai dati piatti ai dati strutturati	203
20.2.1 Il nuovo dominio: sequenze e trasduzioni	203
20.3 La memoria nelle reti neurali	204
20.3.1 Approccio 1: memoria finita (IDNN)	204
20.3.2 Approccio 2: unità ricorrenti	205
20.3.3 Sistemi a transizione di stato	206
20.3.4 Una rete ricorrente (Simple RNN)	206
20.3.5 Proprietà	207
20.4 Apprendimento: l'unfolding	207
20.4.1 Modelli avanzati	208
20.5 Approcci collegati	208
20.5.1 Transformer e attenzione	209
20.5.2 Reti ricorrenti randomizzate: le Echo State Network	209
20.6 Verso i domini strutturati	210
21 Apprendimento su dati strutturati e grafi	212
21.1 Perché dati strutturati e grafi	212
21.1.1 Lo scenario	213
21.1.2 Un po' di storia	213
21.2 Deep Graph Networks	213
21.2.1 I grafi considerati	214
21.2.2 Il problema della rappresentazione	214
21.2.3 Trasduzioni su grafi	215
21.2.4 Il message passing	215
21.2.5 Il legame con le CNN	216
21.2.6 La formula generale	216
21.2.7 NN4G e GCN: iterazioni come strati	217
21.2.8 GNN e GraphESN: approccio ricorsivo	218
21.3 Problemi aperti e alcune proposte	218
21.3.1 Efficienza: oltre la backpropagation end-to-end	218
21.3.2 I problemi della profondità: gli "over-*)"	219
21.3.3 Estensioni in corso	220
21.4 Altri approcci: i kernel per strutture	221

Capitolo 1

Introduzione al Machine Learning

La prima lezione ha un doppio scopo: motivare lo studio del **Machine Learning** (ML, in italiano *apprendimento automatico*) collocandolo nel panorama dell'informatica e dell'intelligenza artificiale, e fornire le informazioni pratiche sul corso (prerequisiti, struttura, esame, materiale). In chiusura vengono richiamati i concetti matematici di base che serviranno per tutto il corso.

1.1 Che cos'è il Machine Learning

1.1.1 L'apprendimento come principio universale

L'apprendimento è un principio che accomuna esseri viventi, società e — oggi — macchine. Come osservano Poggio e Shelton (*AI Magazine*, 1999), *il problema dell'apprendimento è probabilmente il cuore stesso del problema dell'intelligenza, sia biologica che artificiale*. Capire come si impara significa quindi capire una parte fondamentale di cosa vuol dire essere intelligenti.

Definizione — Machine Learning

Il **Machine Learning** è un'area teorica e applicativa dell'informatica che unisce due obiettivi: quello dell'intelligenza artificiale di costruire **computer capaci di apprendere**, e quello di disporre di **strumenti adattivi e statistici potenti**, con fondamenti rigorosi nelle scienze computazionali. Operativamente, è l'apprendimento automatico, da parte di un sistema, a partire dall'**esperienza** (un insieme di esempi), finalizzato a risolvere un compito computazionale.

1.1.2 Perché far apprendere le macchine: lusso o necessità?

Il ML non è un vizzo tecnologico ma una **necessità**, per due ragioni principali:

1. **Disponibilità crescente di dati empirici** e bisogno di analizzarli. La scienza stessa sta cambiando paradigma verso un approccio *data-driven*, in cui la conoscenza si estrae dai dati; il ML ha un ruolo centrale e metodologico in questo cambiamento.
2. **È difficile programmare esplicitamente l'adattività e l'intelligenza**. Già Alan Turing lo aveva intuito: per molti compiti scrivere a mano le regole è impraticabile, e l'apprendimento diventa l'unica strada per dotare i sistemi di comportamenti intelligenti.

La combinazione di **grandi quantità di dati**, **potenza di calcolo** (HPC, GPU) e **flessibilità dei modelli di ML** ha aperto una nuova era dell'intelligenza artificiale.

1.1.3 Quando serve il ML

Il ML è la scelta giusta quando per un problema c'è **poca o nessuna conoscenza a priori** (non sappiamo scrivere le regole della soluzione), ma è relativamente **facile disporre di esperienza**, cioè di dati di cui conosciamo il risultato corretto. Esempi classici sono la classificazione delle email come *spam*, il riconoscimento di caratteri, volti o parlato.

Esempio — Riconoscimento di cifre scritte a mano

Ogni cifra è un'immagine di 8×8 pixel. Scrivere a mano le regole che distinguono un "3" da un "8" per tutte le possibili calligrafie è di fatto impossibile. Si raccolgono invece molti esempi di cifre già etichettate e si lascia che il sistema **apprenda un classificatore** f che associa a ogni immagine la classe corretta (0, 1, ..., 9).

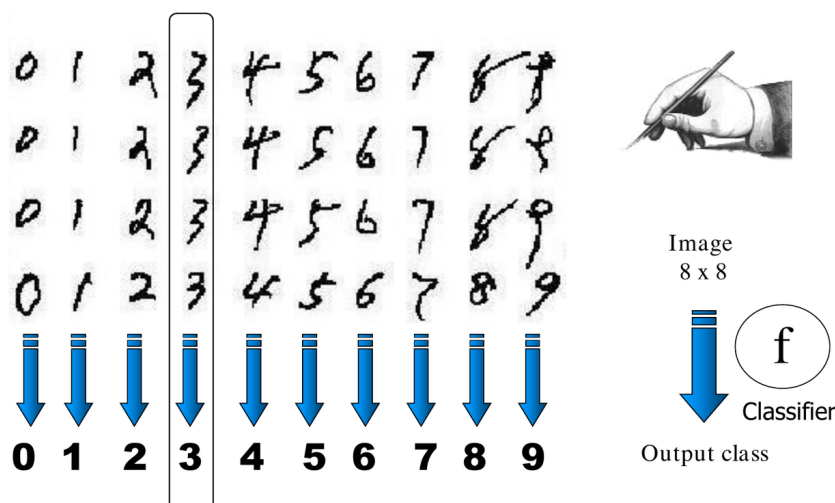


Figura 1.1: Riconoscimento di cifre manoscritte: il classificatore f mappa un'immagine 8×8 nella classe di output.

1.1.4 Aree applicative

Il ML è ormai pervasivo nei sistemi reali (dall'OCR alle interfacce uomo-macchina, ai motori di ricerca) ed è il nucleo di un'area interdisciplinare che dagli anni '80 comprende:

- **Pattern Recognition** (riconoscimento di volti e parlato), **Computer Vision**, **Natural Language Processing** (dalla classificazione di testi all'elaborazione del parlato fino all'AI generativa);
- **Robotica**, sistemi e filtri adattivi, reti di sensori intelligenti, componenti personalizzati;
- **Knowledge Discovery e Data Mining**, Information Retrieval, analisi di dati complessi (medicina, biologia, chimica, web, marketing), previsioni finanziarie.

In alcuni campi — NLP, Pattern Recognition, Computer Vision — l'impatto è stato rivoluzionario, tanto da parlare di una "nuova AI".

1.1.5 Alcuni successi recenti

Il professore mostra alcuni risultati emblematici:

- **Riconoscimento facciale (DeepFace, CVPR 2014)**: combinando reti neurali profonde e altre tecniche di ML, addestrato su quattro milioni di immagini di oltre 4000 persone, il sistema stabilisce se due foto ritraggono la stessa persona con un'accuratezza del 97,25%, appena sotto quella umana (97,53%).
- **AlphaGo (DeepMind, marzo 2016)**: sconfigge il campione mondiale di Go.
- **Medicina**: una rete neurale profonda per la classificazione dei tumori della pelle, addestrata su 130.000 casi (più di quanti un medico ne veda "in molte vite"), raggiunge l'accuratezza di dermatologi certificati (*Nature*, 2017). Al CIML di Pisa, il progetto BrAID applica l'AI alla diagnosi della sindrome di Brugada.
- **Traduzione automatica**: Google Neural Machine Translation (dal 2016) e DeepL (dal 2017), interamente basati su reti neurali.
- **Large Language Models e AI generativa**: ChatGPT (OpenAI, novembre 2022), Bard/LaMDA (Google), modelli open source come Dolly, e sistemi che generano immagini (Stable Diffusion), video, musica.
- Guida autonoma, analisi di dati da sensori, IoT.

Il ML fornisce la base teorica e metodologica della cosiddetta *rivoluzione dell'AI*, con enormi investimenti industriali. Viene considerato una **GPT** (*General-Purpose Technology*): una tecnologia capace di trasformare profondamente le strutture economiche e sociali preesistenti. La figura del "*ML scientist*" è tra le più richieste nel mercato del lavoro.

Nota — Premio Turing 2018

Il 27 marzo 2019 l'ACM ha assegnato il Premio Turing (il "Nobel dell'informatica") a **Yoshua Bengio, Geoffrey Hinton e Yann LeCun** "per le scoperte concettuali e ingegneristiche che hanno reso le reti neurali profonde una componente critica dell'informatica".

Nota — Premi Nobel 2024

Nel 2024 il ML è arrivato anche ai Nobel: - **Fisica**: a **John Hopfield** e **Geoffrey Hinton** per le scoperte e invenzioni fondamentali che permettono il machine learning con le reti neurali artificiali (la rete di Hopfield come memoria associativa, la macchina di Boltzmann). - **Chimica**: a **Demis Hassabis** e **John Jumper** (Google DeepMind) per la predizione della struttura delle proteine con **AlphaFold**, e a **David Baker** per la progettazione computazionale di proteine. Hassabis: "*Ho sempre pensato che, se riuscissimo a costruire l'AI nel modo giusto, potrebbe essere lo strumento definitivo per aiutare gli scienziati a esplorare l'universo che ci circonda.*"

1.1.6 Lo scopo ultimo

Tutto ciò che è potente può essere pericoloso se usato male, ma lo scopo ultimo di AI e ML è **portare benefici alle persone** risolvendo problemi grandi e piccoli, **accelerare il progresso umano** e fornire intelligenza a ogni altro campo scientifico: aumentare le nostre capacità, esaltando la nostra umanità (P. Domingos, *Scientific American*, 2018).

1.2 Perché studiare il Machine Learning

Nel percorso di laurea magistrale il ML serve a:

- conoscere i **principi di base dei processi di apprendimento**, dal punto di vista computazionale;

- conoscere **nuovi paradigmi di calcolo**, come le **reti neurali**: studiate come paradigma computazionale dagli anni '40 con ispirazione neurobiologica, oggi sono un insieme di modelli potenti per l'**approssimazione di funzioni** con capacità predittive, supportati da una solida teoria (la *learning theory*), e sono alla base del **Deep Learning**;
- saper **applicare questi modelli con rigore**;
- avere le basi per tutti i corsi successivi di AI.

1.2.1 Il ML moderno

Il ML è passato dall'essere una **collezione di metodi euristici** (trucchi provenienti da biologia, statistica, reti neurali, logica fuzzy...) alla costruzione di **quadri concettuali generali**. È emersa una nuova comprensione dei principi che governano i processi di apprendimento, con tre conseguenze importanti:

- esprimere modelli diversi in “linguaggi” matematici o informatici diversi **non cambia i principi**;
- campi applicativi diversi **non cambiano i principi**;
- nuovi avanzamenti tecnologici **non cambiano i principi**.

Suggerimento — Il filo conduttore del corso

Il “Machine Learning” in senso proprio è il **nucleo di principi e metodi** per costruire modelli computazionali flessibili e potenti a partire dai dati. I singoli modelli cambiano, i principi restano: è questo che conviene imparare davvero.

1.2.2 Tre punti di vista sul ML

1. **Come metodologia di AI** → costruire sistemi adattivi e intelligenti.
2. **Come apprendimento statistico** (inferenza di ipotesi secondo principi matematici) → costruire potenti sistemi predittivi per l'analisi intelligente dei dati.
3. **Come metodo informatico per aree applicative innovative** → usare i modelli come strumenti per problemi complessi e interdisciplinari.

Il corso li considera tutti, con un **approccio critico**: dei vari metodi si discutono i limiti e l'appropriatezza d'uso, con attenzione ai principi sottostanti. Gli obiettivi pratici sono due: imparare a **sviluppare nuovi modelli** di ML e imparare ad **applicare le metodologie allo stato dell'arte** a problemi di altri campi.

1.3 Informazioni sul corso

1.3.1 Prerequisiti e obiettivi

Nessun corso è strettamente necessario: si parte da zero. Servono però le basi tipiche di una laurea triennale scientifica: **analisi matematica** (funzioni, calcolo differenziale), **notazione e calcolo matriciale**, **elementi di probabilità e statistica**, **algoritmi**.

Il corso introduce i principi e l'analisi critica dei principali paradigmi (modelli e algoritmi, con focus sulle reti neurali) per l'apprendimento dai dati. I concetti vengono introdotti progressivamente, dagli approcci più semplici fino ai modelli allo stato dell'arte, all'interno del quadro

concettuale del ML moderno: l'**apprendimento di funzioni a partire da esempi**. L'attenzione è su:

- analisi critica delle caratteristiche per la progettazione e l'uso del ML;
- **valutazione sperimentale rigorosa**: c'è una grande differenza tra *usare* strumenti di ML e usarli *correttamente*;
- aspetti computazionali dei sistemi di apprendimento.

1.3.2 Struttura del corso

Il corso ha una struttura forte: si parte dai modelli semplici per arrivare allo stato dell'arte, e i concetti fondamentali vengono introdotti attraverso modelli diversi. Il consiglio pratico è duplice: (1) studiare i singoli metodi, (2) cercare le **connessioni e i confronti** tra di essi, così da astrarre i concetti. Il filo rosso è la comprensione del rapporto tra **flessibilità del modello** e **controllo della complessità**, concetto fondamentale per la capacità di **generalizzazione**.

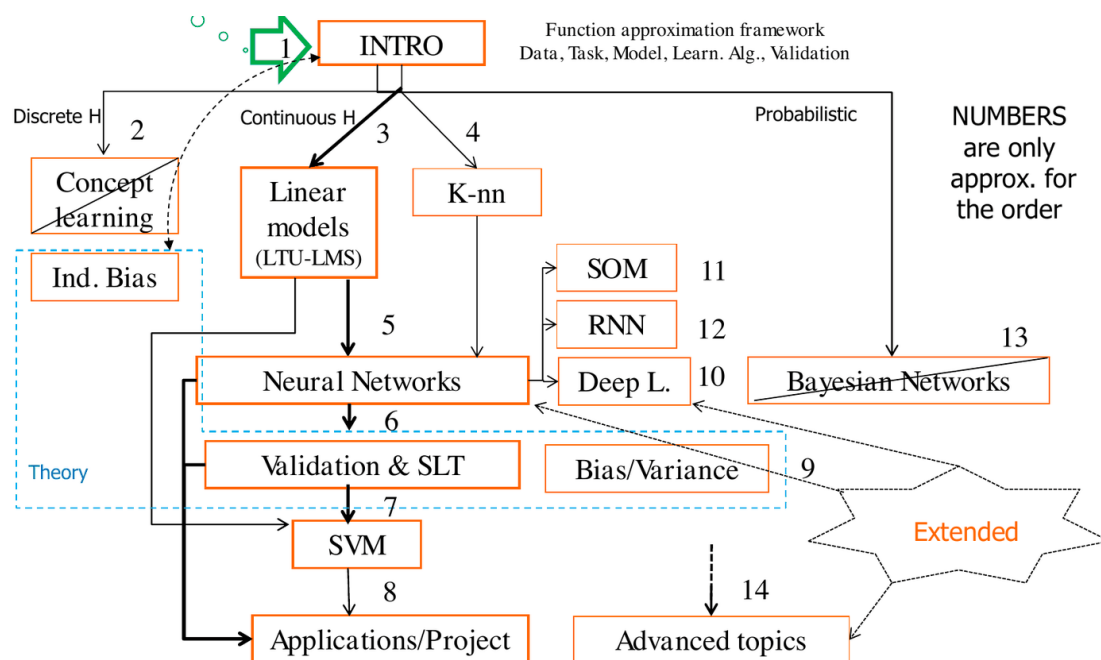


Figura 1.2: La mappa del corso. I numeri indicano approssimativamente l'ordine delle lezioni; il riquadro tratteggiato “Theory” raccoglie le parti teoriche.

La mappa non è solo un ordine delle lezioni: descrive il percorso. Si introducono dei **“mattoni”** (concetti e modelli intermedi) che poi vengono composti per costruire i modelli di ML — ad esempio le reti neurali si costruiscono dai modelli lineari, e un sistema di ML completo si costruisce dalle tecniche di validazione. Per questo i modelli intermedi vanno visti come pezzi di un puzzle che si ricompone alla fine.

Nota — Ordine delle lezioni secondo la mappa

1. Introduzione (dati, task, modello, algoritmo di apprendimento, validazione) — nel quadro dell'**approssimazione di funzioni**.
2. Concept learning e bias induttivo (spazio delle ipotesi discreto).
3. Modelli lineari (LTU, LMS) (spazio delle ipotesi continuo).
4. K-nearest neighbors.
5. Reti neurali.
6. Validazione e Statistical Learning Theory.

7. Support Vector Machines.
8. Applicazioni e progetto.
9. Bias/Varianza.
10. Deep Learning (e, in estensione, SOM, RNN). 11–14. SOM, RNN, (Reti Bayesiane — rimosse dal programma), argomenti avanzati (dati strutturati).

La densità del corso non è uniforme: l'introduzione è lenta e “morbida”, il nucleo (reti neurali, SVM, validazione) è denso e consequenziale, la parte avanzata è veloce ma meno densa. Il progetto (opzionale) viene annunciato circa a metà corso.

1.3.3 Organizzazione pratica

- **Docente:** Prof. Alessio Micheli (Dipartimento di Informatica), professore ordinario, responsabile del gruppo CIML (*Computational Intelligence & Machine Learning*) con oltre 25 anni di ricerca in ML/AI, **presidente della European Neural Network Society** (conferenza annuale ICANN), co-fondatore della IEEE Task Force on Reservoir Computing, associate editor di IEEE TNNLS e *Neural Networks*. Obiettivo del corso: introdurre in 3 mesi al ML attuale, ripercorrendone anche le basi e la storia.
- **Lezioni** (a.a. 2026/27): martedì, mercoledì e giovedì 16–18, Aula E. L'inizio è puntuale per poter fare una pausa.
- **Q&A** con il docente: il martedì dopo la lezione (tempo facoltativo, non parte della lezione) più alcune lezioni speciali di Q&A. Per domande personali si scrive una mail con il tag **[ML-26-Q]** nell'oggetto (obbligatorio).
- **Comunicazioni:** tramite Teams e/o le news di Moodle. Le **registrazioni** delle lezioni sono su Teams (scheda *Shared* → *Recordings*); le lezioni ufficiali sono solo in presenza, lo streaming è informale e senza interazione garantita.
- **Recuperi:** il lunedì pomeriggio (annunciati quando servono) e nel periodo di recupero dal 9 al 18 dicembre 2026 (da considerare prima di programmare le vacanze di Natale). Le lezioni del 15–17 settembre 2026 non si sono tenute e saranno recuperate.

Nota — Chi arriva in ritardo al corso

Tutte le informazioni sono sul **Moodle** del corso (elearning.di.unipi.it, con iscrizione autonoma tramite account Unipi): slide, bibliografia e sezione **FAQ**. Conviene usare anche Teams (news e registrazioni) e gli appunti dei colleghi. Progetto e test di esercizio **non sono obbligatori** per l'esame. Non chiedere al docente ciò che è già nelle FAQ o nella prima lezione.

1.3.4 Esame

Attenzione — Regole aggiornate (a.a. 2026/27)

Le modalità d'esame sono cambiate rispetto agli anni precedenti: il **progetto non è più obbligatorio** e l'esame si basa su uno **scritto obbligatorio**, con orale solo se necessario.

L'esame si articola, in ordine cronologico, in quattro fasi.

1. Test di esercizio durante il corso (opzionali). Alla fine di ogni ciclo di lezioni vengono proposti su Moodle brevi e semplici test di esercizio. Servono per l'**autovalutazione** (come esercizi per casa, con un punteggio che però **non vale per l'esame**), tengono al passo con le lezioni e sono una testimonianza della partecipazione attiva. Hanno una scadenza (tipicamente

una settimana), ma se se ne perde qualcuno si può continuare la serie. Preparano allo scritto, anche se formato, contenuti e profondità sono diversi. Vengono annunciati solo in aula e con un breve messaggio su Teams.

2. Progetto (opzionale). Alla fine del corso (dicembre) si può svolgere un progetto:

- lavoro di gruppo di **2 studenti**, senza eccezioni;
- contenuto: implementazione, applicazione e/o analisi di un metodo o di una tecnica di ML/reti neurali con **valutazione sperimentale** (possibili progetti coordinati con il corso di CM);
- formato: presentazione pubblica di un **poster** in aula (una delle ultime lezioni), più la consegna del materiale sviluppato; è previsto un premio per il miglior poster;
- dopo la presentazione si può sostenere l'esame in qualunque sessione, ma per discutere il progetto i due membri del gruppo devono fare l'esame **nella stessa sessione**.

Il consiglio è di non candidarsi al progetto prima di aver studiato a fondo i contenuti del corso. I dettagli verranno dati in una lezione dedicata.

3. Scritto (obbligatorio). Si svolge dopo il corso, nelle sessioni d'esame (da gennaio e febbraio):

- è obbligatorio **isciversi** all'appello su esami.unipi.it entro la scadenza (verificarla con largo anticipo) e cancellare l'iscrizione se si decide di non partecipare;
- si svolge **solo in presenza**, in aula, nella data, ora e aula indicate su esami.unipi.it; non presentarsi equivale a ritirarsi da quella sessione;
- le domande vanno risposte su **fogli prestampati** con spazio per la risposta libera; bisogna portare penna o matita;
- le risposte richiedono **equazioni, disegni e/o parti descrittive** (quiz e domande aperte);
- non si può usare alcun materiale del corso o esterno, né alcun ausilio elettronico (assistenti virtuali, chatbot...): è vietato l'uso di qualunque dispositivo (portatili, smartphone, braccialetti smart, auricolari, occhiali smart...), pena l'esclusione.

4. Orale (solo se necessario) e verbalizzazione. Per accedere all'orale serve un livello sufficiente nello scritto (la lista degli ammessi è comunicata su Moodle). L'orale si tiene nella stessa sessione dello scritto, in presenza, tipicamente nello studio del docente: nei giorni successivi alla correzione il docente assegna uno slot a ogni studente o gruppo. Se serve (in base all'esito dello scritto), l'orale è un colloquio su **tutto il programma** e può includere la discussione dello scritto e del progetto. Esigenze particolari (priorità, impossibilità di essere in presenza...) si valutano caso per caso su richiesta.

In sintesi — Riepilogo delle regole

La valutazione tiene conto di: **risultato dello scritto** + partecipazione attiva (inclusi i test durante il corso) + attività extra facoltative (progetto, presentazioni volontarie a fine corso) + eventuale orale completo, se richiesto dal docente.

- Scritto **sufficiente** → in uno slot pochi giorni dopo: eventuale orale e **proposta di voto** → se accettata, verbalizzazione.
- Scritto **insufficiente** o voto non accettato → si ripete in una sessione successiva, a scelta tra le molte dell'anno.

Completare l'80% dei test online e un progetto molto buono possono dare un **piccolo bonus** (se lo scritto è già sufficiente o buono), ma **non sono necessari** né per sostenere l'esame né per ottenere il voto massimo. L'indicatore chiave è la **conoscenza finale del ML e la sua profondità**.

Suggerimento — I consigli del professore

- Seguire le lezioni usando le slide come guida e studiare progressivamente durante il corso; le lezioni di Q&A sono un forum di discussione: è una classe, non un insieme di registrazioni.
- Prima **studiare i contenuti** del corso, poi dedicarsi al progetto: aumenta sia l'efficacia sia l'efficienza del lavoro.
- Se qualcosa non è chiaro: niente panico, chiedere al docente (Q&A), collaborare con i colleghi, usare libri e articoli (bibliografia e ricerca personale) e i supporti del corso. Il ML è un campo in continua evoluzione: va studiato!

1.3.5 Materiale e bibliografia

Il riferimento principale sono le slide del corso (su Moodle, spesso pubblicate in anticipo; attenzione a usare l'ultima versione dei file ML-26...-v*), che però vanno integrate con i propri appunti (le slide da sole possono non bastare a ricostruire il filo della lezione) e con i libri. Il docente è disponibile a condividere su Moodle gli appunti degli studenti. I testi principali sono:

- S. Haykin, *Neural Networks and Learning Machines*, Prentice Hall, 3^a ed., 2008;
- T. M. Mitchell, *Machine Learning*, McGraw-Hill, 1997;
- I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, 2016 (gratuito online).

Altri riferimenti utili: Russell–Norvig (*AIMA*), Hastie–Tibshirani–Friedman (*The Elements of Statistical Learning*, copia gratuita online), Cherkassky–Mulier, Bishop (*Pattern Recognition and Machine Learning*, ora **gratuito** dal sito dell'editore; e *Neural Networks for Pattern Recognition*), Duda–Hart–Stork, Shalev-Shwartz–Ben-David (*Understanding Machine Learning*, gratuito online).

Per le **librerie software**: F. Chollet, *Deep Learning with Python*; A. Géron, *Hands-on Machine Learning with Scikit-Learn, Keras & TensorFlow*; A. Zhang, Z. Lipton, M. Li, A. Smola, *Dive into Deep Learning* (Cambridge University Press, 2023, gratuito online, con esempi in PyTorch, NumPy/MXNet, JAX e TensorFlow). Le librerie principali verranno introdotte più avanti dall'assistente del corso. Attenzione: esistono moltissimi libri e blog tecnici, bisogna badare alla qualità scientifica.

Nota — Supporti al corso

- Un **assistente del corso** per aiutare nello sviluppo del progetto (da confermare).
- Un **chatbot tutor** (basato su ChatGPT), sperimentale, per il corso ML 2026: serve a chiarire dubbi, approfondire concetti e avere indicazioni su slide, definizioni ed esercizi, sempre sulla base dei materiali ufficiali del corso. Dettagli e link saranno pubblicati nelle news di Moodle.

In parallelo è molto utile il corso di **Computational Mathematics for Learning and Data Analysis** (CM), che fornisce le basi matematiche profonde dei metodi di apprendimento (non è però obbligatorio per seguire ML).

1.4 Richiami matematici

Per seguire il corso servono alcuni concetti di base: calcolo in più variabili (derivate parziali, gradiente), probabilità (densità, media, varianza, normale, probabilità condizionate e con-

giunte), calcolo matriciale (prodotto scalare, inversa, norme). Argomenti come il problema dei minimi quadrati lineari con la SVD e i moltiplicatori di Lagrange possono essere approfonditi in parallelo. Riferimenti consigliati: appendice A di AIMA, capitoli 2–4 del *Deep Learning book*, e *Mathematics for Machine Learning* (2020).

La notazione usata è: x scalare, $\mathbf{x}$ vettore, X matrice.

1.4.1 Prodotto scalare e norme

Definizione — Prodotto scalare (*inner/dot product*)

$$\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + a_2 b_2 + \dots + a_n b_n = \sum_{i=1}^n a_i b_i = |\mathbf{a}| |\mathbf{b}| \cos \theta$$

Notazioni equivalenti: $\mathbf{a} \cdot \mathbf{b} = \mathbf{a}^T \mathbf{b} = \mathbf{a}^t \mathbf{b} = \langle \mathbf{a}, \mathbf{b} \rangle$, e anche solo $\mathbf{ab}$ se il contesto è chiaro.

La relazione con il coseno dà l'intuizione geometrica: il prodotto scalare misura **quanto due vettori puntano nella stessa direzione**. È massimo quando sono paralleli, nullo quando sono ortogonali, negativo quando puntano in versi opposti. Questa lettura tornerà di continuo (ad esempio nei modelli lineari, dove $\mathbf{w}^T \mathbf{x}$ misura l'“allineamento” dell'input con i pesi).

La **norma euclidea** (modulo, lunghezza) di un vettore è

$$\|\mathbf{x}\|_2 = \sqrt{\mathbf{x}^T \mathbf{x}} = \sqrt{\sum_i x_i^2} = d(\mathbf{x}, \mathbf{0}),$$

spesso scritta semplicemente $\|\mathbf{x}\|$ o $|\mathbf{x}|$. Altre norme utili sono la **norma L^1** , $\|\mathbf{x}\|_1 = \sum_i |x_i|$, e la **norma del massimo**, $\|\mathbf{x}\|_\infty = \max_i |x_i|$.

Più in generale, un **prodotto interno** $\langle \cdot, \cdot \rangle$ è una forma bilineare simmetrica che associa a una coppia di vettori uno scalare:

$$\langle v, w \rangle = \langle w, v \rangle, \quad \langle v + w, u \rangle = \langle v, u \rangle + \langle w, u \rangle, \quad \langle kv, w \rangle = k \langle v, w \rangle.$$

Vale la **disuguaglianza di Cauchy-Schwarz**: $|\langle x, y \rangle| \leq \|x\| \cdot \|y\|$ per ogni $x, y \in V$.

Nota — Tensori

Un **tensore**, nella visione semplificata usata in ML, è un array multidimensionale di numeri disposti su una griglia regolare con un numero variabile di assi (ad esempio $X_{i,j,k}$ ha 3 indici). In ML si usa soprattutto come comodità di notazione e per ragionare sul calcolo su GPU; in matematica i tensori hanno proprietà più ricche (legate ai cambi di base).

1.4.2 Derivate parziali e gradiente

La **derivata parziale** generalizza la derivata a funzioni di più variabili: è la derivata rispetto a una sola variabile tenendo fisse tutte le altre. Per $f(x_1, x_2, x_3)$ si calcolano $\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \frac{\partial f}{\partial x_3}$.

Definizione — Gradiente

Il **gradiente** di f è il vettore delle sue derivate parziali:

$$\nabla f = \text{grad } f = \left(\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n} \right) = \sum_{i=1}^n \frac{\partial f}{\partial x_i} \mathbf{e}_i$$

dove gli $\mathbf{e}_i$ sono i versori ortogonali lungo le direzioni coordinate.

Il significato geometrico è fondamentale per tutto il corso: in un punto, il gradiente è un vettore che **punta nella direzione di massima pendenza**, e la sua norma indica **quanto è ripida** la pendenza. Il gradiente indica quindi dove la funzione cresce più rapidamente, mentre il suo opposto $-\nabla f$ indica la direzione in cui **decresce** più rapidamente. È esattamente questa l'idea alla base della **discesa del gradiente** (*gradient descent*), che useremo per addestrare i modelli minimizzando una funzione di errore.

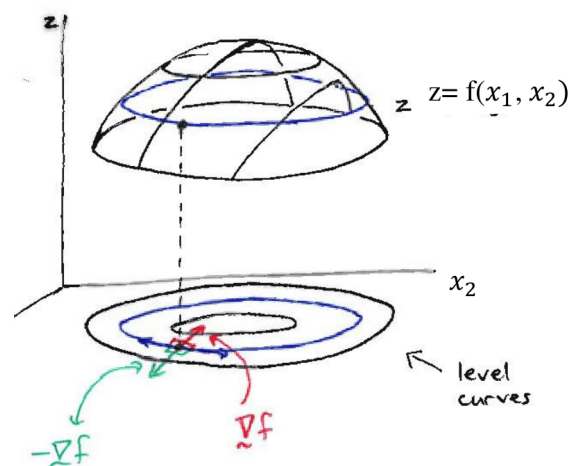


Figura 1.3: Gradiente su una superficie: ∇f punta verso la crescita di f , $-\nabla f$ verso la decrescita; entrambi sono perpendicolari alle curve di livello.

Un punto in cui il gradiente è nullo si dice **punto stazionario**. Può essere un minimo locale, un massimo locale o un punto di sella: il gradiente nullo da solo non basta a dire di quale caso si tratta.

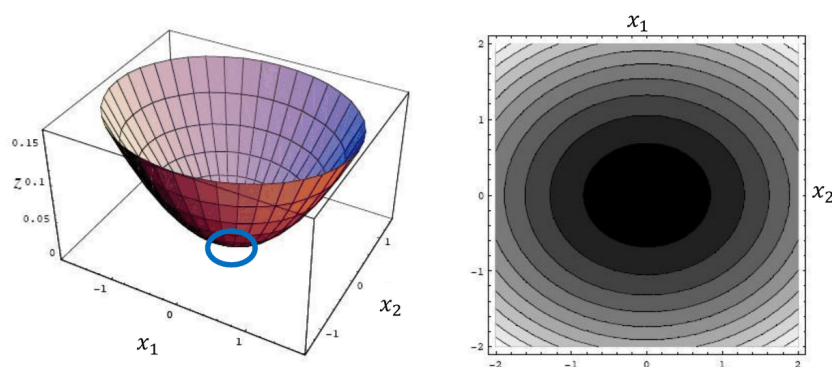


Figura 1.4: Un minimo: nel punto più basso del paraboloide il gradiente è zero.

1.4.3 Densità di probabilità: la gaussiana

La **distribuzione normale** con media μ e varianza σ^2 ha densità di probabilità (la “funzione gaussiana”)

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-(x-\mu)^2/(2\sigma^2)}.$$

Con $\mu = 0$ e $\sigma^2 = 1$ si ottiene la **normale standard**.

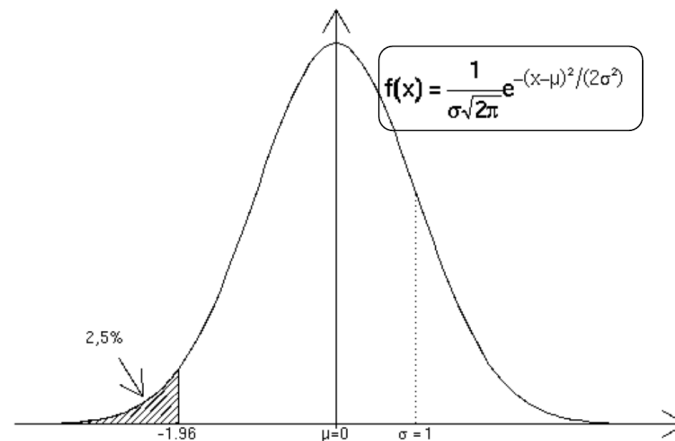


Figura 1.5: Densità della normale standard: l’area a sinistra di $-1,96$ vale il 2,5%.

1.5 Sintesi e prossimi passi

In sintesi — Sintesi

Il ML è l’apprendimento automatico di un compito a partire da esempi: è necessario quando i dati abbondano ma le regole esplicite mancano. Il corso lo affronta come **approssimazione di funzioni a partire da esempi**, costruendo i modelli per “mattoni” successivi, con attenzione ai principi (che non cambiano al variare dei modelli) e alla valutazione rigorosa. Il concetto chiave che attraversa tutto il corso è il compromesso tra **flessibilità del modello** e **controllo della complessità**, da cui dipende la **generalizzazione**.

Le lezioni successive completano l’introduzione: nelle lezioni 2–3 si vedono l’utilità del ML, l’apprendimento come approssimazione di funzioni con un esempio guida, e le componenti di progetto di un sistema di ML (task, spazio delle ipotesi, bias induttivo, funzione di loss, generalizzazione); nella lezione 4 si approfondiscono generalizzazione e validazione.

Domande — Possibili domande d’esame

- Perché il ML è una necessità e non un lusso? In quali condizioni è appropriato usarlo?
- Che cos’è il gradiente e perché $-\nabla f$ indica la direzione di massima decrescita?
- Che cos’è un punto stazionario? È sempre un minimo?

Capitolo 2

Il ML nei curricula della magistrale

Questo pacco di slide è un “intermezzo” della prima lezione, che il professore chiede di leggere autonomamente. Descrive come il corso di Machine Learning si inserisce nella **Laurea Magistrale in Informatica** dell’Università di Pisa e nei suoi curricula. Non contiene materiale d’esame, ma aiuta a capire il ruolo del corso nel percorso di studi.

2.1 I curricula della magistrale

Dal 2017 la magistrale in Informatica è organizzata in quattro curricula:

- **Artificial Intelligence** (AI);
- **Data and Knowledge**, dal 2021 **Big Data Technologies** (BD);
- **ICT Solutions Architect** (ICT);
- **Software: Programming, Principles, and Technologies** (SW).

L’organizzazione per curricula permette di specializzarsi in un’area (con un diploma supplementare che certifica il curriculum), offre corsi metodologici di base all’inizio del percorso e mira a mostrare come due anni di studio in più siano utili per il futuro professionale. Regolamenti, istruzioni per gli esami e modalità di cambio di curriculum o di piano di studi sono sul sito del dipartimento.

2.2 Il ruolo del Machine Learning

Il ML è collegato a **tutti e quattro** i curricula, ma è un corso **caratterizzante per il curriculum AI**, dove fa parte della base metodologica per costruire sistemi adattivi e intelligenti insieme a:

- *Artificial Intelligence Fundamentals* (6 CFU);
- *Computational Mathematics for Learning and Data Analysis* (9 CFU);
- *Machine Learning* (9 CFU);
- *Parallel and Distributed Systems: Paradigms and Models* (9 CFU).

Gli altri caratterizzanti del curriculum AI sono *Human Language Technologies*, *Intelligent Systems for Pattern Recognition* e *Smart Applications*, affiancati da gruppi di esami a scelta (Algorithm Engineering, Data Mining, Mobile and Cyber-Physical Systems, Information Retrieval, Computational Neuroscience, Robotics, Semantic Web, ecc.).

Il ML è il perno dell’**area dei sistemi intelligenti**: molti corsi della magistrale (Human Language Technologies, Intelligent Systems for Pattern Recognition, Smart Applications, Computational Neuroscience, Robotics, Data Mining, Information Retrieval, Bioinformatics, Big Data Analytics...) ne usano direttamente i metodi.

Nota — Piano di studi del curriculum AI (dal 2021)

60 CFU di corsi caratterizzanti, 27 CFU da gruppi di esami a scelta (un esame da 9 CFU e tre da 6 CFU) e almeno 9 CFU a libera scelta, che si possono coprire anche con due esami da 6 CFU.

2.3 Sinergia con Computational Mathematics

Il professore consiglia di seguire in parallelo **ML** e **CM** (*Computational Mathematics for Learning and Data Analysis*): il primo fornisce i metodi, il secondo le basi matematiche, e i due corsi si stimolano a vicenda. CM non è comunque un prerequisito: molti studenti di altri corsi di laurea o curricula hanno seguito ML senza CM.

2.4 Note per categorie particolari di studenti

Attenzione — Per chi viene dalla triennale di Pisa (corso IIA)

Le prime sei lezioni sembreranno “facili”, ma **non coincidono** con i contenuti di IIA: contengono parti nuove e concetti matematici diversi. Il cambio di prospettiva è netto: in IIA i modelli semplici erano lo *strumento* per introdurre i concetti di ML, qui i modelli e i principi allo stato dell’arte sono *l’oggetto* di studio. Dalla lezione 5–6 in poi tutto è nuovo.

Per gli studenti del vecchio esame **AA1** (320AA, 6 CFU), che non esiste più: chi deve passare da AA1 a ML (9 CFU) sostiene un’integrazione su una lista di argomenti indicata dal professore; chi deve fare il percorso inverso deve contattare il docente.

Per qualsiasi informazione il riferimento è il Prof. Alessio Micheli (micheli@di.unipi.it), del gruppo *Computational Intelligence & Machine Learning* del Dipartimento di Informatica.

Capitolo 3

Concetti fondamentali del ML

Le lezioni 2 e 3 forniscono la **panoramica e la terminologia** del Machine Learning prima di entrare nei modelli veri e propri. Si parte dal quadro generale — l'apprendimento come approssimazione di una funzione sconosciuta a partire da esempi — per poi descrivere gli “ingredienti” di un sistema di ML (dati, task, modello, algoritmo di apprendimento, validazione) e tre concetti cruciali: il **bias induttivo**, la **funzione di loss** e la **generalizzazione**. Molti concetti qui solo accennati verranno ripresi e approfonditi nel resto del corso.

3.1 Il Machine Learning in ambito computazionale

Restringendoci al quadro computazionale, il ML studia **principi, metodi e algoritmi per l'apprendimento e la predizione**: il sistema apprende dall'esperienza (dati noti) per affrontare un compito computazionale, costruendo un **modello** (o **ipotesi**) da usare per fare predizioni.

Il quadro specifico più diffuso, e quello adottato dal corso, è il seguente: **inferire un modello, cioè una funzione, a partire da un insieme di esempi, in modo che sia capace di generalizzare**, ovvero di dare risposte accurate su dati nuovi.

3.1.1 Quando usare il ML

Il ML va usato con **opportunità** (quando serve davvero) e **consapevolezza** (dei suoi requisiti e limiti). I modelli predittivi sono utili quando:

- non c'è una teoria (o c'è poca conoscenza) che spieghi il fenomeno;
- i dati sono **incerti, rumorosi o incompleti**, e questo ostacola la formalizzazione di una soluzione esatta.

In cambio, il ML richiede due cose: una **fonte di esperienza** per l'addestramento (dati *representativi*) e una certa **tolleranza sulla precisione** dei risultati, che non saranno mai garantiti esatti.

Il ML è complementare alle tecniche tradizionali (modelli analitici basati su conoscenza pregressa, algoritmi e programmazione imperativa, AI classica). È particolarmente adatto quando:

- **la conoscenza è troppo difficile da formalizzare** a mano: gli esseri umani riconoscono i volti ma non sanno descrivere *come* lo fanno; lo stesso vale per il riconoscimento vocale;
- **la conoscenza umana non basta**: ad esempio prevedere la forza di legame di molecole con proteine;
- serve un **comportamento personalizzato**: filtrare email o pagine web secondo le preferenze dell'utente, interfacce uomo-macchina individualizzate.

Le grandi sfide del ML sono costruire **sistemi autonomi intelligenti** (robotica, interazione uomo-robot, motori di ricerca), fornire **strumenti potenti per l'analisi dei dati** (gli stru-

menti del *data scientist*) e aprire **nuove aree applicative** interdisciplinari, in un'epoca in cui il progresso scientifico è sempre più *data-driven*.

3.2 L'apprendimento come approssimazione di funzioni

Il corso adotta una visione specifica ma molto diffusa: **l'apprendimento è l'approssimazione di una funzione sconosciuta a partire da esempi**. Il vantaggio è che task diversi (classificazione, regressione, ...) vengono trattati in un quadro uniforme — matematicamente, quello degli spazi di funzioni (spazi di Hilbert) — che permette una formulazione rigorosa.

3.2.1 L'esempio guida: riconoscimento di cifre

Riprendiamo l'esempio della prima lezione. L'input è una collezione di immagini di cifre scritte a mano (matrici 8×8 di valori); il problema è costruire un modello che, ricevuta un'immagine, **predica** la cifra. Si tratta di un **problema di classificazione** con dieci classi.

È un caso tipico di successo del ML: è **difficile formalizzare esattamente** la soluzione (le calligrafie variano, ci sono rumore e dati ambigui), mentre è **relativamente facile raccogliere esempi etichettati**.

Definizione — Machine Learning (definizione estesa)

Il ML studia e propone metodi per costruire (**inferire**) dipendenze, funzioni o ipotesi a partire da esempi di dati osservati, che: - si **adattino** agli esempi noti (*fit*); - siano in grado di **generalizzare**, con accuratezza ragionevole, su dati nuovi; secondo risultati **verificabili**, sotto condizioni e criteri statistici e computazionali, tenendo conto dell'**espressività** e della **complessità algoritmica** dei modelli e degli algoritmi di apprendimento.

3.2.2 Esempi di coppie $x \rightarrow f(x)$

In tutti i casi seguenti si vuole inferire una funzione generale partendo da dati noti, cioè da un **training set** (TR) di coppie $\langle x, f(x) \rangle$:

Problema	Input x	Output $f(x)$
Riconoscimento della scrittura	dati del movimento della penna	lettera dell'alfabeto
Diagnosi medica	proprietà del paziente (sintomi, esami)	malattia o terapia consigliata
Riconoscimento facciale	bitmap del volto	nome della persona
Rilevamento spam	messaggio email	spam / non spam
<i>Protein folding</i>	sequenza di amminoacidi (stringa di lunghezza variabile)	coordinate 3D degli atomi (sequenza di vettori)
<i>Drug design</i>	molecola (grafo di atomi e legami)	forza di legame con la proteasi dell'HIV (numero reale)

Gli ultimi due esempi mostrano che input e output possono essere **dati complessi**: stringhe, sequenze, grafi.

3.3 Gli ingredienti di un sistema di ML

Un sistema di ML predittivo si può descrivere attraverso i suoi componenti, che costituiscono anche una **guida alle scelte progettuali chiave**.

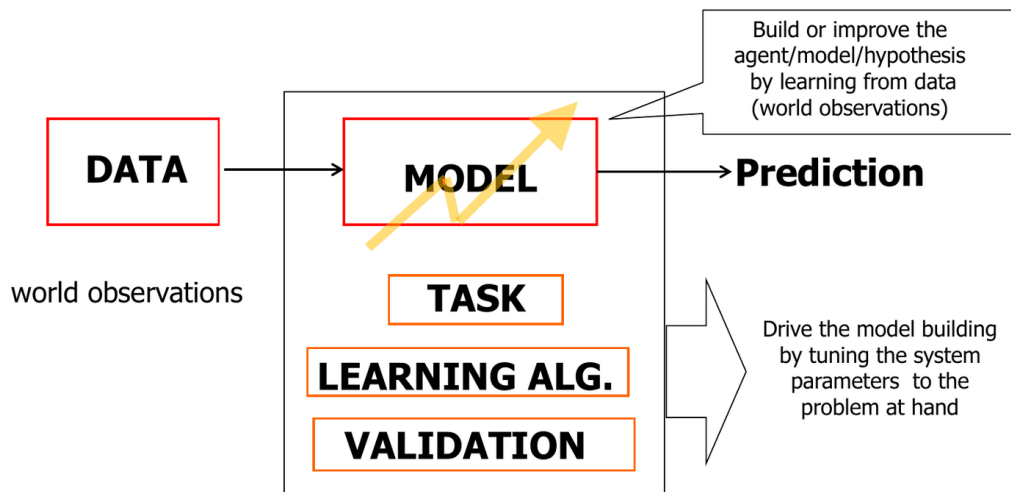


Figura 3.1: Panoramica di un sistema di ML predittivo e dei suoi “ingredienti”.

- **Dati:** le osservazioni del mondo, cioè l’esperienza disponibile.
- **Task:** lo scopo dell’applicazione, che definisce cosa vogliamo ottenere.
- **Modello:** l’agente/ipotesi che viene costruito o migliorato apprendendo dai dati.
- **Algoritmo di apprendimento:** guida la costruzione del modello regolando i parametri del sistema sul problema.
- **Validazione:** valuta la qualità del modello ottenuto.

3.3.1 Dati

I dati rappresentano i fatti disponibili. Il primo problema è quello della **rappresentazione**: come catturare la struttura degli oggetti analizzati.

Nel caso **piatto** (*flat*, linguaggio attributo-valore) ogni oggetto è un **vettore di dimensione fissa di proprietà (feature)**, e l’intero dataset è una tabella di tuple. Gli attributi possono essere **categorici/discreti** (colore: giallo, rosso) o **continui** (peso, costo), e possono esserci **dati mancanti** (indicati con “?”).

Nota — Terminologia sui dati

- Ogni riga $\mathbf{x}$ (vettore, in grassetto) è un **esempio**, *pattern*, istanza, campione.
- La **dimensione del dataset** è il numero di esempi, indicato con l .
- La **dimensione dell’input** è il numero di feature, indicato con n .
- x_i (o x_j) è tipicamente la i -esima feature (attributo, componente) di $\mathbf{x}$.
- $\mathbf{x}_p$ è tipicamente il p -esimo pattern (riga) del dataset.
- $x_{p,i}$ è l’attributo i del pattern p .

I dati possono essere **pre-elaborati**: scalatura delle variabili, codifica, selezione delle feature.

Codifica delle variabili categoriche

Per usare variabili categoriche in modelli numerici bisogna codificarle:

- per **due classi** si usa 0/1 oppure $-1/+1$;
- per **più classi** si può usare 1, 2, 3, ..., ma questa codifica introduce un **grado di somiglianza** artificiale (la classe 1 risulta “più vicina” alla 2 che alla 3). È adatta solo per variabili **categoriche ordinate** (es. piccolo, medio, grande);
- per simboli senza ordine si usa la codifica **1-of-k** (o **one-hot**): ogni simbolo diventa un vettore con un solo 1, ad esempio $A = (1, 0, 0)$, $B = (0, 1, 0)$, $C = (0, 0, 1)$. Tutti i simboli sono così equidistanti tra loro. Si usa sia per l’input sia per l’output, ed è utile per il progetto.

Dati strutturati

Molti dati non sono naturalmente vettori: **sequenze** (liste), **alberi**, **grafi**, dati multi-relazionali. Esempi: immagini, serie temporali, stringhe di un linguaggio, DNA e proteine, relazioni gerarchiche, molecole, la rete di link delle pagine web. Il problema è trovare una rappresentazione naturale per essi (se ne parlerà nella parte avanzata del corso).

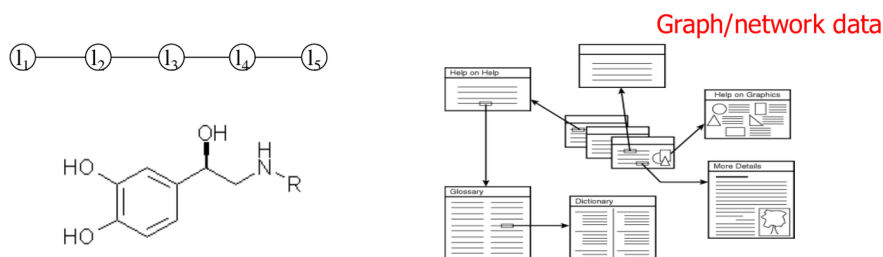


Figura 3.2: Dati strutturati: sequenze, molecole e grafi.

Rumore, outlier e feature selection

- Il **rumore** (noise) è l’aggiunta di fattori esterni al segnale informativo: è dovuto alla casualità delle misure, non alla legge sottostante (es. rumore gaussiano).
- Gli **outlier** sono valori insoliti, incoerenti con la maggior parte delle osservazioni (es. errori di misura anomali). Si gestiscono rilevandoli e rimuovendoli in pre-elaborazione oppure usando metodi di modellazione **robusti**.
- La **feature selection** seleziona un piccolo numero di feature informative, fornendo una rappresentazione ottimale dell’input per il problema.

3.3.2 Task

Il task definisce lo scopo dell’applicazione: quale conoscenza vogliamo ottenere, qual è la natura utile del risultato, quali informazioni sono disponibili. Distinguiamo:

- **task predittivi** (classificazione, regressione): approssimazione di funzioni — sono il focus del corso;
- **task descrittivi** (cluster analysis, regole di associazione): trovare sottoinsiemi o gruppi di dati non classificati.

Apprendimento supervisionato

Definizione — Apprendimento supervisionato

Dati: esempi di training nella forma $\langle \text{input}, \text{output} \rangle = \langle \mathbf{x}, d \rangle$ (esempi **etichettati**) per una funzione sconosciuta f , nota solo nei punti degli esempi. Il **target** d (indicato anche con t o y) è fornito da un “insegnante” secondo $f(\mathbf{x})$.

Obiettivo: trovare una buona approssimazione di f , cioè un’**ipotesi** h utilizzabile per predire su dati mai visti $\mathbf{x}'$ — un’ipotesi che **generalizzi**.

Il target può essere:

- **categorico** → **classificazione**: $f(\mathbf{x}) \in \{1, 2, \dots, K\}$ (funzione a valori discreti);
- **numerico** → **regressione**: $f(\mathbf{x}) \in \mathbb{R}$ o $\mathbb{R}^K$ (funzione a valori reali continui).

Grazie al formalismo dell’approssimazione di funzioni, classificazione e regressione hanno una **visione unificata**. In terminologia statistica gli input sono le **variabili indipendenti** e gli output le **variabili dipendenti** o **risposte**.

Apprendimento non supervisionato

Nell’apprendimento **non supervisionato** non c’è insegnante: il training set contiene solo dati **non etichettati** $\langle \mathbf{x} \rangle$. L’obiettivo è ad esempio trovare **raggruppamenti naturali** nei dati (**clustering**, cioè partizionare i dati in sottoinsiemi di elementi “simili”), fare **riduzione di dimensionalità**, visualizzazione o pre-elaborazione, oppure **modellare la densità** dei dati.

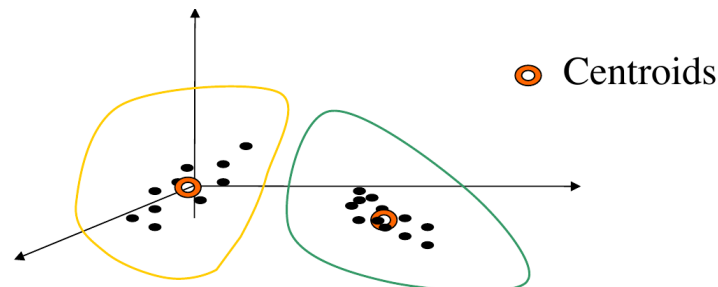


Figura 3.3: Clustering: partizione dei dati in gruppi, ognuno rappresentato dal suo centroide.

Classificazione

Nella classificazione i pattern sono visti come membri di una classe, e l’obiettivo è assegnare a ogni pattern la classe corretta. Se le classi sono **due**, $f(\mathbf{x})$ è una **funzione booleana**: si parla di **classificazione binaria** o **concept learning** (vero/falso, 0/1, $-1/+1$, negativo/positivo). Con **più di due** classi si ha un problema **multi-classe**.

Geometricamente, la classificazione può essere vista come la **suddivisione dello spazio degli input in regioni di decisione**. L’esempio più semplice è il separatore lineare in $\mathbb{R}^2$: una retta (in generale un iperpiano) che divide i punti della classe 1 da quelli della classe 0.

L’iperpiano separatore è l’insieme dei punti $\mathbf{x}$ tali che

$$\mathbf{w}^T \mathbf{x} + w_0 = w_1 x_1 + w_2 x_2 + w_0 = 0,$$

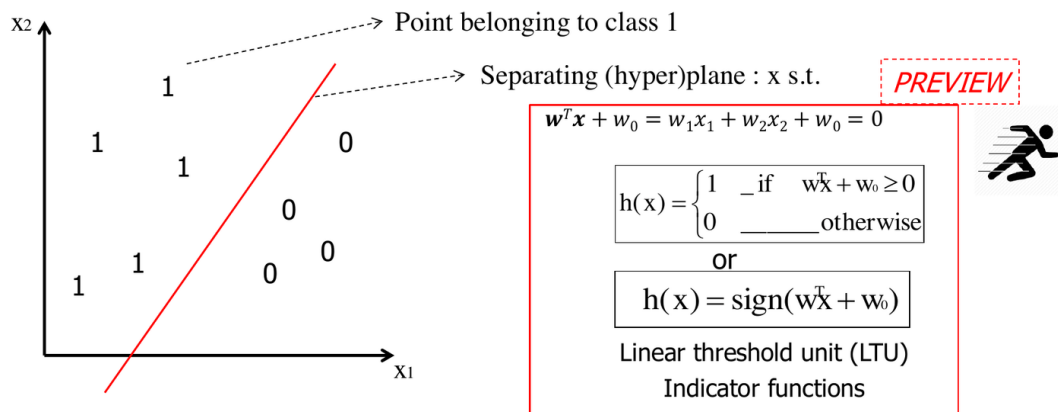


Figura 3.4: Un separatore lineare nel piano: anticipazione della Linear Threshold Unit.

e il classificatore è

$$h(\mathbf{x}) = \begin{cases} 1 & \text{se } \mathbf{w}^T \mathbf{x} + w_0 \geq 0 \\ 0 & \text{altrimenti} \end{cases} \quad \text{oppure} \quad h(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x} + w_0).$$

Questo modello si chiama **Linear Threshold Unit** (LTU) e sarà studiato in dettaglio più avanti. Funzioni di questo tipo, a valori in $\{0, 1\}$, sono dette **funzioni indicatrici**. L'insieme H di tutte le dicotomie (divisioni in due classi) indotte dagli iperpiani è il suo spazio delle ipotesi.

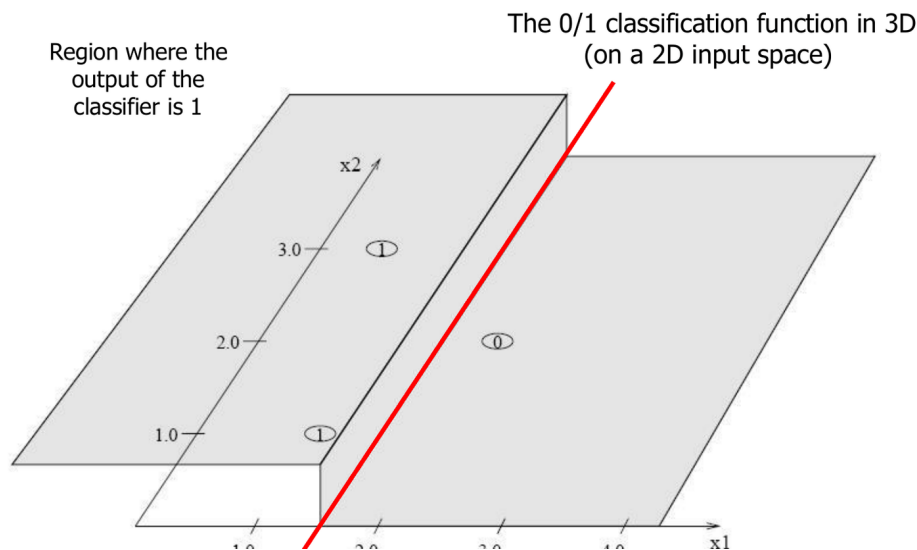


Figura 3.5: La stessa funzione di classificazione vista in 3D: è una funzione “a gradino” che vale 1 in una regione e 0 nell'altra.

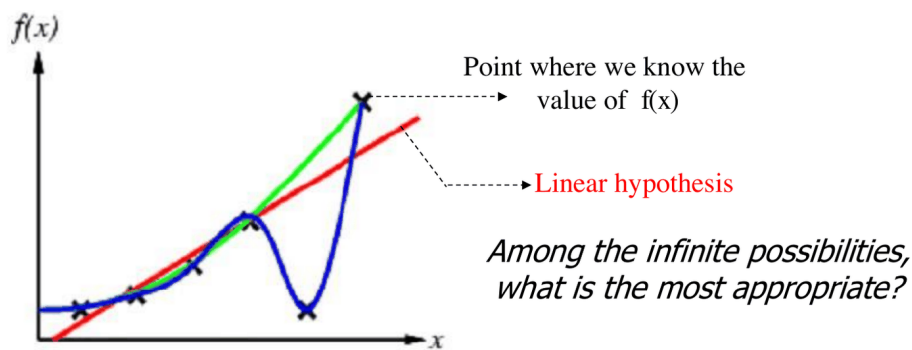
Regressione

La **regressione** è la stima di una funzione a valori reali sulla base di un insieme finito di **campioni rumorosi**: si conoscono coppie $(\mathbf{x}, f(\mathbf{x}) + \text{rumore})$. È una forma di *curve fitting* (in una dimensione nell'esempio, a k dimensioni in generale).

Esempio — Esercizio di regressione

Dati i punti $(1; 2,1)$, $(2; 3,9)$, $(3; 6,1)$, $(4; 8,4)$, $(5; 9,8)$, qual è f ? Viene spontaneo ipotizzare $f(x) = 2x$, che ha errori piccoli su tutti i punti. Ma come abbiamo fatto a sceglierla? E come lo farebbe una macchina (ad esempio una rete neurale)? È esattamente ciò che l'algoritmo

di apprendimento deve formalizzare.



An example (**linear hypothesis**): $h_w(x) = w_1 x + w_0 = 0.2x - 0.4$

Figura 3.6: Tra le infinite funzioni compatibili con i dati, qual è la più appropriata?

La figura pone la domanda chiave: tra le infinite possibilità, quale funzione scegliere? La curva blu passa esattamente per tutti i punti ma oscilla in modo innaturale; la retta rossa ($h_w(x) = w_1 x + w_0 = 0.2x - 0.4$) commette piccoli errori ma è molto più semplice. Questo dilemma è il cuore del problema della **generalizzazione**.

Altri paradigmi

- **Semi-supervised learning:** combina esempi etichettati e non etichettati.
- **Self-supervised learning:** è un apprendimento supervisionato in cui le etichette vengono **generate automaticamente dalla struttura stessa dei dati**, senza etichettatura manuale. Il modello viene **pre-addestrato** su grandi quantità di dati non etichettati, imparando rappresentazioni generali, che poi si adattano a task specifici con il **fine-tuning**. Esempi attuali, alla base degli LLM:
 - la **predizione della parola successiva** in un testo, cioè l'apprendimento self-supervised **autoregressivo** della famiglia GPT (*Generative Pre-trained Transformer*, OpenAI);
 - la ricostruzione di **parole mascherate** (BERT, Google);
 - per le immagini, i **masked autoencoder** (Meta), che ricostruiscono porzioni nascoste dell'immagine.
- **Reinforcement learning:** apprendimento con un “critico” che dice giusto/sbagliato. L'algoritmo apprende una **politica** su come agire date le osservazioni del mondo; ogni azione ha un impatto sull'ambiente, che fornisce un feedback. Non ci sono esempi passo-passo; l'obiettivo è il *decision-making*, ed è molto usato nella AI moderna.

3.3.3 Modello

Il **modello** cattura le relazioni tra i dati (in base al task) attraverso un “linguaggio” (numerico, simbolico, ...), legato alla rappresentazione usata. Soprattutto, **il modello definisce la classe di funzioni che la macchina può implementare: lo spazio delle ipotesi**. Ad esempio, l'insieme delle funzioni $h(\mathbf{x}, \mathbf{w})$ al variare del parametro $\mathbf{w}$.

Definizione — Terminologia di base

- **Esempio di training** (supervisionato): coppia $(\mathbf{x}, f(\mathbf{x}) + \text{rumore})$, dove $\mathbf{x}$ è il vettore delle feature e $d = f(\mathbf{x}) + \text{rumore}$ è il **target**.
- **Funzione target** f : la vera funzione (sconosciuta).
- **Ipotesi** h : una funzione proposta, che si ritiene simile a f ; un'espressione in un dato linguaggio che descrive le relazioni tra i dati.
- **Spazio delle ipotesi** H : l'insieme di tutte le ipotesi che l'algoritmo di apprendimento può, in linea di principio, produrre.

Alcuni esempi di rappresentazioni diverse:

- **Modelli lineari**: H è uno spazio parametrizzato in modo **continuo**, e ogni assegnamento di $\mathbf{w}$ è un'ipotesi diversa. Esempi: il classificatore binario $h(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x} + w_0)$ o la regressione lineare semplice $h_{\mathbf{w}}(x) = w_1 x + w_0$ (es. $2x + 150$).
- **Regole simboliche**: H è **discreto**, ad esempio “se $x_1 = 0$ e $x_2 = 1$ allora $h(\mathbf{x}) = 1$, altrimenti 0”.
- **Modelli probabilistici**: stimano $p(\mathbf{x}, y)$.
- **K-nearest neighbors** (regressione): predice la media dei valori y dei vicini più prossimi (modello basato sulla memoria).
- **Reti neurali**: al di là dell'ispirazione biologica, sono un modello computazionale capace di approssimare relazioni complesse e **non lineari** tra input e output. Anche esse, ancora una volta, definiscono una **classe di funzioni**.

I principali paradigmi (linguaggi per H) sono:

Paradigma	Esempi
Simbolici / a regole (H discreto)	congiunzioni di letterali, alberi di decisione, grammatiche induttive, algoritmi evolutivi, Inductive Logic Programming
Sub-simbolici (H continuo)	analisi discriminante lineare, regressione lineare multipla, LTU, reti neurali, metodi kernel (SVM)
Probabilistici / generativi	modelli parametrici tradizionali, reti bayesiane, Naïve Bayes, modelli di Markov, HMM
Instance-based	nearest neighbor

Alcuni modelli possono essere espressi in più linguaggi diversi.

Teorema — No Free Lunch Theorem

Non esiste un metodo di apprendimento universalmente “migliore” (senza alcuna conoscenza, per tutti i problemi): se un algoritmo ottiene risultati superiori su alcuni problemi, deve pagarli con risultati inferiori su altri (Devroye 1982; Wolpert e Macready 1997).

Per questo il corso fornisce sia un **insieme di modelli** sia gli **strumenti critici per confrontarli**. Tuttavia i modelli *non* sono tutti equivalenti: differiscono molto per **flessibilità** (capacità, in linea di principio, di approssimare funzioni arbitrarie, non solo lineari) e per il **controllo della complessità**. Modelli flessibili e principi per controllarne la complessità sono il nucleo del ML.

3.3.4 Algoritmo di apprendimento

L'algoritmo di apprendimento, sulla base di dati, task e modello, esegue una **ricerca (euristica) nello spazio delle ipotesi H** per trovare l'ipotesi migliore, cioè la migliore approssimazione della funzione target sconosciuta. Tipicamente cerca la h con **errore minimo**: i parametri liberi del modello vengono adattati al task (il miglior w nei modelli lineari, le migliori regole nei modelli simbolici...). Nell'esempio di regressione, la scelta $w_1 = 2$, $w_0 = 0$ è il risultato di questa ricerca.

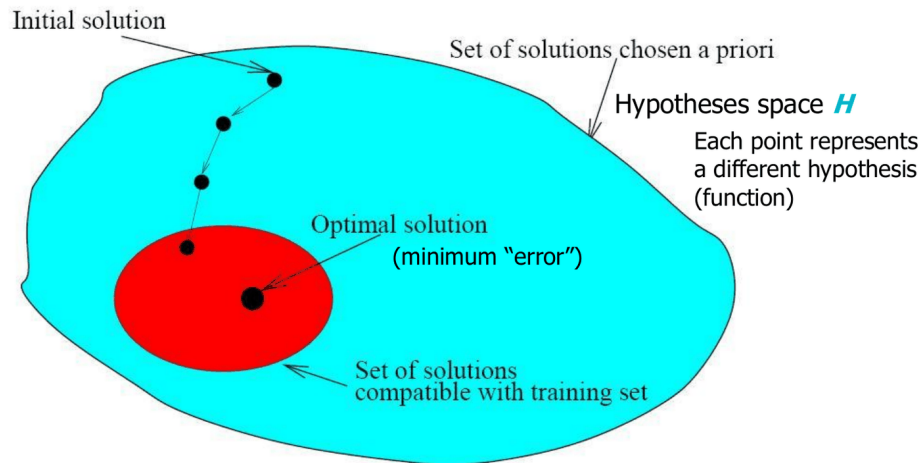


Figura 3.7: L'apprendimento come ricerca nello spazio delle ipotesi: ogni punto è una funzione diversa; si parte da una soluzione iniziale e si procede, tipicamente con ricerca locale, verso quella a errore minimo.

H può non coincidere con l'insieme di tutte le funzioni possibili e la ricerca non può essere esaustiva: servono delle **assunzioni**, e qui entra in gioco il **bias induttivo**.

A seconda del contesto, “apprendimento” prende nomi diversi: **inferenza** (statistica), **abduzione/induzione** (logica), **adattamento** (biologia, sistemi), **ottimizzazione** (matematica), **addestramento/training** (reti neurali), **approssimazione di funzioni** (matematica), regressione, *curve fitting*.

Dopo questi quattro ingredienti restano tre concetti da discutere: il **bias induttivo**, la **loss** e la **generalizzazione**.

3.4 Il bias induttivo

Per costruire un modello e un algoritmo di apprendimento dobbiamo fare **assunzioni** sulla natura della funzione target. Queste possono riguardare:

- **vincoli sul modello**, cioè sullo spazio delle ipotesi H (l'insieme delle ipotesi che possiamo esprimere): è il **language bias**;
- **vincoli o preferenze nell'algoritmo di apprendimento**, cioè nella strategia di ricerca: è il **search bias**;
- entrambi.

Vedremo che queste assunzioni sono **indispensabili** per ottenere un modello utile, cioè capace di generalizzare. Lo studiamo con esempi in spazi di ipotesi discreti, imparando un **concetto** (una funzione booleana), come in Mitchell cap. 2 (es. $h_{cat}(x) = 1$ se l'animale x è un gatto, 0 altrimenti).

3.4.1 Imparare funzioni booleane: un problema mal posto

Supponiamo di voler imparare una funzione booleana sconosciuta $y = f(x_1, x_2, x_3, x_4)$ da 7 esempi.

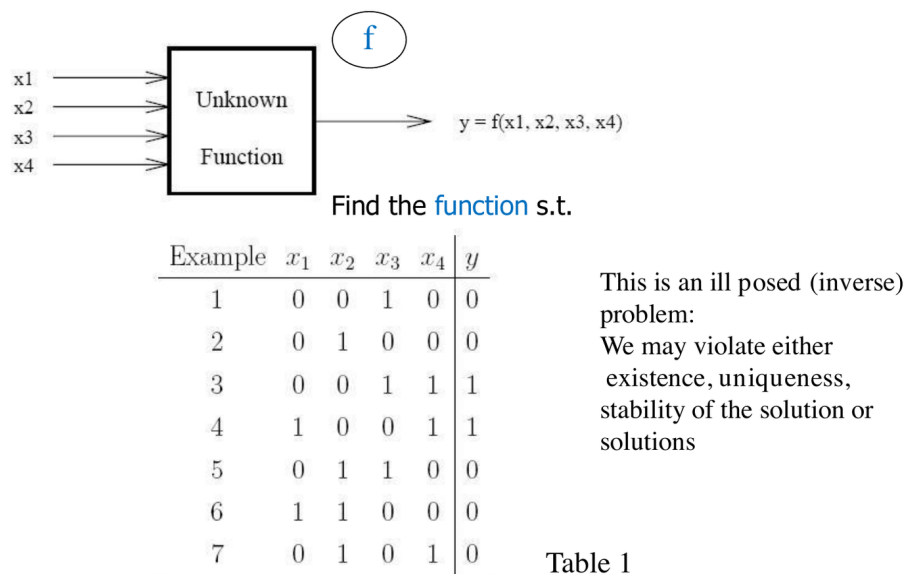


Figura 3.8: Apprendere una funzione booleana di quattro variabili da sette esempi.

Si tratta di un **problema (inverso) mal posto**: la soluzione può violare esistenza, unicità o stabilità. Con 4 input binari esistono $2^4 = 16$ possibili istanze di input e quindi $2^{16} = 65536$ possibili funzioni booleane. Non possiamo sapere quale sia quella corretta finché non vediamo **tutte** le coppie input-output: dopo 7 esempi restano ancora $2^9 = 512$ possibilità (le 9 istanze non viste possono avere output qualsiasi).

In generale, per input e output binari con dimensione dell'input n :

$$|H| = 2^{\text{\#istanze di input}} = 2^{2^n}.$$

Un modello che usa tutto questo spazio è una **lookup table**: un *rote learner* che memorizza gli esempi e classifica $\mathbf{x}$ solo se coincide con un esempio già visto (altrimenti “nessuna risposta”). **Nessun bias induttivo** → **nessuna generalizzazione**.

3.4.2 Restringere lo spazio: le regole congiuntive

Introduciamo un **language bias**: consideriamo solo **regole congiuntive**, cioè congiunzioni (AND) di letterali, come $h_1 = l_2$, $h_2 = l_1 \wedge l_2$, $h_3 = \text{true}$, $h_4 = \neg l_1 \wedge l_2$. Ad esempio “se $x_2 = 1$ allora $h(\mathbf{x}) = 1$, altrimenti 0”.

Il numero di ipotesi semanticamente distinte crolla: per ognuna delle n posizioni possiamo avere l_i , $\neg l_i$ oppure “non importa”, da cui 3^n ipotesi, più 1 perché tutte le congiunzioni che contengono $l_i \wedge \neg l_i$ equivalgono a “false”. Con $n = 4$ si passa da 65536 a $3^4 + 1 = 82$ ipotesi.

Definizione — Consistenza e Version Space

Un'ipotesi h è **consistente** con il training set TR se $h(\mathbf{x}) = d(\mathbf{x})$ per ogni esempio $\langle \mathbf{x}, d(\mathbf{x}) \rangle \in TR$.

Il **version space** $VS_{H,TR}$ è il sottoinsieme delle ipotesi di H consistenti con tutti gli esempi di training.

In questo spazio ridotto esistono algoritmi efficienti (Mitchell cap. 2) che trovano il version space senza enumerare tutte le ipotesi.

3.4.3 L'unbiased learner

L'assunzione congiuntiva è però troppo restrittiva: se il concetto target non è in H non può essere rappresentato. Ad esempio “se $x_1 = 1$ **oppure** $x_2 = 1$ allora 1” non è una congiunzione.

L'idea opposta è scegliere un H capace di esprimere **ogni concetto insegnabile**: l'insieme di tutti i sottoinsiemi dello spazio degli input X , cioè l'insieme delle parti $\mathcal{P}(X)$ (disgiunzioni, congiunzioni, negazioni). Con $n = 10$ input binari, $|X| = 2^{10} = 1024$ e $|\mathcal{P}(X)| = 2^{1024} \approx 10^{308}$ concetti — più degli atomi dell'universo. H contiene sicuramente il concetto target. Ma cosa succede alla generalizzazione?

Teorema — Un learner senza bias non può generalizzare

Con un *unbiased learner*, le uniche istanze classificate in modo non ambiguo dal version space sono gli esempi di training stessi: si ottiene di nuovo una **lookup table**.

Dimostrazione. Sia x_i un'istanza non vista (di test) e sia $h \in VS$. Consideriamo h' identica a h ovunque tranne che in x_i , dove $h'(x_i) \neq h(x_i)$. Poiché H contiene *tutte* le funzioni, $h' \in H$; e poiché h e h' coincidono sul training set, anche $h' \in VS$. Quindi per ogni ipotesi del VS che classifica x_i come positivo ce n'è un'altra che lo classifica negativo: ogni istanza non vista è classificata 1 da esattamente metà delle ipotesi del VS e 0 dall'altra metà. Il VS non può dare alcuna risposta.

Esempio — Esempio banale

Un solo input binario x , $H = \{x, \neg x, 0, 1\}$ (tutte le funzioni possibili). Il TR contiene solo $x = 0 \mapsto d = 0$. Il version space è $\{x, 0\}$: entrambe valgono 0 in $x = 0$. Per il test $x = 1$, però, x risponde 1 e 0 risponde 0. Non c'è modo di decidere, a meno di usare tutto X come training set.

Suggerimento — Futilità dell'apprendimento senza bias

Un learner che non fa **alcuna assunzione a priori** sull'identità del concetto target **non ha alcuna base razionale** per classificare istanze non viste. Il bias (di restrizione o di preferenza) non serve solo per l'efficienza: è **necessario per la generalizzazione**. Per imparare senza bias bisognerebbe mostrare ogni singola istanza di X come esempio. Il bias però non ci dice ancora *quale* soluzione generalizza meglio.

3.4.4 Sistemi induttivi e deduttivi equivalenti

Un modo elegante di vedere il ruolo del bias: un sistema induttivo (algoritmo di apprendimento che, usando lo spazio delle ipotesi H , riceve esempi e una nuova istanza e produce una classificazione o “non so”) è **equivalente a un sistema deduttivo** — un dimostratore di teoremi — che riceve gli stessi esempi e la stessa istanza **più il bias induttivo come assioma aggiuntivo**. Il bias induttivo è quindi esattamente l'insieme di assunzioni che rende “logicamente deducibile” la classificazione prodotta dal learner.

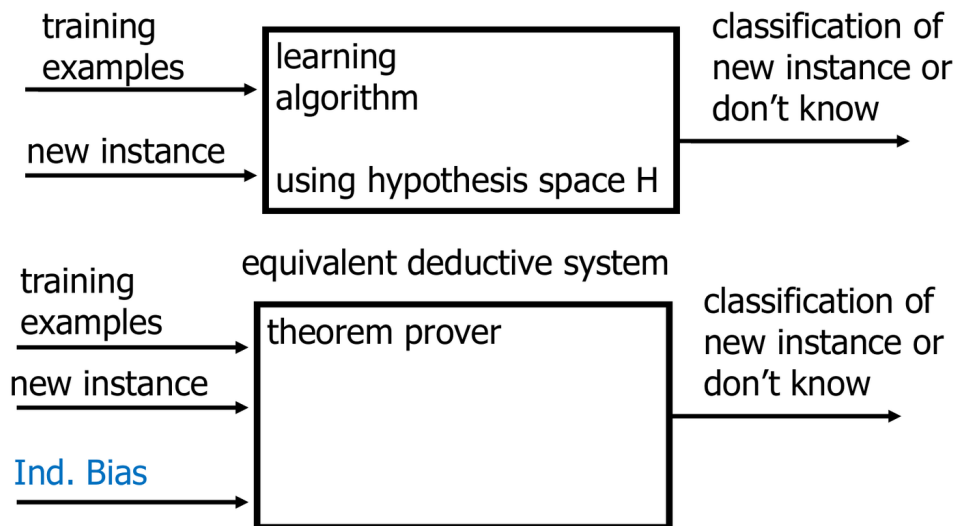


Figura 3.9: Il bias induttivo rende un sistema induttivo equivalente a un sistema deduttivo.

3.4.5 Language bias o search bias?

Nel ML moderno si preferisce in genere il **search bias**. Si usano approcci flessibili, con spazi di ipotesi molto espressivi e capacità di approssimazione universale (reti neurali, alberi di decisione), evitando il language bias e quindi **senza escludere a priori** la funzione target. Il bias induttivo resta, ma è affidato alla **strategia di ricerca** dell'algoritmo di apprendimento, in pratica usando una ricerca **incompleta** (che privilegia certe soluzioni rispetto ad altre).

In sintesi — Conclusioni sul bias induttivo

- Apprendere senza bias non permette di estrarre regolarità dai dati: si ottiene una lookup table, senza generalizzazione.
- **Ogni approccio di ML allo stato dell'arte ha un bias induttivo.**
- Il problema è caratterizzare il bias dei diversi modelli e algoritmi.

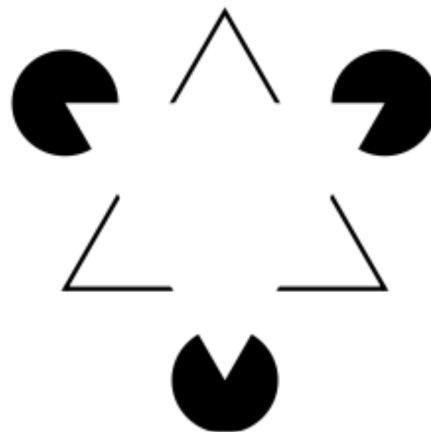


Figura 3.10: Il triangolo di Kanizsa: un esempio di bias percettivo del nostro sistema visivo, che “completa” forme non presenti.

3.5 Task e funzioni di loss

Abbiamo detto di voler trovare una “buona” approssimazione di f . Ma come si misura la qualità dell’approssimazione? Il modello produce $h(\mathbf{x})$ per l’input $\mathbf{x}$, e vogliamo misurare la “distanza” tra $h(\mathbf{x})$ e il target d . Si usa una **funzione di loss** (“interna”) $L(h_{\mathbf{w}}(\mathbf{x}), d)$ calcolata sul singolo pattern: un valore alto indica un’approssimazione scarsa.

Definizione — Errore (Rischio, Loss)

L’**errore** è un valore atteso della loss L , ad esempio la sua media sugli l campioni:

$$\text{Loss}(h_{\mathbf{w}}) = E(\mathbf{w}) = \frac{1}{l} \sum_{p=1}^l L(h_{\mathbf{w}}(\mathbf{x}_p), d_p).$$

Serve come **funzione obiettivo** da minimizzare durante il training e come misura di controllo dell’errore in fase di test. Per ora Errore, Rischio e Loss sono considerati sinonimi; le differenze verranno precisate più avanti.

La funzione L cambia a seconda del task. Vediamo i casi principali (utili come riferimento futuro).

3.5.1 Regressione: errore quadratico

- **Output:** $d_p = f(\mathbf{x}_p) + e$ (funzione reale più errore casuale).
- H : un insieme di funzioni a valori reali.
- **Loss:** l’**errore quadratico** $L(h_{\mathbf{w}}(\mathbf{x}_p), d_p) = (d_p - h_{\mathbf{w}}(\mathbf{x}_p))^2$.
- La media sul dataset è il **Mean Squared Error** (MSE).

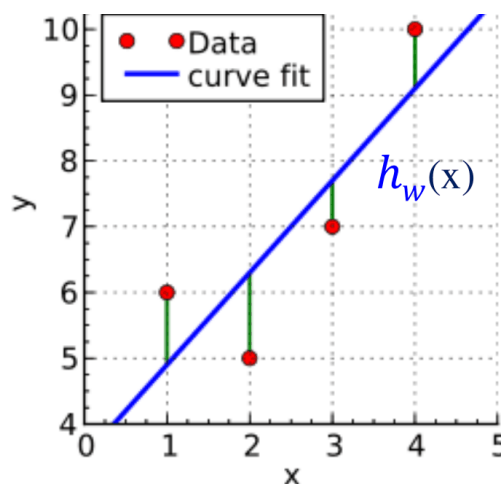


Figura 3.11: LMSE è la media dei quadrati dei segmenti verdi, cioè delle distanze verticali tra i dati (in rosso) e la retta $h_w(x) = w_1x + w_0$ (in blu).

$$E(\mathbf{w}) = \frac{1}{l} \sum_{p=1}^l (y_p - h_{\mathbf{w}}(\mathbf{x}_p))^2$$

dove $\mathbf{w}$ sono i parametri liberi del modello lineare (nella figura i target sono indicati con y).

3.5.2 Classificazione: loss 0/1

- **Output:** ad esempio $\{0, 1\}$.
- H : un insieme di funzioni indicatrici.
- **Loss 0/1:**

$$L(h_w(\mathbf{x}_p), d_p) = \begin{cases} 0 & \text{se } h_w(\mathbf{x}_p) = d_p \\ 1 & \text{altrimenti} \end{cases}$$

- La media sul dataset è la **percentuale di pattern classificati male**: se 20 pattern su 100 sono sbagliati si ha il 20% di errore, cioè l'80% di **accuratezza**.

3.5.3 Clustering e vector quantization (anticipazione)

- **Obiettivo:** partizionare in modo ottimo una distribuzione sconosciuta nello spazio degli input in regioni (cluster), ognuna approssimata da un **centro** o **prototipo**.
- H : un insieme di *vector quantizer* $\mathbf{x} \mapsto c(\mathbf{x})$, che mappano lo spazio continuo in uno discreto.
- **Loss:** la **distorsione quadratica**, che misura la vicinanza del pattern al centroide del suo cluster:

$$L(h(\mathbf{x}_p)) = (\mathbf{x}_p - h(\mathbf{x}_p)) \cdot (\mathbf{x}_p - h(\mathbf{x}_p)) = \|\mathbf{x}_p - h(\mathbf{x}_p)\|^2.$$

3.5.4 Stima di densità (anticipazione)

- **Obiettivo:** stimare una densità da una classe assunta (metodi generativi, parametrici).
- **Output:** una densità, ad esempio una normale $p(\mathbf{x} | \mu, \sigma^2)$.
- H : un insieme di densità (qui μ e σ^2 sono i due parametri sconosciuti).
- **Loss:** $L(h(\mathbf{x}_p)) = -\ln h(\mathbf{x}_p)$, legata alla **massimizzazione della (log-)verosimiglianza**: minimizzare la somma dei $-\ln$ equivale a massimizzare il prodotto delle probabilità dei dati osservati.

3.6 Generalizzazione e validazione

Questo è il **concetto fondamentale del corso**.

- **L'apprendimento** è la ricerca di una buona funzione in uno spazio di funzioni a partire da dati noti, tipicamente minimizzando un errore/loss.
- “Buona” rispetto all'**errore di generalizzazione**: quanto accuratamente il modello predice su **dati nuovi** (loss misurata su dati nuovi).

Attenzione — La generalizzazione è il punto cruciale

È facile *usare* strumenti di ML; è molto più difficile usarli *bene*. La differenza sta tutta nella generalizzazione.

Si distinguono due fasi:

1. **Fase di apprendimento** (*training, fitting*): si costruisce il modello dai dati noti (i dati di training, più il bias).

2. **Fase predittiva o di test** (*deployment*, uso del modello in inferenza): si applica il modello a nuovi esempi. Si prendono nuovi input $\mathbf{x}'$, si calcola la risposta del modello e la si confronta con il target d' che il modello **non ha mai visto**: così si valuta la capacità di generalizzazione.

Nota — Performance nel ML

Nel ML, “prestazione” significa **accuratezza di generalizzazione** (o accuratezza predittiva), stimata dall'errore calcolato su un **test set** tenuto da parte (*hold out*).

La **Statistical Learning Theory** (Vapnik) studia le condizioni matematiche sotto cui un modello è in grado di generalizzare; se ne vedranno le basi nella prossima lezione.

La valutazione delle prestazioni di un sistema di ML coincide con la valutazione della sua accuratezza predittiva: in una parola, **validazione! validazione! validazione!** Le tecniche di validazione servono sia a **valutare** la generalizzazione (*model assessment*) sia a **gestirla** (*model selection*), e saranno l'oggetto della prossima lezione.

Nota — Il costo dell'inferenza

Anche la sola fase di inferenza (il *mapping* input $\rightarrow$ output) può essere costosa quando le richieste sono milioni, come nei servizi di Google: per questo sono stati progettati chip dedicati come le **Tensor Processing Unit** (TPU), pensate originariamente per TensorFlow.

In sintesi — Sintesi

Il ML costruisce un'ipotesi h che approssima una funzione sconosciuta f a partire da esempi. I suoi ingredienti sono **dati** (rappresentazione, codifica, rumore), **task** (supervisionato: classificazione e regressione; non supervisionato: clustering, densità), **modello** (che definisce lo spazio delle ipotesi H), **algoritmo di apprendimento** (ricerca in H dell'ipotesi a errore minimo) e **validazione**. Senza **bias induttivo** non c'è generalizzazione; nel ML moderno si preferisce il *search bias* con modelli molto espressivi. La qualità si misura con una **loss** che dipende dal task (quadratica, 0/1, distorsione, $-\ln$), ma ciò che conta davvero è l'errore sui **dati nuovi**.

Domande — Possibili domande d'esame

- Che cos'è il bias induttivo? Qual è la differenza tra language bias e search bias?
- Dimostrare che un *unbiased learner* non può generalizzare.
- Quante sono le ipotesi con n input binari senza vincoli? E con sole regole congiuntive?
- Che cos'è il version space?
- Quali loss si usano per regressione, classificazione, clustering e stima di densità?
- Cosa si intende per generalizzazione e come si stima?

Capitolo 4

Generalizzazione e validazione (introduzione)

La quarta lezione conclude l'introduzione al ML affrontando il tema centrale del corso: la **generalizzazione**. Attraverso un esempio concreto (il fitting di polinomi) si vedono i fenomeni di **underfitting** e **overfitting**, si dà una prima formalizzazione con la **Statistical Learning Theory** e la **VC-dimension**, e si introducono le tecniche di base di **validazione**, le misure di accuratezza per la classificazione e il ciclo di progettazione di un sistema di ML.

4.1 Il ML in breve

Riepiloghiamo gli ingredienti visti nelle lezioni precedenti:

- **Dati**: l'esperienza disponibile, rappresentata come vettori, strutture, ecc.
- **Task**: supervisionato (classificazione, regressione), non supervisionato, ... Ad esempio, dati esempi etichettati, trovare una buona approssimazione della f sconosciuta.
- **Modello**: descrive le relazioni tra i dati e definisce la classe di funzioni che la macchina può implementare (lo spazio delle ipotesi).
- **Algoritmo di apprendimento**: dati task e modello, esegue una ricerca (euristica) nello spazio delle ipotesi valide sui dati, adattando i parametri liberi del modello.
- **Validazione**: valuta la capacità di generalizzazione dell'ipotesi trovata.

Il messaggio di fondo resta: *facile usare strumenti di ML contro uso corretto e consapevole del ML*.

4.2 I problemi fondamentali del ML

4.2.1 Un problema mal posto

Inferire funzioni generali da dati noti è un **problema mal posto**: in linea di principio la soluzione non è unica, e con dati finiti non possiamo aspettarci di trovare la soluzione esatta. Per questo si lavora con uno **spazio delle ipotesi ristretto** (ricordiamo il bias induttivo). Due domande guidano lo studio dei modelli:

- **Cosa possiamo rappresentare?** (se una funzione non è rappresentabile, non può nemmeno essere appresa);
- **Cosa possiamo apprendere?** (domanda secondaria rispetto alla prima).

4.2.2 Generalizzazione e overfitting

Distinguiamo la **fase di apprendimento**, in cui si costruisce il modello (incluso il training), dalla **fase di predizione**, in cui si valuta la funzione appresa su campioni nuovi (la capacità di generalizzazione).

Definizione — Ipotesi dell'apprendimento induttivo

Qualsiasi ipotesi h che approssima bene f sugli esempi di training approssimerà bene f anche su istanze nuove, mai viste.

Il punto interrogativo che il professore mette accanto a questa ipotesi è voluto: **non è sempre vera**, ed è proprio qui che nasce l'overfitting.

Definizione — Overfitting

Un learner fa **overfitting** sui dati se restituisce un'ipotesi $h \in H$ con errore vero (di generalizzazione, *rischio*) R ed errore empirico (di training) E , quando esiste un'altra $h' \in H$ con

$$E' > E \quad \text{ma} \quad R' < R.$$

In altre parole, h' si adatta *peggio* ai dati di training ma è *migliore* sui dati nuovi.

L'aspetto critico diventa quindi la **stima dell'accuratezza**, che si può affrontare in modo **teorico** (Statistical Learning Theory) o **empirico** (errore su training e test, tecniche di cross-validation).

4.3 Un caso di studio: il fitting polinomiale

Per vedere concretamente il ruolo della complessità del modello si usa un esempio di regressione con un modello parametrico: lo spazio delle ipotesi sono i **polinomi di grado M** ,

$$h_{\mathbf{w}}(x) = w_0 + w_1x + w_2x^2 + \dots + w_Mx^M = \sum_{j=0}^M w_jx^j.$$

La complessità dell'ipotesi cresce con il grado M ; l è il numero di esempi.

Attenzione — Esempio artificiale

È un task semplificato e irrealistico: una sola variabile di input, e conosciamo in anticipo la funzione target. Serve solo a visualizzare i concetti.

La funzione target è $\sin(2\pi x)$ a cui si aggiunge **rumore gaussiano**: per questo i campioni non stanno esattamente sulla curva verde "vera".

Per trovare i parametri migliori si minimizza la **somma degli errori quadratici**:

$$E(\mathbf{w}) = \sum_{p=1}^l (y_p - h_{\mathbf{w}}(x_p))^2,$$

dove p indica l'esempio, y_p il suo target, $h_{\mathbf{w}}(x_p)$ l'uscita del modello nel punto x_p (qui x è una sola variabile, $n = 1$).

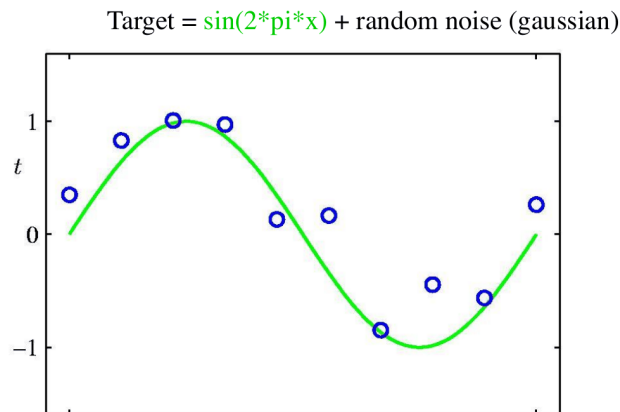


Figura 4.1: La funzione target $\sin(2\pi x)$ (in verde) e i campioni rumorosi di training (in blu).

4.3.1 Effetto del grado del polinomio

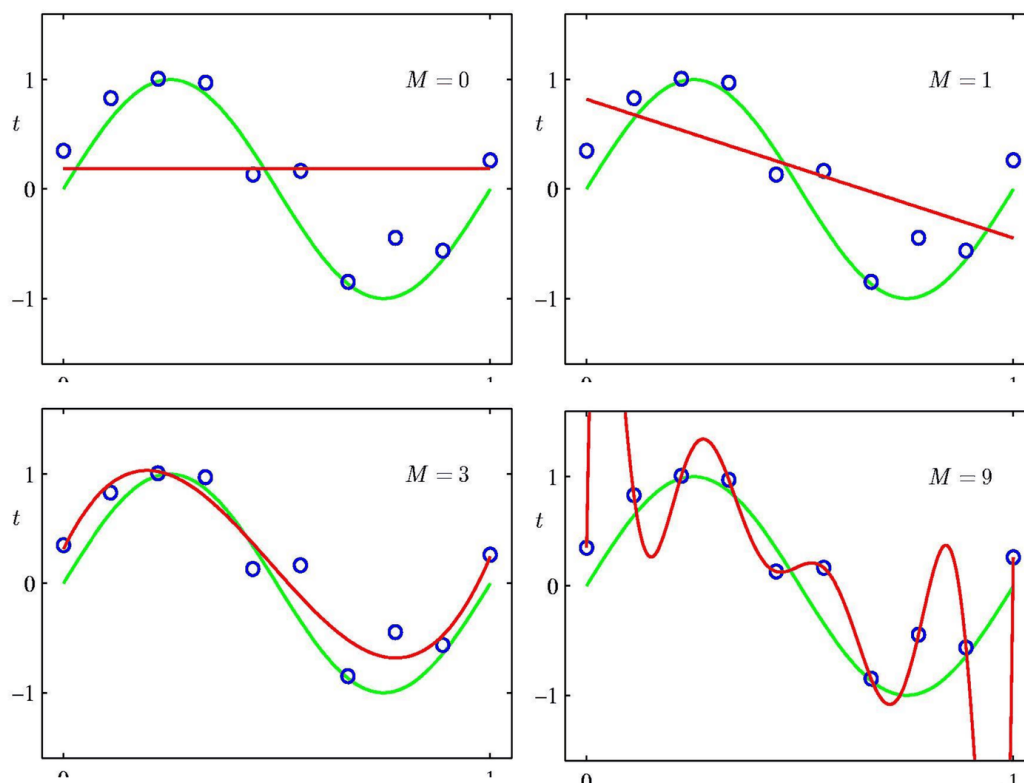


Figura 4.2: Fitting con polinomi di grado $M = 0, 1, 3, 9$ (in rosso) rispetto alla funzione vera (in verde).

- $M = 0$ (costante) e $M = 1$ (retta): il modello è **troppo semplice** rispetto alla funzione target e non riesce a coglierne l'andamento. È l'**underfitting**.
- $M = 3$: più flessibilità è utile, e la curva approssima bene il seno.
- $M = 9$: con 10 punti e 10 parametri il polinomio passa **esattamente** per tutti i dati: $E(\mathbf{w}) = 0$ sul training! Ma la curva oscilla violentemente tra i punti e rappresenta malissimo la funzione vera: il modello è troppo complesso e **impara anche il rumore**. È l'**overfitting**. E l'errore sul test set?

4.3.2 Errore di training e di test al variare di M

Per confrontare i modelli si usa il **Root-Mean-Square error**, $E_{RMS} = \sqrt{2E(\mathbf{w}^*)/l}$, dove $E(\mathbf{w}^*)$ è l'errore del modello addestrato. La divisione per l permette di confrontare dataset di dimensione diversa, e la radice riporta l'errore sulla stessa scala del target.

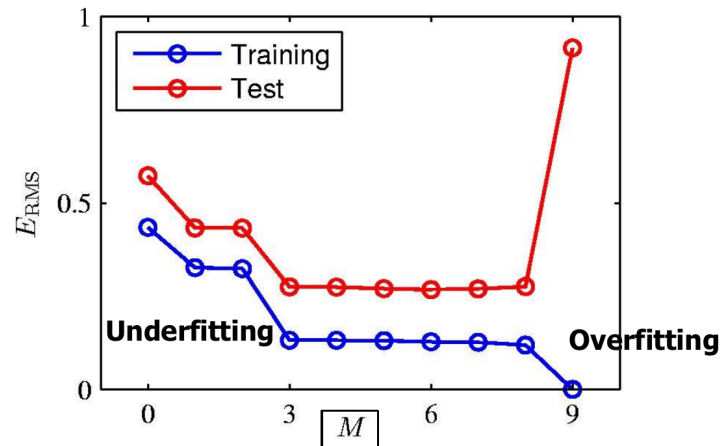


Figura 4.3: Errore RMS di training e di test al variare del grado M : zona di underfitting a sinistra, di overfitting a destra.

L'errore di training **decresce sempre** all'aumentare di M , perché un modello più flessibile può adattarsi meglio ai dati. L'errore di test invece prima decresce, poi resta stabile e infine **esplode** per $M = 9$: è la firma dell'overfitting.

Anche i **coefficienti** ottimi $\mathbf{w}^*$ sono rivelatori:

	$M = 0$	$M = 1$	$M = 3$	$M = 9$
w_0^*	0,19	0,82	0,31	0,35
w_1^*		-1,27	7,99	232,37
w_2^*			-25,43	-5321,83
w_3^*			17,37	48568,31
w_4^*				-231639,30
w_5^*				640042,26
w_6^*				-1061800,52
w_7^*				1042400,18
w_8^*				-557682,99
w_9^*				125201,43

Suggerimento — Pesi enormi = overfitting

Nel polinomio di grado 9 i coefficienti diventano **enormi** e di segno alterno: si compensano a vicenda per far passare la curva esattamente per ogni punto, producendo oscillazioni fortissime. Questa osservazione è alla base della **regolarizzazione** (che vedremo con i modelli lineari e le reti neurali): penalizzare pesi grandi è un modo per controllare la complessità.

4.3.3 Effetto della quantità di dati

Cosa succede se, a parità di modello ($M = 9$), aumentiamo i dati?

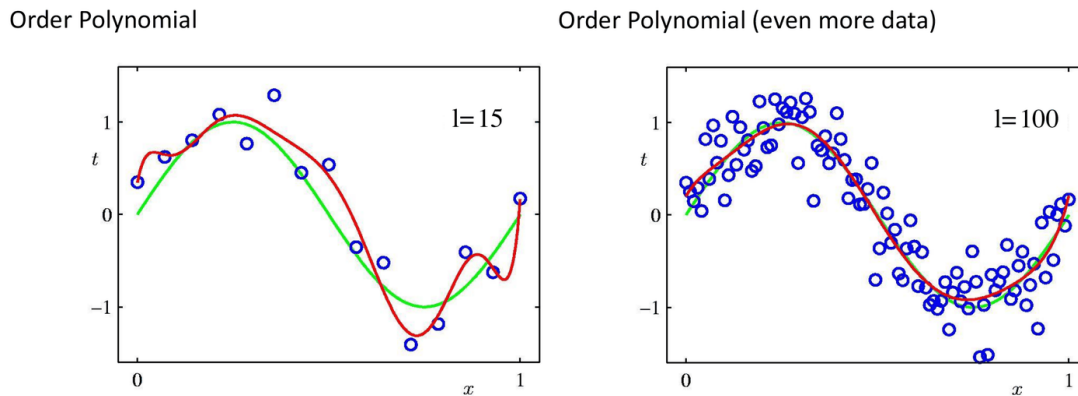


Figura 4.4: Polinomio di grado 9 con $l = 15$ e $l = 100$ esempi: con più dati l'overfitting si riduce.

Con 15 esempi il polinomio di grado 9 oscilla ancora, ma molto meno; con 100 esempi segue quasi perfettamente il seno. **Con più dati si possono usare modelli più complessi senza cadere nell'overfitting.**

In sintesi — Cosa abbiamo imparato

La capacità di generalizzazione (misurata come rischio o errore di test) va studiata in relazione a: - l'**errore di training** e le zone di underfitting e overfitting; - la **complessità del modello**; - il **numero di dati**.

La **Statistical Learning Theory** è la teoria generale che lega questi tre elementi.

4.4 Verso la Statistical Learning Theory

4.4.1 Il setting formale (semplificato)

Vogliamo approssimare una funzione sconosciuta $f(\mathbf{x})$, dove il target è $d = f(\mathbf{x}) + \text{rumore}$.

Idealmente vorremmo minimizzare la **funzione di rischio** (errore vero), calcolata su **tutti** i dati possibili:

$$R = \int L(d, h(\mathbf{x})) dP(\mathbf{x}, d),$$

dove $P(\mathbf{x}, d)$ è la distribuzione di probabilità (sconosciuta) dei dati e L è una loss, ad esempio $L(h(\mathbf{x}), d) = (d - h(\mathbf{x}))^2$. Si cerca quindi $h \in H$ che minimizzi R .

Ma abbiamo solo un dataset finito $TR = \{(\mathbf{x}_p, d_p)\}_{p=1}^l$. Allora si minimizza il **rischio empirico** (errore di training), trovando i valori migliori dei parametri liberi:

$$R_{emp}(h, TR) = \frac{1}{l} \sum_{p=1}^l (d_p - h(\mathbf{x}_p))^2.$$

Questo è il principio induttivo di **Empirical Risk Minimization** (ERM). La domanda cruciale è: **possiamo usare R_{emp} per approssimare R ?**

4.4.2 VC-dimension e bound sul rischio

La **VC-dimension** (Vapnik-Chervonenkis) è una misura della **complessità** di H , cioè della sua flessibilità nell'adattarsi ai dati (per modelli lineari o polinomi è legata al numero di parametri; sarà definita formalmente più avanti).

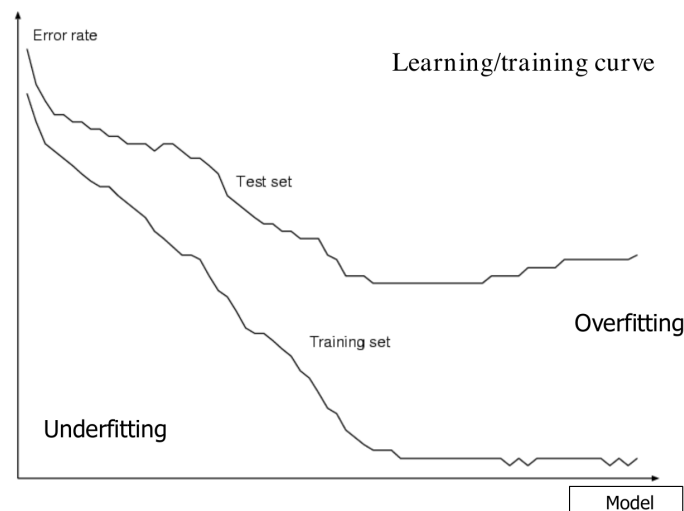


Figura 4.5: Comportamento tipico dell'apprendimento al crescere della complessità del modello.

Teorema — VC-bound

Con probabilità $1 - \delta$ vale:

$$\underbrace{\bar{R}}_{\text{rischio garantito}} \leq R_{emp} + \underbrace{\varepsilon\left(\frac{1}{l}, VC, \frac{1}{\delta}\right)}_{\text{VC-confidence}}$$

dove la **VC-confidence** ε è una funzione che **cresce con la VC-dimension** e **decrece con il numero di dati l** (e con δ).

Il parametro δ è la **confidenza**: regola la probabilità che il bound valga (ad esempio $\delta = 0,01$ significa che il bound vale con probabilità 0,99).

Questo bound **spiega** underfitting e overfitting:

- **più dati** (l alto) $\rightarrow$ VC-confidence più bassa, e il bound si avvicina a R : il rischio empirico diventa una buona stima di quello vero;
- **modello troppo semplice** (VC-dim bassa) $\rightarrow$ ε piccolo, ma R_{emp} alto: **underfitting**;
- **modello troppo complesso** (VC-dim alta, l fissato) $\rightarrow$ R_{emp} basso, ma ε cresce, e con esso il bound su R : **overfitting**.

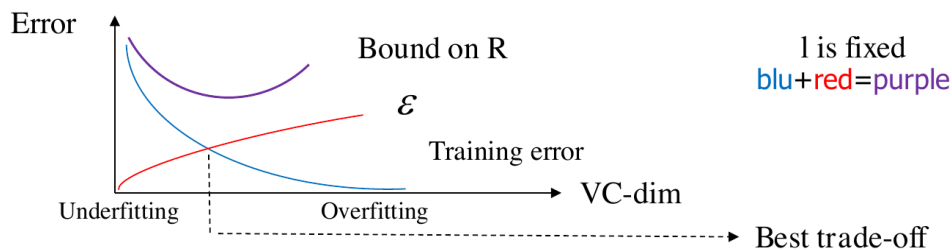


Figura 4.6: Structural Risk Minimization: il bound su R (viola) è la somma dell'errore di training (blu) e della VC-confidence (rossa); il suo minimo è il miglior compromesso.

Definizione — Structural Risk Minimization (SRM)

Invece di minimizzare solo R_{emp} , si **minimizza il bound**, cioè la somma di errore di training e VC-confidence. Questo formalizza il concetto di **controllo della complessità del modello**: un compromesso tra complessità (VC-dim) e accuratezza sul training (fitting).

Un esempio concreto di bound è

$$R \leq R_{emp} + \varepsilon \left(\frac{VC}{l}, -\frac{\ln \delta}{l} \right),$$

e ne esistono diverse formulazioni per diverse classi di funzioni e task. In parole semplici: **si può approssimare bene f dagli esempi purché si abbia un buon numero di dati e la complessità del modello sia adatta al problema**. Bisogna adattarsi ai dati il più possibile per evitare l'underfitting (alto R_{emp}), ma non troppo, per evitare l'overfitting (dovuto alla crescita della VC-confidence).

Nota — Oltre il compromesso classico

Fenomeni più recenti come la **double descent**, la sovra-parametrizzazione e il *benign overfitting* (tipici delle reti profonde) sembrano sfidare questa visione e verranno discussi più avanti nel corso.

4.4.3 Perché la SLT è importante

La SLT permette di inquadrare formalmente il problema della generalizzazione e dell'overfitting, fornendo **upper bound analitici** al rischio R indipendentemente dal tipo di algoritmo o dai dettagli del modello. Mostra che il ML è **ben fondato**: il rischio si può limitare analiticamente, e pochi concetti sono fondamentali. Ha inoltre portato a nuovi modelli (le **SVM**) che considerano direttamente il controllo della complessità, e spiega la differenza principale tra il ML e i metodi di ottimizzazione puri (che forniscono le tecniche per fare *fitting*): nel ML l'obiettivo non è minimizzare l'errore sui dati, ma sui dati *futuri*.

Restano aperte domande pratiche: come misurare la complessità? Come trovare il miglior compromesso tra fitting e complessità?

Domande — Esercizi proposti dal professore

- Perché un errore di training nullo non implica necessariamente una buona soluzione?
- Collegare underfitting e overfitting all'interpretazione della disuguaglianza della SLT e all'esempio dei polinomi.
- Guardando la definizione di overfitting, individuare h e h' sul grafico del bound SLT. (Suggerimento: h sta a destra del minimo, con R_{emp} più basso ma bound più alto; h' sta vicino al minimo.)
- Il rumore è la causa dell'overfitting, o lo è il compromesso di complessità? Si può avere overfitting anche con dati perfettamente puliti? (Sì: anche senza rumore, un polinomio di grado molto alto interpolando pochi punti di una funzione liscia può oscillare molto tra un punto e l'altro; il problema è la complessità eccessiva rispetto ai dati disponibili.)

4.5 Validazione

La valutazione delle prestazioni di un sistema di ML è la valutazione della sua **accuratezza predittiva**. Attenzione: **le prestazioni sui dati di training danno una valutazione troppo ottimistica**. Da qui il mantra “validazione, validazione, validazione”. Qui si dà un’introduzione; la validazione sarà ripresa in lezioni dedicate.

4.5.1 I due obiettivi della validazione

Definizione — Model selection e model assessment

- **Model selection:** stimare le prestazioni (errore di generalizzazione) di **diversi modelli** per scegliere il migliore. Include la ricerca dei migliori **iperparametri** (es. il grado del polinomio). *Restituisce un modello.*
- **Model assessment:** scelto il modello finale, stimarne l’errore di predizione (rischio) su **nuovi dati di test**, come misura della qualità del modello scelto. *Restituisce una stima.*

Attenzione — Regola d’oro

Tenere **separati gli obiettivi** e usare **insiemi di dati separati** per ciascuno. Se il test set viene usato per scegliere il modello, la stima finale non è più affidabile (è ottimistica).

Nel mondo ideale avremmo un grande training set (per trovare l’ipotesi migliore), un grande validation set per la model selection e un test set esterno molto grande. Con dataset finiti, spesso piccoli, si può solo **stimare** la generalizzazione.

4.5.2 Hold-out

Nella tecnica di base, **hold-out**, il dataset D viene partizionato in tre insiemi **disgiunti**:

- **Training set (TR):** usato per eseguire l’algoritmo di apprendimento;
- **Validation set (VL)** o *selection set*: usato per scegliere il modello migliore (es. tuning degli iperparametri);
- **Test set (TS):** usato **solo** per la model assessment, mai per scegliere o regolare h .

TR e VL insieme formano il **development/design set**; il test set (ad esempio il 25–30% dei dati) viene tenuto da parte.

4.5.3 K-fold cross-validation (anticipazione)

L’hold-out può sfruttare male i dati, soprattutto se sono pochi. Nella **K-fold cross-validation**:

1. si divide D in k sottoinsiemi mutuamente esclusivi $D_1, \dots, D_k$;
2. per ogni i si addestra su $D \setminus D_i$ e si valuta su D_i ;
3. si combinano i k risultati.

Così tutti i dati vengono usati sia per l’addestramento sia per la validazione (o il test). Si può applicare sia per lo split di validazione sia per quello di test. Le questioni aperte sono quante fold usare (3, 5, 10, fino a *leave-one-out*), il costo computazionale spesso elevato e la combinazione con altri schemi (validation set, doppia K-fold). Sarà ripresa in dettaglio più avanti.

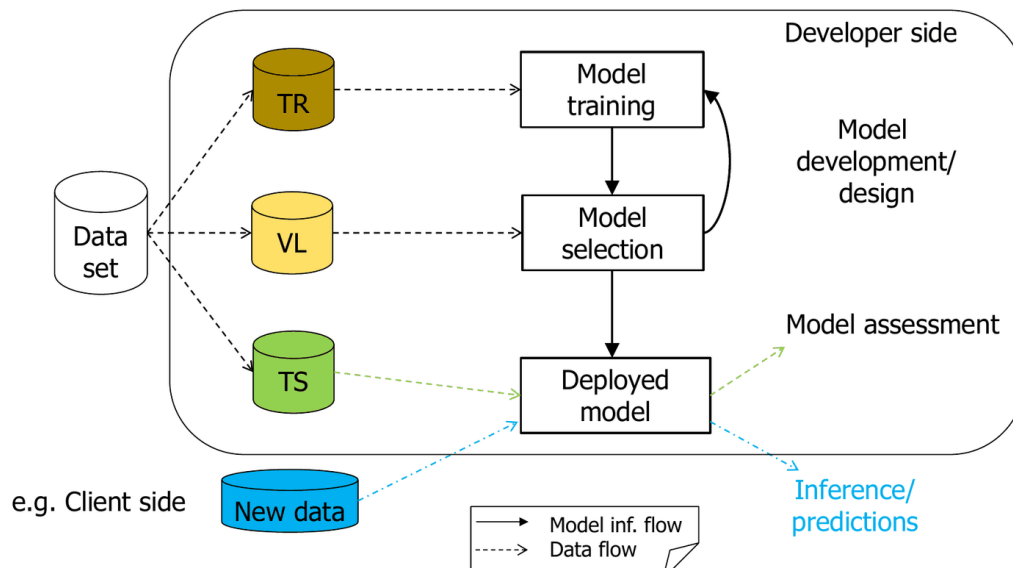


Figura 4.7: Ruolo di TR, VL e TS: sviluppo del modello lato sviluppatore, uso in inferenza lato cliente.

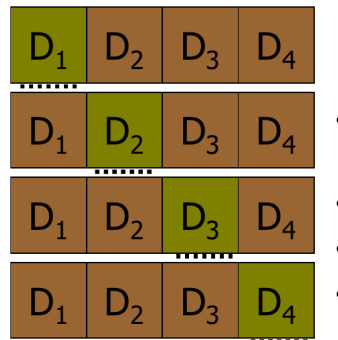


Figura 4.8: 4-fold cross-validation: a turno ogni fold fa da insieme di validazione.

4.6 Accuratezza nella classificazione

4.6.1 Matrice di confusione

Per un classificatore binario si confrontano classe reale e classe predetta:

Reale \ Predetto	Positivo	Negativo
Positivo	TP (veri positivi)	FN (falsi negativi)
Negativo	FP (falsi positivi)	TN (veri negativi)

Da cui:

- **Accuratezza** = $\frac{TP + TN}{\text{totale}}$: percentuale di pattern classificati correttamente;
- **Sensibilità** (*True Positive rate, Recall*) = $\frac{TP}{TP + FN}$: quanti positivi veri vengono riconosciuti;
- **Specificità** (*True Negative rate*) = $\frac{TN}{FP + TN} = 1 - FPR$: quanti negativi veri vengono riconosciuti;
- **Precisione** = $\frac{TP}{TP + FP}$: quanti dei predetti positivi sono davvero positivi.

Un falso positivo equivale a un **falso allarme**.

Attenzione — Attenzione all'accuratezza

- Per la classificazione binaria, un'accuratezza del 50% è quella di un predittore **casuale** (lancio di una moneta).
- Con **dati sbilanciati** (es. 99% di positivi) esiste un classificatore banale che risponde sempre "positivo" e ha il 99% di accuratezza senza aver imparato nulla. In questi casi l'accuratezza da sola è fuorviante.

4.6.2 Curva ROC

La **curva ROC** mostra il *True Positive rate* (sensibilità) in funzione del *False Positive rate* (1-specificità) al variare della soglia di decisione del classificatore.

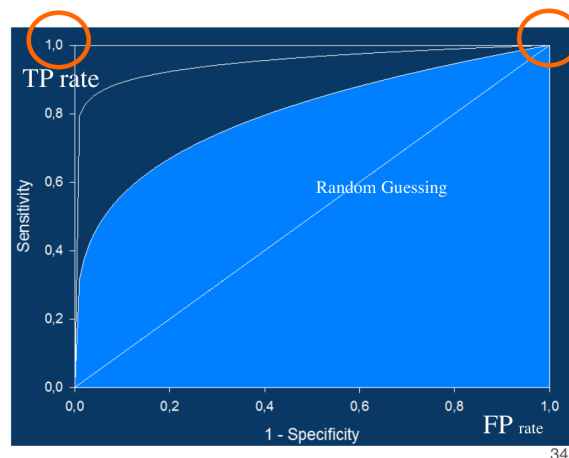


Figura 4.9: Curva ROC: la diagonale corrisponde al classificatore casuale; curve migliori hanno un'area sottesa (AUC) maggiore.

La **diagonale** corrisponde al classificatore peggiore (casuale). Le curve migliori salgono rapidamente verso l'angolo in alto a sinistra (TPR alto con FPR basso) e hanno un'**AUC** (*Area Under the Curve*) più alta: il classificatore ideale ha $AUC = 1$, quello casuale 0,5.

4.7 Il ciclo di progettazione

1. **Raccolta dei dati:** selezione, integrazione, pulizia. Serve un insieme di esempi abbastanza **grande e rappresentativo** per training e test.
2. **Rappresentazione dei dati:** dipende dal dominio e sfrutta la conoscenza a priori dell'esperto; comprende feature selection, rilevamento degli outlier, scalatura delle variabili, gestione dei dati mancanti. **Spesso è la fase più critica** per il successo complessivo.
3. **Scelta del modello:** formulazione del problema e delle ipotesi. Bisogna conoscere i **limiti di applicabilità** del modello e controllarne la complessità.
4. **Costruzione del modello** (il cuore del ML): tramite l'algoritmo di apprendimento sui dati di training.
5. **Valutazione:** la prestazione è l'**accuratezza predittiva**; si aggiungono l'interpretazione dei risultati, la spiegazione dei dati e l'estrazione di conoscenza.
6. **Deployment.**

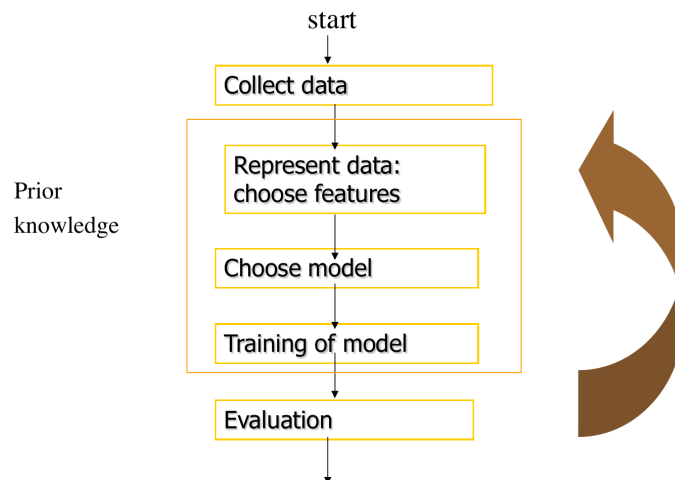


Figura 4.10: Il ciclo di progettazione di un sistema di ML: un processo iterativo guidato dalla conoscenza a priori.

Il ciclo è iterativo: i risultati della valutazione possono richiedere di tornare alle fasi precedenti.

4.8 Errori di interpretazione

Per ogni modello statistico (inclusi Data Mining e ML):

- si assume (spesso) una **causalità** e serve un insieme di dati **rappresentativo** del fenomeno. Il ML non funziona con variabili scorrelate o fenomeni casuali (lotterie); input non informativi portano a modelli e risultati scadenti;
- **la causalità non si può inferire dalla sola analisi dei dati**: in Florida le persone sono in media più anziane che negli altri stati USA, ma non è il clima a farle vivere più a lungo (ci si trasferisce lì da pensionati). La dipendenza statistica può avere ragioni esterne ai dati.

Specifico per il ML: **modelli potenti aumentano il rischio**, perché si adattano anche a dati “spazzatura”; e risultati **non validati bene** possono portare a predizioni e interpretazioni fuorvianti.

In sintesi — Sintesi

La generalizzazione dipende dal compromesso tra **complessità del modello** e **quantità di dati**. Modelli troppo semplici fanno **underfitting**, troppo complessi fanno **overfitting** (errore di training basso, di test alto). La SLT formalizza tutto con il **VC-bound** $R \leq R_{emp} + \epsilon(1/l, VC, 1/\delta)$ e suggerisce la **Structural Risk Minimization**. In pratica la generalizzazione si stima con la **validazione**: *model selection* (su VL) e *model assessment* (su TS), sempre su dati separati.

Domande — Possibili domande d’esame

- Definire formalmente l’overfitting.
- Descrivere l’esempio del fitting polinomiale: cosa succede al variare di M e di l ?
- Scrivere il VC-bound e spiegare come giustifica underfitting e overfitting.
- Differenza tra rischio R e rischio empirico R_{emp} ; cos’è il principio ERM?
- Differenza tra model selection e model assessment; perché servono insiemi separati?
- Definire accuratezza, sensibilità, specificità, precisione; cos’è la curva ROC?

Capitolo 5

Modelli lineari e K-nearest neighbors

Dopo l'introduzione si passa ai primi modelli veri e propri, che occupano due estremi opposti del panorama del ML. Da una parte i **modelli lineari**: rigidi, con pochi parametri, fondati sulla matematica classica ma già ricchi di concetti moderni (funzione di loss, discesa del gradiente, regolarizzazione). Dall'altra il **K-nearest neighbors**: estremamente flessibile, locale, senza un vero modello da addestrare. Il confronto tra i due mostra concretamente il compromesso tra flessibilità e controllo della complessità.

Con i modelli lineari si passa da uno spazio delle ipotesi discreto (concept learning) a uno **continuo**, ancora ristretto ma parametrizzato da numeri reali.

5.1 Notazione sui dati

I dati sono organizzati in una matrice X di dimensione $l \times n$: l righe (i pattern, $p = 1, \dots, l$) e n colonne (le feature, $i = 1, \dots, n$).

- $\mathbf{x}$ (in grassetto) è un generico pattern, cioè una riga della tabella: esempio, istanza, campione, vettore di input;
- x_i o x_j (scalare) è la componente i o j di un pattern $\mathbf{x}$ (omettendo l'indice del pattern);
- $\mathbf{x}_p$ è il p -esimo pattern (riga);
- $x_{p,i}$, scritto anche $(\mathbf{x}_p)_i$, è la componente i del pattern p ;
- per il target si usa y_p (o d_p, t_p), con $p = 1, \dots, l$.

Quando il contesto è chiaro alcuni indici vengono omessi.

5.2 Modelli lineari

Il modello lineare è stato il pilastro della statistica. Come scrive Hastie: *“nonostante i grandi progressi delle moderne tecniche di regressione non parametrica, i modelli lineari restano importanti, e dobbiamo capirli bene”*. Si parte dalla forma più semplice, lineare nelle variabili di input. Il modello lineare è anche una **baseline** naturale: la prima domanda davanti a un problema è “è un problema lineare?”.

5.2.1 Regressione lineare univariata

Riprendiamo l'esercizio dell'introduzione: dati i punti (1; 2,1), (2; 3,9), (3; 6,1), (4; 8,4), (5; 9,8) avevamo “indovinato” $h(x) = 2x$. Ora vogliamo trovare i parametri in modo **sistematico**.

Nel caso univariato (una variabile di input x , una di output y) si assume il modello

$$h_{\mathbf{w}}(x) = w_1 x + w_0,$$

dove w_0, w_1 sono coefficienti reali detti **pesi** (*weights*) o parametri liberi. Si tratta di adattare ai dati una retta. Lo spazio delle ipotesi è infinito (i $\mathbf{w}$ sono continui), ma la matematica classica

fornisce una soluzione elegante che risale a Gauss e Legendre (circa 1795). Sorprendentemente, con questo strumento di base si può già “apprendere”, e contiene molti concetti rilevanti del ML moderno.

Apprendimento tramite Least Mean Squares

Addestrare significa trovare $\mathbf{w}$ che **minimizza l'errore empirico** sui dati di training: per ora ci concentriamo solo su R_{emp} (il controllo della complessità verrà dopo). Dato un insieme di l esempi (x_p, y_p) , cerchiamo $h_{\mathbf{w}}(x) = w_1 x + w_0$ che minimizza la **somma dei quadrati dei residui**:

$$\text{Loss}(h_{\mathbf{w}}) = E(\mathbf{w}) = \sum_{p=1}^l (y_p - h_{\mathbf{w}}(x_p))^2 = \sum_{p=1}^l (y_p - (w_1 x_p + w_0))^2.$$

Per avere la media si divide per l . È il metodo dei **minimi quadrati** (*Least (Mean) Squares*, LMS), cioè $\arg \min_{\mathbf{w}} E(\mathbf{w})$ in norma L^2 .

Suggerimento — Perché i minimi quadrati?

Ogni retta candidata produce dei **residui**, cioè le distanze verticali tra i punti (x_p, y_p) e la retta. Rette diverse hanno residui diversi: minimizzare la somma dei loro quadrati è un modo naturale di trovare la retta che meglio approssima i dati. In generale, i minimi quadrati sono l'approccio standard per risolvere in modo approssimato i **sistemi sovradeterminati**, cioè con più equazioni (gli esempi) che incognite (i parametri).

Come si risolve

Un minimo locale è un **punto stazionario**, dove il gradiente è nullo:

$$\frac{\partial E(\mathbf{w})}{\partial w_i} = 0, \quad i = 1, \dots, n+1.$$

Nel caso semplice con due parametri si cercano w_0, w_1 tali che $\frac{\partial E}{\partial w_0} = 0$ e $\frac{\partial E}{\partial w_1} = 0$. Poiché la loss è **convessa** (una parabola in due dimensioni), non ci sono minimi locali spuri e la soluzione esiste in forma chiusa:

$$w_1 = \frac{\sum_p x_p y_p - \frac{1}{l} \sum_p x_p \sum_p y_p}{\sum_p x_p^2 - \frac{1}{l} \left(\sum_p x_p \right)^2} = \frac{\text{Cov}[x, y]}{\text{Var}[x]}, \quad w_0 = \bar{y} - w_1 \bar{x},$$

dove $\bar{x} = \frac{1}{l} \sum_p x_p$ e $\bar{y} = \frac{1}{l} \sum_p y_p$ sono le medie.

Calcolo del gradiente per un pattern

Per un singolo pattern (x, y) , usando le regole di derivazione ($\frac{\partial k}{\partial w} = 0$, $\frac{\partial w}{\partial w} = 1$, $\frac{\partial f(w)^2}{\partial w} = 2f(w) \frac{\partial f(w)}{\partial w}$, e “la derivata della somma è la somma delle derivate”):

$$\frac{\partial E(\mathbf{w})}{\partial w_i} = \frac{\partial (y - h_{\mathbf{w}}(x))^2}{\partial w_i} = 2(y - h_{\mathbf{w}}(x)) \frac{\partial (y - (w_1 x + w_0))}{\partial w_i},$$

da cui

$$\frac{\partial E(\mathbf{w})}{\partial w_0} = -2(y - h_{\mathbf{w}}(x)), \quad \frac{\partial E(\mathbf{w})}{\partial w_1} = -2(y - h_{\mathbf{w}}(x))x.$$

La quantità $\delta = y - h(x)$, l'errore sul pattern, verrà chiamata **delta**. Per l pattern basta sommare; poi si estende a input multidimensionali.

Esempio — Esercizio svolto

Con i dati $(1; 2,1), (2; 3,9), (3; 6,1), (4; 8,4), (5; 9,8)$: $\bar{x} = 3$, $\bar{y} = 6,06$; $\sum x_p y_p = 2,1 + 7,8 + 18,3 + 33,6 + 49 = 110,8$; $\sum x_p^2 = 55$. Quindi $w_1 = \frac{110,8 - 5 \cdot 3 \cdot 6,06}{55 - 45} = \frac{110,8 - 90,9}{10} = 1,99$ e $w_0 = 6,06 - 1,99 \cdot 3 = 0,09$. La retta trovata, $h(x) \approx 1,99x + 0,09$, conferma l'intuizione $h(x) = 2x$.

5.2.2 Notazione per input multidimensionali

Con $\mathbf{x}$ e $\mathbf{w}$ vettori colonna, il modello lineare è

$$\mathbf{w}^T \mathbf{x} + w_0 = w_0 + w_1 x_1 + w_2 x_2 + \dots + w_n x_n = w_0 + \sum_{i=1}^n w_i x_i.$$

w_0 si chiama **intercetta**, soglia, **bias** o offset. Conviene includere una componente costante $x_0 = 1$ nell'input, così che

$$\mathbf{x}^T = [1, x_1, \dots, x_n], \quad \mathbf{w}^T = [w_0, w_1, \dots, w_n], \quad h(\mathbf{x}_p) = \mathbf{x}_p^T \mathbf{w} = \sum_{i=0}^n x_{p,i} w_i.$$

Il modello lineare è quindi semplicemente un **prodotto scalare** tra input e pesi. (Nelle reti neurali la trasposta viene spesso omessa.)

5.3 Classificazione con modelli lineari

5.3.1 Il problema

Consideriamo 200 punti in $\mathbb{R}^2$, generati da una distribuzione sconosciuta, 100 per ciascuna di due classi. Possiamo costruire una regola che predica il colore di punti futuri? I dati potrebbero essere generati da una gaussiana per classe (con medie diverse), oppure da una miscela di tante gaussiane a bassa varianza.

5.3.2 Iperpiani e regioni di decisione

Gli stessi modelli usati per la regressione possono essere usati per la **classificazione**, con target categorici (0/1 o $-1/+1$). L'espressione $\mathbf{w}^T \mathbf{x}$ definisce un **iperpiano**; si sfrutta il fatto che $\mathbf{w}^T \mathbf{x}$ assume valori positivi da un lato e negativi dall'altro per decidere a quale classe appartiene un punto. L'obiettivo è trovare, tramite l'apprendimento, un $\mathbf{w}$ che dia una buona accuratezza.

In due dimensioni, $\mathbf{w}^T \mathbf{x} = w_0 + w_1 x_1 + w_2 x_2 = 0$ è una retta: il **confine di decisione** (*decision boundary*).

Per ottenere un classificatore si applica una **funzione soglia**:

$$h(\mathbf{x}) = \begin{cases} 1 & \text{se } \mathbf{w}^T \mathbf{x} + w_0 \geq 0 \\ 0 & \text{altrimenti} \end{cases} \quad (\text{output in } [0, 1]), \quad h(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x} + w_0) \quad (\text{output in } [-1, +1]).$$

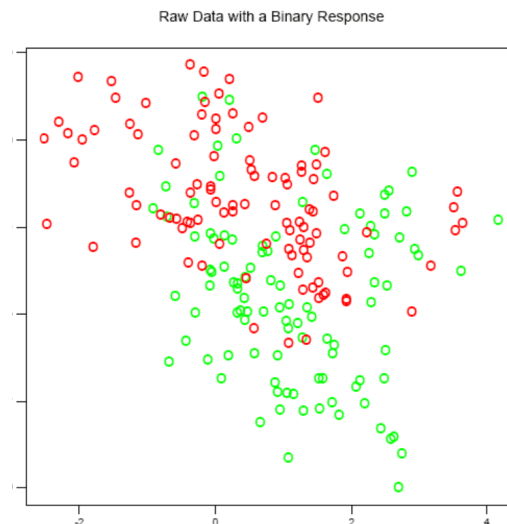


Figura 5.1: Il problema di esempio (Hastie-Tibshirani-Friedman): 200 punti di due classi.

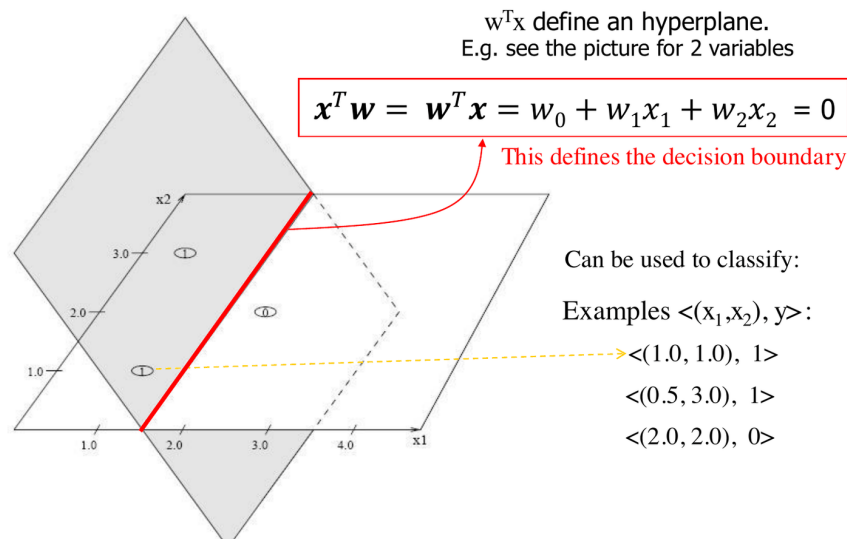


Figura 5.2: Liperpiano $\mathbf{w}^T \mathbf{x}$ e la sua intersezione con il piano degli input: il confine di decisione.

Includendo w_0 in $\mathbf{w}$: $h(\mathbf{x}_p) = \text{sign}(\mathbf{x}_p^T \mathbf{w}) = \text{sign}\left(\sum_{i=0}^n x_{p,i} w_i\right)$. È la **Linear Threshold Unit (LTU)**.

Nota — Il bias come soglia

Dire $\mathbf{w}^T \mathbf{x} + w_0 \geq 0$ equivale a dire $\mathbf{w}^T \mathbf{x} \geq -w_0$. Le due forme individuano la stessa regione positiva, ma la seconda mette in evidenza il ruolo del bias: $-w_0$ è il **valore di soglia** che la combinazione pesata degli input deve superare per “attivare” l’uscita +1.

5.3.3 Usare il modello: due esempi

Esempio — Terremoti o esplosioni nucleari? (AIMA)

Dati sismici (1982–1990, Asia): x_1 è la magnitudo delle onde di volume, x_2 quella delle onde di superficie. Un algoritmo di apprendimento trova il confine $-4,9 + 1,7x_1 - x_2 = 0$. Per classificare un nuovo evento (6, 3):

$$h(6, 3) = \text{sign}(-4,9 + 1,7 \cdot 6 - 1 \cdot 3) = \text{sign}(2,3) = +1 \Rightarrow \text{esplosione nucleare.}$$

(Ottenere nuovi esempi di training, in questo caso, è decisamente costoso.)

Find h s.t. given (x_1, x_2) return 0/-1 for Earthquakes and 1 for Nuclear Explosion

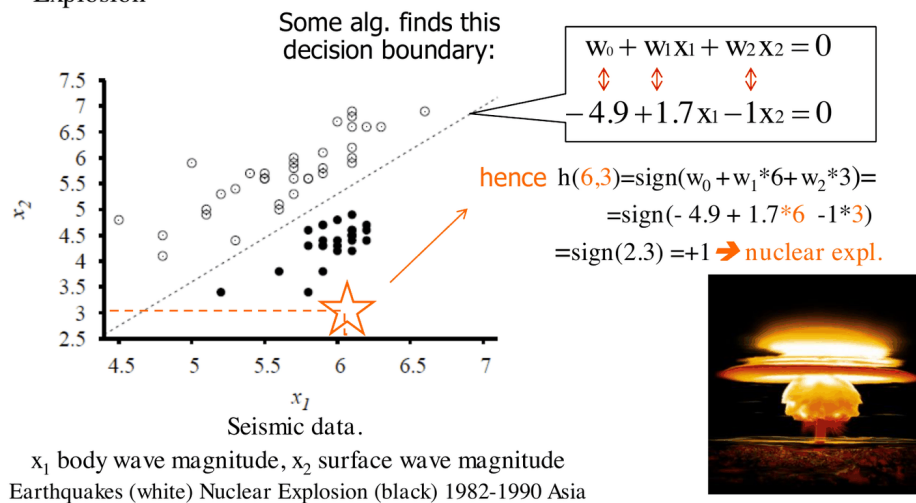
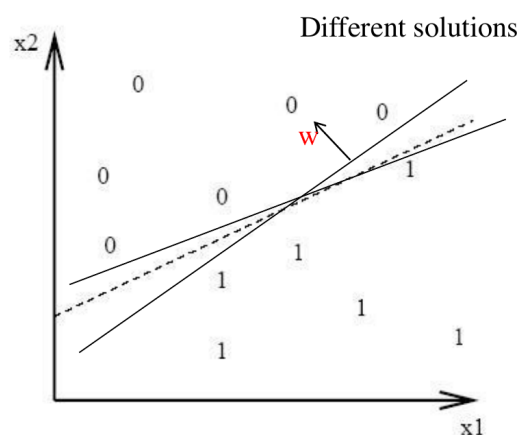


Figura 5.3: Classificazione di dati sismici con un confine di decisione lineare.

Esempio — Filtro antispam

Si vuole $h(\text{mail}) = +1$ per lo spam e -1 altrimenti. Le feature $\phi_k(\text{mail})$ sono ad esempio la presenza (0/1) di parole o frasi (“free money”) o la lunghezza (intero): è la rappresentazione *bag of words*. Ogni peso w_k esprime il contributo della feature alla predizione: positivo per “free money”, negativo per “.edu” o “unipi”. La classificazione è $h_w(\mathbf{x}) = \text{sign}(\sum_k w_k \phi_k(\mathbf{x}))$: se la combinazione pesata è positiva, è spam.

5.3.4 Proprietà utili dell’iperpiano



A linearly separable problem

Figura 5.4: Un problema linearmente separabile ammette molti iperpiani separatori; $\mathbf{w}$ è ortogonale all’iperpiano.

- Se $w_0 = 0$ la retta passa per l’origine.

- Se $n > 2$ il confine è un **iperpiano**.
- **Libertà di scala**: moltiplicando $\mathbf{w}$ per una costante K il confine non cambia.
- $\mathbf{w}$ è **ortogonale all'iperpiano**: presi due punti $\mathbf{x}_a, \mathbf{x}_b$ sull'iperpiano, $\mathbf{w}^T \mathbf{x}_a + w_0 = 0$ e $\mathbf{w}^T \mathbf{x}_b + w_0 = 0$; sottraendo si ha $\mathbf{w}^T (\mathbf{x}_a - \mathbf{x}_b) = 0$, cioè $\mathbf{w}$ è ortogonale a ogni vettore che giace sull'iperpiano.
- Se esiste un iperpiano separatore, **ne esistono molti**, e quindi anche molti algoritmi per trovarli.

L'equazione della retta si può riscrivere come $x_2 = -x_1 \frac{w_1}{w_2} - \frac{w_0}{w_2}$: provare a disegnarla con valori diversi di $\mathbf{w}$ è un buon esercizio.

5.4 Il problema di apprendimento

5.4.1 Formulazione

Dati l esempi $(\mathbf{x}_p, y_p)$ con $y_p \in \{0, 1\}$ o $\{-1, +1\}$ e una loss L , si cerca $\mathbf{w}$ che minimizzi il rischio empirico

$$R_{emp} = \frac{1}{l} \sum_{p=1}^l L(h(\mathbf{x}_p), y_p).$$

Per la classificazione, usare direttamente la loss 0/1 su $\text{sign}(\mathbf{w}^T \mathbf{x})$ rende il problema difficile: la funzione è **costante a tratti**, quindi il gradiente è zero quasi ovunque e non dà indicazioni su come muoversi, e si finisce in problemi combinatori.

5.4.2 Sostituire la loss con una funzione liscia

L'idea è sostituire la loss 0/1 con una funzione **liscia e differenziabile**, come l'errore quadratico, applicata direttamente all'uscita lineare $\mathbf{w}^T \mathbf{x}$ (non a $h(\mathbf{x})$, che contiene il segno):

$$E(\mathbf{w}) = \sum_{p=1}^l (y_p - \mathbf{x}_p^T \mathbf{w})^2 = \|\mathbf{y} - X\mathbf{w}\|^2.$$

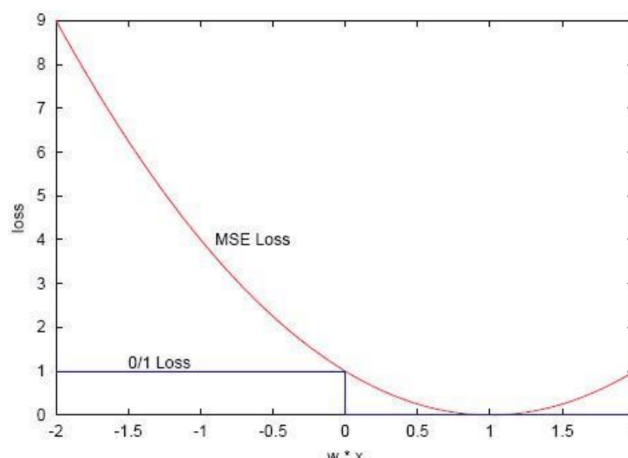


Figura 5.5: Loss 0/1 e loss quadratica per un pattern con target $y = 1$: entrambe sono minimizzate nella zona $\mathbf{w}^T \mathbf{x} > 0$.

Perché funziona? Con target $y_p = 1$, minimizzare $(1 - \mathbf{x}_p^T \mathbf{w})^2$ spinge $\mathbf{x}_p^T \mathbf{w}$ verso 1, quindi positivo: niente errore di classificazione. Con $y_p = -1$ lo spinge verso -1 . Entrambe le loss

sono minimizzate nella stessa zona (a destra nel grafico). La funzione è **quadratica**, quindi un minimo esiste sempre (ma non è necessariamente unico). Lo stesso approccio vale identico per la regressione.

Si vedono ora due algoritmi di apprendimento, entrambi basati su LMS e validi sia per regressione sia per classificazione:

1. un **approccio diretto** basato sulle **equazioni normali**;
2. un **approccio iterativo** basato sulla **discesa del gradiente**.

5.4.3 Approccio diretto: le equazioni normali

Derivando $E(\mathbf{w})$ rispetto alla componente w_j :

$$\frac{\partial E(\mathbf{w})}{\partial w_j} = \sum_{p=1}^l 2(y_p - \mathbf{x}_p^T \mathbf{w}) \frac{\partial (y_p - \mathbf{x}_p^T \mathbf{w})}{\partial w_j}.$$

Nel prodotto scalare $\mathbf{x}_p^T \mathbf{w} = x_{p,1}w_1 + x_{p,2}w_2 + \dots$ l'unico termine che dipende da w_j è $x_{p,j}w_j$, la cui derivata è $x_{p,j}$. Quindi:

$$\frac{\partial E(\mathbf{w})}{\partial w_j} = -2 \sum_{p=1}^l (y_p - \mathbf{x}_p^T \mathbf{w}) x_{p,j} = -2 \sum_{p=1}^l \delta_p x_{p,j},$$

dove $\delta_p = y_p - \mathbf{x}_p^T \mathbf{w}$ è l'errore sul pattern p . Questa forma tornerà nelle reti neurali.

Imponendo il gradiente nullo e passando in notazione matriciale si ottengono le **equazioni normali**:

$$(X^T X) \mathbf{w} = X^T \mathbf{y}.$$

Teorema — Soluzione ai minimi quadrati

Se $X^T X$ è non singolare, la soluzione è unica:

$$\mathbf{w} = (X^T X)^{-1} X^T \mathbf{y} = X^+ \mathbf{y},$$

dove X^+ è la **pseudoinversa di Moore-Penrose** (definita anche se X non è invertibile). Altrimenti le soluzioni sono infinite, e si può scegliere quella a **norma minima** di $\mathbf{w}$.

In pratica si usa la **decomposizione ai valori singolari** (SVD): $X = U \Sigma V^T$ implica $X^+ = V \Sigma^+ U^T$, dove Σ^+ si ottiene sostituendo ogni elemento diagonale non nullo con il suo reciproco. Applicando la SVD si calcola direttamente $\mathbf{w} = X^+ \mathbf{y}$, ottenendo la soluzione a norma minima. **Questo è l'algoritmo di apprendimento** dell'approccio diretto; è implementato in tutte le librerie numeriche (Numerical Recipes, Armadillo, NumPy, R, Matlab...), con molte varianti per efficienza e stabilità.

5.4.4 Approccio iterativo: la discesa del gradiente

Perché cercare altri approcci se esiste la soluzione diretta? Un approccio iterativo permette di ottenere:

- soluzioni **più efficienti** (quella diretta è cubica nella dimensione della matrice);
- **regolarizzazione** (per ridurre la complessità del modello);

- migliore approssimazione con dati rumorosi, fermando la ricerca **prima del minimo**;
- soprattutto, un metodo applicabile anche a **modelli non lineari**. È la base degli approcci fondamentali che vedremo (reti neurali).

L'idea

Il gradiente $\frac{\partial E}{\partial w_j} = -2 \sum_p (y_p - \mathbf{x}_p^T \mathbf{w}) x_{p,j}$ indica la direzione di **salita** dell'errore. Muovendosi nel verso opposto ($\Delta \mathbf{w} = -\nabla E(\mathbf{w})$) si scende verso il minimo. È una **ricerca locale**: si parte da un vettore di pesi iniziale e lo si modifica iterativamente per diminuire l'errore (*steepest descent*).

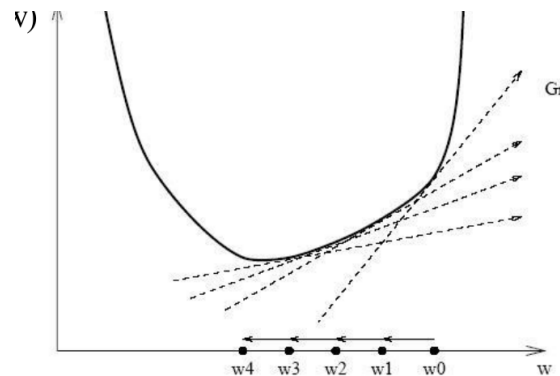
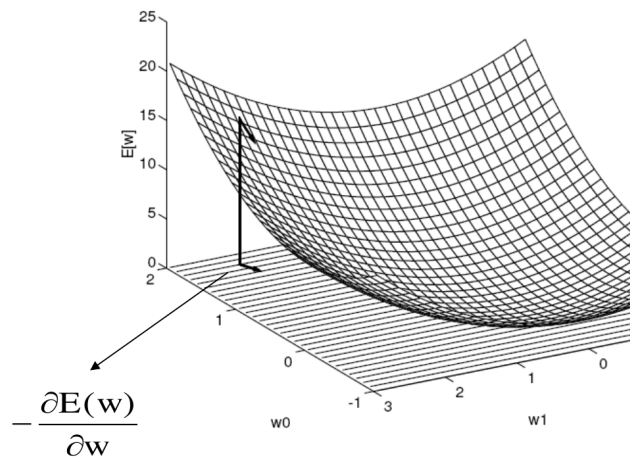


Figura 5.6: Discesa del gradiente in una dimensione: a ogni passo il peso si sposta nel verso opposto alla pendenza.

Con due pesi, la superficie d'errore $E([w_0, w_1]^T)$ è un **paraboloide** (funzione quadratica convessa), e il gradiente negativo è la nostra “bussola” per trovare il minimo.



Our "compass" to find the minimum

Figura 5.7: La superficie d'errore di un modello lineare con due pesi: ogni punto del piano (w_0, w_1) è un'ipotesi.

In generale il vettore $\Delta \mathbf{w}$ ha una componente per ogni peso:

$$\Delta \mathbf{w} = -\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}} = \left(-\frac{\partial E}{\partial w_0}, -\frac{\partial E}{\partial w_1}, \dots, -\frac{\partial E}{\partial w_n} \right)^T,$$

e si può lavorare in spazi multidimensionali senza bisogno di visualizzarli.

La regola di apprendimento (delta rule)

Ci si muove iterativamente con una **regola di apprendimento**:

$$\mathbf{w}_{new} = \mathbf{w} + \eta \Delta \mathbf{w},$$

dove η (eta) è il **learning rate** (*step size*), che regola la velocità della discesa.

In sintesi — Algoritmo di discesa del gradiente

1. Inizializzare $\mathbf{w}$ con valori piccoli; fissare $0 < \eta < 1$.
2. Calcolare $\Delta \mathbf{w} = -\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}}$.
3. Aggiornare $\mathbf{w}_{new} = \mathbf{w} + \eta \Delta \mathbf{w}$.
4. Ripetere dal passo 2 fino a convergenza o finché $E(\mathbf{w})$ è “abbastanza piccolo”.

Note:

- usare $\Delta \mathbf{w}/l$ corrisponde al *least mean squares* (dividere per l sarà il caso standard);
- η regola un compromesso **velocità/stabilità**; può essere ridotto gradualmente verso zero per garantire la convergenza evitando oscillazioni attorno al minimo (molte varianti verranno viste più avanti).

Batch e on-line

- **Versione batch**: il gradiente è la somma su **tutti** gli l pattern, e i pesi si aggiornano dopo aver visto tutto il training set (un'**epoca**). La stima del gradiente è più “precisa”.
- **Versione on-line** (o **stocastica**, SGD): i pesi si aggiornano dopo **ogni** pattern, con il gradiente del singolo pattern:

$$\frac{\partial E_p(\mathbf{w})}{\partial w_j} = -2(y_p - \mathbf{x}_p^T \mathbf{w}) x_{p,j} = -\Delta_p w_j.$$

L'output del secondo pattern è quindi calcolato con pesi già aggiornati dal primo, e così via. Fa progressi a ogni esempio, può essere più veloce, ma richiede un η più piccolo.

Esistono casi intermedi (**mini-batch**), che vedremo più avanti.

Domande — Esercizio: quale curva è lenta, quale instabile, quale buona?

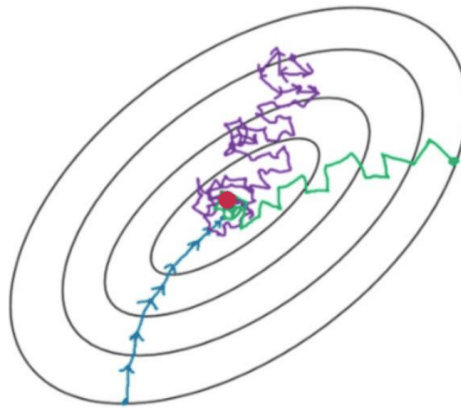
La curva **verde** scende molto lentamente (learning rate troppo piccolo); la **blu** presenta picchi e oscillazioni (learning rate troppo grande, instabile); la **rossa** scende rapidamente e si stabilizza (buona).

Interpretazione: una regola di correzione dell'errore

Ricapitolando,

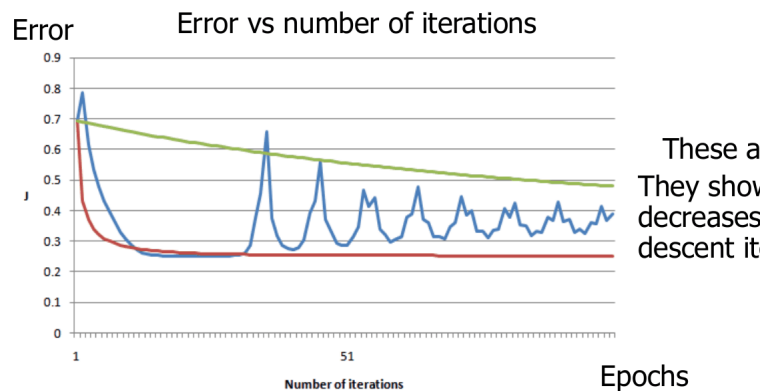
$$\Delta w_j = 2 \sum_{p=1}^l \underbrace{(y_p - \mathbf{x}_p^T \mathbf{w})}_{\delta_p} x_{p,j}, \quad \mathbf{w}_{new} = \mathbf{w} + \eta \Delta \mathbf{w}$$

(la costante 2 si può omettere, assorbendola in η). È una regola di **correzione dell'errore**, detta **regola di Widrow-Hoff** o **delta rule**: ogni w_j cambia **in proporzione all'errore** (target – output).



Paths over the error surface by Batch or On-line version

Figura 5.8: Percorsi sulla superficie d'errore: batch (blu, regolare) e on-line (viola e verde, più irregolari).



Exercise: 1 is slow, 1 is unstable, 1 is good: which one?

Figura 5.9: Curve di apprendimento: l'errore in funzione delle iterazioni (epoche).

- Se l'errore è 0, non c'è correzione.
- Con input $x_j > 0$: se l'errore è positivo (output troppo basso), delta positivo $\rightarrow w_j$ aumenta $\rightarrow$ l'output aumenta $\rightarrow$ l'errore diminuisce.
- Con input $x_j > 0$: se l'errore è negativo (output troppo alto), delta negativo $\rightarrow w_j$ diminuisce $\rightarrow$ l'output diminuisce $\rightarrow$ l'errore diminuisce.
- Con input negativo i segni si invertono, con input nullo il peso non cambia: l'input "decide" quanto e in che verso ogni peso è responsabile dell'errore.

Si impara dagli errori precedenti: *"cercando e sbagliando si impara"* (Goethe).

Nell'esempio, l'input è (1, 0, 1) con pesi (-0,2; 0,5; -0,2) e $w_0 = 0$: $\mathbf{w}^T \mathbf{x} = -0,4$, quindi l'uscita è -1 ma il target è +1. Il delta è positivo: w_1 e w_3 (i cui input valgono 1) vengono aumentati, mentre w_2 (input 0) resta invariato. Dopo la correzione ($w_1 = w_3 = +0,1$) si ha $\mathbf{w}^T \mathbf{x} = 0,2 > 0$ e l'uscita è corretta.

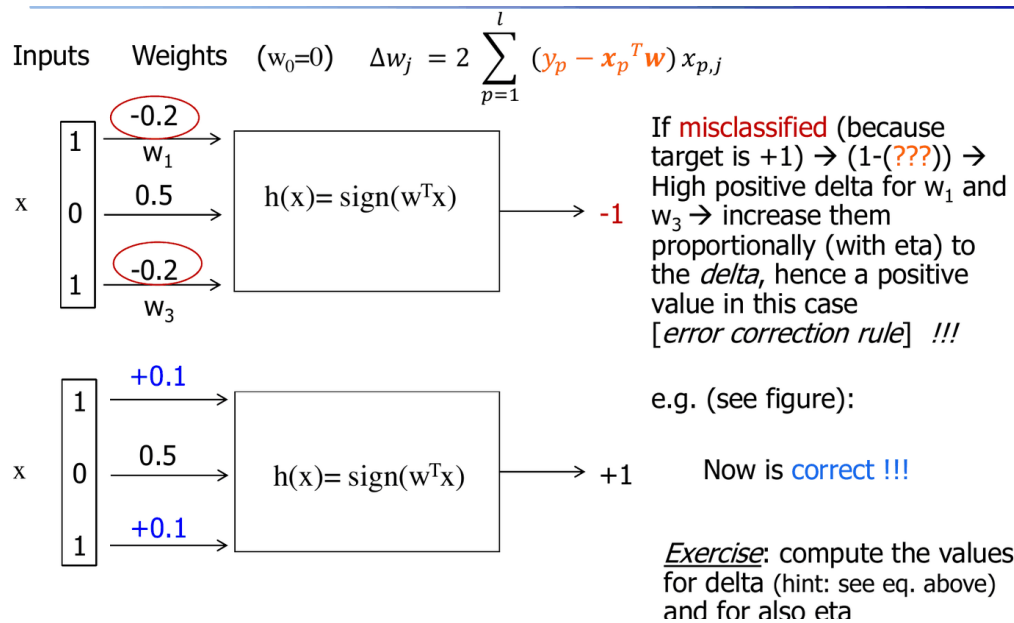


Figura 5.10: La delta rule all'opera: i pesi degli input attivi vengono aumentati finché il pattern viene classificato correttamente.

Suggerimento — Perché la discesa del gradiente è così importante

È un metodo di ricerca locale semplice ed efficace che permette di esplorare uno spazio delle ipotesi **infinito e continuo**. Si applica sempre quando H è continuo e la loss è differenziabile, **non solo ai modelli lineari**: sarà la base dell'addestramento di reti neurali e deep learning. Esistono molti miglioramenti (metodi di Newton e quasi-Newton, gradiente coniugato...).

5.4.5 Riepilogo per la classificazione

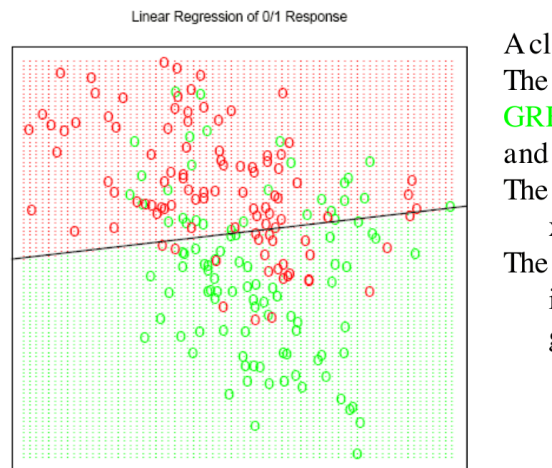
- Il modello viene addestrato sul TR con LMS applicato a $\mathbf{w}^T \mathbf{x}$, tramite la stessa discesa del gradiente usata per la regressione.
- Il modello viene **usato** per classificare applicando la soglia: $h(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x})$.
- L'errore si può valutare come **errore di classificazione** (loss 0/1, numero di pattern sbagliati), non solo come MSE:

$$\text{mean_err} = \frac{1}{l} \sum_{p=1}^l L(h(\mathbf{x}_p), d_p) = \frac{\text{num_err}}{l}, \quad \text{accuratezza} = \frac{l - \text{num_err}}{l}.$$

5.4.6 Torniamo al problema: è una buona soluzione?

Il confine $\{\mathbf{x} \mid \mathbf{x}^T \mathbf{w} = 0.5\}$ è lineare e sembra commettere molti errori sul training. È una cattiva soluzione? Dipende da come sono stati generati i dati:

- **Scenario 1:** ogni classe è generata da una gaussiana con componenti scorrelate, stessa varianza e medie diverse. In questo caso la regola lineare è **quasi ottima**: la sovrapposizione tra le classi è inevitabile (dovuta al rumore nei dati).
- **Scenario 2:** ogni classe è una miscela di 10 gaussiane. In questo caso il modello lineare è **troppo rigido**, e servono i modelli successivi.



The *decision boundary* is $\{\mathbf{x} \mid \mathbf{x}^T \mathbf{w} = 0\}$.
seems to make many errors on the training data.

Figura 5.11: Soluzione lineare al problema di esempio: classi codificate come GREEN = 0, RED = 1, confine $\mathbf{x}^T \mathbf{w} = 0,5$.

Nota — Minimi quadrati e massima verosimiglianza

I minimi quadrati corrispondono al criterio di **massima verosimiglianza** se gli errori sperimentali hanno distribuzione normale.

5.4.7 Bias induttivo del modello lineare

- **Language bias:** H è l'insieme delle funzioni lineari, che può essere molto restrittivo e rigido.
- **Search bias:** ricerca ordinata guidata dalla minimizzazione dei minimi quadrati. Si potrebbe preferire un metodo diverso, che ad esempio restringa i valori dei parametri, ottenendo soluzioni con proprietà diverse (in particolare rispetto alla generalizzazione).

Anche per un modello “semplice” le possibilità sono molte: serve un approccio fondato su principi (la teoria del ML).

5.5 Limiti dei modelli lineari

5.5.1 Regressione di problemi non lineari

Se la funzione target non è lineare (come il seno dell'esempio polinomiale), una retta è una soluzione **troppo povera**: è il caso $M = 1$ già visto, in piena zona di underfitting.

5.5.2 Classificazione: separabilità lineare

Definizione — Separabilità lineare

Due insiemi di punti nel piano sono **linearmente separabili** se possono essere separati completamente da una singola retta. In generale, due gruppi sono linearmente separabili in uno spazio n -dimensionale se possono essere separati da un iperpiano di dimensione $n - 1$.

Il confine di decisione lineare fornisce soluzioni esatte **solo** per insiemi linearmente separabili.

Il modello lineare può rappresentare ad esempio le **congiunzioni**. La congiunzione $x_1 \wedge x_2 \wedge x_4$ si realizza con $1 \cdot x_1 + 1 \cdot x_2 + 0 \cdot x_3 + 1 \cdot x_4 \geq 2,5$ (vale solo se tutti e tre sono 1); l'AND a due variabili con $x_1 + x_2 \geq 1,5$. I pesi w di queste soluzioni possono essere appresi.

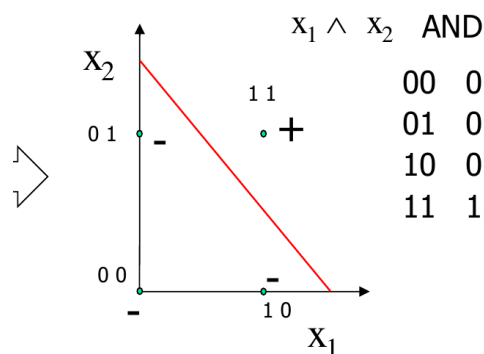


Figura 5.12: L'AND è linearmente separabile.

Ma non tutto è separabile:

- **3 punti:** si può sempre trovare un iperpiano per ogni assegnamento delle etichette? Sì, **se non sono allineati**. Se sono allineati, l'etichettatura "1-0-1" (0 in mezzo) non è separabile.
- **4 punti:** no. Esiste un'etichettatura per cui nessun classificatore lineare è perfetto: lo **XOR** ($00 \mapsto 0$, $01 \mapsto 1$, $10 \mapsto 1$, $11 \mapsto 0$), in cui i positivi stanno su una diagonale e i negativi sull'altra.

Nota — Anticipazione

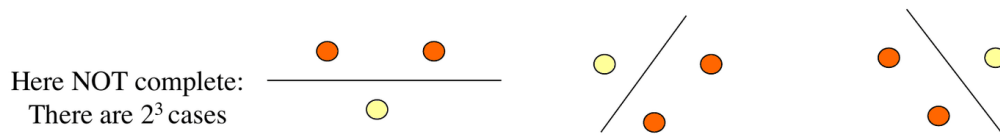
Questo "contare quante etichettature si riescono a separare" è esattamente l'idea della **VC-dimension**: un iperpiano nel piano separa tutte le etichettature di 3 punti (non allineati), ma non di 4. La VC-dimension delle rette nel piano è quindi 3.

5.6 Estendere il modello lineare

5.6.1 "Lineare" nei parametri, non negli input

Nei modelli statistici parametrici, "**lineare**" **non si riferisce alla retta**, ma al modo in cui i **coefficienti** w compaiono nell'equazione. Si possono quindi usare input trasformati come $x, x^2, x^3, \dots$ per modellare relazioni non lineari tra input e output, mantenendo **la stessa macchina di apprendimento** (minimi quadrati). È proprio la regressione polinomiale dell'introdu-

- Given **3 points**, can we always find a separation plane for every assignment of $f(\mathbf{x})$?
No, 3 aligned points with 0 in the middle and others 1; yes if they are not aligned (existence!).



- Given **4 points**, can we always find a separation plane for every assignment of $f(\mathbf{x})$?
No (XOR)
 we can find a labeling such that the linear classifier fails to be perfect

		XOR
		00 0
		01 1
		10 1
		11 0

Exercise: try to redo it by yourself!

Figura 5.13: Limiti della separazione lineare: tre punti non allineati sono sempre separabili, lo XOR su quattro punti no.

zione:

$$h_{\mathbf{w}}(x) = w_0 + w_1x + w_2x^2 + \dots + w_Mx^M = \sum_{j=0}^M w_jx^j.$$

5.6.2 Linear Basis Expansion (LBE)

Definizione — Espansione in basi lineare

$$h_{\mathbf{w}}(\mathbf{x}) = \sum_{k=0}^K w_k \phi_k(\mathbf{x})$$

dove le $\phi_k : \mathbb{R}^n \rightarrow \mathbb{R}$ sono trasformazioni fissate dell'input. Si aumenta il vettore di input con nuove variabili derivate da $\mathbf{x}$.

Esempi di ϕ :

- rappresentazione **polinomiale**: $\phi(\mathbf{x}) = x_j^2$, $\phi(\mathbf{x}) = x_jx_i$, ...;
- trasformazioni non lineari di singoli input: $\log(x_j)$, $\sqrt{x_j}$, ...;
- trasformazioni di più input: $\phi(\mathbf{x}) = \|\mathbf{x}\|$;
- spline, ...

Ad esempio: $h(\mathbf{x}) = w_1x_1 + w_2x_2 + w_3 \log(x_2) + w_4 \log(x_3) + w_5(x_2x_3) + w_0$.

Tipicamente il numero di parametri diventa $K > n$. Il modello è **lineare nei parametri** (e nelle ϕ), non in $\mathbf{x}$: si usa lo stesso algoritmo di apprendimento. Vale per la regressione e anche per la classificazione (basta applicare la soglia a $\sum_k w_k \phi_k(\mathbf{x})$).

Attenzione — Pro e contro della basis expansion

Pro: modella relazioni più complesse, è più espressivo.

Contro: - con molte funzioni di base si rischia facilmente l'**overfitting**, quindi servono metodi di controllo della complessità (intesa come flessibilità, cioè VC-dim, non come costo computazionale); - la **maledizione della dimensionalità** (*curse of dimensionality*): il volume dello spazio cresce così in fretta che i dati disponibili diventano sparsi e non bastano più

a sostenere la complessità del modello; - le ϕ sono **fissate prima** di vedere i dati. L'alternativa sono modelli adattivi, non lineari nei parametri, come le **reti neurali** (che imparano le ϕ) e le **SVM**.

Resta poi aperta la domanda: quali ϕ scegliere? (approcci “a dizionario”).

5.7 Regularizzazione: il controllo della complessità

Come controllare la complessità del modello? Tra i molti approcci c'è il **coefficient shrinkage** (“restringimento” dei coefficienti).

5.7.1 Ridge regression (regularizzazione di Tikhonov)

Si aggiunge alla loss un termine che **penalizza i pesi grandi**:

$$\text{Loss}(\mathbf{w}) = \underbrace{\sum_{p=1}^l (y_p - \mathbf{x}_p^T \mathbf{w})^2}_{\text{termine d'errore } (R_{emp})} + \underbrace{\lambda \|\mathbf{w}\|^2}_{\text{termine di penalità}}, \quad \|\mathbf{w}\|^2 = \sum_j w_j^2.$$

λ è l'**iperparametro di regularizzazione**, un piccolo valore positivo scelto in fase di **model selection**. Il modello risultante è più “liscio” (*smooth*): si favoriscono modelli con pesi piccoli o nulli, cioè che usano meno termini, quindi meno complessi.

Nota — Loss ed Errore non sono più sinonimi

Da qui in poi **Loss** indica la funzione obiettivo usata per l'addestramento (errore + penalità), mentre **Errore E** indica la misura dell'errore del modello (il termine sui dati). Finora li avevamo trattati come equivalenti.

Soluzione

- **Approccio diretto:** $\mathbf{w} = (X^T X + \lambda I)^{-1} X^T \mathbf{y}$. La matrice $X^T X + \lambda I$ è **sempre invertibile** (per $\lambda > 0$): un vantaggio anche numerico.
- **Approccio a gradiente:** si segue ancora il gradiente negativo della loss. La derivata del termine di penalità rispetto a w_j è $2\lambda w_j$, quindi la nuova regola è

$$\mathbf{w}_{new} = \mathbf{w} + \eta \Delta \mathbf{w} - 2\lambda \mathbf{w},$$

dove $\Delta \mathbf{w}$ è il gradiente negativo del solo termine d'errore (moltiplicato per η), mentre la penalità è moltiplicata direttamente per λ . È la tecnica del **weight decay**: anche con gradiente d'errore nullo, a ogni passo ogni peso si riduce di una frazione del suo valore.

Il compromesso governato da λ

- λ **piccolo**: minimizzando la loss conta soprattutto il termine d'errore; si ottiene un modello troppo complesso (pesi di norma alta), con rischio di **overfitting**.
- λ **grande**: conta soprattutto la penalità; l'errore sui dati può crescere troppo, e si va verso l'**underfitting**.

Il grande vantaggio è che si ha una **realizzazione concreta del controllo della complessità**, facile da implementare e di applicabilità generale.

Suggerimento — Tikhonov e SLT

La penalità spinge tutti i pesi verso valori piccoli (alcuni anche a zero), realizzando un controllo della complessità: si ottiene un modello con VC-dimension più bassa (o più adatta), e il compromesso è regolato da **un solo parametro**, λ . Nel grafico del bound SLT, aumentare λ equivale a spostarsi verso sinistra (meno complessità): un λ ben scelto porta vicino al minimo del bound su R .

Effetto sul polinomio di grado 9

Riprendiamo il polinomio di grado 9 con 10 punti, che senza regolarizzazione ($\lambda = 0$) aveva errore di training nullo e overfitting fortissimo.

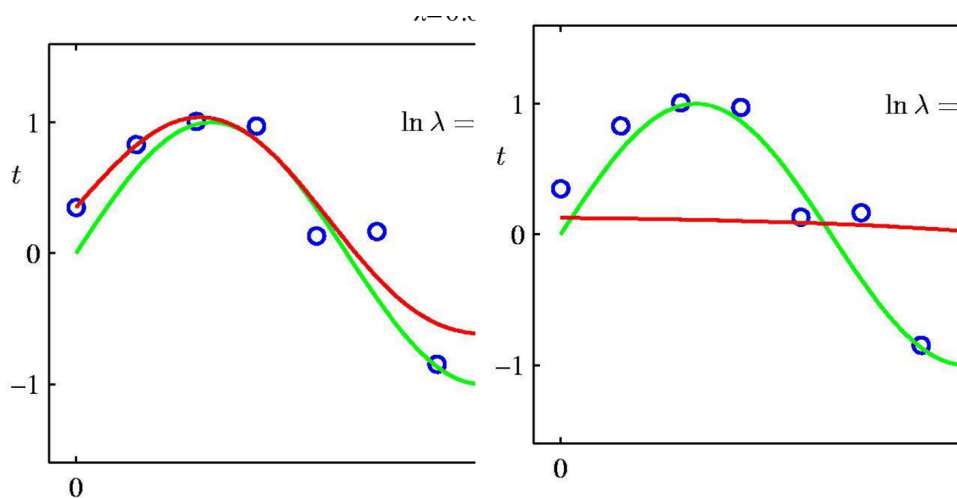


Figura 5.14: Polinomio di grado 9 regolarizzato: con $\ln \lambda = -18$ (sinistra) la curva è liscia e segue bene la funzione vera; con $\ln \lambda = 0$ (destra) è troppo piatta.

Con un λ adatto ($\ln \lambda = -18$, cioè $\lambda \approx 1,5 \cdot 10^{-8}$) il modello funziona bene: è **peggiore sul training** ma riduce l'overfitting. Con λ troppo alto ($\ln \lambda = 0$, cioè $\lambda = 1$) si finisce in underfitting.

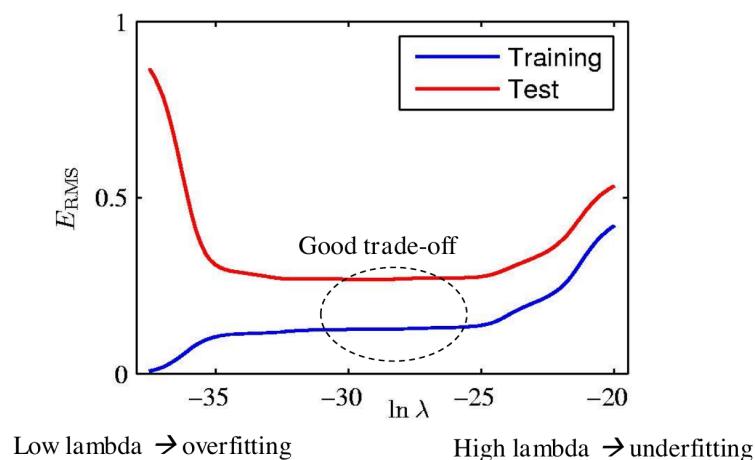


Figura 5.15: E_{RMS} di training e test al variare di $\ln \lambda$: a sinistra overfitting, a destra underfitting, al centro il buon compromesso.

Anche i coefficienti mostrano l'effetto della regolarizzazione:

	$\ln \lambda = -\infty (\lambda = 0)$	$\ln \lambda = -18$	$\ln \lambda = 0$
w_0^*	0,35	0,35	0,13
w_1^*	232,37	4,74	-0,05
w_2^*	-5321,83	-0,77	-0,06
w_3^*	48568,31	-31,97	-0,05
w_4^*	-231639,30	-3,89	-0,03
w_5^*	640042,26	55,28	-0,02
w_6^*	-1061800,52	41,32	-0,01
w_7^*	1042400,18	-45,95	-0,00
w_8^*	-557682,99	-91,53	0,00
w_9^*	125201,43	72,68	0,01

I pesi enormi del caso non regolarizzato si riducono drasticamente: la regolarizzazione “doma” le oscillazioni.

5.7.2 Altre tecniche di regolarizzazione

- **Ridge regression** (norma $\|\cdot\|_2$): penalizza il quadrato dei pesi e tende a rendere **tutti** i pesi più piccoli.
- **Lasso** (norma $\|\cdot\|_1$): penalizza il valore assoluto e tende a portare **alcuni pesi esattamente a zero** (lasciandone altri grandi) → una forma di **feature selection**. Purtroppo il valore assoluto rende la loss non differenziabile (in zero), e servono altri approcci.
- **Elastic net**: usa entrambe le norme.

Suggerimento — Perché L1 azzera i pesi e L2 no?

Il gradiente di λw^2 è $2\lambda w$: diventa piccolissimo quando w è vicino a zero, quindi la spinta verso zero si attenua e il peso resta piccolo ma non nullo. Il gradiente (sub-gradiente) di $\lambda|w|$ è invece $\lambda \text{sign}(w)$: una spinta **costante** verso zero, anche per pesi già piccoli, che li porta esattamente a zero se il termine d'errore non li “difende”.

5.7.3 Altri miglioramenti (opzionali)

- **Data augmentation**: aggiungere rumore agli input insegna al modello a ignorare variazioni irrilevanti (robustezza e regolarizzazione); l'oversampling aiuta con dataset sbilanciati.
- **Input derivati**: si usa un piccolo numero di nuove variabili, combinazioni lineari degli input (Principal Component Regression, Partial Least Squares).

5.7.4 Classificazione multi-classe (opzionale)

Due approcci semplici:

- codifica **1-of-K** delle classi (es. rosso, verde, blu → (0, 0, 1), (0, 1, 0), (1, 0, 0)) e un modello lineare per ciascuna componente;
- **OVA** (*one-vs-all*): si addestrano K classificatori binari, ognuno distingue una classe da tutte le altre; per classificare si sceglie quello con l'uscita più alta (più positiva);

- **AVA** (*all-vs-all, one-vs-one*): un classificatore per ogni coppia di classi ($K(K - 1)/2$ classificatori distinti); vince la classe con la massima somma di uscite o con più voti. Ogni classificatore si addestra su meno dati.

Con molte classi possono verificarsi fenomeni di **mascheramento** (una classe “nascosta” dalle altre). Esistono approcci più sofisticati e modelli con uscite multiple native. Altri classificatori lineari noti sono la **Linear Discriminant Analysis** e la **regressione logistica** (che modella direttamente $P(y | \mathbf{x})$).

5.7.5 Uno sguardo avanti

- Il **Perceptron** (Rosenblatt, 1958, di ispirazione biologica) minimizza solo le classificazioni errate ed è la base delle **reti neurali**: più unità organizzate in strati, che realizzano un'**espansione in basi adattiva** (le ϕ vengono apprese), cioè *representation learning*, addestrate con la discesa del gradiente.
- Le **SVM** (Vapnik, 1996) regolarizzano tramite la **massimizzazione del margine** tra le classi e allargano lo spazio delle feature con basis expansion (es. polinomi).

Entrambe realizzano un'approssimazione non lineare flessibile per classificazione e regressione.

In sintesi — Lezioni apprese dai modelli lineari

- È possibile **apprendere regolando parametri liberi w** , cercando in uno spazio continuo guidati da una funzione di loss.
- Si formula il problema come **LMS** e si derivano due algoritmi: **diretto** (equazioni normali, SVD) e **iterativo** (discesa del gradiente, delta rule).
- I modelli lineari hanno un forte **language bias** (solo problemi linearmente separabili).
- Due concetti chiave del filo rosso del corso: la **basis expansion** (più **flessibilità**) e la **regolarizzazione (controllo della complessità)**. Li ritroveremo, in forme diverse, nelle reti neurali e nelle SVM.

5.8 K-nearest neighbors

Ora ci si sposta all'estremo opposto: da un approccio rigido a uno **molto flessibile** (e locale).

5.8.1 Apprendimento eager e lazy

Rispetto al *timing* dell'apprendimento distinguiamo:

- **eager** (“avido”): si analizzano i dati di training e si costruisce un'**ipotesi esplicita** (come il modello lineare);
- **lazy** (“pigro”): si memorizzano i dati e si aspetta un punto di test; solo allora si costruisce un'**ipotesi ad hoc** per classificare quel punto.

5.8.2 1-nearest neighbor

Definizione — Algoritmo 1-NN

1. Memorizzare i dati di training $\langle \mathbf{x}_p, y_p \rangle$, $p = 1, \dots, l$.
2. Dato un input $\mathbf{x}$ (di dimensione n), trovare l'esempio di training più vicino:

$$i(\mathbf{x}) = \arg \min_p d(\mathbf{x}, \mathbf{x}_p),$$

ad esempio con la distanza euclidea $d(\mathbf{x}, \mathbf{x}_p) = \sqrt{\sum_{t=1}^n (x_t - x_{p,t})^2} = \|\mathbf{x} - \mathbf{x}_p\|$.

3. Restituire y_i .

Il 1-NN è **molto flessibile**: sul training set commette **zero errori** (ogni punto è il vicino più prossimo di sé stesso). Il confine di decisione non è lineare ed è molto irregolare, forse inutilmente rumoroso (ad esempio nello scenario 1, dove sappiamo che la soluzione ottima è quasi lineare). E sui dati di test?

5.8.3 K-nearest neighbors

Un modo naturale di classificare un nuovo punto è guardare i suoi **vicini** e farne la media:

$$\text{avg}_k(\mathbf{x}) = \frac{1}{k} \sum_{\mathbf{x}_i \in N_k(\mathbf{x})} y_i,$$

dove $N_k(\mathbf{x})$ è l'insieme dei k pattern più vicini a $\mathbf{x}$ secondo la distanza d . Se nell'intorno di $\mathbf{x}$ una classe domina chiaramente, è probabile che anche $\mathbf{x}$ vi appartenga: la regola di classificazione è il **voto a maggioranza** tra i membri di $N_k(\mathbf{x})$. Per target in $\{0, 1\}$:

$$h(\mathbf{x}) = \begin{cases} 1 & \text{se } \text{avg}_k(\mathbf{x}) > 0,5 \\ 0 & \text{altrimenti.} \end{cases}$$

Per la **regressione** si usa direttamente la media dei target dei k vicini.

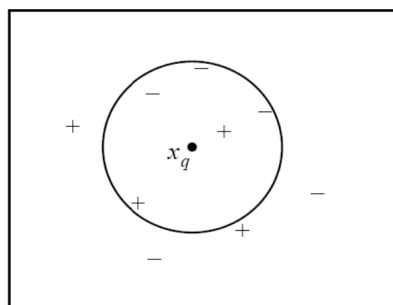


Figura 5.16: 1-NN restituisce “+” per x_q , 5-NN restituisce “-”: aumentare k “liscia” la decisione, utile con dati rumorosi.

Con $k = 15$ il modello è ancora molto flessibile, ma commette qualche errore sul training; il confine è ancora irregolare ma meno, e si **adatta alle densità locali** delle classi.

Il K-NN usa implicitamente il **diagramma di Voronoi**: ogni cella contiene tutti i punti più vicini a un certo pattern che a qualsiasi altro, e i lati delle celle sono i punti equidistanti da due pattern. Il 1-NN assegna a ogni cella l'etichetta del suo pattern.

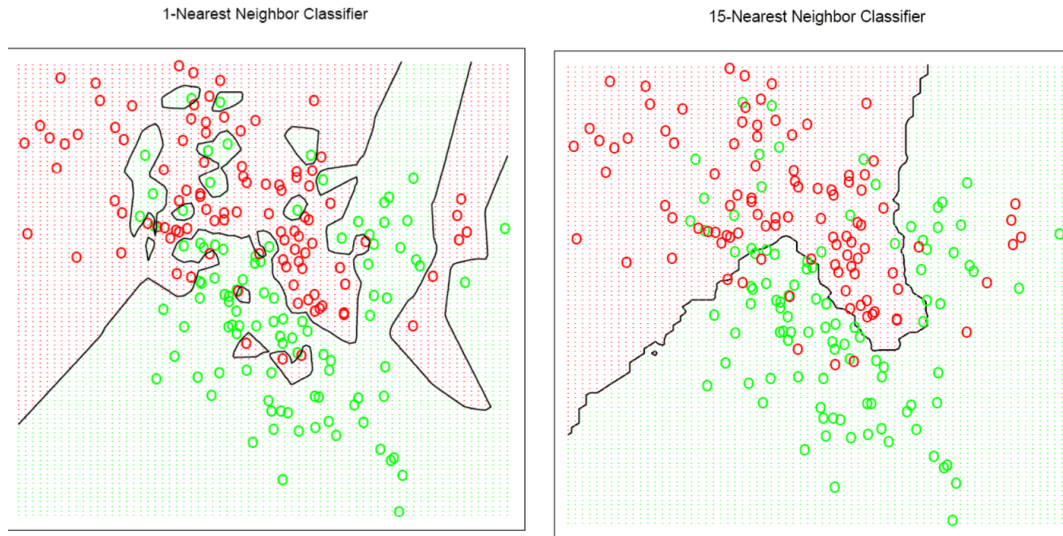


Figura 5.17: Confini di decisione del 1-NN (sinistra) e del 15-NN (destra) sullo stesso problema.

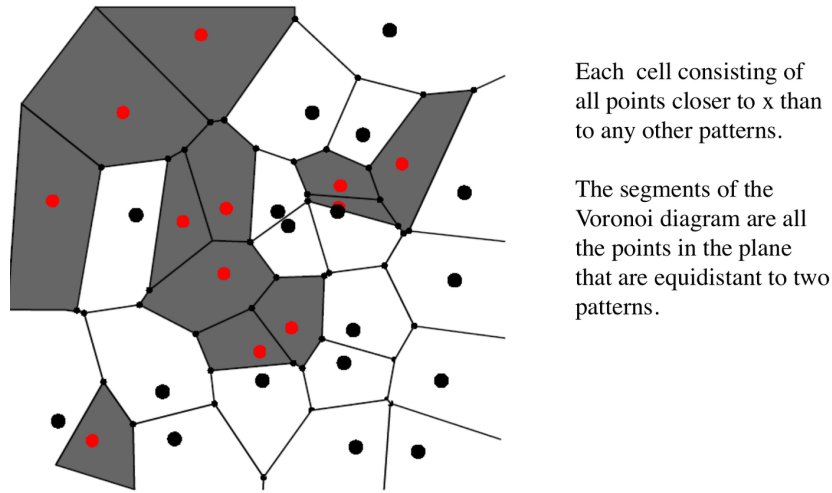


Figura 5.18: Diagramma di Voronoi: il confine del 1-NN segue i lati delle celle tra punti di classi diverse.

5.8.4 Varianti

Multi-classe: si restituisce la classe più frequente tra i k vicini:

$$h(\mathbf{x}) = \arg \max_v \sum_{\mathbf{x}_i \in N_k(\mathbf{x})} \mathbb{1}_{v, y_i}, \quad \mathbb{1}_{v, y_i} = \begin{cases} 1 & \text{se } v = y_i \\ 0 & \text{altrimenti.} \end{cases}$$

Distanza pesata: i vicini più vicini contano di più:

$$h(\mathbf{x}) = \arg \max_v \sum_{\mathbf{x}_i \in N_k(\mathbf{x})} \frac{\mathbb{1}_{v, y_i}}{d(\mathbf{x}, \mathbf{x}_i)^2},$$

e se $d = 0$ per qualche $\mathbf{x}_i$ si restituisce direttamente y_i .

5.8.5 Un estremo del ML

Il K-NN non costruisce un'ipotesi globale valida per tutte le istanze: **non c'è un modello da addestrare**, ma bisogna memorizzare gli esempi. Fa stime **locali** (con funzioni localmente costanti) invece di un'approssimazione lineare globale. È un metodo *lazy*, basato su memoria, sulle istanze, sulle distanze.

5.8.6 K-NN contro modello lineare

	Lineare	K-NN
Flessibilità	rigido (bassa varianza)	flessibile (alta varianza)
Timing	eager	lazy
Tipo	parametrico	instance-based
Parametri	$n + 1$ (3 nell'esempio)	circa l/k "parametri effettivi"

Con k piccolo bastano pochi punti per cambiare il confine: la flessibilità si paga. Si potrebbe pensare che il K-NN abbia un solo parametro (k), ma realisticamente ha circa l/k **parametri effettivi** (Hastie-Tibshirani-Friedman): è come se dividesse lo spazio in l/k regioni, ognuna con la sua "media".

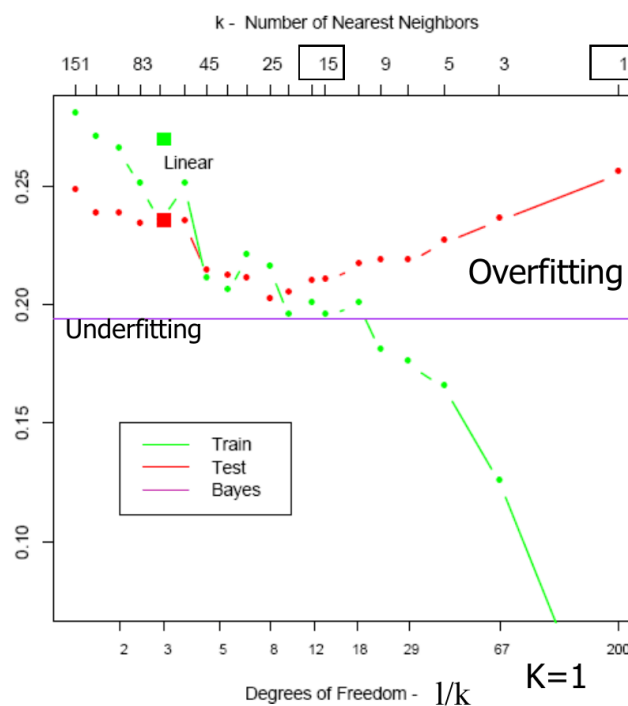


Figura 5.19: Curve di errore del K-NN al variare di k (training di 200 punti, test di 10000). I quadratini sono il modello lineare, la linea viola l'errore ottimo di Bayes.

Spostandosi da k grande a $k = 1$ (cioè aumentando l/k) si passa dall'**underfitting** all'**overfitting**: l'errore di training scende fino a zero, quello di test ha un minimo per valori intermedi. È di nuovo il grafico del bound SLT, con k nel ruolo del controllo della complessità: più flessibilità permette di trovare il risultato migliore, **se** la complessità viene controllata.

5.8.7 Il classificatore ottimo di Bayes

Definizione — Classificatore di Bayes ed errore di Bayes

Se si conosce la densità $P(\mathbf{x}, y)$, la scelta ottima è assegnare la classe più probabile:

$$h(\mathbf{x}) = \arg \max_v P(v | \mathbf{x}), \quad v \in \{C_1, \dots, C_K\}.$$

Il suo tasso d'errore, detto **Bayes rate**, è il **minimo errore raggiungibile** data la distribuzione dei dati (assumendo nota la densità generatrice).

Il K-NN approssima direttamente questa soluzione: il voto a maggioranza in un intorno è esattamente una stima di $\arg \max_v P(v | \mathbf{x})$, con due rilassamenti: la probabilità condizionata **in un punto** diventa probabilità condizionata **in un intorno** del punto (approssimazione locale), e le probabilità sono stimate con le **proporzioni** nel campione di training.

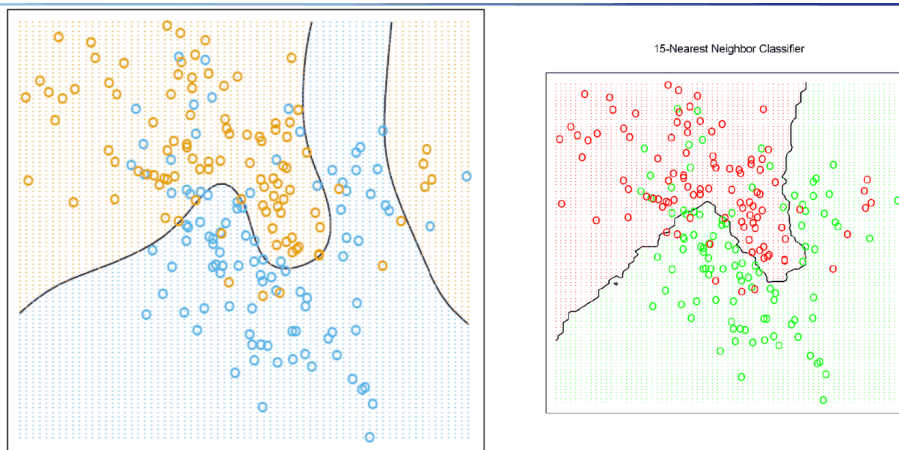


Figura 5.20: Confine di Bayes ottimo (sinistra) e confine del 15-NN (destra): il 15-NN è molto vicino all'ottimo, coerentemente con il minimo dell'errore di test nella figura precedente.

5.8.8 Bias induttivo del K-NN

1. **La distanza assunta** stabilisce quali esempi sono simili: si assume che la classificazione di un punto sia simile a quella dei vicini **secondo la metrica scelta**. Si possono usare metriche diverse dall'euclidea; dati simbolici richiedono metriche ad hoc (es. la **distanza di Hamming** tra stringhe della stessa lunghezza: il numero di posizioni in cui differiscono).
2. **Località**: approssimazione locale, con un'assunzione di *smoothness* locale (da confrontare più avanti con il bias del deep learning).

Il bias legato alla metrica è spesso sottovalutato nei libri, ma è cruciale.

5.8.9 Limiti del K-NN

Scala delle variabili e metrica

La scelta della scala dipende dalla conoscenza del dominio. Se le variabili devono contribuire allo stesso modo, bisogna fare attenzione alle differenze di range: **riscalare i dati** (es. media zero e varianza unitaria) equivale a **cambiare la metrica**. La scalatura può cambiare del tutto quale sia il vicino più prossimo: il K-NN è fragile anche rispetto a un pre-processing elementare.

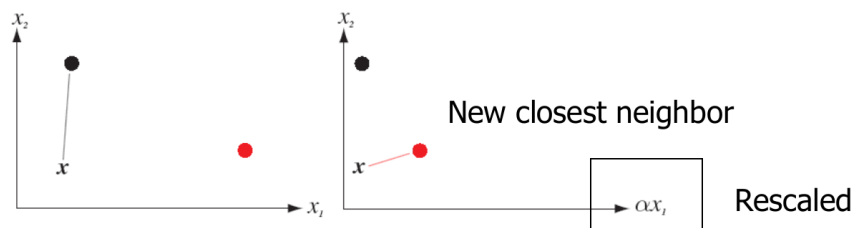


Figura 5.21: Riscalando una variabile cambia il vicino più prossimo.

Costo computazionale

Il K-NN costruisce l'approssimazione locale **per ogni nuovo esempio** da predire: il costo computazionale è **spostato nella fase di predizione**. Per ogni input si calcolano le distanze da tutti i vettori memorizzati (tempo proporzionale al numero di pattern), anche se esistono algoritmi di ricerca di prossimità e indicizzazione ad hoc. Anche il costo in **spazio** è alto (tutti i dati di training).

Interpretabilità

I modelli K-NN offrono poca interpretazione, soggettiva e dipendente dalla metrica.

La maledizione della dimensionalità

Il K-NN funziona bene se si trova un insieme significativo di dati vicini a ogni $\mathbf{x}$, cioè con un **campionamento denso**. Quando ci sono molte variabili di input (dimensione n alta) fallisce spesso per la **curse of dimensionality**, che ha molte manifestazioni. Tre in particolare influenzano direttamente la generalizzazione.

1. In alta dimensione è difficile trovare punti vicini. Diventa difficile raccogliere k osservazioni “vicine” (cioè simili) a un punto di query: gli intorni tendono a essere spazialmente grandi e le stime non sono più locali; ridurre la dimensione dell'intorno significa ridurre k , e la varianza della stima aumenta (overfitting).

Consideriamo dati uniformi nel cubo unitario e un sottocubo che deve contenere una frazione r dei dati. Il suo lato deve essere $r^{1/n}$ (perché volume = lato ^{n}).

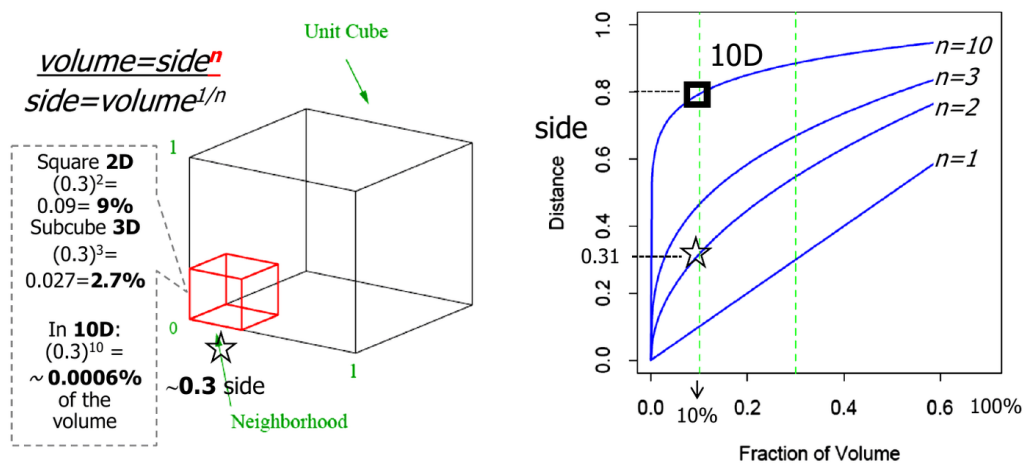


Figura 5.22: Maledizione della dimensionalità: lato del sottocubo necessario per catturare una frazione r del volume, al variare della dimensione n .

- Per catturare il 10% dei dati: in 2D basta un lato di $0,1^{1/2} \approx 0,32$ (il 30% del range di ogni variabile); in 10D serve $0,1^{1/10} \approx 0,8$, cioè l'**80% del range** di ogni coordinata! Anche per l'1% del volume in 10D serve il 63% del range. L'intorno non è più "locale" e i vicini non sono più simili.
- Viceversa, fissato il lato a 0,3: in 1D si cattura il 30% dei dati, in 2D il 9%, in 3D il 2,7%, in 10D circa lo **0,0006%**. Quel lato può non bastare a trovare k dati, a meno di usare k piccolo (e quindi rischiare overfitting).

2. Bassa densità di campionamento. La densità di campionamento è proporzionale a $l^{1/n}$. Se 100 punti bastano a stimare una funzione in $\mathbb{R}^1$, per ottenere un'accuratezza simile in $\mathbb{R}^{10}$ ne servono $100^{10} = 10^{20}$.

3. Feature irrilevanti (*curse of noisy*). Se il target dipende solo da poche delle molte feature (es. 2 su 20), la somiglianza tra pattern può essere dominata dal gran numero di feature irrilevanti, e il "vicino" trovato non è davvero simile. Il problema cresce con la dimensionalità. Rimedi: **pesare le feature** secondo la loro rilevanza (allungando gli assi lungo alcune dimensioni; i pesi si possono cercare con una costosa model selection o altri metodi) oppure fare **feature selection** (eliminare variabili, riducendo la dimensione dell'input).

5.8.10 Scelte di progetto del K-NN

- La **metrica** d (euclidea, Hamming, Manhattan, pesi sulle feature...): spesso è la chiave del successo di un'applicazione.
- Il valore di k , che controlla underfitting e overfitting.
- Spesso è necessario selezionare un **sottoinsieme dei dati** (un insieme di prototipi, ad esempio tramite clustering) e un **sottoinsieme delle feature**.

Estensioni ad altri modelli locali: *kernel smoothers*, regressione lineare locale, metodi a prototipi, *case-based reasoning*.

In sintesi — Lezioni generali dal K-NN

- Una varianza troppo bassa è povera (modelli lineari rigidi), una troppo alta è pericolosa (K-NN).
- **Smoothing**: nel K-NN si ottiene aumentando k .
- **Curse of dimensionality**: il volume dello spazio cresce così in fretta che i dati diventano sparsi.
- Le curve di errore al variare di k sono un'altra istanza del grafico del bound SLT.
- Il bias induttivo del K-NN è legato alla metrica.

5.8.11 E adesso?

Il K-NN non costruisce un vero "modello appreso": non estrae regolarità né sintetizza conoscenza, e non soddisfa l'obiettivo dell'apprendimento (poter "dimenticare" gli esempi dopo aver costruito il modello). Nelle prossime lezioni si cercano modelli **compatti** come la LTU (tutta la conoscenza in pochi parametri) ma **flessibili** come il K-NN, con un adeguato supporto al controllo della complessità: le **reti neurali**.

Domande — Possibili domande d'esame

- Formulare il problema di apprendimento per un modello lineare con LMS, per regressione e classificazione.

- Derivare il gradiente dell'errore quadratico e le equazioni normali.
- Descrivere l'algoritmo di discesa del gradiente; differenza tra batch e on-line; ruolo di η .
- Spiegare la delta rule come regola di correzione dell'errore.
- Perché per la classificazione si usa l'errore quadratico su $\mathbf{w}^T \mathbf{x}$ invece della loss 0/1?
- Cos'è la separabilità lineare? Perché lo XOR non è linearmente separabile?
- Cos'è la linear basis expansion? Pro e contro.
- Scrivere la loss di Tikhonov, la soluzione diretta e la regola di weight decay; ruolo di λ .
- Differenza tra regolarizzazione L1 e L2.
- Descrivere il K-NN; confrontarlo con il modello lineare; cos'è il classificatore di Bayes?
- Spiegare la maledizione della dimensionalità.

Capitolo 6

Reti neurali (parte 1): dal neurone al MLP

Con le reti neurali si entra nel cuore del corso. Questa prima parte parte dall'ispirazione biologica, introduce il neurone artificiale e il **Perceptron** con il suo algoritmo di apprendimento e il **teorema di convergenza**, confronta il Perceptron con l'LMS, introduce le funzioni di attivazione sigmoidali e arriva al **Multi-Layer Perceptron** (MLP), analizzato come funzione flessibile, come espansione in basi adattiva e come approssimatore universale. Si chiude con i problemi che l'apprendimento in una rete pone e che saranno risolti dalla **backpropagation**.

6.1 Le reti neurali come strumento di ML

Le reti neurali si studiano con due obiettivi diversi:

- **modellare sistemi biologici** e processi di apprendimento (filosofia, psicologia, neurobiologia, scienze cognitive, neuroscienze computazionali): qui il **realismo biologico** è essenziale;
- **costruire sistemi e algoritmi di ML efficaci** (statistica, AI, fisica, matematica, ingegneria), spesso perdendo lo stretto realismo biologico: qui contano le proprietà **computazionali e algoritmiche**.

Nel corso ci interessano le **ANN** (*Artificial Neural Networks*) come **strumento flessibile di ML**, nel senso dell'approssimazione di funzioni: una rete realizza una funzione matematica $h(\mathbf{x})$ con proprietà speciali.

Pur essendo una “macchina storica”, le reti neurali restano un approccio potente:

- apprendono da esempi;
- sono **approssimatori universali** (teorema di Cybenko): approcci flessibili per funzioni arbitrarie, anche non lineari;
- gestiscono rumore e dati incompleti, con prestazioni che **degradano gradualmente** in condizioni avverse;
- trattano dati reali e discreti, per regressione e classificazione;
- non sono un singolo modello ma un **paradigma** (nella classe degli approcci sub-simbolici).

La “filosofia” di fondo è il **connessionismo**: comportamenti complessi (mentali, comportamentali) che **emergono** dall'interazione di molte unità computazionali semplici interconnesse.

6.2 L'ispirazione biologica

Il cervello umano ha più di 10^{10} neuroni, ciascuno con 10^4 – 10^5 connessioni. Un neurone risponde in circa un millisecondo, eppure riconosciamo un'immagine in circa 0,1 secondi: al massimo un centinaio di passi di calcolo **seriali**. Ne segue che il cervello deve eseguire una

computazione **massicciamente parallela**. (Si noti che il riconoscimento facciale, così facile per noi, è complesso per i computer convenzionali.)

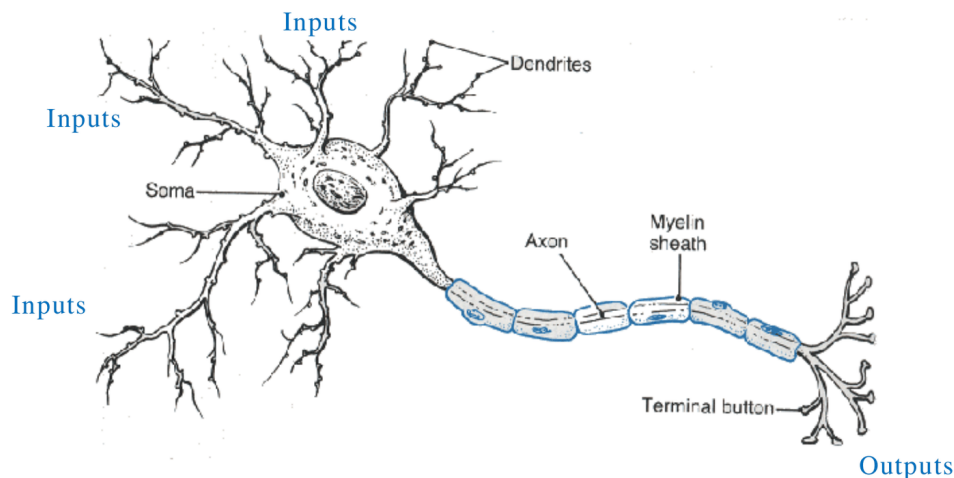


Figura 6.1: Il neurone biologico: dendriti (input), soma, assone e terminali (output).

Il funzionamento, in sintesi: il neurone attraversa cicli di carica e scarica (ioni sodio e potassio); gli input dai dendriti modificano il potenziale della cellula; superata una **soglia**, viene generato uno **spike** (un impulso di tensione), e l'informazione è codificata anche negli intervalli di tempo tra gli spike.

Il segnale si propaga lungo l'assone fino alle **sinapsi**, dove passa (in forma chimica, tramite **neurotrasmettitori**) al neurone successivo. Le sinapsi possono essere **eccitatorie** o **inibitorie**, e soprattutto cambiano con l'apprendimento: la loro forza (il "peso") si rinforza in risposta agli stimoli. È la **plasticità** del sistema nervoso.

Definizione — Apprendimento hebbiano (Hebb, 1949)

Una sinapsi si rafforza quando gli input del neurone (e gli output correlati) si ripetono. Lo stimolo in ingresso rinforza la sinapsi, quindi **i pesi si avvicinano agli input**.

6.3 Il neurone artificiale

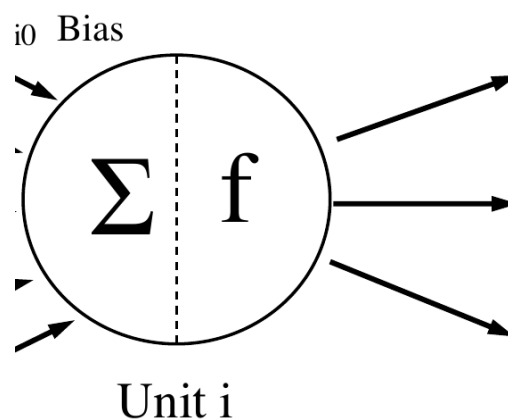


Figura 6.2: L'unità di elaborazione (neurone artificiale).

Un'**unità** (nodo, neurone) riceve input da sorgenti esterne o da altre unità. Ogni connessione ha un **peso** w , parametro libero modificabile dall'apprendimento (l'analogo della forza

sinaptica). L'unità i calcola:

$$net_i(\mathbf{x}) = \sum_j w_{ij} x_j, \quad o_i(\mathbf{x}) = f(net_i(\mathbf{x})).$$

- La somma pesata net_i è l'**input netto** (*net input*) dell'unità i ; include il bias w_{i0} tramite l'input costante $x_0 = 1$.
- f è la **funzione di attivazione** (lineare, a soglia, sigmoide...).

Attenzione — Notazione dei pesi

w_{ij} è il peso **dell'unità i** , cioè della connessione **dall'input/unità j verso l'unità i** (non il contrario). Alcune fonti (ad esempio Wikipedia nella sezione sulla backpropagation, e alcune librerie) usano la convenzione opposta; nel corso si usa sempre quella tradizionale: il primo indice è l'unità che riceve.

Tre funzioni di attivazione classiche:

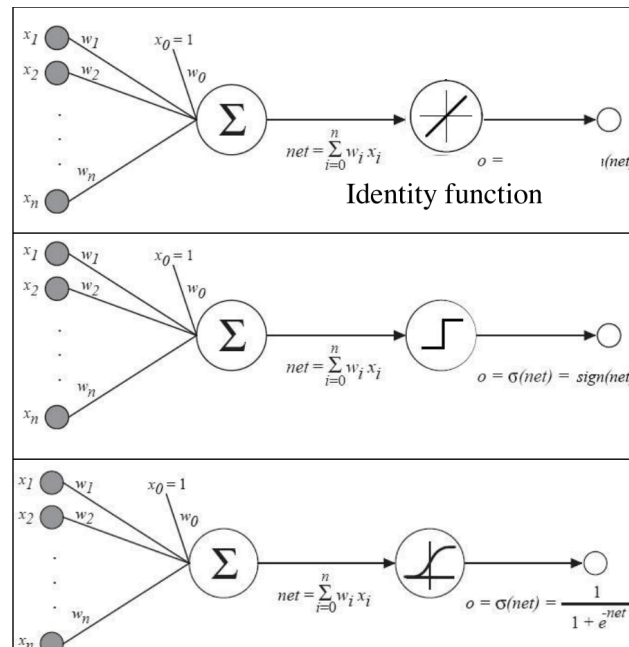


Figura 6.3: Funzione di attivazione lineare (identità), a soglia (Perceptron) e logistica (sigmoide).

1. **lineare** (identità): $h(\mathbf{x}) = \sum_i w_i x_i$;
2. **a soglia** (gradino): è il **Perceptron**, cioè la LTU già vista;
3. **logistica** (sigmoide): vedi più avanti.

6.4 Il Perceptron

Il **Perceptron** fu proposto da Frank Rosenblatt (1957–1960) per la classificazione di pattern, simulando la percezione umana: un “occhio” di fotocellule collegato a unità che calcolano una combinazione pesata con soglia.

Il singolo neurone è un'unità computazionale semplicissima. Minsky: “*per il tipo di riconoscimento che sa fare, è una macchina così semplice che sarebbe sorprendente se la natura non la usasse da qualche parte*”. Il Perceptron è un modello di importanza storica e **paradigmatico** per

- Frank Rosenblatt (1957-1958, 1960, ...)

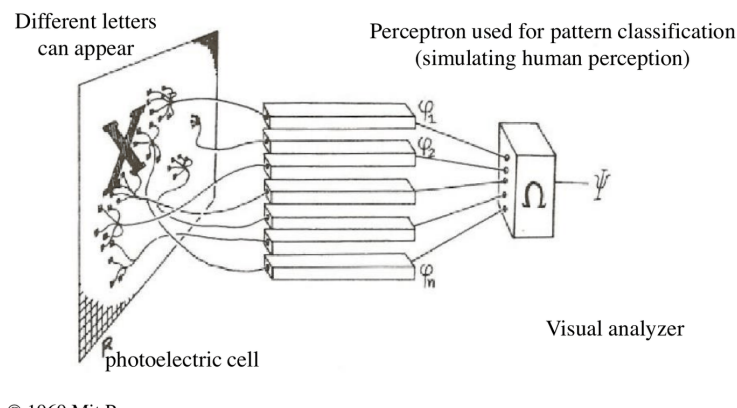


Figura 6.4: Il Perceptron originale, usato per riconoscere lettere.

il passaggio dai modelli lineari a quelli non lineari: componendo e collegando perceptron si ottengono le reti **MLP** (*Multi-Layer Perceptron*).

6.4.1 Le reti di McCulloch e Pitts (1943)

Il primo modello, “*A logical calculus of the ideas immanent in nervous activity*” (1943), si chiedeva cosa calcola una rete data e se una rete può calcolare una data formula logica.

- I neuroni hanno due stati: attivo (1) e non attivo (0).
- Tutte le sinapsi sono equivalenti e caratterizzate da un numero reale (la forza w), positivo per connessioni **eccitatorie** e negativo per **inibitorie**.
- Un neurone i si attiva quando la somma dei pesi w_{ij} provenienti dai neuroni j attivi, più un bias, supera zero.

Input e output binari: sostanzialmente è la LTU/Perceptron.

Rappresentare funzioni booleane

Con $net = \mathbf{w}^T \mathbf{x} = \sum_{i=0}^n w_i x_i$ e uscita 1 se $net > 0$, 0 (o -1) altrimenti:

- **AND:** $w_1 = w_2 = 1$, $w_0 = -1,5$ (serve che entrambi gli input valgano 1 per superare 1,5);
- **OR:** $w_1 = w_2 = 1$, $w_0 = -0,5$ (basta un input a 1).

(Esercizio: il NOT si ottiene con $w_1 = -1$, $w_0 = 0,5$: per $x_1 = 0$ il net è $0,5 > 0$, per $x_1 = 1$ è $-0,5$.)

Lo XOR e le reti a due strati

Lo **XOR** non è linearmente separabile: nessun singolo perceptron lo può rappresentare (Minsky e Papert, 1969). Si può però scomporre:

$$x_1 \oplus x_2 = x_1 \bar{x}_2 + \bar{x}_1 x_2 = \bar{h}_1 \cdot h_2, \quad \text{con } h_1 = x_1 \cdot x_2 \text{ (AND)}, \quad h_2 = x_1 + x_2 \text{ (OR)}.$$

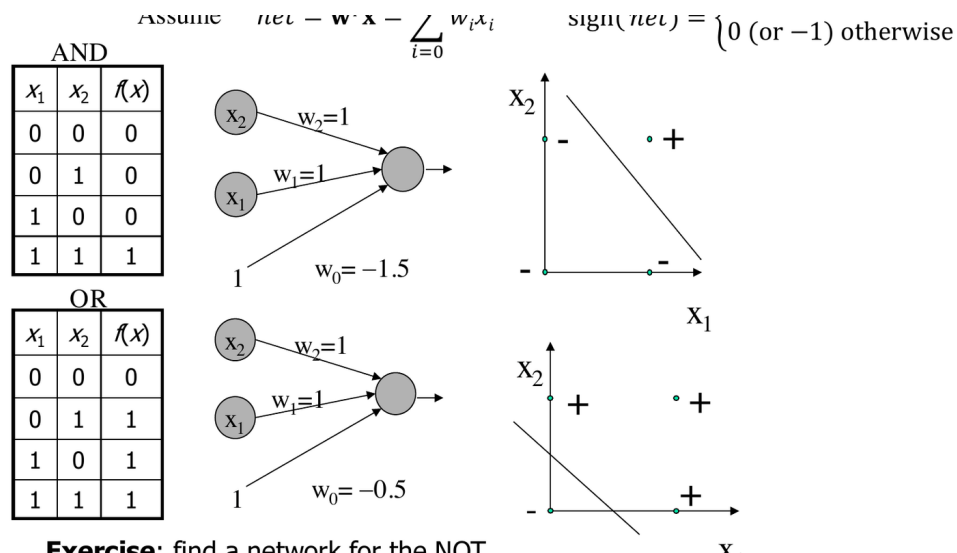


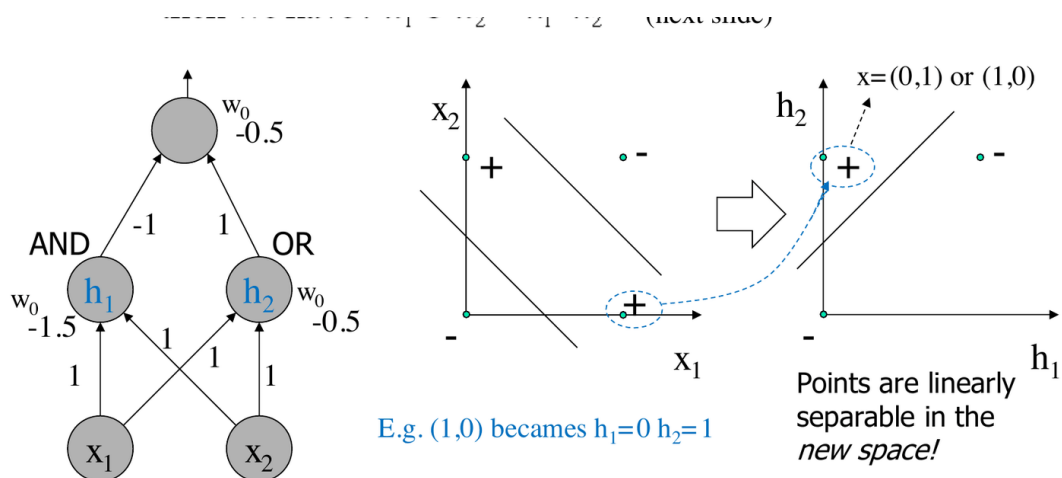
Figura 6.5: AND e OR realizzati da un singolo perceptron.

Teorema — XOR come composizione di AND e OR

Per De Morgan, $\overline{h_1} = \overline{a \wedge b} = \bar{a} \vee \bar{b}$. Quindi

$$\bar{h}_1 \wedge h_2 = (\bar{a} \vee \bar{b}) \wedge (a \vee b) = (\bar{a} \wedge a) \vee (\bar{a} \wedge b) \vee (a \wedge \bar{b}) \vee (b \wedge \bar{b}) = (\bar{a} \wedge b) \vee (a \wedge \bar{b}) = a \oplus b.$$

Si costruisce così una rete a **due strati**: due unità nascoste calcolano h_1 (AND) e h_2 (OR), e un'unità di uscita calcola $\bar{h}_1 \wedge h_2$ (pesi -1 da h_1 e $+1$ da h_2 , bias $-0,5$).

Figura 6.6: Lo XOR con una rete a due strati: nello spazio delle unità nascoste (h_1, h_2) i punti diventano linearmente separabili.**6.4.2 Lo strato nascosto come rappresentazione**

La figura mostra il concetto chiave: lo strato nascosto produce una **ri-rappresentazione interna** degli input. Il punto $(1,0)$, che nello spazio originale sta nell'angolo in basso a destra, diventa $h_1 = 0$, $h_2 = 1$: i due punti positivi $(0,1)$ e $(1,0)$ vengono mappati **nello stesso punto** $(0,1)$ dello spazio nascosto, e il problema diventa linearmente separabile per l'unità di uscita.

Suggerimento — Il concetto più importante delle reti neurali

Sviluppare **feature di alto livello** negli strati nascosti è il fattore chiave delle reti neurali: la rappresentazione nello strato nascosto rende più facile il compito dello strato di uscita. Questa composizione di operazioni intermedie può essere estesa su **molti strati di astrazione**, e — a differenza di quanto visto qui, dove i pesi sono scelti a mano — nelle reti neurali queste rappresentazioni **si apprendono**. È l'idea alla base del *representation learning* e del **deep learning**.

In sintesi — Proprietà delle reti di perceptron

- I perceptron rappresentano AND, OR, NOT (e quindi NAND, NOR) → **ogni funzione booleana** può essere rappresentata da una rete di perceptron.
- **Due livelli** bastano (più livelli possono essere più efficienti).
- Un singolo strato non basta (Minsky e Papert, 1969): limiti dovuti alla separabilità lineare.
- Finora però **non abbiamo alcun algoritmo di apprendimento**, nemmeno per una singola unità.

6.5 Apprendimento per una singola unità

Storicamente ci sono due famiglie di metodi:

1. **Adaline** (*Adaptive Linear Neuron*, Widrow e Hoff): durante il training l'unità è **lineare**; si usano la soluzione diretta LMS o la discesa del gradiente (esattamente come nella lezione sui modelli lineari). Serve per regressione e, con la soglia, classificazione. È l'approccio che generalizzeremo agli MLP.
2. **Perceptron** (Rosenblatt): durante il training l'unità è **non lineare** (con funzione a soglia). Solo classificazione; se ne studiano le capacità e la convergenza.

Per costruire un classificatore lineare abbiamo quindi tre algoritmi: LMS diretto, LMS a gradiente e ora l'algoritmo del Perceptron (e altri verranno, come le SVM).

6.5.1 L'algoritmo di apprendimento del Perceptron

L'obiettivo è **minimizzare il numero di pattern classificati male**: trovare $\mathbf{w}$ tale che $\text{sign}(\mathbf{w}^T \mathbf{x}) = d$. È un algoritmo **on-line** (un passo per ogni pattern).

In sintesi — Perceptron Learning Algorithm

1. Inizializzare i pesi (a zero o a piccoli valori casuali).
2. Scegliere un learning rate $\eta \in (0, 1)$.
3. Finché non è soddisfatta la condizione di arresto (es. i pesi non cambiano più), per ogni pattern $(\mathbf{x}, d)$ con $d \in \{+1, -1\}$:
 - calcolare $\text{out} = \text{sign}(\mathbf{w}^T \mathbf{x})$;
 - se $\text{out} = d$, **non cambiare i pesi**;
 - se $\text{out} \neq d$, aggiornare: $\mathbf{w}_{\text{new}} = \mathbf{w} + \eta d \mathbf{x}$.

In pratica si aggiunge $\eta \mathbf{x}$ se $\mathbf{w}^T \mathbf{x} \leq 0$ e $d = +1$ (falso negativo), si sottrae $\eta \mathbf{x}$ se $\mathbf{w}^T \mathbf{x} > 0$ e $d = -1$ (falso positivo). In forma equivalente:

$$\mathbf{w}_{new} = \mathbf{w} + \frac{1}{2} \eta (d - out) \mathbf{x},$$

perché $d - out$ vale 0 se la classificazione è corretta e ± 2 se è sbagliata.

Attenzione — Differenza con LMS

La formula sembra quella dell'LMS, ma qui out contiene il **segno**: $out = \text{sign}(\mathbf{w}^T \mathbf{x})$, mentre nell'LMS l'errore era calcolato su $\mathbf{w}^T \mathbf{x}$ (senza soglia).

Vista geometrica

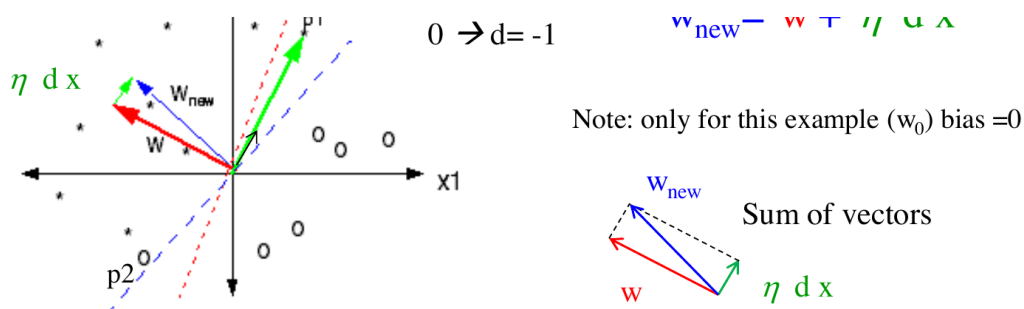


Figura 6.7: Aggiornamento del Perceptron: $\mathbf{w}$ viene spostato nella direzione del pattern p_1 mal classificato.

Prima dell'aggiornamento $\mathbf{w}$ (che punta verso la regione positiva) classifica male sia p_1 sia p_2 . Scegliendo p_1 , che ha target $d = +1$, $\mathbf{w}$ viene spostato di un po' nella direzione di p_1 (somma vettoriale $\mathbf{w} + \eta d \mathbf{x}$), e il nuovo confine è migliore. (Esercizio: se si sceglie p_2 , con $d = -1$, $\mathbf{w}$ viene allontanato da p_2 .)

6.5.2 Delta rule: due letture

- La forma $\mathbf{w}_{new} = \mathbf{w} + \eta d \mathbf{x}$ si può leggere come **apprendimento hebbiano**: $\mathbf{w}$ si muove verso $\mathbf{x}$.
- La forma $\mathbf{w}_{new} = \mathbf{w} + \eta (d - out) \mathbf{x} = \mathbf{w} + \eta \delta \mathbf{x}$ è una regola di **correzione dell'errore** (delta rule, Widrow-Hoff): il peso cambia in proporzione all'errore.

In termini di neuroni: *la modifica di un peso sinaptico è proporzionale al prodotto del segnale d'errore per il segnale di input che eccita la sinapsi*. Il calcolo è facile quando il segnale d'errore δ è **direttamente misurabile**, cioè quando conosciamo la risposta desiderata per l'unità. (Questo sarà il problema nelle reti con strati nascosti.)

6.5.3 Rappresentare e apprendere

Il Perceptron può **rappresentare** confini di decisione lineari, quindi risolve i problemi linearmente separabili. Ma riesce sempre a **imparare** la soluzione? Sì: il Perceptron con il suo algoritmo è sempre in grado di apprendere ciò che può rappresentare.

Nota — Un risultato storico

Il teorema di convergenza è una pietra miliare: un modello **ispirato alla biologia**, con capacità computazionali ben definite e **dimostre matematicamente**.

6.5.4 Il teorema di convergenza del Perceptron**Teorema — Teorema di convergenza del Perceptron**

Se il problema è **linearmente separabile**, l'algoritmo del Perceptron converge (classificando correttamente tutti i pattern) in un **numero finito di passi**, indipendentemente dal punto di partenza (anche se la soluzione finale non è unica e dipende dal punto di partenza). Se il problema non è separabile, l'algoritmo può essere instabile: sviluppa **cicli**, con pesi non necessariamente ottimali.

Nella dimostrazione omettiamo la T nei prodotti scalari tra vettori.

Preliminari 1: ridursi a pattern tutti positivi

Siano $(\mathbf{x}_i, d_i)$ gli esempi di training, $d_i \in \{+1, -1\}$, $i = 1, \dots, l$. Se il problema è linearmente separabile esiste una soluzione $\mathbf{w}^*$ tale che

$$d_i(\mathbf{w}^* \mathbf{x}_i) \geq \alpha, \quad \text{con } \alpha = \min_i d_i(\mathbf{w}^* \mathbf{x}_i) > 0.$$

Quindi $\mathbf{w}^*(d_i \mathbf{x}_i) \geq \alpha$. Definendo $\mathbf{x}'_i = d_i \mathbf{x}_i$, $\mathbf{w}^*$ è una soluzione del problema originale se e solo se è soluzione del problema $(\mathbf{x}'_i, +1)$ con tutti i target positivi:

- (se) $\mathbf{w}^*$ risolve il problema originale $\Rightarrow d_i(\mathbf{w}^* \mathbf{x}_i) \geq \alpha \Rightarrow \mathbf{w}^* \mathbf{x}'_i \geq \alpha \Rightarrow \mathbf{w}^*$ risolve $(\mathbf{x}'_i, +1)$;
- (solo se) il ragionamento inverso.

Basta quindi dimostrare il teorema per un problema con **solli pattern positivi**: “ribaltare” i pattern negativi non cambia nulla.

Preliminari 2: forma dei pesi dopo q errori

Assumiamo $\mathbf{w}(0) = \mathbf{0}$, $\eta = 1$ e definiamo $\beta = \max_i \|\mathbf{x}_i\|^2$. Con soli pattern positivi, la regola diventa $\mathbf{w}(j) = \mathbf{w}(j-1) + \mathbf{x}_{i_j}$ a ogni errore. Dopo q errori (tutti falsi negativi):

$$\mathbf{w}(q) = \sum_{j=1}^q \mathbf{x}_{i_j},$$

dove i_j indica i pattern classificati male (ad esempio gli indici 2, 8, 9, 14 del training set; lo stesso pattern può comparire più volte).

Idea della dimostrazione

Si trovano un **limite inferiore** a $\|\mathbf{w}(q)\|^2$ che cresce come q^2 e un **limite superiore** che cresce come q . Poiché una funzione quadratica supera prima o poi una lineare, i due limiti sono compatibili solo per q fino a un certo valore massimo: il numero di errori (e quindi di aggiornamenti) è finito.

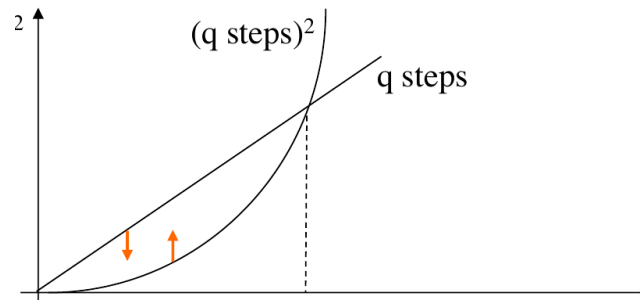
n 2^{na} ed. pages 139-141]

Figura 6.8: Idea della dimostrazione: il limite inferiore quadratico e il limite superiore lineare si incontrano in q_{max} .

Limite inferiore

$$\mathbf{w}^{*T} \mathbf{w}(q) = \mathbf{w}^{*T} \sum_{j=1}^q \mathbf{x}_{i_j} \geq q\alpha,$$

perché ogni termine $\mathbf{w}^{*T} \mathbf{x}_{i_j}$ è almeno α . Per la **disuguaglianza di Cauchy-Schwarz**, $\|\mathbf{a}\|^2 \|\mathbf{b}\|^2 \geq (\mathbf{a}^T \mathbf{b})^2$; con $\mathbf{a} = \mathbf{w}^*$ e $\mathbf{b} = \mathbf{w}(q)$:

$$\|\mathbf{w}^*\|^2 \|\mathbf{w}(q)\|^2 \geq (\mathbf{w}^{*T} \mathbf{w}(q))^2 \geq (q\alpha)^2 \Rightarrow \|\mathbf{w}(q)\|^2 \geq \frac{(q\alpha)^2}{\|\mathbf{w}^*\|^2}.$$

Limite superiore

Usando $\|\mathbf{a} + \mathbf{b}\|^2 = \|\mathbf{a}\|^2 + 2\mathbf{a}^T \mathbf{b} + \|\mathbf{b}\|^2$:

$$\|\mathbf{w}(q)\|^2 = \|\mathbf{w}(q-1) + \mathbf{x}_{i_q}\|^2 = \|\mathbf{w}(q-1)\|^2 + 2\mathbf{w}(q-1)^T \mathbf{x}_{i_q} + \|\mathbf{x}_{i_q}\|^2.$$

Poiché $\mathbf{x}_{i_q}$ è stato classificato male (è il q -esimo errore), $\mathbf{w}(q-1)^T \mathbf{x}_{i_q} \leq 0$: eliminando questo termine non positivo si ottiene

$$\|\mathbf{w}(q)\|^2 \leq \|\mathbf{w}(q-1)\|^2 + \|\mathbf{x}_{i_q}\|^2,$$

e iterando (con $\mathbf{w}(0) = \mathbf{0}$):

$$\|\mathbf{w}(q)\|^2 \leq \sum_{j=1}^q \|\mathbf{x}_{i_j}\|^2 \leq q\beta.$$

Conclusione

Mettendo insieme i due limiti:

$$q\beta \geq \|\mathbf{w}(q)\|^2 \geq \frac{(q\alpha)^2}{\|\mathbf{w}^*\|^2} \Rightarrow q \leq \frac{\beta \|\mathbf{w}^*\|^2}{\alpha^2}.$$

Il numero di errori (e quindi di aggiornamenti) è limitato da una costante: l'algoritmo converge in un **numero finito di passi**. ■

Suggerimento — Interpretazione del bound

Il numero di passi cresce con β (quanto sono “grandi” i pattern) e decresce con α^2 : α misura quanto il problema è “ben separato” (il margine minimo della soluzione $\mathbf{w}^*$). Problemi con classi vicine all’iperpiano separatore richiedono più passi. L’idea di **margine** tornerà centrale nelle SVM.

6.6 Perceptron contro LMS

Le due regole sembrano simili ($\mathbf{w}_{new} = \mathbf{w} + \eta \delta \mathbf{x}$), ma:

- l’LMS è derivato (tramite il gradiente) **senza** funzione a soglia, minimizzando l’errore dell’unità lineare: $\delta = d - \mathbf{w}^T \mathbf{x}$;
- il Perceptron usa l’uscita con soglia: $\delta = d - \text{sign}(\mathbf{w}^T \mathbf{x})$.

Naturalmente il modello addestrato con LMS si può usare per classificare applicando la soglia ($h(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x})$, LTU).

Una conseguenza importante: **l’LMS può classificare male anche problemi linearmente separabili**. L’LMS non minimizza il numero di errori della LTU, ma l’errore quadratico, e modifica i pesi **anche per i pattern già classificati correttamente**. Un pattern corretto ma “lontano” dall’iperpiano (con $\mathbf{w}^T \mathbf{x}$ molto maggiore di 1) produce comunque un errore quadratico grande che “tira” l’iperpiano.

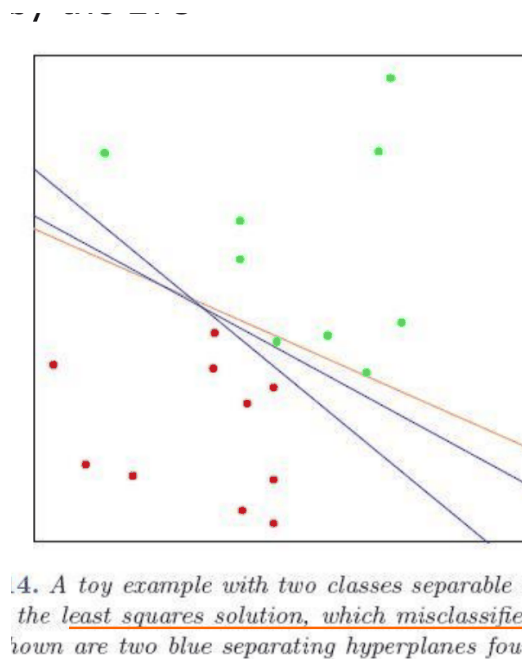


Figura 6.9: Un problema separabile: la soluzione LMS (arancione) sbaglia un punto, il Perceptron (blu, due diverse inizializzazioni) separa tutto (Hastie et al., Fig. 4.14).

Perceptron Learning Algorithm	LMS
Minimizza le classificazioni errate ($out = \text{sign}(\mathbf{w}^T \mathbf{x})$)	Minimizza $E(\mathbf{w})$ con $out = \mathbf{w}^T \mathbf{x}$

Perceptron Learning Algorithm	LMS
Converge sempre per problemi separabili, in un numero finito di passi, a un classificatore perfetto	Convergenza asintotica , anche per problemi non separabili
Non converge se il problema non è separabile	Non sempre zero errori di classificazione, anche su problemi separabili
Difficile da estendere a reti di unità	Estendibile a reti con l'approccio a gradiente

L'ultima riga è decisiva: per costruire reti neurali serve un approccio basato sul **gradiente**, che richiede funzioni differenziabili.

6.7 Funzioni di attivazione sigmoidali

Oltre alla lineare e alla soglia, si usano funzioni **non lineari** “a schiacciamento” (*squashing*) come la **sigmoide logistica**, che assume valori continui nell'intervallo limitato $[0, 1]$:

$$f_{\sigma}(x) = \frac{1}{1 + e^{-ax}}.$$

È una **versione liscia e differenziabile della funzione soglia**. Il parametro a è la **pendenza** (*slope*).

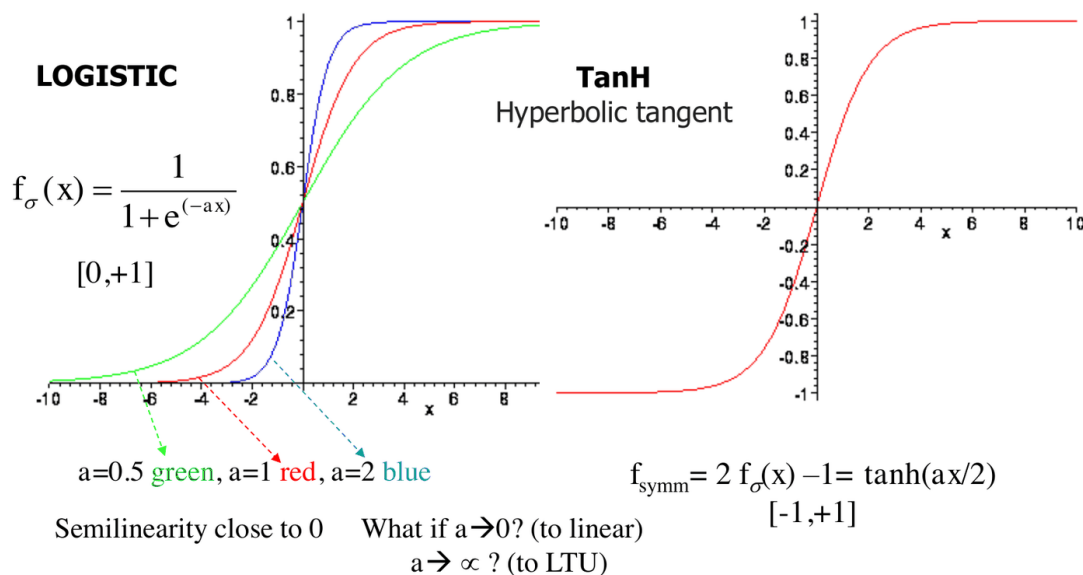


Figura 6.10: Sigmoide logistica (sinistra) per diversi valori di a e tangente iperbolica (destra).

La versione simmetrica è la **tangente iperbolica**, con valori in $[-1, +1]$:

$$f_{\text{symm}}(x) = 2f_{\sigma}(x) - 1 = \tanh(ax/2).$$

Vicino a zero la sigmoide è quasi lineare (**semi-linearità**). Al variare di a :

- per $a \rightarrow 0$ la funzione tende a una funzione **lineare** (molto piatta);
- per $a \rightarrow \infty$ tende alla funzione **a gradino** (LTU).

Per la classificazione, con la logistica un'uscita $\geq 0,5$ corrisponde alla classe positiva e $< 0,5$ alla classe zero/negativa (per la tanh la soglia è 0). La soglia si può spostare (studiando l'effetto su falsi positivi/negativi o con una curva ROC), e si può anche definire una **zona di rifiuto** attorno alla soglia per evitare decisioni fragili.

6.7.1 Altre funzioni di attivazione

- **Funzioni a base radiale (RBF)**: $f(x) = e^{-ax^2}$ applicata a $\|\mathbf{w} - \mathbf{x}_{input}\|$, cioè una gaussiana centrata sui pesi $\rightarrow$ **reti RBF**.
- **Softmax**: per il caso con uscite multiple (vedi lezioni successive).
- **Neuroni stocastici**: uscita +1 con probabilità $P(net)$, -1 altrimenti $\rightarrow$ **macchine di Boltzmann** e modelli della meccanica statistica.
- Approssimazioni **lineari a tratti** della tanh, per calcoli efficienti (anche con shift binari).
- **ReLU (Rectified Linear Unit)**: $f(x) = \max(0, x)$. È diventata la scelta predefinita per i modelli profondi; se ne parlerà nel deep learning.
- **Softplus**, la sua approssimazione liscia: $f(x) = \ln(1 + e^x)$.

Molte funzioni di attivazione hanno prestazioni simili in pratica.

6.7.2 Derivate

- La derivata dell'identità è 1.
- La derivata della funzione a gradino **non è definita** (in zero) ed è nulla altrove: è esattamente il motivo per cui non si usa con LMS.
- Per le sigmoidi (con $a = 1$):

$$\frac{df_{\sigma}(x)}{dx} = f_{\sigma}(x)(1 - f_{\sigma}(x)), \quad \frac{d \tanh(x)}{dx} = 1 - \tanh^2(x).$$

(Con a generico: $\frac{df_{\sigma}}{dx} = a f_{\sigma}(x)(1 - f_{\sigma}(x))$.)

Nota — Derivazione della derivata della logistica

$f_{\sigma}(x) = (1 + e^{-x})^{-1}$, quindi $f'_{\sigma}(x) = \frac{e^{-x}}{(1+e^{-x})^2} = \frac{1}{1+e^{-x}} \cdot \frac{e^{-x}}{1+e^{-x}} = f_{\sigma}(x)(1 - f_{\sigma}(x))$, perché $\frac{e^{-x}}{1+e^{-x}} = 1 - \frac{1}{1+e^{-x}}$. È comodo: la derivata si calcola dal valore della funzione stessa.

6.7.3 LMS con un'unità sigmoidale

Poiché la logistica è una soglia liscia e differenziabile, possiamo derivare un algoritmo LMS calcolando il gradiente dell'errore quadratico come per le unità lineari: si passa da $o(\mathbf{x}) = \mathbf{x}^T \mathbf{w}$ a $o(\mathbf{x}) = f_{\sigma}(\mathbf{x}^T \mathbf{w})$, e si minimizza

$$E(\mathbf{w}) = \sum_{p=1}^l (d_p - f_{\sigma}(\mathbf{x}_p^T \mathbf{w}))^2.$$

Il gradiente si ottiene con la **regola della catena**, $\frac{\partial f}{\partial x} = \frac{\partial f}{\partial g} \frac{\partial g}{\partial x}$, usando come variabili ausiliarie $g = net$ e poi $g = out$. Per un pattern p con $net_p = \mathbf{x}_p^T \mathbf{w}$:

$$\frac{\partial E_p}{\partial w_j} = \frac{\partial E_p}{\partial o_p} \cdot \frac{\partial o_p}{\partial net_p} \cdot \frac{\partial net_p}{\partial w_j} = -2(d_p - o_p) \cdot f'_{\sigma}(net_p) \cdot x_{p,j}.$$

Sommando su tutti i pattern:

$$\frac{\partial E(\mathbf{w})}{\partial w_j} = -2 \sum_{p=1}^l (d_p - o(\mathbf{x}_p)) f'_\sigma(\text{net}(\mathbf{x}_p)) x_{p,j} = -2 \sum_{p=1}^l \delta_p x_{p,j}.$$

Questa derivata risponde alla domanda: **quanto una variazione di w_j influenza la variazione di E ?**

La discesa del gradiente è identica a quella per l'unità lineare, con un nuovo delta:

$$\mathbf{w}_{\text{new}} = \mathbf{w} + \eta \delta_p \mathbf{x}_p, \quad \delta_p = (d_p - o(\mathbf{x}_p)) f'_\sigma(\text{net}(\mathbf{x}_p)).$$

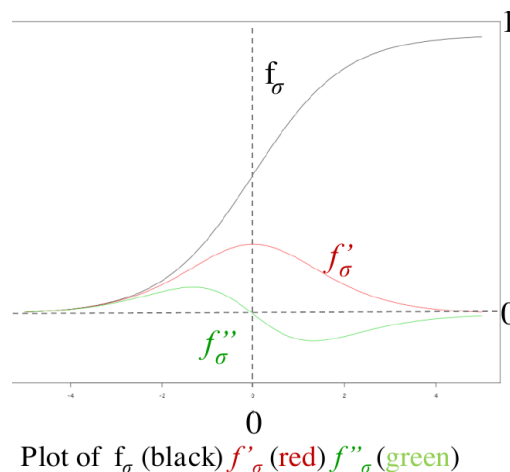


Figura 6.11: La logistica f_σ (nero), la sua derivata f'_σ (rosso) e la derivata seconda f''_σ (verde).

È ancora una regola di **correzione dell'errore**, con alcune osservazioni:

- la pendenza a influenza l'ampiezza del passo di discesa;
- f'_σ è **massima per net vicino a 0** (zona quasi lineare): lì i delta possono essere grandi;
- f'_σ è **minima nei casi saturi** (f vicino a 0 o 1): delta piccoli, pesi che cambiano lentissimamente. Bisogna **evitare una saturazione prematura**, ad esempio **partendo con pesi piccoli**.

Suggerimento — Un ponte tra LMS e Perceptron

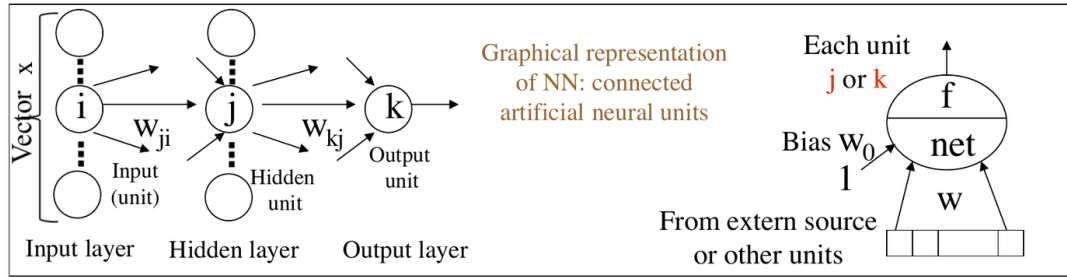
Quando un pattern è classificato correttamente “con decisione” (uscita saturata vicino al target), $f'_\sigma \approx 0$ e la correzione è quasi nulla, proprio come nel Perceptron, che non corregge i pattern corretti. Usando f_σ si approssima quindi non solo la LTU, ma anche l'algoritmo del Perceptron.

6.8 Le reti neurali: il Multi-Layer Perceptron

Tutti gli ingredienti sono pronti. Un **MLP** si può vedere in due modi:

- **A.** come una **rete di unità interconnesse**;
- **B.** come una **funzione flessibile $h(\mathbf{x})$** , composta da funzioni non lineari annidate.

A. A network of units



B. A flexible function

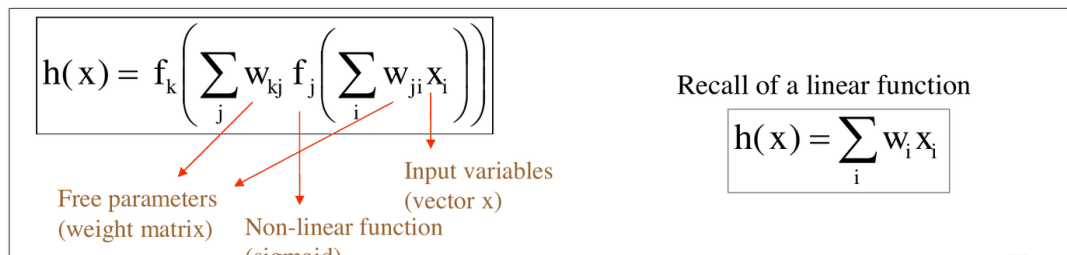


Figura 6.12: Le due viste di una rete feedforward con uno strato nascosto: rete di unità (A) e funzione flessibile (B).

6.8.1 Vista A: una rete di unità

In un'architettura MLP:

- le unità sono connesse da collegamenti pesati e organizzate in **strati** (*layer*);
- lo **strato di input** è solo la sorgente dell'input x : carica (copia) il pattern, senza calcolare *net* né f ;
- lo **strato nascosto** (*hidden*) proietta sullo strato di uscita (o su un altro strato nascosto);
- lo **strato di uscita** produce $h(x)$.

6.8.2 Vista B: una funzione flessibile

Una rete a due strati (uno nascosto e uno di uscita) calcola

$$h(x) = f_k \left(\sum_j w_{kj} f_j \left(\sum_i w_{ji} x_i \right) \right),$$

dove x_i sono le variabili di input, w i parametri liberi (matrici dei pesi) e f funzioni non lineari (sigmoidi). Si confronti con la funzione lineare $h(x) = \sum_i w_i x_i$: la rete è una composizione di funzioni lineari e non lineari.

6.8.3 Componenti di una rete neurale

Una rete si descrive tradizionalmente con:

- il tipo di **unità** (net e funzione di attivazione);
- l'**architettura** (numero di unità, topologia, numero di strati);
- l'**algoritmo di apprendimento**.

6.8.4 Notazione uniforme per le unità

Per una generica unità t (che può essere nascosta, j , o di uscita, k):

$$net_t(\mathbf{x}) = \sum_u w_{tu} o_u, \quad o_t(\mathbf{x}) = f_t(net_t(\mathbf{x})),$$

dove u indica una generica sorgente di input (che può essere i o j). Caricando il pattern nello strato di input, si può usare il simbolo o sia per gli input sia per le uscite delle unità nascoste: l'input all'unità t dalla sorgente u (attraverso la connessione w_{tu}) è o_u . Questa notazione sarà usata nella derivazione della backpropagation.

Esempio — Esercizio (dalla lezione)

1. Per i pesi w_{ji} (input $\rightarrow$ nascosta), l'input o_u è $o_i = x_i$, la componente del pattern.
2. Per i pesi w_{kj} (nascosta $\rightarrow$ uscita), l'input o_u è o_j , l'uscita dell'unità nascosta j .
3. w_{t0} è il **bias** di ciascuna unità t , con input costante $o_0 = 1$.
4. Unità nascosta: $o_j = f_j(\sum_i w_{ji} x_i)$; unità di uscita: $o_k = f_k(\sum_j w_{kj} o_j)$.

6.8.5 Architettura e processing feedforward

L'**architettura** definisce la topologia delle connessioni. La rete a due strati descritta sopra è l'MLP standard, **completamente connesso**; si possono avere più strati nascosti e anche connessioni che “saltano” strati.

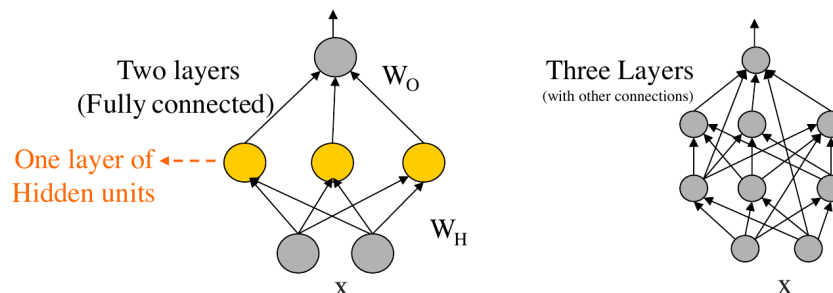


Figura 6.13: Architetture MLP: due strati completamente connessi (sinistra), tre strati con altre connessioni (destra).

L'elaborazione **feedforward** di un pattern procede dall'input all'uscita:

1. si carica il pattern $\mathbf{x}$ nello strato di input;
2. si calcolano le uscite di tutte le unità del primo strato nascosto;
3. poi del secondo strato nascosto, e così via;
4. si calcolano le uscite dello strato di uscita, cioè $h(\mathbf{x})$;
5. si può ora calcolare l'errore (delta) in uscita.

Nota — Reti ricorrenti

Le reti **feedforward** hanno un'unica direzione input $\rightarrow$ output. Le **reti ricorrenti** aggiungono connessioni di feedback (cicli): i self-loop danno alla rete proprietà dinamiche e una **memoria** delle computazioni passate, estendendo il modello all'elaborazione di **sequenze** (e dati strutturati). Saranno trattate in una lezione dedicata.

6.9 Perché le reti neurali sono flessibili

Tre domande guidano l'analisi: perché le reti sono un modello flessibile (e si possono vedere come LBE)? La flessibilità è teoricamente fondata? Come si apprendono i pesi?

6.9.1 Quali task

Lo spazio delle ipotesi è lo **spazio continuo di tutte le funzioni** rappresentabili assegnando valori ai pesi di un'architettura data. A seconda delle unità di uscita, la rete tratta:

- **classificazione**, con uscita sigmoideale;
- **regressione**, con uscita lineare.

Con **più unità di uscita** si ottengono regressione multipla o classificatori multi-classe (ad esempio tre uscite che danno 0,2; 0,7; 0,1 per tre classi).

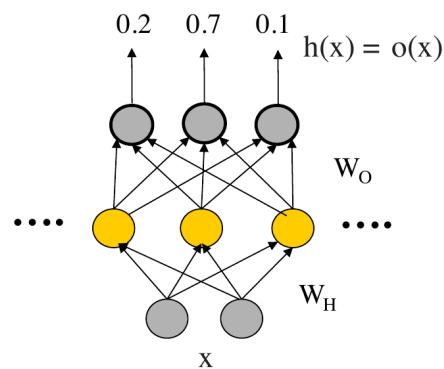


Figura 6.14: Una rete con uscite multiple per un problema multi-classe.

6.9.2 La rete come espansione in basi adattive

La rete calcola $h(\mathbf{x}) = f_k \left(\sum_j w_{kj} f_j \left(\sum_i w_{ji} x_i \right) \right)$. Unità e architettura sono solo una rappresentazione grafica del flusso dei dati. Ogni termine

$$\phi_j(\mathbf{x}, \mathbf{w}) = f_j \left(\sum_i w_{ji} x_i \right)$$

può essere visto come calcolato da un'unità nascosta indipendente, oppure come una **funzione di base ϕ di una LBE**. Confrontiamo:

- **LBE:** $h(\mathbf{x}) = \sum_j w_j \phi_j(\mathbf{x})$ (regressione) o $h(\mathbf{x}) = f \left(\sum_j w_j \phi_j(\mathbf{x}) \right)$ (classificazione), con le ϕ fissate a priori e il modello lineare nei parametri.
- **Rete neurale:** $h(\mathbf{x}) = f_k \left(\sum_j w_{kj} \phi_j(\mathbf{x}, \mathbf{w}) \right)$, dove ora le ϕ dipendono dai pesi e quindi si adattano ai dati durante l'addestramento.

Suggerimento — Le reti neurali come basi adattive

Le reti neurali sono un approccio a **funzioni di base adattive**: le basi stesse vengono adattate ai dati (apprendendo i $\mathbf{w}$ dentro le ϕ). Tutte le basi hanno lo stesso tipo (dato dalla funzione di attivazione). $h(\mathbf{x})$ è una funzione non lineare di somme pesate di modelli lineari trasformati in modo non lineare, con il miglioramento cruciale dell'**adattività**. (Da

confrontare più avanti con le SVM, un'altra forma di LBE.)

Ogni unità nascosta calcola una nuova **feature derivata non lineare**, in modo adattivo. In altre parole: la capacità rappresentativa del modello è legata alla presenza di uno **strato nascosto con attivazioni non lineari**, che trasforma l'input nella **rappresentazione interna** della rete. L'apprendimento definisce una rappresentazione interna adatta, cioè nuove feature nascoste dei dati, che permettono al modello di estrarre le **statistiche di ordine superiore** rilevanti per approssimare la funzione target.

Attenzione — Le unità nascoste devono essere non lineari

Un MLP con unità **lineari** equivale a una rete a **un solo strato**: la composizione di funzioni lineari è lineare. Con f identità, $h(\mathbf{x}) = \sum_j w_{kj} \sum_i w_{ji} x_i = \sum_i \left(\sum_j w_{kj} w_{ji} \right) x_i = \sum_i w'_i x_i$.

Il prezzo della non linearità: il modello è **non lineare nei parametri w** , e l'addestramento diventa un problema di **ottimizzazione non lineare** (non convesso).

6.9.3 L'approssimazione universale

Teorema — Teorema di approssimazione universale (Cybenko 1989, Hornik et al.)

Una rete con **un solo strato nascosto** (con attivazioni logistiche) e uscita lineare può approssimare arbitrariamente bene **ogni funzione continua** (su ipercubi), purché ci siano abbastanza unità nascoste: dati f ed $\varepsilon > 0$, esiste h tale che $|f(\mathbf{x}) - h(\mathbf{x})| < \varepsilon$ per ogni $\mathbf{x}$ nell'ipercubo. Più in generale, un MLP può approssimare arbitrariamente bene ogni mappatura input-output, con abbastanza unità nascoste.

(Si può pensare a una generalizzazione dell'approssimazione con serie di Fourier finite.)

Attenzione — È un teorema di esistenza

Il teorema dice che la rete **esiste**, ma non fornisce né l'**algoritmo** per trovarla né il **numero di unità** necessario.

Restano quindi due questioni fondamentali: **come apprendere** (la backpropagation, prossima lezione) e **come scegliere l'architettura**.

6.9.4 Potere espressivo e VC-dimension

Il potere espressivo di una rete dipende dal **numero di unità** e dalla loro **configurazione**. Il numero di unità è legato alla VC-dimension: le capacità della rete dipendono dal numero di parametri (in un MLP completamente connesso, circa il numero di input per il numero di unità nascoste, più le unità nascoste per le uscite), ma anche dal **valore** dei pesi:

- pesi = 0 $\rightarrow$ VC-dim minima;
- pesi piccoli $\rightarrow$ si lavora nella parte lineare della sigmoide $\rightarrow$ VC-dim piccola;
- pesi grandi $\rightarrow$ modello più complesso.

Poiché l'addestramento parte da pesi vicini a 0 e i pesi crescono durante il training, la rete è uno **spazio delle ipotesi di dimensione variabile**: la VC-dim effettiva cresce durante l'apprendimento. (Questo sarà la base dell'*early stopping*.)

6.9.5 Quanti strati? (anticipazione)

Il teorema di approssimazione universale dice che uno strato nascosto basta, ma non garantisce che basti un numero **piccolo** di unità. Esistono risultati di tipo “*no flattening*” (sull’efficienza, non sull’espressività): funzioni che con un solo strato nascosto richiederebbero un numero **esponenziale** di unità (rispetto alla dimensione dell’input), mentre con più strati ne bastano molte meno. Ma è facile addestrare un MLP con molti strati? Lo vedremo nel deep learning.

6.9.6 Il bias induttivo delle reti neurali

Per le reti addestrate con backpropagation, il bias induttivo è legato alle proprietà di **smoothness** delle funzioni: piccole variazioni dell’input producono piccole variazioni dell’output (ad esempio, derivata prima localmente limitata). È un’assunzione molto comune nel ML, ed è ragionevole: se la funzione target non è liscia (ad esempio un generatore di numeri casuali), la generalizzazione è comunque impossibile.

6.10 Verso l’algoritmo di apprendimento

L’algoritmo di apprendimento adatta i pesi della rete per ottenere la migliore approssimazione della funzione target, minimizzando come sempre una funzione d’errore (o loss) sui dati di training.

6.10.1 Il problema del credit assignment

- Il Perceptron ha un algoritmo di apprendimento, ma non rappresenta tutte le funzioni booleane.
- Una rete di perceptron rappresenta ogni funzione booleana, ma serve un algoritmo per addestrarla.

Cosa cambia? Il **problema del credit assignment**: quanta “responsabilità” dell’errore attribuire alle unità nascoste? Per le unità di uscita conosciamo la risposta desiderata e quindi l’errore; per le unità nascoste **non sappiamo quale dovrebbe essere la loro uscita**, quindi il segnale d’errore non è direttamente misurabile. In più, il modello è non lineare nei pesi.

Minsky e Papert (1969) ritenevano il problema troppo difficile; fu risolto da diversi ricercatori con l’algoritmo di **backpropagation**, reso popolare da Rumelhart, Hinton e Williams nel libro *PDP* (1986): fu il **rinascimento** delle reti neurali.

6.10.2 Il loading problem

Definizione — Loading problem

Data una rete e un insieme di esempi, esiste un insieme di pesi per cui la rete è consistente con gli esempi? (“Caricare” i dati di training nei parametri liberi della rete.)

Il loading problem è **NP-completo** (Judd, 1990): non è noto un algoritmo polinomiale. In pratica, però, le reti si addestrano in tempi ragionevoli con la backpropagation, anche se la soluzione ottima non è garantita.

6.10.3 L'idea chiave: estendere la discesa del gradiente

La discesa del gradiente che minimizza una loss **si estende agli MLP**, purché loss e funzioni di attivazione siano **differenziabili**: si tratta di trovare il delta per **ogni** unità della rete. Il problema resta lo stesso: dati l esempi $(\mathbf{x}_p, d_p)$ e una loss L (ad esempio $L(h(\mathbf{x}_p), d_p) = (d_p - h_{\mathbf{w}}(\mathbf{x}_p))^2$), trovare $\mathbf{w}$ che minimizzi

$$E(\mathbf{w}) = R_{emp} = \frac{1}{l} \sum_{p=1}^l (d_p - h(\mathbf{x}_p))^2$$

calcolandone il gradiente. Servono: loss differenziabile, attivazioni differenziabili e una rete per seguire il flusso dell'informazione.

Proprietà della backpropagation:

- è semplice grazie alla **forma compositiva** del modello;
- usa solo quantità **locali** a ogni unità, preservandone la modularità;
- è **efficiente**: $O(\#W)$ invece di $O(\#W^2)$, grazie alla fattorizzazione dei delta;
- sulla plausibilità biologica il dibattito è aperto (si è ipotizzato che il cervello usi un'approssimazione locale subottima della BP).

La derivazione completa è nella prossima lezione (e nelle note [07 - Note sulla backpropagation](#)).

6.10.4 Questioni pratiche nell'addestramento

Il modello è spesso **sovra-parametrizzato** e l'ottimizzazione è **non convessa** e potenzialmente instabile. Le questioni da affrontare (dopo la backpropagation) sono: valori iniziali dei pesi, learning rate, on-line/batch; overfitting e regolarizzazione; scalatura dell'input e rappresentazione dell'output; numero di unità nascoste; minimi multipli; criteri di arresto.

In sintesi — Sintesi

Il neurone artificiale calcola $o = f(\sum_j w_j x_j)$. Il **Perceptron** (soglia) ha un algoritmo di apprendimento che **converge in un numero finito di passi** per problemi linearmente separabili, ma non rappresenta lo XOR. Reti di perceptron a due strati rappresentano ogni funzione booleana grazie alla **ri-rappresentazione interna** dello strato nascosto. Sostituendo la soglia con una **sigmoide** differenziabile si può usare la discesa del gradiente. L'MLP è una funzione annidata $h(\mathbf{x}) = f_k(\sum_j w_{kj} f_j(\sum_i w_{ji} x_i))$, interpretabile come **espansione in basi adattiva**, ed è un **approssimatore universale**. Per addestrarlo bisogna risolvere il **credit assignment** delle unità nascoste: lo fa la backpropagation.

Domande — Possibili domande d'esame

- Descrivere il neurone artificiale e le principali funzioni di attivazione.
- Come si rappresentano AND, OR, NOT con un perceptron? Come si risolve lo XOR?
- Scrivere l'algoritmo di apprendimento del Perceptron.
- Enunciare e dimostrare il teorema di convergenza del Perceptron.
- Differenze tra algoritmo del Perceptron e LMS; perché l'LMS può sbagliare su problemi separabili?
- Derivare la regola di aggiornamento per una singola unità sigmoideale.
- Perché le unità nascoste devono essere non lineari?

- In che senso una rete neurale è una LBE adattiva?
- Enunciare il teorema di approssimazione universale e discuterne i limiti.
- Cos'è il problema del credit assignment? E il loading problem?

Capitolo 7

Note sulla backpropagation

Queste note contengono la **derivazione completa dell'algoritmo di backpropagation**, l'algoritmo che risolve il problema del *credit assignment* nelle reti multistrato introdotto in [06 - Reti neurali \(parte 1\) - dal neurone al MLP](#). Il professore insiste su un punto: **la derivazione va rifatta da sé, non memorizzata**. Solo così si è in grado di ricalcolarla per loss, attivazioni o architetture diverse — ed è questo il vero obiettivo. Implementarla nel progetto è il modo migliore per capirla a fondo.

Riferimenti: Rumelhart, Hinton, Williams, *Parallel Distributed Processing* (1986), cap. 8; Mitchell, *Machine Learning*, cap. 4; Haykin, *Neural Networks*.

7.1 Strumenti di calcolo differenziale

La discesa del gradiente richiede derivate di funzioni composte. Servono tre regole:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial x} \quad (\text{regola della catena}), \quad D[f(g(x))] = f'(g(x)) \cdot g'(x), \quad D[f \cdot g] = f' \cdot g + f \cdot g'.$$

La regola della catena è utile perché **scompone** il calcolo, mettendo in evidenza ciò che si può calcolare immediatamente senza perdere il filo generale.

7.2 Il problema

L'architettura di base è una rete feedforward completamente connessa (MLP). Si usano indici diversi per gli strati: i per l'input, j per lo strato nascosto, k per l'uscita.

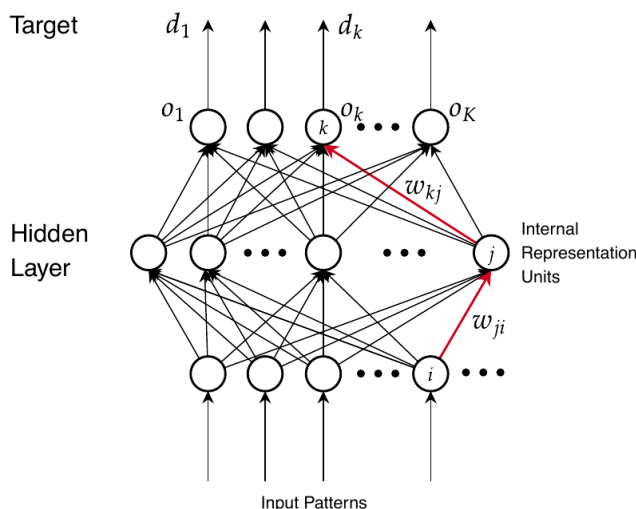


Figura 7.1: La rete di riferimento: input i , unità nascoste j , unità di uscita k .

Definizione — Il problema della backpropagation

Stimare il **contributo delle unità nascoste all'errore** in uscita. Lo si fa calcolando la **regola delta generalizzata** (*Generalized Delta Rule*).

Il training set è $TR = \{(\mathbf{x}^1, \mathbf{d}^1), \dots, (\mathbf{x}^l, \mathbf{d}^l)\}$ (apprendimento supervisionato). Ogni unità di input i carica la componente x_i del pattern: $o_i = x_i$.

Errore totale della rete sul training set è

$$E_{tot} = \sum_p E_p, \quad E_p = \frac{1}{2} \sum_{k=1}^K (d_k - o_k)^2,$$

dove E_p è l'errore sul pattern p calcolato su tutte le K unità di uscita. (Il fattore $\frac{1}{2}$ serve solo a semplificare la derivata.) Per alleggerire la notazione, d'ora in poi l'indice p viene omissso: o_k sta per $o_k(\mathbf{x}_p)$ e d_k per $d_{p,k}$.

Obiettivo: trovare i pesi $\mathbf{w}$ che minimizzano E_{tot} (per ora solo il termine sui dati). È ancora una minimizzazione ai minimi quadrati (LMS). **Idea:** scendere lungo la superficie di E_{tot} seguendo il gradiente.

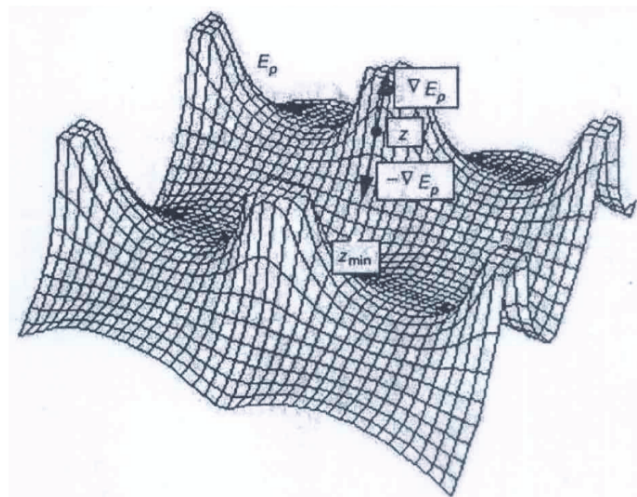


Figura 7.2: Un paesaggio di E_{tot} nello spazio di due pesi: seguendo $-\nabla E$ dal punto Z si raggiunge Z_{min} . A differenza del modello lineare, la superficie non è convessa.

7.3 L'algoritmo iterativo

Si calcola la loss, se ne calcola il gradiente, si aggiornano tutti i pesi; si ripete fino a convergenza o a un altro criterio di arresto.

In sintesi — Algoritmo 1 — Backpropagation

Inizializzare tutti i pesi $\mathbf{w}$ della rete e η . Calcolare le uscite delle unità ed E_{tot} . **Finché** $E_{tot} > \epsilon$ (o altro criterio): - per ogni $w \in \mathbf{w}$: $\Delta w = -\frac{\partial E_{tot}}{\partial w}$ (**passo 1**); - $w_{new} = w + \eta \Delta w + \dots$ (**passo 2**); - ricalcolare le uscite ed E_{tot} .

Esistono le versioni stocastica (on-line) e batch. Bisogna distinguere due aspetti:

1. il **calcolo del gradiente** $\partial E_{tot} / \partial w$, per ogni peso della rete: è ciò che fa la backpropagation;

2. la **regola di aggiornamento** dei pesi: backprop standard, Quickprop, R-prop, ...

Ci concentriamo sul passo 1:

$$\Delta w = -\frac{\partial E_{tot}}{\partial w} = -\sum_p \frac{\partial E_p}{\partial w} \stackrel{def}{=} \sum_p \Delta_p w.$$

7.4 La derivazione

7.4.1 Il gradiente per un peso generico

Consideriamo un peso generico w_{tu} , che collega l'input proveniente da un'unità generica u (che può essere i o j) all'unità generica t . Poiché

$$net_t = \sum_s w_{ts} o_s, \quad o_t = f_t(net_t),$$

applicando la regola della catena attraverso net_t :

$$\Delta_p w_{tu} = -\frac{\partial E_p}{\partial w_{tu}} = -\underbrace{\frac{\partial E_p}{\partial net_t}}_{\delta_t} \cdot \underbrace{\frac{\partial net_t}{\partial w_{tu}}}_{o_u} = \delta_t \cdot o_u.$$

Infatti $\frac{\partial net_t}{\partial w_{tu}} = \frac{\partial \sum_s w_{ts} o_s}{\partial w_{tu}} = o_u$: tutti i termini della somma sono costanti rispetto a w_{tu} tranne quello con $s = u$. (Gli o_s sono gli input dell'unità t .)

Abbiamo definito il **delta dell'unità t** :

$$\delta_t = -\frac{\partial E_p}{\partial net_t}.$$

Espandiamolo ancora con la regola della catena, passando per $o_t = f_t(net_t)$:

$$\delta_t = -\frac{\partial E_p}{\partial net_t} = -\frac{\partial E_p}{\partial o_t} \cdot \frac{\partial o_t}{\partial net_t} = -\frac{\partial E_p}{\partial o_t} \cdot f'_t(net_t).$$

Resta da calcolare $-\frac{\partial E_p}{\partial o_t}$, e qui si distinguono due casi: t è un'**unità di uscita** k , oppure un'**unità nascosta** j .

7.4.2 Caso 1: unità di uscita ($t = k$)

L'uscita o_k compare direttamente in E_p :

$$-\frac{\partial E_p}{\partial o_k} = -\frac{\partial \frac{1}{2} \sum_{r=1}^K (d_r - o_r)^2}{\partial o_k} = (d_k - o_k),$$

perché solo il termine $r = k$ dipende da o_k , e la sua derivata è $\frac{1}{2} \cdot 2(d_k - o_k) \cdot (-1)$. Quindi:

$$\delta_k = (d_k - o_k) \cdot f'_k(net_k)$$

È esattamente il delta già visto per la singola unità sigmoidale: l'errore è direttamente misurabile perché conosciamo il target.

7.4.3 Caso 2: unità nascosta ($t = j$)

Per un'unità nascosta non c'è un target: o_j non compare direttamente in E_p . Però o_j influenza E_p **attraverso tutte le unità di uscita** a cui è collegata: ogni net_k dipende da o_j . Applicando la regola della catena con una somma su tutte le k :

$$-\frac{\partial E_p}{\partial o_j} = \sum_{k=1}^K \underbrace{-\frac{\partial E_p}{\partial net_k}}_{\delta_k} \cdot \frac{\partial net_k}{\partial o_j} = \sum_{k=1}^K \delta_k w_{kj},$$

poiché $\frac{\partial net_k}{\partial o_j} = \frac{\partial \sum_s w_{ks} o_s}{\partial o_j} = w_{kj}$. Quindi:

$$\delta_j = \left(\sum_{k=1}^K \delta_k w_{kj} \right) \cdot f'_j(net_j)$$

Suggerimento — Il punto cruciale

Il termine $-\frac{\partial E_p}{\partial net_k}$ **lo abbiamo già calcolato**: è δ_k ! Il delta di un'unità nascosta si ottiene quindi **propagando all'indietro** i delta delle unità di uscita, pesati con gli stessi pesi w_{kj} usati nella propagazione in avanti. Da qui il nome **back-propagation**: l'errore "risale" la rete. Questo risolve il credit assignment: la "colpa" di un'unità nascosta è la somma delle colpe delle unità a cui contribuisce, pesata da quanto contribuisce.

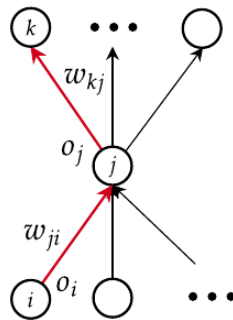


Figura 7.3: Usare il grafo della rete per localizzare il calcolo: ogni termine coinvolge solo unità e pesi adiacenti.

Alcune osservazioni sul caso 2:

- esprime la variazione di E_p considerando **tutte** le unità di uscita o_k ;
- ogni o_k (e net_k) dipende da o_j : per questo compare la somma su k ;
- lo stesso risultato si può ottenere direttamente dalla definizione di E_p (per esercizio), ma questa forma è più istruttiva perché mostra il flusso delle derivate parziali **sulla rete**.

7.5 Sintesi delle formule

Per due strati abbiamo derivato:

$$\Delta_p w_{tu} = \delta_t \cdot o_u,$$

dove δ_t è il **segnale d'errore** disponibile all'unità t e o_u è l'input a t proveniente da u (attraverso la connessione w_{tu}). In particolare:

Peso	Aggiornamento	Delta
w_{kj} (nascosta $\rightarrow$ uscita)	$\Delta_p w_{kj} = \delta_k \cdot o_j$	$\delta_k = (d_k - o_k) f'_k(net_k)$
w_{ji} (input $\rightarrow$ nascosta)	$\Delta_p w_{ji} = \delta_j \cdot o_i$	$\delta_j = (\sum_k \delta_k w_{kj}) f'_j(net_j)$

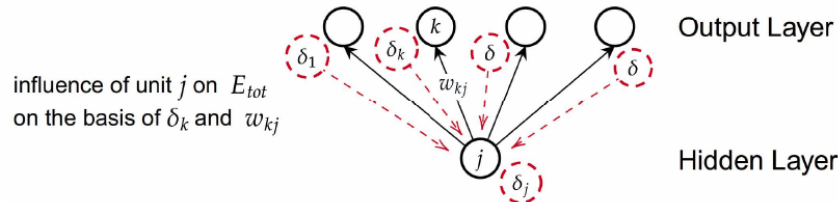


Figura 7.4: Retropropagazione dei delta: l'influenza dell'unità nascosta j su E_{tot} si calcola dai δ_k dello strato superiore e dai pesi w_{kj} .

7.5.1 Generalizzazione a m strati nascosti

La derivazione vale per **qualsiasi numero di strati nascosti**: i delta non arrivano solo dallo strato di uscita, ma da qualsiasi strato superiore h a cui l'unità è collegata. In generale, **i delta arrivano da tutte le unità a cui l'unità corrente è connessa**:

$$\delta_j = \left(\sum_{h \in \text{successori}(j)} \delta_h w_{hj} \right) f'_j(net_j).$$

La regola di aggiornamento per ogni pattern (omettendo p) è

$$w_{tu}^{new} = w_{tu} + \eta \delta_t o_u.$$

Usando $\Delta_p w$ per ogni pattern si ottiene la versione **on-line**; per la versione **batch** si sommano i contributi su tutti i pattern prima di aggiornare.

7.6 Il ciclo di addestramento completo

1. **Calcolo in avanti (forward)**: uscite di tutte le unità, strato per strato.
2. **Calcolo degli errori e dei delta nello strato di uscita.**
3. **Propagazione all'indietro** dei delta dallo strato di uscita agli strati nascosti.
4. **Aggiornamento dei pesi.**
5. Si ricomincia, fino al criterio di arresto.

Attenzione — Non dimenticare i bias

Calcolo e aggiornamento vanno applicati anche ai **bias** di tutte le unità: si assume la prima componente dell'input di ogni unità pari a 1, con il bias come peso w_{t0} . Quindi $\Delta w_{t0} = \delta_t \cdot 1$.

7.7 Proprietà e interpretazione

- **Calcoli solo locali:** ogni unità usa direttamente solo informazioni vicine (le unità sotto e sopra), non strati lontani; il resto arriva indirettamente tramite il flusso dei delta. In altre parole: nonostante il calcolo sia locale, la modifica di un peso ha un **effetto globale** sulla rete. Qual è l'influenza di ciascun w_{tu} sull'errore globale? Questo effetto complesso è stato **scomposto** in componenti semplici tramite i delta locali a ogni unità.
- **Visione come processo di diffusione:** la backpropagation è una propagazione locale spazio-temporale dai “genitori” dell'unità all'unità e dall'unità ai suoi “figli”.
- **Efficienza — la “fattorizzazione magica”:** il delta δ_t di ogni unità si calcola **una sola volta** e si riusa per **tutti** i suoi pesi w_{tu} . Il costo è quindi proporzionale al **numero di pesi**, non al suo quadrato. Con modelli da miliardi di pesi la differenza è enorme: 10^9 contro $(10^9)^2 = 10^{18}$ operazioni, che sarebbe infattibile.

Suggerimento — Tre livelli di lettura della derivazione

1. I singoli **passaggi matematici**.
2. L'**interpretazione** delle variazioni locali di una quantità rispetto alle altre (il significato delle derivate parziali).
3. Il **quadro generale**: la scomposizione serve a trovare i valori delta di ogni livello, scomponendo l'errore **sulla rete** che si ha davanti.

Attenzione — Terminologia corretta

Non si dice “rete backprop”: la rete neurale è il **modello**. La backpropagation è un **algoritmo di addestramento** basato sulla retropropagazione degli errori, cioè sul calcolo diretto del gradiente attraverso la rete.

Esempio — Esempio numerico (per verificare la comprensione)

Rete con un input $x = 1$, una unità nascosta sigmoideale e una di uscita lineare, senza bias; pesi $w_{ji} = 0,5$, $w_{kj} = 1$, target $d = 1$, $\eta = 0,1$. - Forward: $net_j = 0,5$, $o_j = \sigma(0,5) \approx 0,622$; $o_k = 1 \cdot 0,622 = 0,622$. - Delta di uscita (lineare, $f' = 1$): $\delta_k = (1 - 0,622) \cdot 1 = 0,378$. - Delta nascosto: $\delta_j = (\delta_k w_{kj}) \sigma'(net_j) = 0,378 \cdot 0,622 \cdot (1 - 0,622) \approx 0,089$. - Aggiornamenti: $\Delta w_{kj} = \eta \delta_k o_j \approx 0,1 \cdot 0,378 \cdot 0,622 \approx 0,0235$; $\Delta w_{ji} = \eta \delta_j x \approx 0,0089$. Entrambi i pesi aumentano, facendo crescere l'uscita verso il target.

Domande — Possibili domande d'esame

- Derivare la backpropagation per un MLP a due strati con errore quadratico.
- Cos'è δ_t ? Come si calcola per un'unità di uscita e per un'unità nascosta?
- Perché la backpropagation ha costo lineare nel numero dei pesi?
- Come cambia la derivazione con una loss diversa o con più strati nascosti?

Capitolo 8

Reti neurali (parte 2): addestramento in pratica

Con la backpropagation (07 - [Note sulla backpropagation](#)) sappiamo calcolare il gradiente per ogni peso della rete. Questa lezione affronta tutte le questioni pratiche che servono per addestrare davvero un MLP: inizializzazione, minimi multipli, versioni on-line/batch/mini-batch, learning rate e momentum, criteri di arresto, overfitting e regolarizzazione, numero di unità (con il Cascade Correlation), rappresentazione di input e output. Chiude una sintesi di pregi e difetti delle reti neurali e i primi consigli per il progetto (benchmark MONK).

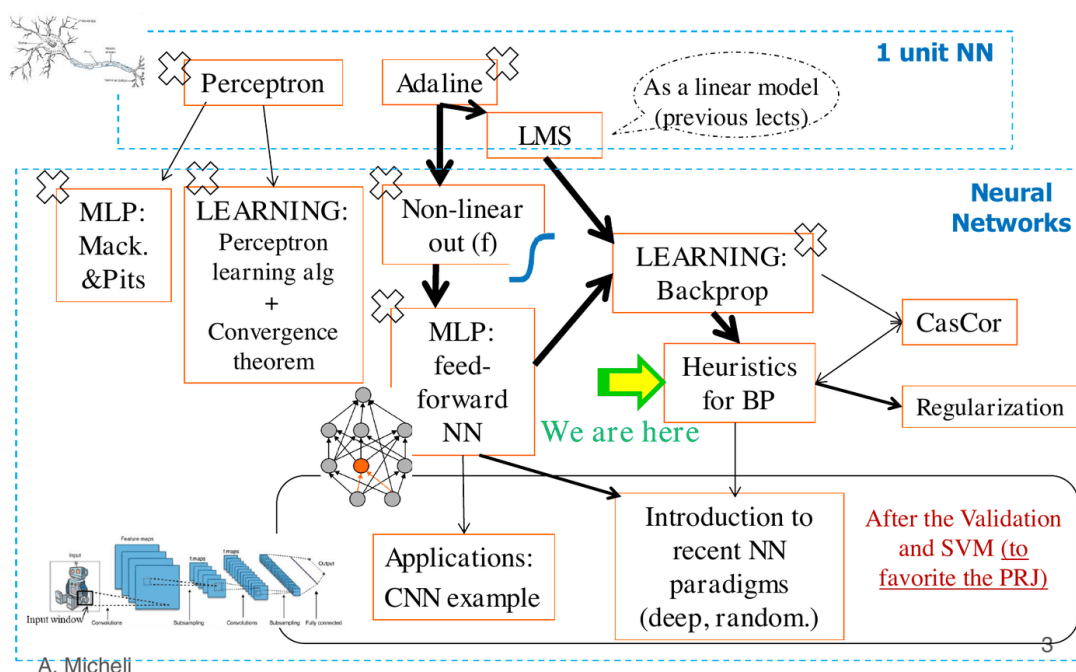


Figura 8.1: Zoom sul blocco "reti neurali" del corso.

8.1 Filosofia

Il ML si è evoluto da una collezione di trucchi euristici a quadri concettuali generali: il professore preferisce non introdurre troppi dettagli euristici. Molte delle tecniche che seguono si possono esplorare sperimentalmente nel progetto; alcune hanno un fondamento teorico (la regolarizzazione) e si applicano in un quadro più generale. Quelle indicate con (!) sono molto comuni e **vanno usate** nel progetto.

Attenzione — Diffidare delle "regole empiriche"

Evitare le *rules of thumb* trovate sui blog, a meno che non siano dimostrate in articoli scientifici o non se ne sia verificato sperimentalmente il vantaggio.

Suggerimento — Un'interpretazione utile

La backpropagation è un **percorso nello spazio dei pesi** (sulla superficie della loss). Il percorso dipende da: i dati, il modello, il **punto di partenza** (pesi iniziali), la **velocità di convergenza** e il **punto finale** (regola di arresto). Tutte queste scelte costituiscono un **controllo della ricerca** nello spazio delle ipotesi.

L'algoritmo di base resta la discesa del gradiente: pesi iniziali piccoli, $\Delta \mathbf{w} = -\partial E / \partial \mathbf{w}$ (calcolato con la backprop: fase forward fino alle uscite, poi fase backward per i delta), $\mathbf{w}_{new} = \mathbf{w} + \eta \Delta \mathbf{w}$, ripetere. Il modello è spesso **sovra-parametrizzato** e l'ottimizzazione è **non convessa** e potenzialmente instabile: da qui le questioni che seguono. Gli **iperparametri** sono i valori che bisogna fissare per eseguire l'addestramento.

8.2 Valori iniziali dei pesi

Suggerimento — (!) Inizializzazione

Inizializzare i pesi con **valori casuali vicini a zero**, ad esempio nell'intervallo $[-0,7; +0,7]$ (per dati standardizzati).

Da evitare:

- **tutti zero**: i delta delle unità nascoste sarebbero tutti nulli (dipendono dai pesi verso l'uscita) e la rete non imparerebbe;
- **valori alti**: le sigmoidi partono **saturate**, con derivate quasi nulle e apprendimento lentissimo;
- **tutti uguali**: le unità nascoste restano **simmetriche** (ricevono gli stessi aggiornamenti e calcolano tutte la stessa cosa). La casualità serve a **rompere la simmetria**.

Si può tenere conto del **fan-in** (numero di input di un'unità), ad esempio con un intervallo pari a $\text{range} \cdot 2 / \text{fan-in}$, ma non se il fan-in è troppo grande e non per le unità di uscita (altrimenti i delta partono vicini a zero). Molto popolare è l'inizializzazione di **Glorot e Bengio** (2010): bias a 0 e pesi da una distribuzione uniforme in $[-1/a, +1/a]$ con $a = \sqrt{\text{fan-in}}$ (o, in risultati successivi, a dipendente anche dal fan-out).

Nota — (!) Per il progetto

Una rete molto piccola (pochissime unità) può dipendere fortemente dall'inizializzazione.

8.3 Minimi multipli

La loss **non è convessa**: ci sono molti minimi locali, e il risultato dipende dai pesi iniziali. Quindi:

Suggerimento — (!) Più inizializzazioni

Provare diverse configurazioni iniziali casuali (ad esempio 5–10 o più *trial*). Per valutare il modello si riporta la **media** dei risultati (media degli errori) e la **varianza**. Se si vuole un'unica risposta, si può scegliere la soluzione con il più basso errore di validazione (o la

mediana), oppure sfruttare i diversi punti finali con una risposta **di comitato** (media delle uscite o voto). Più avanti si vedrà perché gli *ensemble* sono vantaggiosi in generale.

Fermarsi in un minimo locale con errore troppo alto non è un grosso problema: si vede l'errore finale e si può ricominciare. Spesso un “**buon**” **minimo locale è sufficiente**.

8.3.1 Serve davvero il minimo globale?

Due osservazioni fondamentali:

1. Nel ML **non cerchiamo né il minimo locale né quello globale di R_{emp}** , ma il minimo di R , che non possiamo calcolare. Spesso ci si ferma in un punto che non ha nemmeno gradiente nullo.
2. La rete è uno **spazio delle ipotesi di dimensione variabile**: durante l'addestramento la VC-dim cresce, l'errore di training scende verso zero (o il minimo globale) mentre la rete diventa troppo complessa. Bisogna fermarsi **prima** di questa condizione di *overtraining* (e quindi overfitting), indipendentemente dalla questione minimi locali/globali.

8.4 On-line, batch e mini-batch

Ricordiamo (vedi i modelli lineari):

- **Batch**: si sommano i gradienti di tutti i pattern di un'**epoca** (un ciclo completo di presentazione del training set) e poi si aggiornano i pesi.
- **On-line/stocastico**: si aggiorna $\mathbf{w}$ dopo ogni pattern p con $\Delta_p \mathbf{w}$, senza aspettare la somma. Progredisce a ogni esempio, può essere più veloce, ma richiede η più piccolo.

In formule, per il peso w_{tu} :

$$\text{batch: } -\frac{\partial E}{\partial w_{tu}} = -\sum_{p=1}^l \frac{\partial E_p}{\partial w_{tu}} = \sum_{p=1}^l \delta_{p,t} o_{p,u}, \quad \text{on-line: } -\frac{\partial E_p}{\partial w_{tu}} = \delta_{p,t} o_{p,u}.$$

In sintesi — Batch e on-line a confronto

Batch — 1) fissare $\mathbf{w}_{init}$ ed η ; 2) calcolare $\Delta \mathbf{w}$ sull'intero training set (la somma su p è dentro questo passo); 3) $\mathbf{w}_{new} = \mathbf{w} + \eta \Delta \mathbf{w}$; 4) ripetere fino a convergenza.

On-line — 1) fissare $\mathbf{w}_{init}$ ed η ; per ogni pattern p : 2) calcolare $\Delta_p \mathbf{w}$; 3) $\mathbf{w}_{new} = \mathbf{w} + \eta \Delta_p \mathbf{w}$; 4) ripetere (epoca dopo epoca) fino a convergenza.

Il batch dà una stima **più accurata** del gradiente. Nell'on-line il gradiente di un singolo pattern è un'approssimazione **rumorosa** del gradiente complessivo, da cui il nome *stochastic gradient descent*: la discesa procede a zig-zag, il che **può aiutare a sfuggire ai minimi locali**.

Suggerimento — (!) Shuffling

Nella versione on-line bisogna presentare i pattern in **ordine diverso** a ogni epoca (di solito **mescolandoli casualmente**), per evitare una deriva sistematica nella discesa.

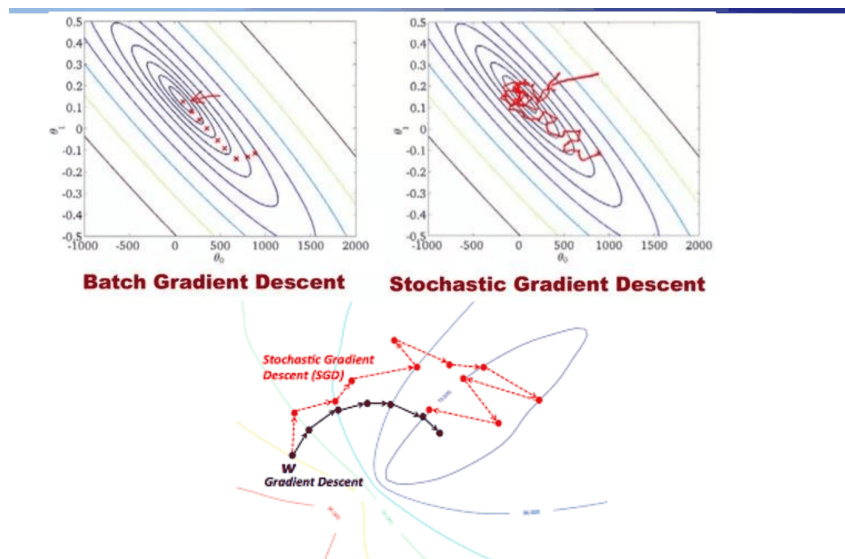


Figura 8.2: Discesa del gradiente batch e stocastica.

8.4.1 Mini-batch

Nel **mini-batch** si divide ogni epoca in parti: si sommano i gradienti di mb pattern ($1 < mb < l$) prima di aggiornare i pesi, e si ripete finché tutti i dati dell'epoca sono stati usati. L'aggiornamento avviene quindi a ogni mini-batch invece che a ogni pattern o a ogni epoca.

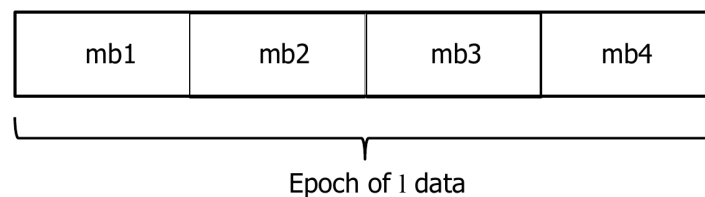


Figura 8.3: Un'epoca suddivisa in mini-batch.

On-line	Mini-batch (SGD)	Batch
non sfrutta il parallelismo tra esempi può essere instabile il rumore può avere un effetto regolarizzante	compromesso; si adatta alla memoria della GPU	stima più accurata del gradiente più lento (aspetta tutti i dati)

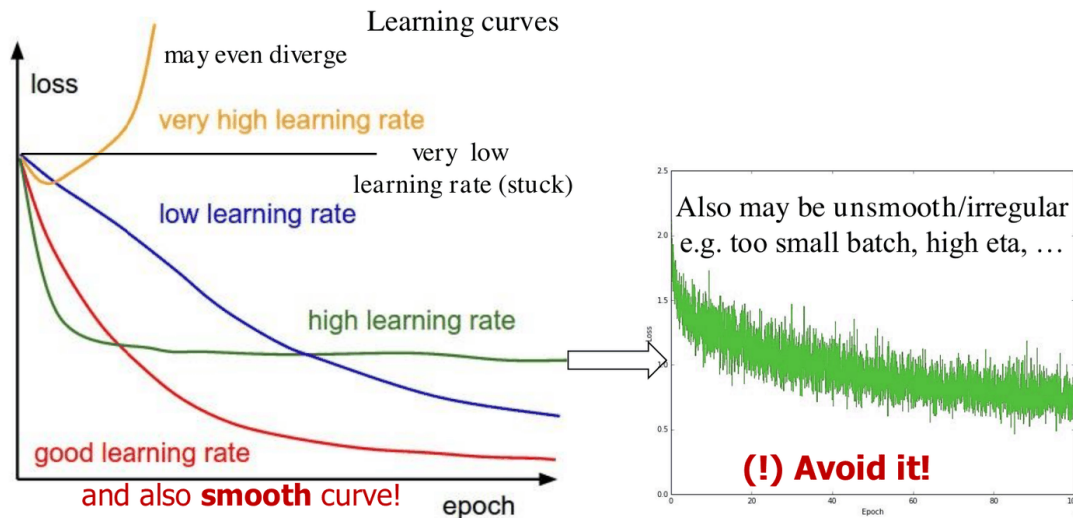
Il **mini-batch SGD** usa campionamento casuale (shuffling) per una stima non distorta del gradiente. Con training set enormi o dati in streaming si può anche fare a meno delle epoche. Poiché il costo di un aggiornamento dipende da mb e non da l , lo SGD con mini-batch è un fattore chiave per estendere l'apprendimento a dataset di milioni di esempi.

8.5 Il learning rate η

- **Batch**: gradiente più accurato $\rightarrow$ si può usare η più alto.
- **On-line**: più veloce ma più instabile $\rightarrow \eta$ più piccolo.
- In generale: η **alto** è veloce ma può essere instabile; η **basso** è stabile ma può essere troppo lento. Tipicamente si cerca in $[0,01; 0,5]$ con LMS.

Suggerimento — (!) Guardare sempre la curva di apprendimento

Il grafico dell'errore durante l'addestramento permette di verificare il comportamento nelle prime fasi di progettazione. Qui si valuta la **qualità** della curva e la **velocità** di convergenza (il valore assoluto dell'errore dipende anche dalla capacità del modello e dagli altri iperparametri).



Here just looking to the optimization “failures” (on the TR set),

Figura 8.4: Effetto del learning rate sulle curve di apprendimento (errore di training).

Una curva irregolare può dipendere da batch troppo piccoli o da η troppo alto: va evitata. Si cerca una curva che scende rapidamente **e in modo liscio**. (Qui si guardano solo i “fallimenti” dell’ottimizzazione sul training set, non l’errore di generalizzazione.)

8.5.1 LMS: dividere per l

Suggerimento — (!) Usare la media dei gradienti

Conviene usare la **media** dei gradienti sull’epoca (somma diviso l): rende l’approccio uniforme rispetto al numero di dati (è il *least mean squares*). Equivale a usare η/l con $0 < \eta < 1$.

Attenzione: confrontando on-line e batch con LMS, l’on-line richiede un η **molto più piccolo** (dello stesso fattore $1/l$) per essere comparabile. Per il mini-batch (se si divide per mb) va ritarato η , oppure si divide sempre per l .

8.5.2 Migliorare la discesa

Momentum (!)

$$\Delta \mathbf{w}_{new} = -\eta \frac{\partial E(\mathbf{w})}{\partial \mathbf{w}} + \alpha \Delta \mathbf{w}_{old}, \quad \mathbf{w}_{new} = \mathbf{w} + \Delta \mathbf{w}_{new},$$

con $0 < \alpha < 1$ (ad esempio 0,5–0,9). Qui η è incluso in $\Delta \mathbf{w}$, e dopo ogni passo si salva $\Delta \mathbf{w}_{old} = \Delta \mathbf{w}_{new}$.

È il **metodo della “palla pesante”** (*heavy ball*): a ogni passo si aggiunge una frazione dello spostamento precedente, come se i pesi avessero un’inerzia.

- **Più veloce nei plateau:** se il gradiente ha lo stesso segno in iterazioni consecutive, il momentum aumenta il passo.
- **Smorza le oscillazioni:** nelle direzioni in cui il gradiente cambia segno i contributi si compensano.
- **Effetto inerzia:** permette di usare η più alti.

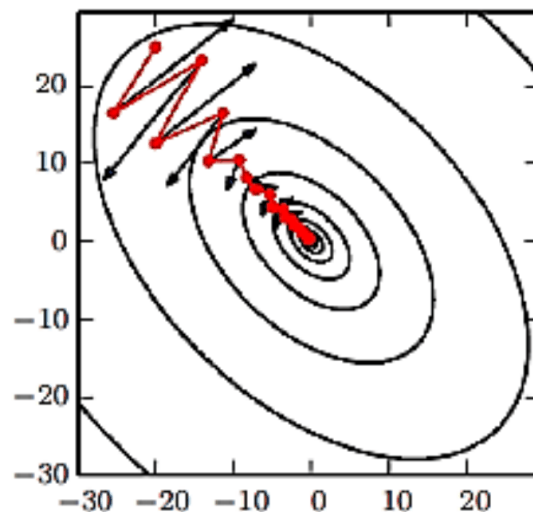


Figura 8.5: Effetto del momentum in un “canyon” della superficie d’errore (Hessiana mal condizionata): il percorso rosso attraversa la valle invece di rimbalzare tra le pareti.

Il momentum nasce per il batch. Nell’on-line $\Delta \mathbf{w}_{old}$ è quello del **pattern precedente**, e il momentum diventa una **media mobile dei gradienti passati** che liscia i gradienti stocastici all’interno dell’epoca (analogamente per il mini-batch). Il significato è quindi diverso dal batch, dove si combinano gradienti calcolati sugli stessi esempi in epoche diverse.

Nesterov momentum: si applica prima il momentum ($\mathbf{w} + \alpha \Delta \mathbf{w}_{old}$), si valuta il gradiente in questo punto intermedio, e poi si applica il $\Delta \mathbf{w}$ (momentum più nuovo gradiente) ai pesi originali. Migliora la velocità di convergenza nel batch (non nel caso stocastico).

Learning rate variabile

Con il mini-batch il gradiente **non tende a zero** vicino al minimo (per via del rumore di campionamento), quindi un η fisso va evitato. Ad esempio si decresce linearmente fino all’iterazione τ :

$$\eta_s = \left(1 - \frac{s}{\tau}\right) \eta_0 + \frac{s}{\tau} \eta_\tau, \quad s \leq \tau,$$

e poi si usa $\eta = \eta_\tau$ costante e piccolo (ad esempio $\eta_\tau \approx 1\%$ di η_0 , τ qualche centinaio di passi). η_0 si sceglie con il compromesso instabilità/blocco (prove preliminari o grid search).

Learning rate adattivi

Algoritmi che adattano automaticamente il learning rate durante l’addestramento, anche separatamente **per ogni parametro**: **AdaGrad**, **RMSProp**, **Adam** (molto popolare e spesso robusto con i valori di default, ma con cautela). Si combinano con il momentum. Nelle reti con molti strati, η può anche dipendere dallo strato o dal fan-in.

8.5.3 Oltre lo SGD di base

Lo SGD è efficiente (lineare nel numero di parametri) ma dipende dal numero di dati e di epoche. Esistono molte euristiche: **Quickprop** (salta al minimo di una parabola che approssima localmente la loss), **R-prop** (usa solo il **segno** del gradiente, non il suo valore, utile quando il gradiente svanisce negli strati profondi).

I **metodi del secondo ordine** usano l'Hessiana (derivate seconde) per avere informazioni sulla curvatura e scendere meglio (metodo di Newton), ma l'Hessiana è costosissima. Ci sono versioni approssimate: Gauss-Newton, Levenberg-Marquardt, metodi *Hessian-free* come il **gradiente coniugato**, metodi quasi-Newton come **BFGS**.

Nota — Punti di sella

Un punto di sella ha gradiente (quasi) nullo ma non è un minimo. In spazi ad alta dimensione i punti di sella crescono esponenzialmente rispetto ai minimi locali. Il metodo di Newton cerca (e “salta verso”) i punti a gradiente nullo, quindi è più sensibile ai punti di sella: questo può spiegare perché i metodi del secondo ordine non hanno sostituito la discesa del gradiente per le reti neurali, mentre la discesa del gradiente empiricamente riesce a sfuggire ai punti di sella. In pratica troviamo abbastanza in fretta valori bassi della loss, utili soprattutto per generalizzare.

Suggerimento — In pratica

Per l'ottimizzazione gli approcci di base (quelli con (!)), se applicati correttamente, di solito bastano. L'esperienza con tecniche note conta più di metodi sofisticati che non si sanno gestire. La tecnica di ottimizzazione è rilevante solo se ci sono ostacoli all'addestramento: **la validazione conta più dell'ottimizzazione sul training set**, perché ottimizzare sui dati di training non è il nostro obiettivo. Il punto di partenza (anche didattico) è **SGD con momentum** (e regolarizzazione).

8.6 Criteri di arresto

- Il criterio base è l'errore usato (errore medio $< \epsilon$): è il migliore se si conosce la tolleranza dei dati (conoscenza dell'esperto), ma spesso non è così. Varianti: tolleranza massima invece che media; per la classificazione, il numero di errori.
- Criteri “interni”: pesi che non cambiano più, gradiente quasi nullo (norma euclidea $< \epsilon$), errore che non diminuisce più in modo significativo in un'epoca (es. meno dello 0,1%). Attenzione: possono essere **prematuri** (ad esempio con η troppo piccolo), quindi si osservano k epoche (**patience**).
- (!) Fermarsi comunque dopo un numero eccessivo di epoche, per sfuggire a convergenze troppo lente, ma **non** fermarsi a un numero arbitrario di epoche fissato a priori.
- E non necessariamente fermarsi con un errore di training molto basso...

8.7 Overfitting e regolarizzazione

8.7.1 Fermarsi al minimo?

Tipicamente **non** vogliamo il minimizzatore globale di $R_{emp}(\mathbf{w})$, perché è probabilmente una soluzione in overfitting: è una differenza fondamentale rispetto ai metodi di ottimizzazione (CM). Il nostro scopo principale è il **controllo della complessità** per ottenere la migliore

generalizzazione, tramite regolarizzazione (esplicita con un termine di penalità, o implicita con l'early stopping), più la model selection (cross-validation) per trovare il compromesso.

8.7.2 Come nasce l'overfitting in una rete

- Si parte con **pesi piccoli e casuali** (rottura di simmetria).
- La mappatura input→output è quasi **lineare** (le sigmoidi lavorano nella zona lineare): il numero effettivo di parametri liberi (e la VC-dim) è quasi quello di un perceptron.
- Man mano che l'ottimizzazione procede, le unità nascoste tendono a **saturare**, aumentando il numero effettivo di parametri e le non linearità: la VC-dim cresce.

È di nuovo lo **spazio delle ipotesi di dimensione variabile**.

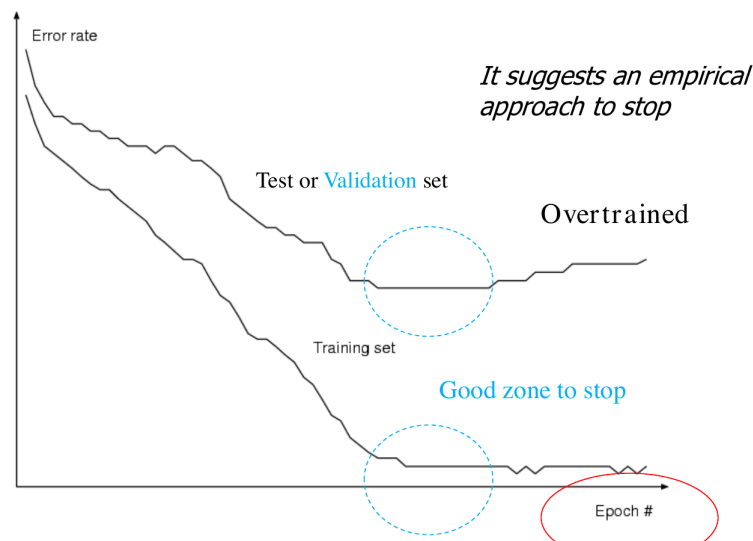


Figura 8.6: Comportamento tipico della backpropagation: suggerisce un criterio empirico per fermarsi.

La curva va letta insieme al grafico del VC-bound (fondamento teorico). Come intervenire:

1. **Early stopping:** usare un **validation set** per decidere quando fermarsi, cioè quando l'errore di validazione inizia a salire. È vago: bisogna osservare più epoche prima di decidere (*patience*) ed eventualmente tornare indietro (*backtracking*). Poiché il numero effettivo di parametri cresce durante l'addestramento, **fermarsi prima equivale a limitare la complessità effettiva**.
2. **Regolarizzazione** della loss: il nostro strumento principale.
3. Metodi di **pruning** (più avanti).

8.7.3 Regolarizzazione (!)

Durante l'addestramento i pesi crescono: possiamo ottimizzare la loss tenendo conto del loro valore. L'approccio fondato (di nuovo Tikhonov) aggiunge un termine di penalità:

$$\text{Loss}(\mathbf{w}) = \sum_p (d_p - o(\mathbf{x}_p))^2 + \lambda \|\mathbf{w}\|^2, \quad \|\mathbf{w}\|^2 = \sum_i w_i^2 \text{ su tutti i pesi della rete.}$$

L'effetto è il **weight decay**: si aggiunge circa $2\lambda w$ al gradiente di ogni peso,

$$\mathbf{w}_{\text{new}} = \mathbf{w} + \eta \Delta \mathbf{w} - 2\lambda \mathbf{w}$$

(il 2 si può omettere). λ è in genere molto piccolo (es. 0,01) e si sceglie in **model selection** (cross-validation). Applicata a un modello lineare è la ridge regression; esistono penalità più sofisticate (ad esempio per la *weight elimination*).

Nota — Errore e Loss

Con un termine di penalità conviene distinguere: **Loss** è la funzione obiettivo usata per l'addestramento (MSE + penalità); **Errore/Rischio** è il termine sui dati (MSE), che misura l'errore del modello. (!) È l'errore che va riportato in tabelle e grafici, perché è la misura utile per chi usa il modello.

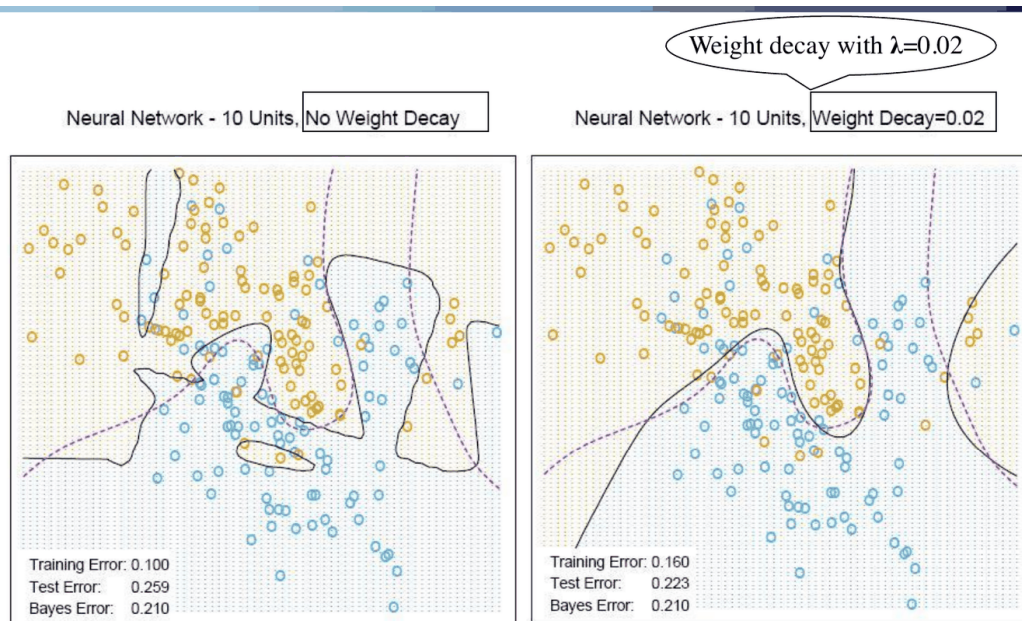


Figura 8.7: Effetto del weight decay su una rete da 10 unità (Hastie-Tibshirani-Friedman): il confine diventa più regolare e l'errore di test si avvicina a quello di Bayes (0,210).

Fraintendimenti frequenti

Attenzione — La regolarizzazione non serve a stabilizzare la convergenza

La regolarizzazione **non** è una tecnica per rendere più stabile la curva di apprendimento: serve a **controllare la complessità** del modello (misurata dalla VC-dim, legata al numero e al valore dei pesi). Se la curva è instabile, il problema è altrove (learning rate, batch...).

Domande — Meglio early stopping o regolarizzazione?

Early stopping è un approccio **empirico** che richiede un validation set per decidere quando fermarsi, sacrificando una parte dei dati. La regolarizzazione nella loss è un approccio **fondato**: permette alla curva di validazione di seguire quella di training (purché non si abbia underfitting per un λ troppo alto), e in quel caso non serve (strettamente) l'early stopping e ci si può fermare a convergenza. Si possono anche usare entrambi: l'early stopping non interviene se l'errore di validazione non sale.

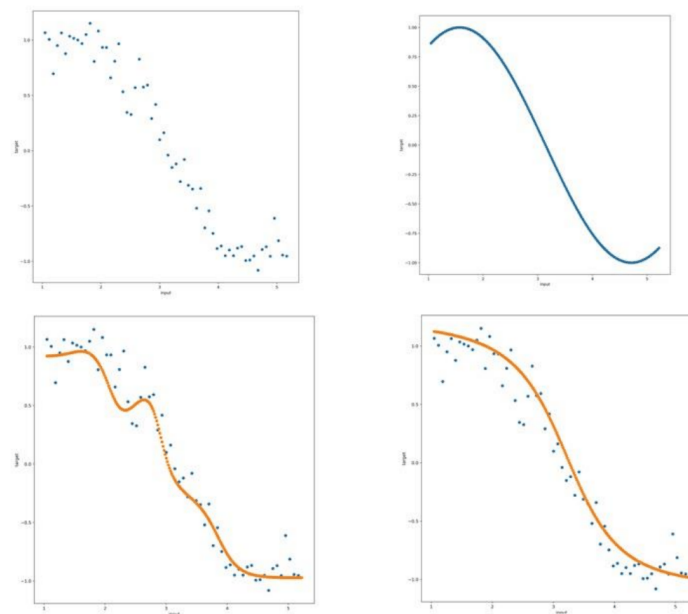
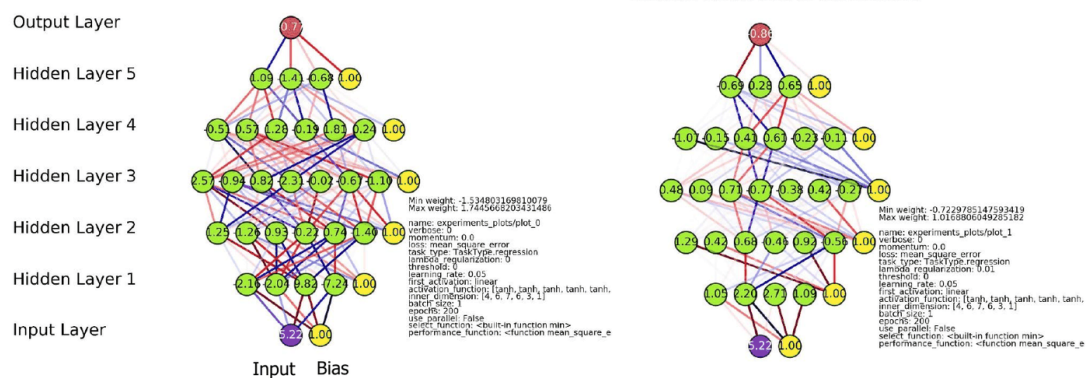
NN output $\lambda=0$ Regularized ($\lambda=0.01$)

Figura 8.8: Regressione univariata: rete senza regolarizzazione ($\lambda = 0$) e regolarizzata ($\lambda = 0,01$).



NN trained with $\lambda=0$:
all the weights are significantly
higher (or lower) than 0.

Regularized ($\lambda=0.01$):
most of the weights are close to zero
(light color $\rightarrow$ lower in module).
Hence lower model **complexity**!

Figura 8.9: I pesi della rete senza regolarizzazione (sinistra) e con regolarizzazione (destra): con $\lambda = 0,01$ la maggior parte dei pesi è vicina a zero, quindi la complessità del modello è minore.

Dettagli

- Spesso il **bias** w_0 è **escluso** dal regolarizzatore (includerlo renderebbe i risultati dipendenti da traslazioni/scalature del target), oppure ha un proprio coefficiente.
- Tipicamente si applica nella versione batch. Nell'on-line/mini-batch la penalità viene applicata a ogni passo, quindi molte volte per epoca: per un confronto equo col batch si userebbe $\lambda \cdot mb/l$. (Se λ si sceglie con la model selection, questo avviene automaticamente.)
- Un'altra tecnica è il **dropout** (più avanti).

8.7.4 Mettere tutto insieme: momentum e weight decay

Con la loss regolarizzata, la regola con momentum diventa (separando i due termini della loss e usando η solo per il termine d'errore, come già fatto nei modelli lineari):

$$\Delta w_{tu} = -\eta \frac{\partial \text{Loss}}{\partial w_{tu}} + \alpha \Delta w_{tu}^{old} \doteq \eta \delta_t o_u - \lambda w_{tu} + \alpha \Delta w_{tu}^{old}, \quad w_{tu}^{new} = w_{tu} + \Delta w_{tu}.$$

Così η non moltiplica λ : cambiando uno non si cambia l'altro.

Il momentum deve includere anche la parte di λ ? Spesso sì (Matlab, Caffè...). Ma per rendere **indipendenti tutti e tre gli iperparametri** η , α , λ si può scrivere:

$$\Delta w_{tu} = \eta \delta_t o_u + \alpha \Delta w_{tu}^{old}, \quad w_{tu}^{new} = w_{tu} + \Delta w_{tu} - \lambda w_{tu},$$

separando il weight decay dalla memoria del momentum. Si divide per l solo il gradiente di E (dove c'è la somma sui pattern).

8.8 Numero di unità nascoste

Il numero di unità è legato al **controllo della complessità**, alla dimensione dell'input e alla dimensione del training set. In generale è un problema di **model selection**: il numero di unità, insieme a λ , si sceglie con la cross-validation.

- Troppe poche unità $\rightarrow$ underfitting; troppe $\rightarrow$ overfitting (può verificarsi).
- Il numero di unità può essere più alto se si usa un'adeguata regolarizzazione.

Alternative alla ricerca manuale:

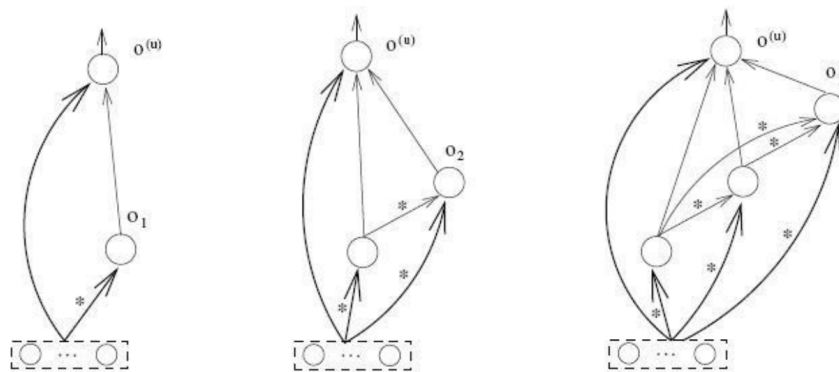
- **approcci costruttivi**: l'algoritmo decide il numero di unità partendo da una rete piccola e aggiungendone;
- **metodi di pruning**: si parte da reti grandi e si eliminano progressivamente pesi o unità.

8.8.1 Cascade Correlation

Tra gli approcci costruttivi (Tower, Tiling, Upstart per la classificazione) il più noto è il **Cascade Correlation** (CC) di Fahlman e Lebiere (1990), per regressione e classificazione. Apprende **sia i pesi sia la topologia** (numero di unità): lo spazio delle ipotesi ha dimensione flessibile, decisa dall'algoritmo, e a ogni passo si addestra una sola unità.

In sintesi — Algoritmo Cascade Correlation

1. Si parte da una rete N_0 **senza unità nascoste** e la si addestra. Se non risolve il problema, si passa a N_1 .
2. In N_1 si aggiunge un'unità nascosta i cui pesi vengono addestrati per **massimizzare la correlazione** tra la sua uscita e l'**errore residuo** della rete N_0 .
3. Dopo l'addestramento, i pesi in ingresso della nuova unità vengono **congelati**; si riaddestrano i pesi rimanenti (strato di uscita).
4. Se la rete non risolve il problema si aggiungono altre unità, ciascuna collegata a **tutti gli input e a tutte le unità nascoste precedenti** (da qui “a cascata”).
5. Si continua finché l'errore residuo soddisfa il criterio di arresto.



© A. Micheli 2003

* = weights frozen after candidate training**Figura 8.10:** Costruzione di una rete Cascade Correlation con fino a 3 unità nascoste.

L'algoritmo alterna la minimizzazione dell'errore totale (LMS, ad esempio backprop sullo strato di uscita) e la **massimizzazione della covarianza** S tra l'uscita o_p della candidata e l'errore residuo $E_{p,k} = o_{p,k} - d_{p,k}$:

$$S = \sum_k \left| \sum_p (o_p - \bar{o})(E_{p,k} - \bar{E}_k) \right|.$$

Il gradiente (con $d|f|/dx = \text{sign}(f) df/dx$ e trascurando la dipendenza delle medie da o_p) è

$$\frac{\partial S}{\partial w_j} = \sum_k \text{sgn}(S_k) \sum_p (E_{p,k} - \bar{E}_k) f'(net_{p,h}) I_{p,j},$$

dove h è la candidata e $I_{p,j}$ il suo input j . Si fa **ascesa** del gradiente (si massimizza S).

Note sul CC:

- il ruolo delle unità nascoste è **ridurre l'errore residuo**: ogni unità risolve un sotto-problema e diventa un **rilevatore di feature permanente**;
- **pool di candidate**: la superficie di S ha molti massimi, quindi si addestra un gruppo di candidate e si sceglie la migliore, evitando massimi locali;
- è un algoritmo **greedy**: converge facilmente e può trovare un numero minimo di unità, ma può andare in overfitting (serve regolarizzazione);
- la loss non deve essere necessariamente “max S ” (per la regressione si può usare LMS).

8.9 Input e output

Input: il pre-processing ha grande effetto.

- **Standardizzazione** (spesso utile): per ogni feature media 0 e deviazione standard 1, $(v - \text{media})/\text{dev. std.}$
- **Rescaling** in $[0, 1]$: $(v - \min)/(\max - \min)$.
- Input categorici: codifica **1-of-K** ($A \rightarrow 100, B \rightarrow 010, C \rightarrow 001$).
- Dati mancanti: attenzione, **0 non significa “nessun input”** se 0 è nel range dei valori.

Output:

- **(!) Regressione:** unità di uscita **lineari** (una o più).
- **Classificazione:** una unità (binaria) o 1-of-K con più uscite. Con la sigmoide si sceglie la soglia per assegnare la classe (con eventuale zona di rifiuto); con 1-of-K vince l'uscita più alta. Spesso la **tanh** (simmetrica) apprende più in fretta. Come target si può usare 0,9 invece di 1 (e $-0,9$ o $0,1$ invece di -1 o 0) per evitare la convergenza asintotica verso la saturazione. Ovviamente il range dei target deve stare nel range delle uscite.
- **Softmax** per targets 0/1 multi-classe: le uscite sommano a 1 e si possono interpretare come probabilità $p(\text{classe} = k \mid \mathbf{x})$:

$$o_k(\mathbf{x}) = \frac{e^{\text{net}_k}}{\sum_{j=1}^K e^{\text{net}_j}}.$$

- Si può usare la **cross-entropy** come loss alternativa (stima di massima verosimiglianza); per un'unità:

$$- \sum_{i \in TR} \{d_i \log(\text{out}(\mathbf{x}_i)) + (1 - d_i) \log(1 - \text{out}(\mathbf{x}_i))\}.$$

Domande — Esercizio per il progetto

Riflettere sul ruolo dei diversi iperparametri e collegarli alla teoria: ad esempio collegare la loss con penalità al VC-bound, per capire come numero di unità, λ e gli altri controllano underfitting e overfitting. Quali sono i più rilevanti per il progetto (e per vincere la competizione)?

8.10 Pregi e difetti delle reti neurali

In sintesi — Pregi

- Metodo flessibile: estende i metodi lineari alla stima di funzioni arbitrarie e costruisce un'**ipotesi esplicita** (a differenza del K-NN).
- Potere espressivo dato da numero di unità, architettura e addestramento.
- Spazio delle ipotesi continuo e ricco, loss differenziabile $\rightarrow$ addestramento con discesa del gradiente.
- Approssimatori universali; gestiscono rumore e dati incompleti con degrado graduale; dati continui e discreti, regressione e classificazione.
- Un **paradigma** ampio (anche apprendimento non supervisionato, memorie associative, approcci stocastici).
- Plausibilità neurale; **rappresentazione distribuita** della conoscenza (compatta in

matrici di pesi); **tolleranza ai guasti**; adattamento a ambienti non stazionari con apprendimento on-line (*continual learning*, con il dilemma stabilità-plasticità).

Attenzione — Punti critici

- **Problema della black box**: è difficile interpretare la conoscenza acquisita ed estrarla per gli esperti umani. È legato al tema attuale della **explainable AI** (XAI). Approcci: proiezione/clustering delle rappresentazioni interne, analisi di sensitività, **Layer-wise Relevance Propagation** (LRP, 2015, anche per reti profonde, che produce mappe di calore sulle parti dell'input rilevanti per la decisione), estrazione di regole simboliche.
- **Progettazione dell'architettura**: servono metodi costruttivi/pruning o approcci diversi (SVM).
- **Dipendenza dal processo di addestramento**: le molte varianti e iperparametri influenzano la soluzione finale.
- Non ideali con **dati mancanti** (si usano rimozione, imputazione...).

Le reti neurali sono indicate quando: l'input è ad alta dimensione (discreto o reale), il task è di classificazione o regressione, i dati possono essere rumorosi, il tempo di addestramento non è critico, la forma della funzione target è sconosciuta, la leggibilità del risultato non è critica, e il calcolo dell'uscita deve essere veloce.

Nota — Note storiche

McCulloch e Pitts (1943), Hebb (1949), Minsky (1954, 1969), von Neumann (1956), Rosenblatt (1958), Widrow e Hoff (1960), Kohonen (1972, 1982), Grossberg (ART, 1980), Hopfield (1982), Rumelhart, Hinton, Williams, Werbos, Parker, LeCun (1975–1985, backprop), Poggio e Girosi (teoria RBF, 1990), Vapnik (SVM, anni '90), deep learning (Hinton, Bengio, LeCun, dal 2007). **Hopfield e Hinton** hanno ricevuto il **Premio Nobel per la Fisica 2024**; Hinton, Bengio e LeCun il Premio Turing 2018.

8.11 Verso il progetto: il benchmark MONK

Non si è ancora pronti per il progetto (mancano la **validazione** e la lezione dedicata), ma si può iniziare a implementare la rete (backprop e regole di aggiornamento) o a esplorare i simulatori.

Verificare la correttezza di un'implementazione è difficile: una rete spesso “funziona” (anche se male) nonostante piccoli errori. Il primo collaudo è il dataset **MONK** (UCI), i cui risultati vanno riportati nel report:

- 3 task di classificazione binaria, piccoli e artificiali, “non difficili”: una piccola rete con poche unità raggiunge accuratezze altissime (fino al 100%) in poco tempo;
- esiste un report con risultati precedenti di MLP e altri modelli;
- **input**: 6 variabili con 2, 3 o 4 valori simbolici ciascuna → codifica **1-of-k** → **17 unità di input**;
- il test set include il training set (cattiva pratica, ma qui irrilevante).

Suggerimento — Consigli per le prime prove sul MONK

- Un'unità di uscita per la classificazione (target e uscite coerenti, 0/1 o -1/ +1); si suggerisce la **tanh** in uscita.

- **Codifica one-hot degli input:** errore frequentissimo se dimenticata!
- Batch con gradiente diviso per il numero di pattern: converge in qualche centinaio di epoche.
- Poche unità nascoste (2–5), come nel manuale: il test funziona se si raggiungono i risultati dello stato dell'arte con poche unità.
- Il MONK è sensibile all'inizializzazione (poche unità): provare intervalli iniziali piccoli (o basati sul fan-in).
- Con approcci on-line usare lo **shuffling** (i pattern sono ordinati per classe).

Buoni risultati sul MONK **non garantiscono** la correttezza del simulatore; risultati cattivi indicano **sicuramente** che codice o impostazioni vanno rivisti. Nei grafici conta la **forma** delle curve (lisce, convergenti), non i valori.

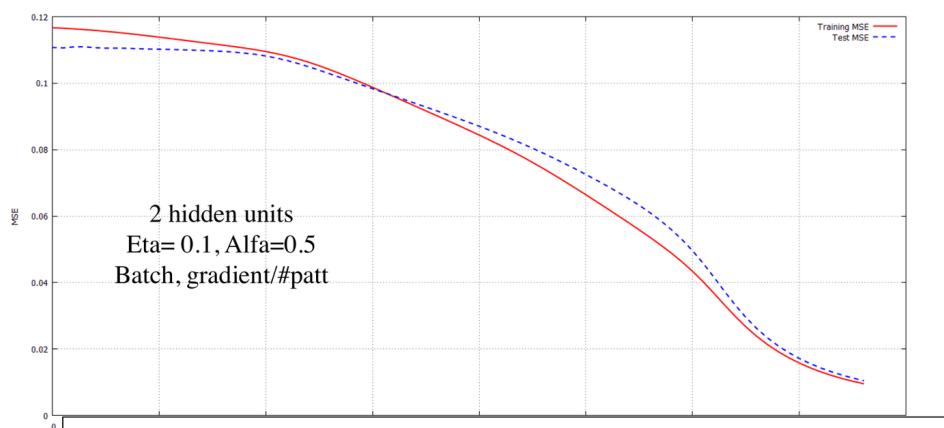


Figura 8.11: MONK2: MSE di training (rosso) e test (blu tratteggiato) con 2 unità nascoste, $\eta = 0,1$, $\alpha = 0,5$.

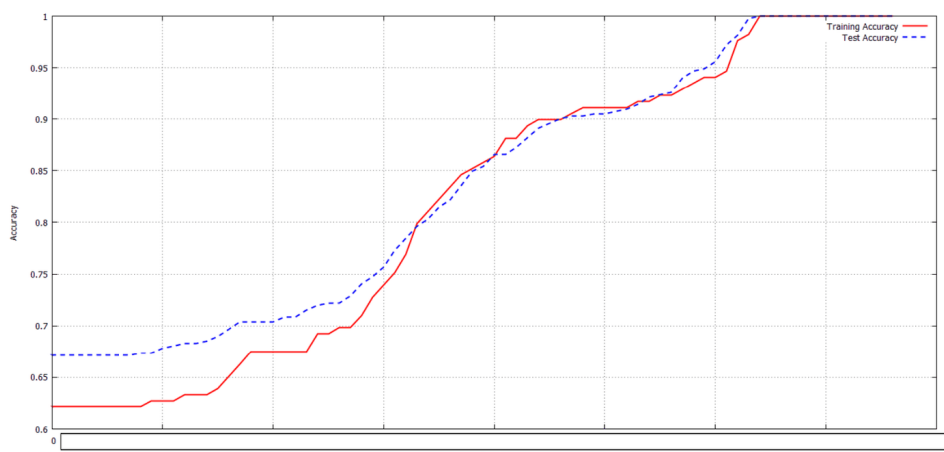


Figura 8.12: MONK2: l'accuratezza raggiunge il 100% anche sul test (caso insolito in generale).

Domande — Cosa succede nella Fig. 8.13?

Nella parte finale l'errore di training continua a scendere mentre quello di test risale leggermente: è l'inizio dell'**overfitting** (MONK3 ha etichette rumorose). La regolarizzazione lo attenua; un λ scelto con la model selection o l'early stopping aiuterebbero.

Per verificare la backpropagation si può usare un esempio passo-passo (attenzione se omette i bias) o una libreria affidabile come “oracolo” (Keras, scikit-learn, PyTorch...). Altri dataset utili

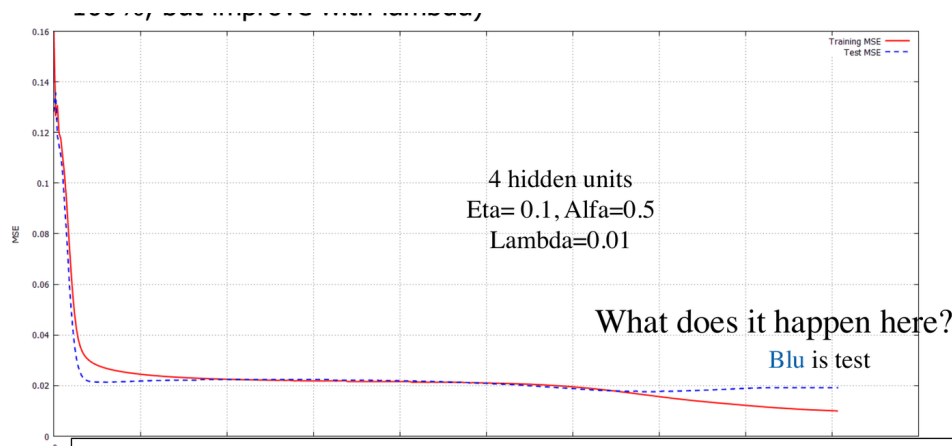


Figura 8.13: MONK3 con regolarizzazione ($\lambda = 0,01$): il MONK3 contiene rumore e può andare in overfitting; il test non arriva al 100%.

si trovano nell'UCI Machine Learning Repository.

Nota — Anteprima del progetto (regole a.a. 2026/27)

Negli anni passati il progetto era obbligatorio (gruppi di 2–3, tipo A con simulatore implementato da zero o tipo B con librerie, competizione ML-CUP). Dall'a.a. 2026/27 il progetto è **opzionale**: gruppi di **2 studenti**, implementazione/applicazione/analisi di un metodo di ML con valutazione sperimentale, presentato come **poster** in una delle ultime lezioni (dicembre). Può dare un piccolo bonus ma non è necessario per l'esame, che si basa sullo scritto (vedi [01 - Introduzione al Machine Learning](#)). I consigli sul MONK restano validi come collaudo di qualsiasi implementazione.

Domande — Possibili domande d'esame

- Come si inizializzano i pesi e perché?
- Differenze tra on-line, batch e mini-batch.
- Ruolo del learning rate; cos'è il momentum e perché funziona?
- Serve trovare il minimo globale? Perché?
- Come si manifesta l'overfitting in una rete neurale? Cos'è l'early stopping?
- Scrivere la loss con weight decay e la regola di aggiornamento con momentum.
- La regolarizzazione migliora la stabilità della convergenza?
- Descrivere il Cascade Correlation.
- Come si rappresentano input e output per regressione e classificazione?

Capitolo 9

Validazione (parte 1): model selection e model assessment

La validazione è stata introdotta in [04 - Generalizzazione e validazione \(introduzione\)](#); queste lezioni la approfondiscono, perché è indispensabile per il progetto e per ogni applicazione seria del ML. Si ribadiscono i due obiettivi (model selection e model assessment), si vede un controesempio che mostra quanto è facile ottenere stime sbagliate, si introducono la **grid search** e la **K-fold cross-validation**, e si discutono campionamento, pochi dati e misure d'errore.

9.1 Premessa: bias e varianza

La decomposizione **bias-varianza** (che avrà una lezione dedicata) è un quadro utile per capire perché stimare le prestazioni di un modello è difficile: mostra il ruolo delle diverse **realizzazioni del training set** e, ancora una volta, il compromesso tra capacità di adattamento (bias) e flessibilità del modello (varianza).

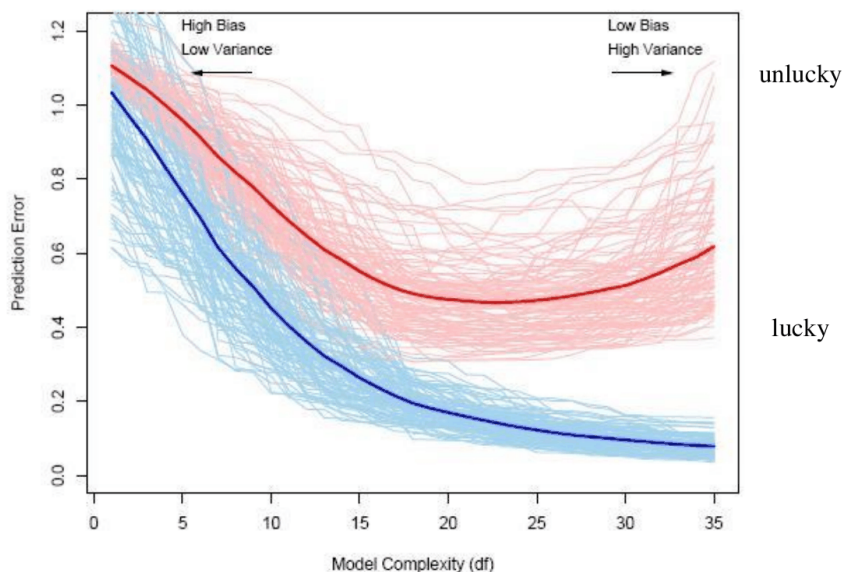


Figura 9.1: Errore di training (azzurro) e di test (rosso) su 100 training set diversi al crescere della complessità (Hastie et al., Fig. 7.1). Con un modello complesso il bias è basso ma la varianza alta: a seconda del campione si può essere “fortunati” o “sfortunati”.

La figura mostra un fatto importante: a parità di complessità, l'errore di test varia molto tra un training set e l'altro, soprattutto per modelli complessi. Una singola misura può essere fortunata o sfortunata.

9.2 Motivazioni

Cerchiamo la soluzione con il minimo errore di predizione (di test), bilanciando **adattamento ai dati di training** e **complessità del modello**. L'errore di training **non è una buona stima** dell'errore di test: all'inizio c'è troppo bias (underfitting, errore di training alto), poi troppa varianza (errore di training basso ma overfitting).

Supponendo di avere un iperparametro θ (implicito o esplicito) che regola la complessità del modello, vogliamo il valore di θ che minimizza l'errore di test. Servono quindi metodi per **stimare l'errore atteso** di un modello (o di ciascun modello di una classe, o di un insieme di modelli).

9.2.1 Come stimare

- **Analiticamente:** criteri AIC e BIC (*Akaike/Bayesian Information Criterion*, limitati a modelli lineari nei parametri), MDL (*Minimum Description Length*), **SRM** (*Structural Risk Minimization*) con la VC-dimension (vedi la lezione sulla SLT).
- **Empiricamente**, sui dati, tramite **ricampionamento** (stima diretta dell'errore fuori campione): **cross-validation** (hold-out, K-fold, ...) e **bootstrap**.

9.3 I due obiettivi della validazione

Attenzione — La slide più importante del corso

Dopo aver addestrato i modelli sul training set:

- **Model selection:** stimare le prestazioni (errore di generalizzazione) di diversi modelli per **scegliere il migliore**. Include la ricerca dei migliori **iperparametri** (grado del polinomio, numero di unità di una rete, λ , η , ...). *Restituisce un modello.*
- **Model assessment:** scelto il modello finale (o una classe di modelli), **stimarne l'errore di predizione** (rischio) su dati di test nuovi, come misura della qualità del modello scelto. *Restituisce una stima.*

Regola d'oro: tenere separati gli obiettivi e usare **insiemi di dati separati** in ogni fase.

9.3.1 Hold-out

Se i dati sono sufficienti si divide il dataset in tre insiemi **disgiunti**, ad esempio 50% TR, 25% VL, 25% TS:

- **TR** (*training set*): per adattare il modello, cioè minimizzare R_{emp} della SLT [**training**];
- **VL** (*validation o selection set*): per scegliere il modello migliore tra modelli e configurazioni di iperparametri diversi [**model selection**];
- **TS** (*test set*): per stimare l'errore di generalizzazione del modello finale, cioè stimare R della SLT [**model assessment**].

TR e VL insieme formano il **development/design set**, usato per costruire il modello finale.

Nota — Due precisazioni

1. La stima fatta sul VL serve **solo** alla model selection: non è una buona stima per l'assessment.
2. I risultati sul TS **non** si possono usare per la model selection. Se lo si fa, quello non è più un test set: chiamiamolo validation set.

9.3.2 Test o model selection?

Cosa succede se il test set viene usato in un ciclo di progettazione ripetuto (provo, guardo il test, cambio, riprovo)? Si sta facendo **model selection**, non una valutazione affidabile, e non potremmo farlo sugli esempi futuri. L'errore sul test così usato è una **stima troppo ottimistica** dell'errore vero. È come un esercizio d'esame di cui si sono viste le soluzioni: non è più un test! Da qui il concetto di **blind test set** (test cieco), usato nelle competizioni di ML.

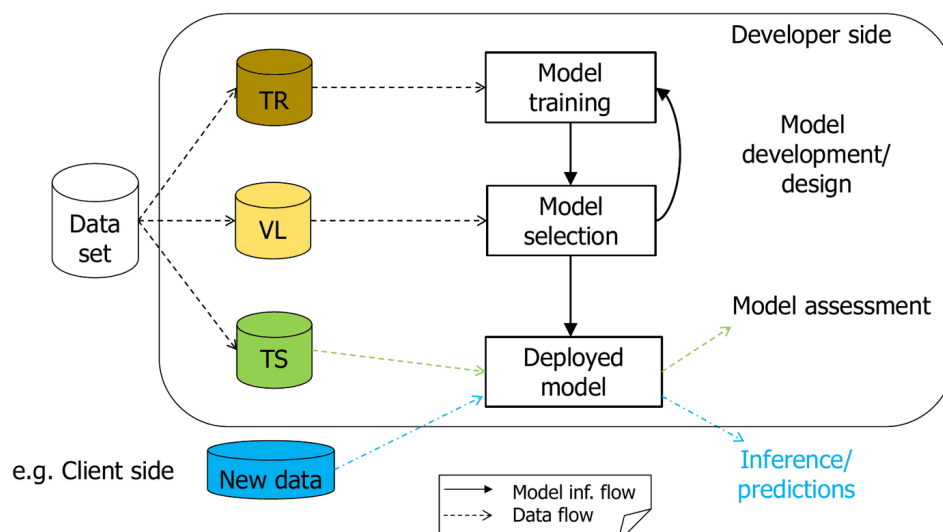


Figura 9.2: Ruolo di TR, VL e TS (ripresa dalla lezione 4).

9.4 Un controesempio istruttivo**Esempio — Il controesempio del target casuale**

- 20–30 esempi, **1000 variabili di input** a valori casuali;
- target **casuale** 0/1.

Si seleziona un modello che usa **una sola** variabile di input, quella che per puro caso “indovina” il target al 99% (o 100%) su qualsiasi successiva suddivisione in TR, VL e TS. Risultato perfetto? Cosa c'è di sbagliato?

Il 99–100% **non** è una buona stima dell'errore di test: quella vera è il **50%** (il target è casuale, nessun modello può fare meglio di una moneta). Su un test set esterno davvero nuovo si otterrebbe il 50%.

Gli errori sono due:

1. la stima dell'errore su TR e VL **non** è una buona stima del rischio;

2. usare l'intero dataset per la selezione delle feature o del modello **pregiudica la stima** (stima distorta, detta *subset selection bias*): il test set è stato usato **implicitamente** all'inizio, nella scelta della variabile. Il test set va separato **in anticipo**, prima di **qualsiasi** model selection, inclusa la **feature selection**.

Con 1000 variabili casuali e soli 20 pattern, è quasi certo che una variabile coincida per caso con il target sulla maggior parte dei pattern. Poiché la scelta è fatta guardando **tutti** i dati, quella variabile “funziona” su qualunque partizione successiva. Su pattern davvero nuovi (TS1, TS2, TS3 nella tabella della slide) la stessa variabile ha accuratezze del 100%, 33%, 66%: puro caso.

Attenzione — Attenzione

Qui si discute la **correttezza della stima**, non la possibilità di risolvere il task. La K-fold CV e altre tecniche **non** risolvono il problema se la selezione è fatta prima della suddivisione (anzi, possono confondere ancora di più).

9.5 Grid search

Gli **iperparametri** sono parametri non appresi direttamente dall'algoritmo; si fissano cercando nello **spazio degli iperparametri**, naturalmente tramite model selection su un validation set.

La **grid search esaustiva** genera tutti i candidati da una griglia di valori. Esempio banale con numero di unità e λ :

Unità λ	0,1	0,01	0,001
1 unità	Res1	Res4	Res7
10 unità	Res2	Res5	Res8
100 unità	Res3	Res6	Res9

Se il migliore risultato (sul validation) è Res3, vince la configurazione (100 unità, $\lambda = 0,1$). Va **automatizzata**, ed è facile da **parallelizzare** (le prove sono indipendenti).

Il costo può essere alto: è il prodotto cartesiano dei valori, cioè $(\text{\#valori})^{\text{\#iperparametri}}$. Con 3 valori per 2 iperparametri sono $3^2 = 9$ prove; con 5–6 iperparametri da 10 valori ciascuno diventa proibitivo. Per questo:

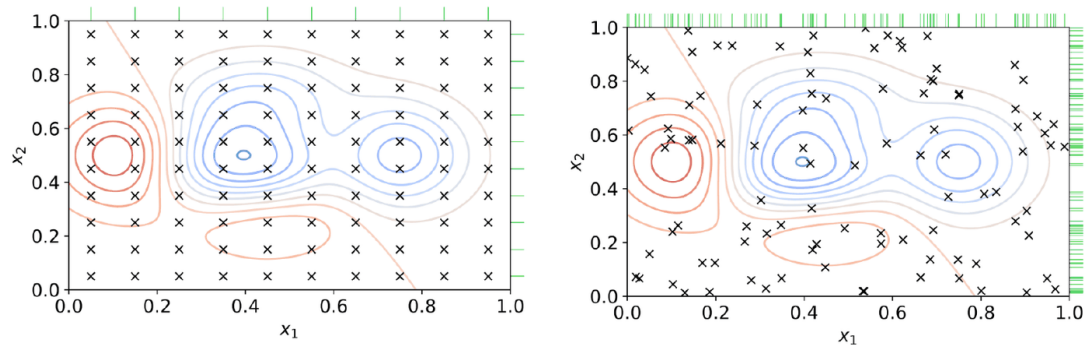
- si possono fissare alcuni iperparametri in una fase sperimentale preliminare, se si mostrano poco rilevanti (con cautela, vedi l'errore della selezione sequenziale nella parte 3);
- si usano **due (o più) livelli di grid search annidate**: prima una ricerca **grossolana** su tutte le combinazioni (ad esempio con valori a crescita esponenziale) per trovare le regioni buone, poi ricerche **più fini** su intervalli sempre più piccoli, sempre considerando **tutti** gli iperparametri significativi insieme.

9.5.1 Alternative alla grid search

La grid search è molto utile per **osservare il comportamento** del modello al variare degli iperparametri (fondamentale per capire), ma è costosa e scala male con il numero di iperparametri. Alternative:

- **ricerca casuale** (Bergstra e Bengio, 2012): evita di ricampionare gli stessi valori di un iperparametro influente quando altri non lo sono, e permette di fissare il **budget** di prove indipendentemente dal numero di iperparametri;
- **ricerca automatica**: approcci bayesiani, evolutivi, metodi di valutazione delle configurazioni (es. Hyperband).

Librerie come Keras Tuner e scikit-learn includono molte alternative (verso l'AutoML), ma non vanno usate acriticamente: la grid search insegna di più al primo approccio.



Grid search for 2 hyperpar.s.: a total of 100 (10x10) different combinations are evaluated and compared. Blue contours indicate regions with strong results, whereas red ones show regions with poor results.

Random search for 2 hyperpar.s.: 100 different random choices are evaluated. The green bars show that more individual values for each hyperparameter are considered compared to a grid search.

Figura 9.3: Grid search e random search con 100 prove ciascuna: la ricerca casuale esplora molti più valori distinti di ciascun iperparametro (barre verdi).

Suggerimento — Perché la random search funziona

Se uno solo dei due iperparametri conta davvero, la grid 10x10 ne prova solo **10 valori distinti** (le altre 90 prove ripetono gli stessi valori variando l'iperparametro inutile), mentre la ricerca casuale ne prova **100**.

9.6 K-fold cross-validation

9.6.1 Quanti dati servono?

L'hold-out richiede dati sufficienti per tre partizioni significative. Quanti? Dipende dalla complessità del modello e dal rapporto segnale/rumore (ne servono pochi per un modello lineare su un task linearmente separabile senza rumore). Come fare se i dati non bastano? E come evitare di dipendere dalla **particolare partizione** scelta? La K-fold CV aiuta.

Nota — Terminologia

“Cross-validation” indica sia un **approccio** alla model selection basato sulla stima diretta dell'errore tramite ricampionamento (anziché analitica), sia più specificamente le **implementazioni** che stabiliscono come dividere i dati per la selezione e per la valutazione (ad esempio la K-fold CV).

9.6.2 Richiamo

Si divide D in K sottoinsiemi mutuamente esclusivi $D_1, \dots, D_K$; per ogni i si addestra su $D \setminus D_i$ e si valuta su D_i . Così tutti i dati vengono usati sia per l'addestramento sia per la valutazione. Non si ottiene un **unico modello** (ce n'è uno per fold), ma si ottiene anche una **varianza** (deviazione standard) sui fold per la propria classe di modelli: il risultato si riporta come **media** $\pm$ **deviazione standard**. Si può usare sia per il validation set sia per il test set.

Questioni: quanti fold (3, 5, 10, ..., *leave-one-out*)? Il costo computazionale è spesso alto. Si può combinare con un validation set o fare una doppia K-fold CV.

9.6.3 Un esempio completo di selezione e valutazione

1. Dividere i dati in **TR** e **TS** (qui con hold-out, ma potrebbe essere una K-fold).
2. **Model selection**: usare una K-fold CV **interna** al TR (che in ogni fold produce nuovi TR e VL) per trovare i migliori iperparametri con una **grid search**. Per ogni cella della griglia (es. $\lambda = 0,1, 20$ unità) si esegue un'intera K-fold CV; si sceglie la configurazione con il **miglior errore medio di validazione** sui fold.
3. Riaddestrare il modello finale sull'**intero TR** originale.
4. **Model assessment**: valutarlo sul **TS esterno**.

(Si può poi riaddestrare di nuovo su tutti i dati? Lo vedremo nella prossima parte.)

9.7 Casi particolari

9.7.1 Campionamento fortunato o sfortunato

Per non dipendere dalla particolare partizione:

- **Stratificazione**: raggruppare la popolazione in sottogruppi omogenei prima di campionare. Per la classificazione: in ogni partizione (TR e TS in hold-out o CV) ogni classe è rappresentata circa nelle stesse proporzioni del dataset completo.
- Controllare la composizione dei fold: l'**ordine dei dati** potrebbe avere un significato (es. dati ordinati per classe o per tempo).
- **Hold-out o CV ripetuti**: ripetere la suddivisione con campionamenti casuali diversi (es. ripetere 10 volte la CV) e mediare i risultati. Utile soprattutto per **confrontare modelli diversi**.

9.7.2 Pochissimi dati

Con pochissimi dati è difficile dire se un campione è rappresentativo. Oltre alla stratificazione, evitare (o tenere in conto nella valutazione): classi o feature mancanti nei dati di training; classi speciali non campionate in TR o TS; outlier noti che influenzano la media dei risultati di test; all'opposto, la selezione di soli casi "facili".

Anche un **blind test set** può essere fuorviante se proviene da una distribuzione diversa, è misurato con scale o tolleranze diverse, non è pulito o pre-elaborato, o richiede **estrapolazione** (dati fuori range).

9.8 Misure d'errore per la valutazione

Classificazione (vedi le prime lezioni): accuratezza (o tasso d'errore, spesso in %), matrice di confusione, specificità, sensibilità, curva ROC.

Regressione, con residuo $r_i = y_i - o_i$ (y target, o uscita):

- **MSE** = $\text{mean}_i[r_i^2]$ e la sua radice **RMS** (o S);
- **errore assoluto medio** (MAE) = $\text{mean}_i[|r_i|]$;
- **errore assoluto massimo** = $\max_i |r_i|$;
- **coefficiente di correlazione** R (e **coefficiente di determinazione** R^2), ad esempio $R = \sqrt{1 - S^2/S_y^2}$, con $S_y^2 = \text{mean}_i[(y_i - \bar{y})^2]$ varianza del target: misura il grado di dipendenza lineare tra y e o , in $[0, 1]$, dove 1 è il meglio;
- grafici **uscite contro target**;
- test statistici e analisi di significatività.

Nota — Bootstrap

Un altro approccio è il **bootstrap**: ricampionamento casuale **con reinserimento**, ripetuto per ottenere sottoinsiemi diversi (di validazione o di test).

In sintesi — Le due regole d'oro

- Il risultato sul **TR** non è una buona stima di quello su VL e TS.
- Il risultato sul **VL** non è una buona stima di quello sul TS.

Quindi: **non usare i risultati del validation set per la stima del rischio** (model assessment), e **non usare i risultati del test set per la model selection**, in nessuna forma. In una sola regola: *tenere separati gli obiettivi e usare insiemi separati in ogni fase.*

Domande — Possibili domande d'esame

- Differenza tra model selection e model assessment; perché servono insiemi separati?
- Descrivere il controesempio del target casuale: quali sono i due errori?
- Cos'è la grid search? Come si riduce il suo costo? Vantaggi della random search.
- Descrivere la K-fold CV e un esempio completo di selezione e valutazione.
- Cos'è la stratificazione?
- Quali misure d'errore si usano per la regressione?

Validazione (parte 2): schemi formali

Questa seconda parte formalizza l'uso corretto della cross-validation per la model selection e/o la stima del rischio, con una notazione precisa (adattata dalle slide di S. Bengio). Sono concetti semplici, ma una notazione rigorosa evita ambiguità. Non è un ricettario: serve a **razionalizzare** e capire in modo sistematico un approccio rigoroso, per poi essere ragionevoli. Il tempo per eseguire gli esperimenti dipende soprattutto dalle scelte di validazione: bisogna evitare sia una valutazione troppo grossolana sia un processo di valutazione infinito. In caso di dubbi, questi schemi sono comunque meglio della fantasia.

Si parla di selezione tra configurazioni di iperparametri (indicate con θ), ma gli stessi schemi valgono per scegliere tra **modelli diversi** (θ può rappresentare anche il tipo di modello).

Suggerimento — Tenere a mente il controesempio

Dalla parte 1: (1) le stime d'errore su TR e/o VL **non** sono buone stime del rischio; (2) usare l'intero dataset per la selezione di feature o modelli **pregiudica** la stima. Negli schemi che seguono si vede esattamente dove questi errori verrebbero commessi.

10.1 Notazione

Dati: $Z_1, \dots, Z_l$ è un campione casuale di una distribuzione sconosciuta $p(z)$, con tutti gli Z_i **indipendenti e identicamente distribuiti** (i.i.d.). $D_r = \{z_1, \dots, z_r\}$ è una particolare istanza con r elementi.

Task: classificazione, $Z = (X, Y) \in \mathbb{R}^n \times \{-1, +1\}$; regressione, $Z = (X, Y) \in \mathbb{R}^n \times \mathbb{R}$.

Loss (“interna”): classificazione $L((x, y), h) = 0$ se $h(x) = y$, 1 altrimenti; regressione $L((x, y), h) = (h(x) - y)^2$.

Obiettivo: minimizzare il **rischio atteso** (errore vero) su H :

$$R(h) = \mathbb{E}_Z[L(z, h)] = \int_Z L(z, h) p(z) dz, \quad h^* = \arg \min_{h \in H} R(h).$$

Il problema: $p(z)$ è sconosciuta e non abbiamo accesso a tutti gli $L(z, h)$.

Rischio empirico su un insieme finito D_r :

$$R_{emp}(h, D_r) = \frac{1}{r} \sum_{i=1}^r L(z_i, h).$$

Attenzione — Uso generale di R_{emp}

Qui R_{emp} è inteso in senso generale, come **surrogato di R** su un insieme finito di dati, usato per il training, la model selection o l'assessment a seconda della partizione. Da non confondere con il R_{emp} della SLT, che è specificamente l'errore di training: in questa notazione è $R_{emp}(h, D_{TR})$.

10.2 Metodologia: identificare l'obiettivo

1. Dare il **miglior modello** ottenibile da un dataset? → serve la **model selection**.
2. Dare le **prestazioni attese** di un modello ottenuto per minimizzazione del rischio empirico? → serve la **model assessment** (stima del rischio).
3. Dare **entrambi**? → servono tutte e due.

Il dataset originale D_l verrà usato per questi diversi scopi.

10.3 Model selection

10.3.1 Hold-out

In sintesi — Model selection con hold-out

- Scegliere uno spazio delle ipotesi con iperparametro θ .
- Dividere D_l in $D_{TR} = \{z_1, \dots, z_{tr}\}$ e $D_{VL} = \{z_{tr+1}, \dots, z_{tr+vl}\}$, con $tr + vl = l$.
- Per ogni valore θ_m [grid search]:
 - $h_{\theta_m}^*(D_{TR}) = \arg \min_{h \in H_{\theta_m}} R_{emp}(h, D_{TR})$ [training];
 - stimare $R(h_{\theta_m}^*)$ con $R_{emp}(h_{\theta_m}^*, D_{VL}) = \frac{1}{vl} \sum_{z_i \in D_{VL}} L(z_i, h_{\theta_m}^*(D_{TR}))$.
- Scegliere $\theta_m^* = \arg \min_{\theta_m} R(h_{\theta_m}^*)$ [miglior risultato sul VL].
- Restituire $h^*(D_l) = \arg \min_{h \in H_{\theta_m^*}} R_{emp}(h, D_l)$ [riaddestramento su tutti gli l dati].

La stima di R sul VL serve **solo** alla model selection e **non viene restituita** come stima del rischio: restituirla sarebbe esattamente l'errore (1) del controesempio.

10.3.2 K-fold cross-validation

In sintesi — Model selection con K-fold CV

- Scegliere uno spazio delle ipotesi con iperparametro θ .
- Dividere D_l in K parti distinte e uguali $D_1, \dots, D_K$; sia $\bar{D}_k$ il complementare di D_k .
- Per ogni valore θ_m [grid search]:
 - per ogni parte D_k [ciclo sui fold]:
 - * $h_{\theta_m}^*(\bar{D}_k) = \arg \min_{h \in H_{\theta_m}} R_{emp}(h, \bar{D}_k)$ [training];
 - * stimare $R(h_{\theta_m}^*(\bar{D}_k))$ con $R_{emp}(h_{\theta_m}^*(\bar{D}_k), D_k) = \frac{1}{|D_k|} \sum_{z_i \in D_k} L(z_i, h_{\theta_m}^*(\bar{D}_k))$;
 - stimare $R(h_{\theta_m}^*(D_l))$ con la media $\frac{1}{K} \sum_k R(h_{\theta_m}^*(\bar{D}_k))$.
- Scegliere $\theta_m^* = \arg \min_{\theta_m} R(h_{\theta_m}^*(D_l))$ [migliore su tutte le CV].
- Restituire $h^*(D_l) = \arg \min_{h \in H_{\theta_m^*}} R_{emp}(h, D_l)$ [riaddestramento].

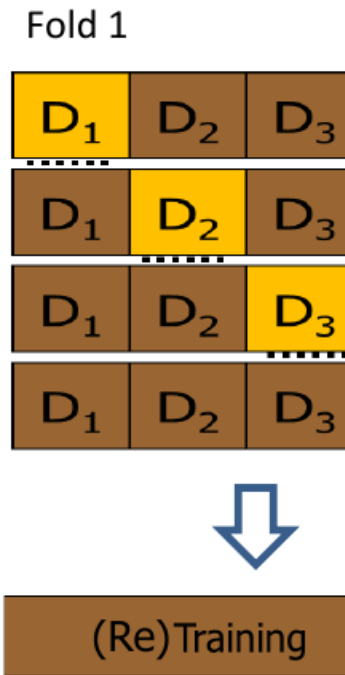


Figura 10.1: K-fold CV per la model selection, seguita dal riaddestramento.

Attenzione — Attenzione all'ordine

Ogni iterazione **riparte da zero**, con addestramento e stima locali alla suddivisione. Se ad esempio si usasse un modello addestrato nello split 1 per valutare lo split 2, si userebbe come validazione dello split 2 una parte (D_2) che era stata usata per addestrare nello split 1.

10.4 Stima del rischio (model assessment)

10.4.1 Hold-out

In sintesi — Model assessment con hold-out

- Dividere D_l in D_{TR} e D_{TS} , con $tr + ts = l$. **Il test set va separato ADESSO** (non dopo): altrimenti si commette l'errore (2) del controesempio.
- $h^*(D_{TR}) = \arg \min_{h \in H} R_{emp}(h, D_{TR})$ [training; può includere una model selection interna su D_{TR}].
- Stimare $R(h^*(D_{TR}))$ con $R_{emp}(h^*(D_{TR}), D_{TS}) = \frac{1}{ts} \sum_{z_i \in D_{TS}} L(z_i, h^*(D_{TR}))$ [**errore di test**].

10.4.2 K-fold cross-validation

In sintesi — Model assessment con K-fold CV

- Dividere D_l in K parti distinte e uguali.
- Per ogni parte D_k [ciclo sui fold]:
 - $h^*(\bar{D}_k) = \arg \min_{h \in H} R_{emp}(h, \bar{D}_k)$ [training; può includere una model selection];

- stimare $R(h^*(\bar{D}_k))$ con $R_{emp}(h^*(\bar{D}_k), D_k)$ [errore di test di ogni iterazione].
- Stimare $R(h^*(D_l))$ con la media $\frac{1}{K} \sum_k R(h^*(\bar{D}_k))$.

Con $K = l$ si ha la **leave-one-out CV**. Non si violano i principi: i risultati di test non vengono mai usati per addestrare o selezionare, in nessuna iterazione.

10.5 Combinare model selection e model assessment

Quando si vogliono sia il modello migliore sia il suo rischio atteso, bisogna combinare i metodi. Ad esempio:

- **hold-out** con training-validation-test (servono tre insiemi separati);
- **K-fold CV + test**: cross-validation sul training set, poi test su un insieme separato;
- **K-fold CV esterna per il test**, con hold-out interno per la model selection;
- **double (nested) cross-validation**: per ogni fold esterno, una cross-validation annidata sugli altri $K - 1$.

Altri aspetti: considerare le **diverse prove** (inizializzazioni diverse di una rete) nel calcolo delle medie su TR, VL, TS. Più in generale: confrontare i risultati con altri metodi, usare **test statistici** per verificarne la significatività, verificare il modello su più dataset.

10.5.1 Hold-out training-validation-test

In sintesi — Schema TR-VL-TS

- Dividere D_l in D_{TR} , D_{VL} , D_{TS} .
- Per ogni θ_m : addestrare $h_{\theta_m}^*(D_{TR})$ e calcolare $R_{emp}(h_{\theta_m}^*(D_{TR}), D_{VL})$.
- Scegliere θ_m^* con il miglior risultato sul VL.
- Riaddestrare $h^*(D_{TR} \cup D_{VL})$ con θ_m^* .
- Stimare R con $\frac{1}{ts} \sum_{z_i \in D_{TS}} L(z_i, h^*(D_{TR} \cup D_{VL}))$.

Il riaddestramento può usare una parte dei dati come VL per l'early stopping.

Domande — Si può riaddestrare ancora su tutto D_l ?

Il riaddestramento finale su tutti i dati (TR+VL+TS) **non usa** i risultati del test per alcuna scelta: il modello finale avrà in generale prestazioni pari o migliori della stima (più dati), che resta valida come stima (leggermente pessimistica) del rischio della procedura. L'importante è che il test non abbia influenzato nessuna decisione.

10.5.2 K-fold CV per la selezione + hold-out per il test

In sintesi — Schema CV + test

- Dividere D_l in D_{TR} (il *design set*) e D_{TS} .
- Per ogni θ_m : stimare $R(h_{\theta_m}^*(D_{TR}))$ con una K-fold CV su D_{TR} (errore di validazione medio sui fold).
- Scegliere θ_m^* con la migliore stima.

- Riaddestrare $h^*(D_{TR})$.
- Stimare R con l'errore su D_{TS} .

10.5.3 K-fold CV per il test con hold-out interno per la selezione

In ogni riga della K-fold esterna, un fold fa da test e il resto viene diviso (arbitrariamente e con shuffling) in TR e VL per la model selection. La descrizione in meta-algoritmo è lasciata per esercizio.

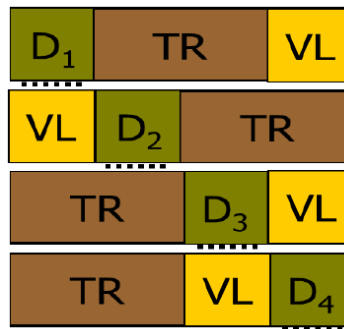


Figura 10.2: K-fold CV esterna per il test, con hold-out TR/VL interno per la model selection (il riaddestramento non è mostrato).

Nota — Nessun modello unico

Questo processo **non fornisce un modello finale**: dà solo una stima del rischio della **classe** di modelli considerata, perché potenzialmente ogni riga seleziona un modello diverso.

10.5.4 Double (nested) K-fold CV

In sintesi — Double K-fold CV

- Dividere D_I in K parti distinte e uguali.
- Per ogni parte D_k [**ciclo esterno**]:
 - scegliere $h^*(\bar{D}_k)$ con una **K-fold CV interna** su $\bar{D}_k$ (con K' eventualmente diverso da K) [**ciclo interno**];
 - stimare $R(h^*(\bar{D}_k))$ con $R_{emp}(h^*(\bar{D}_k), D_k)$ [errore di test per ogni split esterno].
- Stimare $R(h^*(D_I))$ con la media sui fold di test.

Anche qui **non si ottiene un modello finale unico**, perché ogni ciclo esterno può scegliere iperparametri diversi: si ottiene una stima del rischio della classe di modelli/algoritmo, con anche la sua **varianza** (deviazione standard sui fold). Se serve un modello finale unico si esegue una **model selection separata** (hold-out o K-fold CV) e un eventuale riaddestramento finale.

Questo non viola le regole: l'errore di test è già stato stimato e i risultati di test non vengono mai usati per la selezione; il modello finale avrà un errore atteso nell'intervallo stimato. Per questo non è semplicemente vero che “il test viene sempre per ultimo”: qui avviene **prima** dell'ultima model selection (anche se può sembrare anti-intuitivo).

La double CV sfrutta meglio **tutti** i dati per addestramento e valutazione (sia per la selezione sia per la valutazione), al prezzo di un costo computazionale elevato.

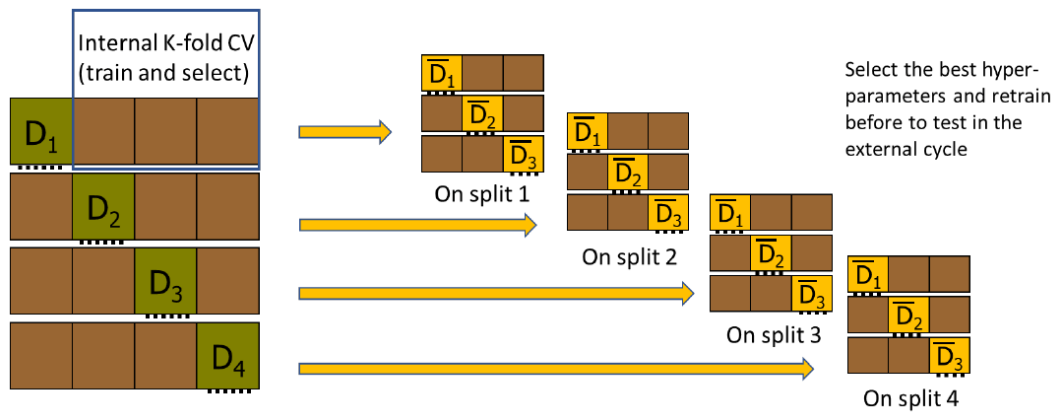


Figura 10.3: Double K-fold CV: in ogni split esterno una CV interna seleziona gli iperparametri.

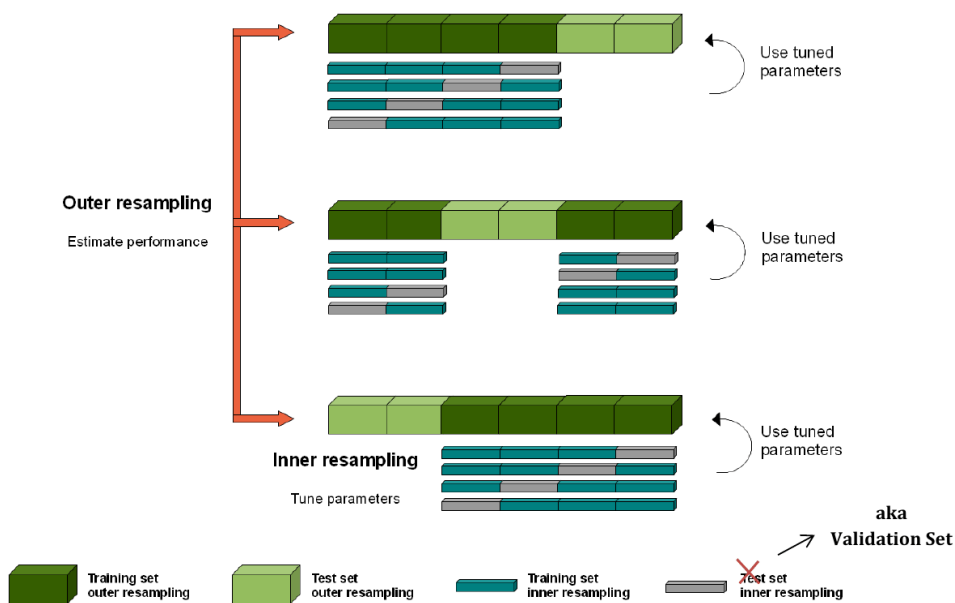


Figura 10.4: Un'altra vista della double CV: resampling esterno per stimare le prestazioni, interno per regolare i parametri.

In sintesi — Sintesi

Prima si identifica l'obiettivo (modello migliore, stima delle prestazioni, o entrambi), poi si sceglie lo schema. In tutti gli schemi corretti: il **test è separato prima** di ogni scelta, le stime sul **VL non vengono restituite** come stime del rischio, ogni iterazione di CV **riparte da zero**. Gli schemi combinati più usati sono TR-VL-TS, CV+test e la double CV.

Domande — Possibili domande d'esame

- Scrivere l'algoritmo di model selection con K-fold CV.
- Scrivere l'algoritmo di model assessment con K-fold CV.
- Descrivere la double (nested) CV. Restituisce un modello?
- Collegare gli errori (1) e (2) del controesempio agli schemi formali.

Nested/Double CV (a further different view, also with the inner hold-out case)

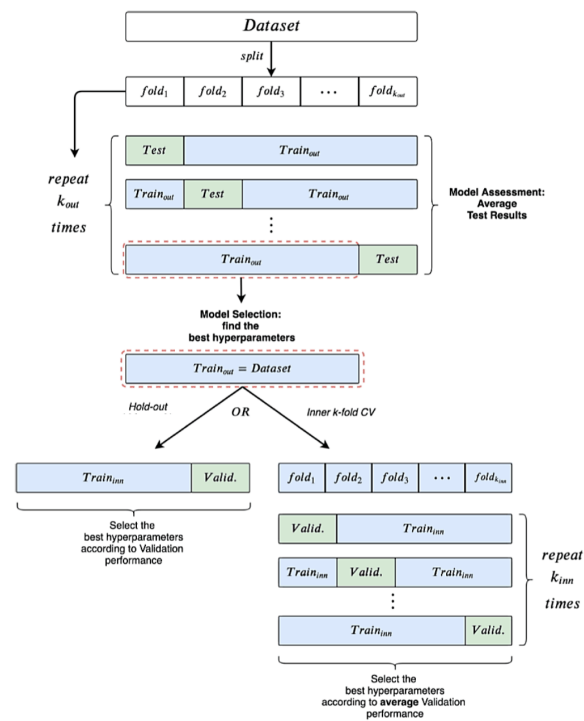


Figura 10.5: Nested CV, anche nella variante con hold-out interno.

Capitolo 11

Validazione (parte 3): errori tipici e FAQ

L'ultima parte sulla validazione raccoglie consigli pratici sull'uso della cross-validation, gli **errori più frequenti** (soprattutto nei progetti) e alcuni esempi di model selection, in parte tratti dalle slide di Andrew W. Moore (CMU).

11.1 Arresto dell'addestramento

11.1.1 Non fissare un numero arbitrario di epoche

Attenzione — Errore frequente

Evitare di fermare l'addestramento di una rete dopo un **numero fisso e arbitrario di epoche**.

Perché:

- se è piccolo, ci si può fermare **troppo presto** (underfitting, o si favoriscono i learning rate alti che richiedono meno epoche);
- se è grande, ci si può fermare **troppo tardi** (overfitting o tempo perso);
- non può andare bene lo stesso valore per tutte le configurazioni e le iterazioni di una K-fold CV;
- e quale numero usare nel riaddestramento finale?

Anche **selezionare il numero di epoche tramite model selection** non è la pratica migliore: è meglio di un numero fisso ma ancora impreciso (per i problemi con la K-fold CV e perché esistono criteri migliori). Le soluzioni che funzionano bene con un sistema **ben regolarizzato** sono quelle viste in [08 - Reti neurali \(parte 2\) - addestramento in pratica](#): guardare la convergenza del training (soglia sulla diminuzione dell'errore o sul gradiente), oppure l'**early stopping** se si può/vuole usare un validation set.

11.1.2 Early stopping

Attenzione — Early stopping e model selection

Il criterio di early stopping (ES) fa **parte della model selection** (non va applicato sul TS): in linea di principio richiede un **VL set ogni volta** che si addestra il modello.

È delicato: usa parte dei dati solo per decidere quando fermarsi, senza dare un valore concreto di iperparametro, e può diventare complesso dentro una CV. Il problema: come scegliere

il punto di arresto per il **riaddestramento finale**, se ogni fold della CV ha un punto di arresto diverso?

- Si potrebbe considerare il numero di epoche come iperparametro e prendere la **media** sui fold. Ma cambiando la quantità di dati nel riaddestramento il punto di arresto ottimo può cambiare (ad esempio con più dati per epoca, e quindi più aggiornamenti per epoca in on-line, servono meno epoche): si rischia sotto- o sovra-addestramento.
- **Molto meglio**: prendere la **media dell'errore di training** nel punto migliore di validazione e usarla come soglia nel riaddestramento, per raggiungere lo stesso livello di fitting (evitando effetti di sotto-addestramento).
- Altrimenti serve un VL set ogni volta: ad esempio CV con ES attivo in ogni fold di validazione (insieme agli altri iperparametri); trovati i migliori iperparametri, si riaddestra sul dataset ricomposto usando ancora una parte dei dati come VL per l'ES.

È una scelta: usare l'ES oppure affidarsi a **modelli regolarizzati** con criteri di arresto standard (es. nessun miglioramento sul training). In ogni caso la regolarizzazione aiuta a evitare un forte sovra-addestramento e riduce la dipendenza dall'ES.

11.2 Inizializzazione casuale e model selection

Inizializzazioni diverse dei pesi portano a **modelli diversi**. Come scegliere quello finale (scelta che va riportata nel report)?

- Si calcolano **media e varianza** dell'errore/accuratezza sulle diverse prove: un caso casuale cadrà in quell'intervallo, quindi il rischio è sotto controllo, a meno di una varianza alta (che comunque è un segnale negativo, vedi la lezione su bias e varianza).
- Si possono usare **tutti** i modelli, con un **ensemble** (media delle uscite o voto).
- Se se ne sceglie **uno**, quella è una **model selection** e serve un VL set: il **migliore** (errore minimo sul VL), la **mediana** (scelta più cauta), oppure uno **casuale** (in questo caso non serve un VL set).
- Si possono anche scartare i casi “sfortunati” guardando solo l'errore di training.

11.3 Selezione sequenziale degli iperparametri

Attenzione — Errore frequente

Selezionare gli iperparametri **uno alla volta**, in sequenza, introduce un **bias legato all'ordine**. Esempio: scegliere il miglior η con 10 unità e poi usarlo anche con 100 unità.

Il procedimento corretto:

1. prove iniziali per determinare gli **intervalli** giusti (ad esempio guardando la stabilità del training) e gli iperparametri più **influenti** (ad esempio guardando la varianza dei risultati);
2. ricerca su una **griglia con tutte le combinazioni** degli iperparametri (più iperparametri e valori → tabella più grande);
3. eventuali griglie **annidate**, da grossolane a fini, su intervalli sempre più piccoli, ma considerando **tutti gli iperparametri significativi insieme**, per tenere conto degli effetti incrociati.

Ma: ragionare, selezionare gli aspetti rilevanti, non affidarsi solo ad approcci automatici sistematici e costosi. I vincoli di tempo del progetto dovrebbero spingere verso schemi semplici e veloci, ma corretti e ragionevoli.

11.4 Cosa fare dopo selezione e valutazione?

Domande — Posso riprogettare il modello in base al risultato sul test?

No: non è il modo corretto e non è davvero vantaggioso (a meno di avere nuovi dati per un nuovo test set). Ripetendo questo procedimento sugli stessi dati si può ottenere qualunque risultato si voglia, adattando il modello a quel dataset: il modello può diventare perfetto su di esso e non nell'uso futuro, e **non si sta più stimando il rischio**.

Non bisogna regolare il predittore (a mano o automaticamente) sulla base dell'errore di test, perché non potremmo farlo sugli esempi futuri. Nelle competizioni il blind test impedisce proprio questo. La valutazione su un test interno resta comunque utile per **verificare il proprio approccio**.

La tentazione di decidere in base ai risultati è un problema “psicologico”, esterno ai metodi di ML. Almeno esserne consapevoli, oppure meglio:

- per fare un **buon modello** bisogna fare una **buona model selection**;
- per stimare le **prestazioni reali** bisogna fare una stima **rigorosa** sul test.

11.5 La cross-validation può andare in overfitting

Un uso intensivo della cross-validation può portare a overfitting? **Sì**, e conserva i problemi già visti. Esempio con la feature selection:

- dataset con 20–30 record e 1000 attributi;
- si provano 1000 modelli di regressione lineare, ognuno con un solo attributo;
- il migliore dei 1000 sembra buono (sul validation set)... ma sarebbe sembrato buono anche con un output **puramente casuale**!

Oppure (uso sbagliato): approssimando un target casuale si trova, con una K-fold CV per il test, un errore del 3% invece del vero 50%, dopo aver scelto le feature per miglior correlazione con i target. **Perché?** La selezione è stata fatta su **tutti** gli esempi.

Cosa fare? Tenere da parte i campioni **prima** di selezionare:

- tenere un **test set aggiuntivo** prima di qualsiasi model selection e verificare che il modello migliore funzioni bene anche su di esso;
- per la model selection con CV: usare “**CV + test**” o la **double CV** (rifacendo la selezione per ogni fold);
- per l'assessment con CV: fare la **feature selection dentro ogni fold**. Così si ottiene la stima corretta, circa il 50% per il task casuale.

11.6 Quale cross-validation?

	Svantaggi	Vantaggi
Test set (hold-out)	varianza: stima inaffidabile delle prestazioni future (fortuna/sfortuna)	economico
Leave-one-out	costoso; ha comportamenti strani	non spreca dati
10-fold	spreca il 10% dei dati; 10 volte più costoso del test set	spreca solo il 10%; solo 10 volte più costoso invece di R volte
3-fold	spreca più dati del 10-fold; più costoso del test set	leggermente meglio del test set
R-fold ($R = l$)	identico al leave-one-out	

11.7 Esempi di model selection

11.7.1 Esempio 1: modelli diversi

i	f_i	TRAINERR	10-FOLD-CV-ERR	VL	Choice
1	f_1				
2	f_2				
3	f_3				
4	f_4				
5	f_5				
6	f_6				

Figura 11.1: Scelta tra sei classi di modelli con la 10-fold CV.

Si sceglie la classe di modelli con il **miglior punteggio di CV**, la si riaddestra su tutti i dati, e quello è il modello predittivo da usare. La scelta si fa guardando **solo l'errore di validazione**, **non** l'errore di training (né una media dei due).

11.7.2 Esempio 2: numero di unità nascoste

Algorithm	TRAINERR	10-FOLD-CV-ERR	VL	Choice
0 hidden units				
1 hidden units				
2 hidden units				
3 hidden units				
4 hidden units				
5 hidden units				

Figura 11.2: Scelta del numero di unità nascoste di una rete con uno strato nascosto.

L'errore di training decresce sempre con il numero di unità, ma l'errore di CV ha un minimo (qui 2 unità). Nella pratica la ricerca si fa su valori come 10, 20, 30... (o con altri passi).

11.7.3 Esempio 3: K-NN

Algorithm	TRAINERR	10-fold-CV-ERR	VL	Choice
$K=1$				
$K=2$				
$K=3$				
$K=4$				
$K=5$				
$K=6$				

Figura 11.3: Scelta di k per il K-NN con leave-one-out CV.

Perché la **leave-one-out** per il K-NN e la 10-fold per le reti? Per ragioni **computazionali**: per il K-NN (e gli altri metodi non parametrici) la LOOCV costa quanto una normale predizione (non c'è un modello da addestrare). E perché fermarsi a $K = 6$? Solo perché le cose sembravano peggiorare al crescere di K ; e non c'è garanzia che un ottimo locale di K rispetto alla LOOCV sia quello globale (la relazione può essere molto irregolare). Per esplorare molti valori si possono usare valori a crescita esponenziale ($K = 1, 2, 4, 8, 16, \dots$) o una ricerca *hill-climbing* partendo da un valore iniziale.

11.8 Conclusioni pratiche

- **Hold-out semplice** è molto usato: a volte il test set è già dato separatamente o “preso” da applicazioni precedenti. Attenzione con pochi dati e controllare i tassi d'errore su TR e VL.
- La **5- o 10-fold CV** è raccomandata come buon compromesso (tra bias e varianza della stima) rispetto alla leave-one-out (Hastie et al.).

Nella prossima lezione si torna alla stima **analitica** dell'errore atteso, con la Structural Risk Minimization e la VC-dimension: offre spunti interessanti per la selezione e il confronto tra modelli (dove conta la **dimensione relativa** degli errori), in alternativa alla valutazione puramente empirica.

Domande — Possibili domande d'esame

- Perché non bisogna fermare l'addestramento dopo un numero fisso di epoche?
- Come si gestisce l'early stopping con la cross-validation e il riaddestramento finale?
- Come scegliere il modello finale con più inizializzazioni casuali?
- Cos'è l'errore della selezione sequenziale degli iperparametri?
- La cross-validation può andare in overfitting? Come evitarlo?
- Confrontare hold-out, K-fold e leave-one-out.

Capitolo 12

Statistical Learning Theory e VC-dimension

Dopo le tecniche empiriche di validazione, si torna alla **stima analitica** dell'errore atteso (di validazione, per la model selection, o di test, per l'assessment), introdotta in [04 - Generalizzazione e validazione \(introduzione\)](#). Questa lezione definisce formalmente la **VC-dimension**, la calcola per alcune classi di ipotesi, presenta il bound della SLT e il principio della **Structural Risk Minimization** (SRM), che porterà alle SVM.

Ricordiamo le alternative per stimare l'errore:

- **analitiche**: AIC, BIC (limitati ai modelli lineari nei parametri), MDL, **SRM con VC-dimension**;
- **per ricampionamento**: cross-validation, bootstrap.

12.1 La VC-dimension

Per ottenere un bound analitico dobbiamo valutare lo spazio delle ipotesi H . Serve una misura della sua **complessità** (capacità, potere espressivo, flessibilità) che funzioni anche per H **infiniti** (come i modelli lineari, con pesi reali): non si può quindi usare $|H|$. L'idea di Vapnik e Chervonenkis è misurare invece **il numero di istanze distinte che H riesce a discriminare completamente**.

Intuitivamente, nel caso della classificazione: *quanto H riesce a discriminare dei punti?* Qual è il **numero massimo di punti che si possono classificare correttamente, senza errori, per tutte le possibili etichettature?**

12.1.1 Shattering

Sia X lo spazio delle istanze, con N punti (attenzione: N qui è il numero di istanze considerate, che può essere diverso dal numero di dati l), e sia H lo spazio delle ipotesi per la classificazione binaria.

- Esistono 2^N possibili **dicotomie** (partizioni, o etichettature dei punti con -1 o $+1$).
- Una dicotomia è **rappresentata** in H se esiste un'ipotesi $h \in H$ che la realizza.

Definizione — Shattering (frammentazione)

H **frammenta** (*shatters*) X se e solo se H può rappresentare **tutte** le possibili dicotomie su X (con errore zero): per ogni possibile etichettatura dei punti esiste un'ipotesi $h \in H$ consistente con essa.

Esempio — Tre punti nel piano e le rette

Consideriamo 3 punti in $\mathbb{R}^2$ e come H l'insieme delle rette, $h(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x} + w_0)$. Una specifica dicotomia (ad esempio due punti a -1 e uno a $+1$) è rappresentabile se esiste una

retta che separa correttamente i punti. Le dicotomie possibili sono $2^3 = 8$.

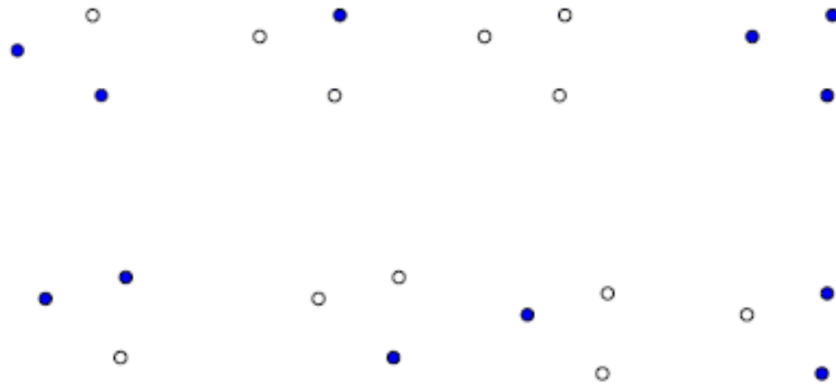


Figura 12.1: Tutte le $2^N = 8$ dicotomie di 3 punti in $\mathbb{R}^2$.

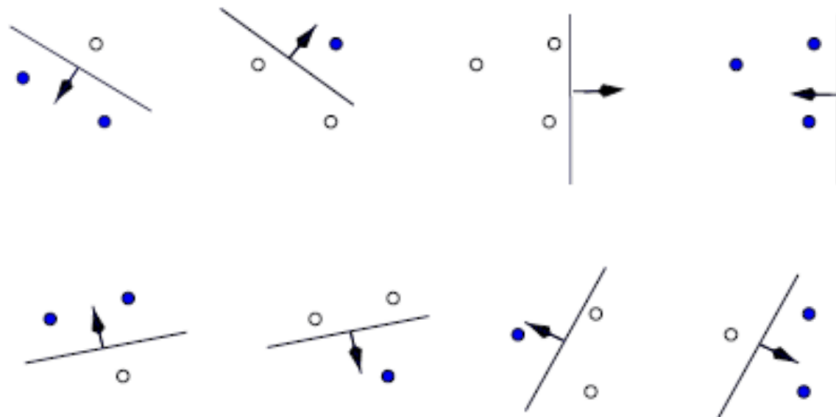


Figura 12.2: Per ogni dicotomia esiste una retta separatrice: le funzioni lineari frammentano questi 3 punti.

12.1.2 Definizione di VC-dimension

Definizione — VC-dimension

La **VC-dimension** di una classe di funzioni H è la **cardinalità massima di un insieme di punti** (in una qualche configurazione) di X che può essere frammentato da H . $VC(H) = p$ significa che:

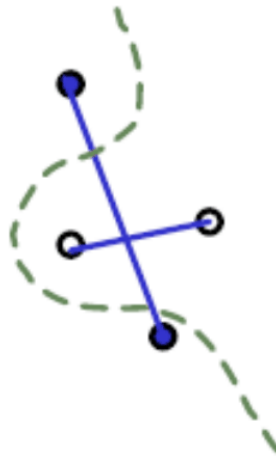
- H frammenta **almeno un** insieme (configurazione) di p punti;
- H **non** frammenta **nessun** insieme di $p + 1$ punti.

Se si possono frammentare insiemi di dimensione arbitrariamente grande, $VC(H) = \infty$.

Attenzione all'asimmetria: per il limite inferiore basta **una** configurazione frammentabile; per il limite superiore bisogna mostrare che **nessuna** configurazione di $p + 1$ punti è frammentabile.

12.1.3 VC-dimension degli iperpiani nel piano

- $VC(H) \geq 3$: l'abbiamo appena mostrato con 3 punti non allineati. Non tutte le configurazioni di 3 punti sono frammentabili: tre punti **allineati** con quello centrale di classe diversa non sono linearmente separabili. Ma basta **una** configurazione frammentabile.
- $VC(H) < 4$: dati 4 punti qualsiasi nel piano, si possono sempre tracciare i 6 segmenti che li collegano a coppie, e due di questi si incrociano (se i 4 punti formano un quadrilatero convesso sono le diagonali; se uno è dentro il triangolo degli altri, si sceglie l'etichettatura con il punto interno di classe diversa). Mettendo nella stessa classe i punti collegati dai segmenti che si incrociano, non si possono separare linearmente: è lo **XOR**.



Conclusion: $VC(H)$

Figura 12.3: Con 4 punti esiste sempre un'etichettatura di tipo XOR non separabile da una retta.

Conclusione: $VC(H) = 3$ per gli iperpiani (LTU) in $\mathbb{R}^2$.

Teorema — VC-dimension degli iperpiani

La VC-dimension della classe degli iperpiani separatori (LTU) in uno spazio n -dimensionale è $n + 1$.

È un limite superiore: vincolando il modello (ad esempio limitando la norma dei pesi, come si vedrà nelle SVM) può diminuire.

12.1.4 Altri esempi

Classe H	VC-dim
Rette in $\mathbb{R}$ (soglie, $\text{sign}(w_1 x + w_0)$)	2
Intervalli su $\mathbb{R}$	2
Cerchi centrati nell'origine in 2D, $\text{sign}(\mathbf{x} \cdot \mathbf{x} - b)$	1
Cerchi centrati nell'origine, $\text{sign}(k \mathbf{x} \cdot \mathbf{x} - b)$	2
Cerchi (raggio e centro liberi) in 2D	3
Cerchi (raggio fisso, centro libero) in 2D	3

Classe H	VC-dim
------------	--------

(Per i cerchi centrati nell'origine con un solo parametro b , il cerchio include i punti vicini all'origine: con due punti a distanze diverse non si può etichettare “+” il più lontano e “-” il più vicino. Aggiungendo k si può invertire l'interno e l'esterno, e la VC-dim sale a 2.)

La VC-dimension può essere difficile da calcolare per modelli con parametri non lineari.

12.1.5 VC-dimension e numero di parametri

La VC-dimension è il numero di parametri liberi? **Sono legati ma non coincidono:**

- nell'esempio dei cerchi, classi con lo **stesso** numero di parametri hanno VC-dim diverse, e viceversa;
- si possono aggiungere parametri **ridondanti** senza cambiare nulla;
- esistono modelli con **un solo parametro e VC-dim infinita** (l'esempio classico è $\text{sign}(\sin(\alpha x))$, che con un α abbastanza grande frammenta qualsiasi insieme di punti opportunamente scelti);
- per il **nearest neighbor** la VC-dim è **infinita** (il 1-NN classifica correttamente qualunque etichettatura del training set).

12.2 Il bound analitico sul rischio

Sia N il numero di dati (l).

Teorema — VC-bound

Con probabilità almeno $1 - \delta$, per ogni $h \in H$ (con $VC < N$):

$$R[h] \leq R_{\text{emp}}[h] + \varepsilon(VC, N, \delta),$$

dove il membro destro è il **rischio garantito** e ε è la **VC-confidence**. Ad esempio, per la loss 0/1:

$$\varepsilon(VC, N, \delta) = \sqrt{\frac{VC \left(\ln \frac{2N}{VC} + 1 \right) - \ln \frac{\delta}{4}}{N}}.$$

Esistono formulazioni diverse per diverse classi di funzioni e task. L'interpretazione è quella già vista: $\varepsilon \rightarrow 0$ al crescere di N , ε cresce con la VC-dim, e il bound su R ha un andamento a U al crescere della VC-dim.

Nella formula si vede bene: ε dipende essenzialmente dal rapporto VC/N (a meno di logaritmi). Raddoppiare i dati ha lo stesso effetto di dimezzare la complessità.

Il bound permette di **stimare l'errore sui dati futuri** basandosi solo sull'errore di training e sulla VC-dimension di H , e la VC-confidence si può calcolare **prima** dell'apprendimento.

Attenzione — Bound “nel caso peggiore”

I bound risultanti sono *worst case*: devono valere per tutte le funzioni e tutti i training set, tranne una frazione δ . Per questo sono spesso molto larghi.

Suggerimento — Una conseguenza pratica

Per molte classi ragionevoli (es. approssimatori lineari) la VC-dim è **lineare nel numero di parametri liberi**. Quindi, per apprendere “bene”, serve un numero di esempi **lineare nella VC-dimension** (e quindi, in questo caso, nel numero di parametri). È una spiegazione teorica di euristiche pratiche come “servono almeno k esempi per parametro”.

12.3 Structural Risk Minimization

La **SRM** usa la VC-dimension come parametro di controllo per minimizzare il bound sul rischio. Assumendo VC-dim finite, si definisce una **struttura annidata** di spazi delle ipotesi ordinati per VC-dim:

$$H_1 \subseteq H_2 \subseteq \dots \subseteq H_n, \quad VC(H_1) \leq \dots \leq VC(H_n).$$

Esempi di strutture:

- reti neurali con numero crescente di **unità nascoste** (circa), ma anche con numero crescente di **epoche** (sappiamo che conta anche la grandezza dei pesi: la rete è uno spazio di dimensione variabile);
- polinomi di **grado** M crescente;
- valori crescenti di c , dove $\|\mathbf{w}\| < c$ per la **regolarizzazione** (o equivalentemente controllato da λ);
- alberi di decisione con numero crescente di nodi (circa).

Gli **iperparametri importanti** sono proprio quelli legati alla VC-dimension.

12.3.1 SRM per la model selection

Al crescere della VC-dim l'errore empirico (di training) **diminuisce** e la VC-confidence **aumenta**. La SRM cerca il compromesso sul bound

$$R \leq R_{emp} + \varepsilon(1/l, VC, 1/\delta),$$

cioè sceglie il modello h con il **miglior bound sul rischio vero**.

Attenzione — Attenzione

La VC-confidence può essere **molto conservativa**: anche centinaia di volte più grande dell'effetto di overfitting osservato empiricamente.

12.3.2 A cosa serve il bound

- Fornisce un **fondamento teorico** al ML basato su principi, indipendente dai dettagli del modello o dell'algoritmo; evidenzia il ruolo del **controllo della complessità**; mostra che la scelta ottima della complessità (sulla struttura) minimizza il rischio atteso: è il **principio induttivo della SRM**.
- Come **stima dell'errore predittivo** è usato raramente:
 - può essere molto largo, ma resta utile per la **model selection**, dove conta la **dimensione relativa** (il *ranking*) degli errori;

3-off on the bound

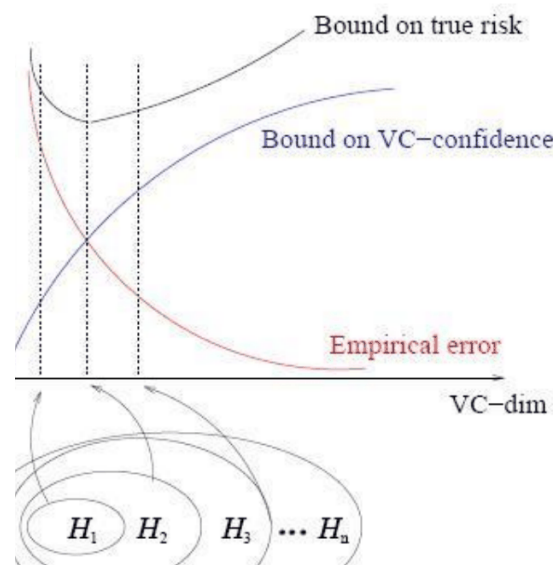


Figura 12.4: Structural Risk Minimization: su una struttura annidata di spazi delle ipotesi si sceglie quello che minimizza il bound sul rischio.

i	H_i	TR err	VC-Conf	Probable upper bound on R	Choice
1	H_1				
2	H_2				
3	H_3				x
4	H_4				
5	H_5				
6	H_6				

Figura 12.5: Model selection con SRM: si sceglie H_3 , con il minimo bound probabile su R .

- è un upper bound probabilistico valido per **tutte** le funzioni della classe, quindi si può cercare nella classe;
 - è difficile calcolare la VC-dim di classi specifiche;
 - il bound grezzo (troppo pessimistico) può non essere adeguato per una valutazione affidabile dell'errore di generalizzazione (**model assessment**); si stanno sviluppando bound più stretti.
- Indica una direzione per sviluppare **nuovi modelli** guidati dalla SRM, verso approcci basati su principi e meno su tentativi empirici.

12.3.3 Due approcci pratici alla SRM

1. **Fissare la struttura/complessità** (e quindi la VC-confidence) e **minimizzare l'errore di training**. Si può usare con le reti neurali. Le euristiche di addestramento possono già realizzare una SRM **implicita** (early stopping, ...); oppure si realizza la SRM con la **re-**

golarizzazione di Tikhonov, minimizzando una loss $R_{emp} +$ termine di complessità, che considera entrambi i termini.

2. **Fissare l'errore di training e minimizzare automaticamente la VC-confidence.** Come? È l'idea delle **SVM** (13 - [Support Vector Machines](#)).

In sintesi — Sintesi

La **VC-dimension** misura la capacità di H come il massimo numero di punti che H riesce a frammentare (classificare correttamente per ogni etichettatura). Per gli iperpiani in $\mathbb{R}^n$ vale $n + 1$; non coincide con il numero di parametri (può essere infinita con un solo parametro, ed è infinita per il NN). Il **VC-bound** $R \leq R_{emp} + \varepsilon(VC, N, \delta)$ lega rischio, errore di training, complessità e numero di dati. La **SRM** sceglie, in una struttura annidata di spazi, quello che minimizza il bound. I bound sono spesso larghi, ma forniscono il fondamento teorico del controllo della complessità.

Domande — Possibili domande d'esame

- Definire shattering e VC-dimension.
- Dimostrare che la VC-dim delle rette nel piano è 3.
- La VC-dim coincide con il numero di parametri liberi? Esempi.
- Scrivere il VC-bound e interpretarne i termini.
- Cos'è la Structural Risk Minimization? Esempi di strutture annidate.
- Quali sono i limiti pratici del bound? A cosa serve comunque?

Capitolo 13

Support Vector Machines

Le **Support Vector Machines** (SVM) sono la risposta alla domanda lasciata aperta dalla SLT (12 - [Statistical Learning Theory e VC-dimension](#)): è possibile fissare l'errore di training e minimizzare **automaticamente** la VC-confidence? L'SVM è ancora una macchina lineare (una LTU), ma sceglie tra tutti gli iperpiani separatori quello che **massimizza il margine**, e questo equivale a controllare la VC-dimension. La lezione procede in quattro passi: SVM lineare *hard margin*, SVM *soft margin* per dati non separabili, **kernel** per problemi non lineari, e infine SVM per la **regressione**.

Nota — Notazione (da Haykin, cap. 6)

N è il numero di esempi (prima l), con indici i e j per i pattern; m è la dimensione del vettore di input (prima n); si usa b invece di w_0 per il bias. Le parti segnate con * nelle slide sono utili per completezza ma non indispensabili per usare il modello (e per un orale standard): in particolare i dettagli della risoluzione con i moltiplicatori di Lagrange.

13.1 Parte I — SVM per la classificazione binaria

Una SVM è:

- una **macchina lineare** (come la LTU o il Perceptron);
- che **massimizza il margine di separazione**;
- e realizza una **Structural Risk Minimization**.

Inizialmente (SVM *hard margin*) si assume che il problema sia **linearmente separabile** e che **non ci siano errori** nei dati.

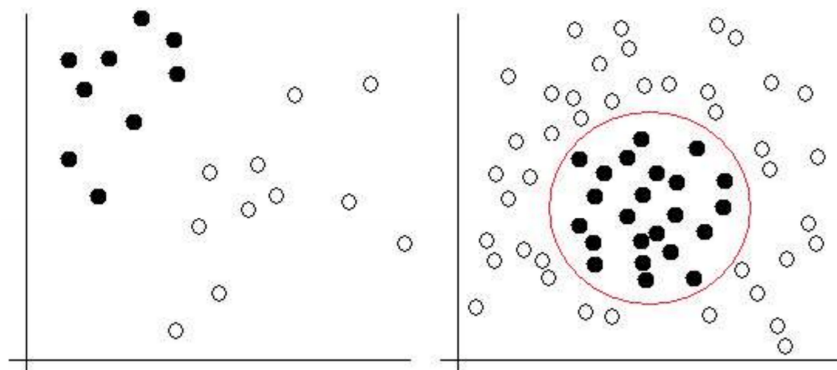


Figura 13.1: Pattern linearmente separabili (sinistra) e non linearmente separabili (destra).

13.1.1 L'iperpiano separatore

Dato il training set $T = \{(\mathbf{x}_i, d_i)\}_{i=1}^N$, si cerca un iperpiano $\mathbf{w}^T \mathbf{x} + b = 0$ che separi gli esempi:

$$\mathbf{w}^T \mathbf{x}_i + b \geq 0 \text{ se } d_i = +1, \quad \mathbf{w}^T \mathbf{x}_i + b < 0 \text{ se } d_i = -1.$$

$g(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$ è la **funzione discriminante** e $h(\mathbf{x}) = \text{sign}(g(\mathbf{x}))$ è l'**ipotesi**.

13.1.2 Il margine di separazione

Definizione — Margine di separazione

Se l'iperpiano è alla stessa distanza dal più vicino esempio positivo e dal più vicino negativo, il **margine di separazione** ρ è il doppio della distanza tra l'iperpiano e il punto più vicino. Lo si può pensare come una “**zona di sicurezza**” attorno al confine.

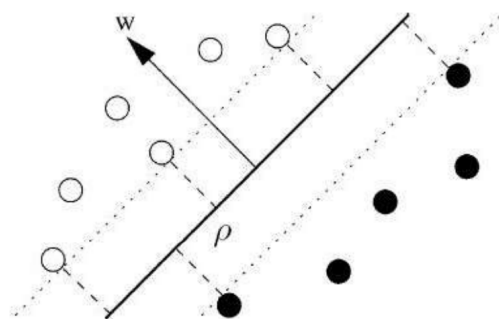


Figura 13.2: Il margine di separazione ρ .

Non tutti gli iperpiani che risolvono il problema sono uguali: al variare dell'iperpiano separatore varia anche il margine.

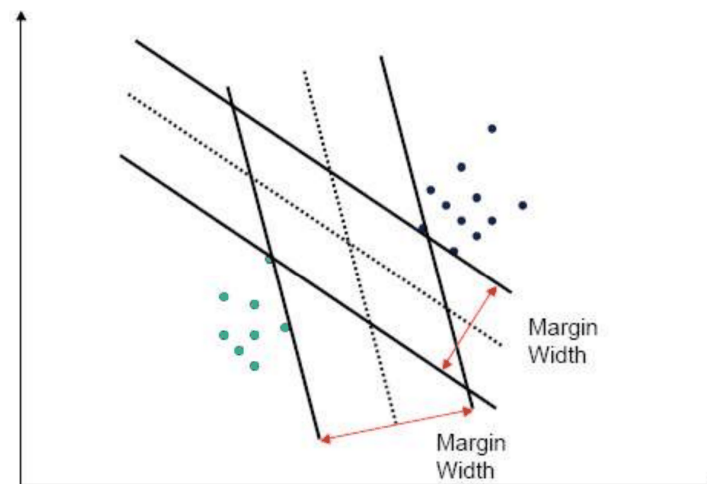


Figura 13.3: Iperpiani separatori diversi hanno margini diversi.

Definizione — Iperpiano ottimo

Iperpiano ottimo $\mathbf{w}_o^T \mathbf{x} + b_o = 0$ (“o” sta per *optimal*) è quello che **massimizza il margine** ρ . Vedremo che $\rho = \frac{2}{\|\mathbf{w}\|}$, quindi

$$\text{massimizzare } \rho \iff \text{minimizzare } \|\mathbf{w}\|.$$

13.1.3 Rappresentazione canonica e support vector

Per la **libertà di scala** (moltiplicando $\mathbf{w}$ e b per una costante l'iperpiano non cambia) possiamo riscrivere $\mathbf{w}$ e b in modo che i punti più vicini all'iperpiano soddisfino $|g(\mathbf{x}_i)| = |\mathbf{w}^T \mathbf{x}_i + b| = 1$. Allora:

$$\mathbf{w}^T \mathbf{x}_i + b \geq 1 \quad \text{se } d_i = +1, \quad \mathbf{w}^T \mathbf{x}_i + b \leq -1 \quad \text{se } d_i = -1,$$

cioè, in forma compatta:

$$d_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 \quad \forall i = 1, \dots, N.$$

Definizione — Support vector

Un **support vector** $\mathbf{x}^{(s)}$ soddisfa il vincolo con l'uguaglianza: $d^{(s)}(\mathbf{w}^T \mathbf{x}^{(s)} + b) = 1$. I support vector sono i punti **più vicini** all'iperpiano, quelli che stanno sul bordo del margine.

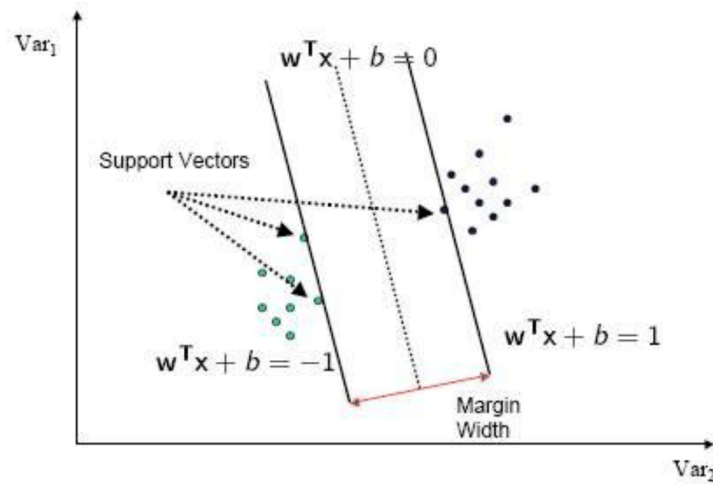


Figura 13.4: I support vector giacciono sulle rette $\mathbf{w}^T \mathbf{x} + b = \pm 1$.

13.1.4 Calcolo del margine

Distanza di un punto dall'iperpiano. Sia r la distanza tra $\mathbf{x}$ e l'iperpiano ottimo. Poiché $\mathbf{w}_o$ è ortogonale all'iperpiano, possiamo scrivere

$$\mathbf{x} = \mathbf{x}_p + r \frac{\mathbf{w}_o}{\|\mathbf{w}_o\|},$$

dove $\mathbf{x}_p$ è la proiezione di $\mathbf{x}$ sull'iperpiano.

Valutando $g(\mathbf{x}) = \mathbf{w}_o^T \mathbf{x} + b_o$:

$$g(\mathbf{x}) = \mathbf{w}_o^T \mathbf{x}_p + b_o + r \mathbf{w}_o^T \frac{\mathbf{w}_o}{\|\mathbf{w}_o\|} = \underbrace{g(\mathbf{x}_p)}_{=0} + r \frac{\|\mathbf{w}_o\|^2}{\|\mathbf{w}_o\|} = r \|\mathbf{w}_o\|,$$

perché $\mathbf{x}_p$ sta sull'iperpiano. Quindi

$$r = \frac{g(\mathbf{x})}{\|\mathbf{w}_o\|}.$$

Margine. Per un support vector positivo $g(\mathbf{x}^{(s)}) = 1$, quindi la sua distanza dall'iperpiano è $r = \frac{1}{\|\mathbf{w}_o\|} = \frac{\rho}{2}$. Da cui:

$$\rho = \frac{2}{\|\mathbf{w}_o\|}.$$

L'iperpiano ottimo massimizza ρ , cioè **minimizza** $\|\mathbf{w}\|$.

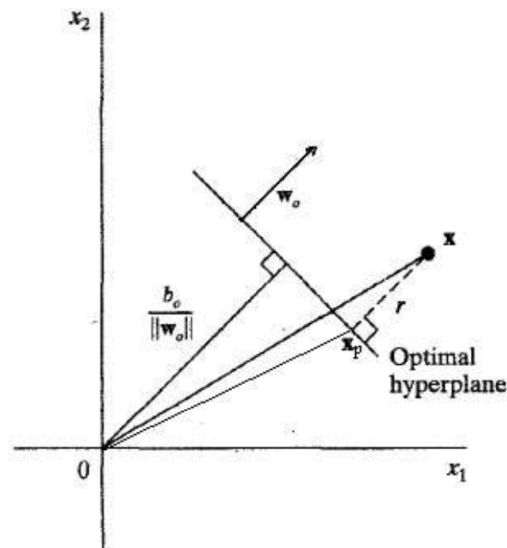


Figura 13.5: Distanza r tra un punto $\mathbf{x}$ e l'iperpiano ottimo.

13.1.5 Il problema di ottimizzazione quadratica (hard margin)

La derivazione formale richiede tecniche di **ottimizzazione vincolata** (assunte note, ad esempio dal corso CM); la programmazione quadratica si considera risolta da pacchetti esistenti. Qui interessa capire come è formulato il problema, la relazione tra support vector e moltiplicatori α , e il risultato finale per $\mathbf{w}_o$.

Definizione — Problema primale — SVM hard margin

Dato $T = \{(\mathbf{x}_i, d_i)\}_{i=1}^N$, trovare $\mathbf{w}$ e b che minimizzano

$$\Psi(\mathbf{w}) = \frac{1}{2} \mathbf{w}^T \mathbf{w} \quad (\text{cioè } \min \|\mathbf{w}\|)$$

soggetto ai vincoli (zero errori di classificazione)

$$d_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 \quad \forall i = 1, \dots, N.$$

- La funzione obiettivo è **quadratica e convessa** in $\mathbf{w}$.
- I vincoli sono **lineari** in $\mathbf{w}$.
- Risolvere questo problema scala con la dimensione dello spazio di input m (il costo effettivo dipende comunque dall'algoritmo del risolutore).

Soluzione con i moltiplicatori di Lagrange*

Si costruisce la **lagrangiana**:

$$J(\mathbf{w}, b, \alpha) = \frac{1}{2} \mathbf{w}^T \mathbf{w} - \sum_{i=1}^N \alpha_i (d_i(\mathbf{w}^T \mathbf{x}_i + b) - 1),$$

con $\alpha_i \geq 0$ i **moltiplicatori di Lagrange**, uno per ogni vincolo del primale. J va **minimizzata** rispetto a $\mathbf{w}$ e b e **massimizzata** rispetto ad α : la soluzione è un **punto di sella** di J .

Condizioni di ottimalità:

$$\frac{\partial J}{\partial \mathbf{w}} = 0 \Rightarrow \mathbf{w} = \sum_{i=1}^N \alpha_i d_i \mathbf{x}_i, \quad \frac{\partial J}{\partial b} = 0 \Rightarrow \sum_{i=1}^N \alpha_i d_i = 0.$$

Condizioni di Kuhn-Tucker. Nel punto di sella vale

$$\alpha_i (d_i (\mathbf{w}^T \mathbf{x}_i + b) - 1) = 0 \quad \forall i.$$

Quindi:

- se $\alpha_i > 0$, allora $d_i (\mathbf{w}^T \mathbf{x}_i + b) = 1$: $\mathbf{x}_i$ è un **support vector**;
- se $\mathbf{x}_i$ non è un support vector, allora $\alpha_i = 0$.

Suggerimento — L'iperpiano dipende solo dai support vector

$\mathbf{w}_o = \sum_{i=1}^{N_s} \alpha_{o,i} d_i \mathbf{x}_i$: la somma si restringe agli N_s support vector. **L'iperpiano dipende solo dai support vector!** Tutti gli altri punti potrebbero essere rimossi senza cambiare la soluzione.

Il problema duale*

Sostituendo le condizioni di ottimalità in J si ottiene il **problema duale**, in cui compaiono solo i moltiplicatori:

Definizione — Problema duale — SVM hard margin

Trovare $\{\alpha_i\}_{i=1}^N$ che massimizzano

$$Q(\alpha) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j d_i d_j \mathbf{x}_i^T \mathbf{x}_j$$

soggetto a $\alpha_i \geq 0$ per ogni i e $\sum_{i=1}^N \alpha_i d_i = 0$.

Gli α si trovano risolvendo il problema di **programmazione quadratica** (QP) o con approcci più recenti ed efficienti (ad esempio **SMO**, *Sequential Minimal Optimization*). Il duale scala con il **numero di esempi** N , meno con la dimensione dello spazio. Si noti che i dati compaiono **solo tramite prodotti scalari** $\mathbf{x}_i^T \mathbf{x}_j$: sarà la chiave dei kernel.

Ottenere $\mathbf{w}_o$ e b_o

1. Risolvere il duale ottenendo $\{\alpha_{o,i}\}$.
2. Calcolare $\mathbf{w}_o = \sum_{i=1}^N \alpha_{o,i} d_i \mathbf{x}_i$.
3. Calcolare $b_o = 1 - \mathbf{w}_o^T \mathbf{x}^{(s)}$ per un support vector positivo $\mathbf{x}^{(s)}$, cioè $b_o = 1 - \sum_{i=1}^N \alpha_{o,i} d_i \mathbf{x}_i^T \mathbf{x}^{(s)}$.

13.1.6 Come si usa

Non serve conoscere esplicitamente $\mathbf{w}_o$: bastano i moltiplicatori (dal duale) e il bias. La superficie di decisione è

$$\mathbf{w}_o^T \mathbf{x} + b_o = 0 \iff \sum_{i=1}^N \alpha_{o,i} d_i \underbrace{\mathbf{x}_i^T \mathbf{x}}_{\text{prodotto scalare}} + b_o = 0.$$

Dato un pattern $\mathbf{x}$: (1) calcolare $g(\mathbf{x}) = \sum_{i=1}^N \alpha_{o,i} d_i \mathbf{x}_i^T \mathbf{x} + b_o$; (2) classificare con il segno di $g(\mathbf{x})$. La somma si può restringere agli N_s support vector.

13.1.7 Perché migliora la generalizzazione?

Su problemi linearmente separabili abbiamo **fissato l'errore di training** (a zero). Minimizzare la norma di $\mathbf{w}$ equivale allora a **minimizzare la VC-dimension**, e quindi il termine di capacità (VC-confidence) ε in

$$R[h] \leq R_{emp}[h] + \varepsilon(VC, N, \delta).$$

Teorema — Teorema di Vapnik

Sia D il diametro della più piccola sfera che contiene i dati $\mathbf{x}_1, \dots, \mathbf{x}_N$. Per la classe degli iperpiani separatori $\mathbf{w}^T \mathbf{x} + b = 0$ con margine ρ , la VC-dimension è limitata da

$$VC \leq \min \left(\left\lceil \frac{D^2}{\rho^2} \right\rceil, m_0 \right) + 1.$$

Poiché $\frac{D^2}{\rho^2} \propto \text{Raggio}^2 \|\mathbf{w}\|^2$, la VC-dim può essere **minore di $m_0 + 1$** (quella degli iperpiani generici) restringendo la ricerca agli iperpiani “regolarizzati” con margine massimo. **Più il margine è grande, più la VC-dim è piccola**, indipendentemente dalla dimensione dello spazio.

13.1.8 Perché è un approccio elegante

Per dati linearmente separabili ci sono molte soluzioni; Vapnik propone l'**iperpiano separatore ottimo** a margine massimo, che fornisce:

- una **soluzione unica** (a differenza dell'algoritmo iterativo del Perceptron) con **zero errori** (a differenza dell'LMS) per il classificatore binario;
- un'**SRM automatizzata**, che minimizza la VC-confidence (massimizzando il margine) direttamente all'interno del processo di ottimizzazione, **senza iperparametri** nel caso separabile;
- l'uso di risolutori di **programmazione quadratica vincolata** (anziché la discesa del gradiente), con una bella forma duale che mostra i support vector e i prodotti scalari tra pattern;
- una soluzione concentrata su dati di training “**selezionati**”, i support vector: pregio, perché non dipende dai punti lontani dal confine; difetto, perché si spera che i punti sul confine non siano i più rumorosi.

Ma cosa fare con dati **rumorosi** o **non linearmente separabili**?

13.2 SVM soft margin: ammettere errori

Se almeno un punto viola la condizione di separazione esatta $d_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1$, si passa al **soft margin**.

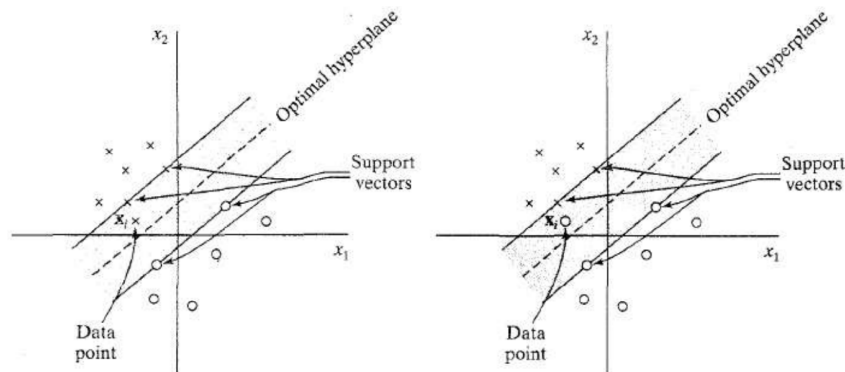


Figura 13.6: Pattern che violano il margine, senza (sinistra) e con (destra) errore di classificazione.

Suggerimento — Il vantaggio del soft margin

Ammettere punti dentro il margine permette di avere un **margine più ampio**: si ottiene tolleranza al rumore e un modello più semplice. (Esercizio: trovare il margine delle figure senza ammettere punti al suo interno.)

13.2.1 Variabili slack

Si introducono variabili non negative $\xi_i \geq 0$, dette **variabili slack**, e i vincoli diventano

$$d_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i \quad \forall i.$$

- $\xi_i = 0$: il punto è fuori dal margine o sul bordo; - $0 < \xi_i \leq 1$: il punto è dentro il margine ma classificato correttamente; - $\xi_i > 1$: il punto è classificato male.

Un support vector soddisfa ora $d_i(\mathbf{w}^T \mathbf{x}_i + b) = 1 - \xi_i$.

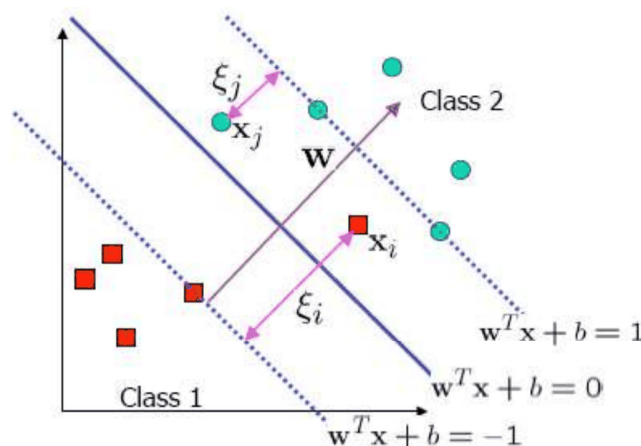


Figura 13.7: Soft margin e variabili slack.

Attenzione: il **teorema di Vapnik non vale più** (l'hard margin non ammette punti nel margine).

13.2.2 Il problema primale soft margin

Definizione — Problema primale — SVM soft margin

Trovare $\mathbf{w}$ e b che minimizzano

$$\Psi(\mathbf{w}, \xi) = \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum_{i=1}^N \xi_i$$

soggetto a $d_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i$ e $\xi_i \geq 0$ per ogni i .

C è un **iperparametro di regolarizzazione** (si perde la SRM completamente automatica!): regola il compromesso tra minimizzazione del rischio empirico e minimizzazione del termine di capacità (VC-confidence).

- C **basso**: molti errori di training ammessi $\rightarrow$ possibile **underfitting**.
- C **alto**: nessun errore ammesso $\rightarrow$ margine più piccolo $\rightarrow$ possibile **overfitting**.

(Per $C \rightarrow \infty$ si ritrova l'hard margin.)

13.2.3 Il duale soft margin*

Definizione — Problema duale — SVM soft margin

Trovare $\{\alpha_i\}$ che massimizzano

$$Q(\alpha) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j d_i d_j \mathbf{x}_i^T \mathbf{x}_j$$

soggetto a $\sum_{i=1}^N \alpha_i d_i = 0$ e $0 \leq \alpha_i \leq C$ per ogni i .

È **identico** al caso hard margin, tranne il vincolo superiore $\alpha_i \leq C$. Le condizioni di Kuhn-Tucker diventano

$$\alpha_i (d_i(\mathbf{w}^T \mathbf{x}_i + b) + \xi_i - 1) = 0, \quad \mu_i \xi_i = 0,$$

dove i μ_i sono i moltiplicatori che impongono $\xi_i \geq 0$. Ne segue:

- $0 < \alpha_i < C \Rightarrow \xi_i = 0$: il punto è **sul bordo** del margine;
- $\alpha_i = C \Rightarrow \xi_i \geq 0$: il punto è **dentro** il margine (o dal lato sbagliato).

Soluzione: $\mathbf{w}_o = \sum_i \alpha_{o,i} d_i \mathbf{x}_i$ e $b_o = d_j - \sum_i \alpha_{o,i} d_i \mathbf{x}_i^T \mathbf{x}_j$ per un pattern j con $0 < \alpha_j < C$ (o la media su tutti questi, per stabilità numerica). I moltiplicatori non nulli corrispondono ai support vector. L'uso è come prima: $g(\mathbf{x}) = \sum_i \alpha_{o,i} d_i \mathbf{x}_i^T \mathbf{x} + b_o$, $h(\mathbf{x}) = \text{sign}(g(\mathbf{x}))$.

13.3 Kernel: SVM per problemi non lineari

13.3.1 Mappare in uno spazio ad alta dimensione

Definizione — Idea

Mappare i dati dallo spazio degli input a uno **spazio delle feature** ad alta dimensione, dove diventano **linearmente separabili**.

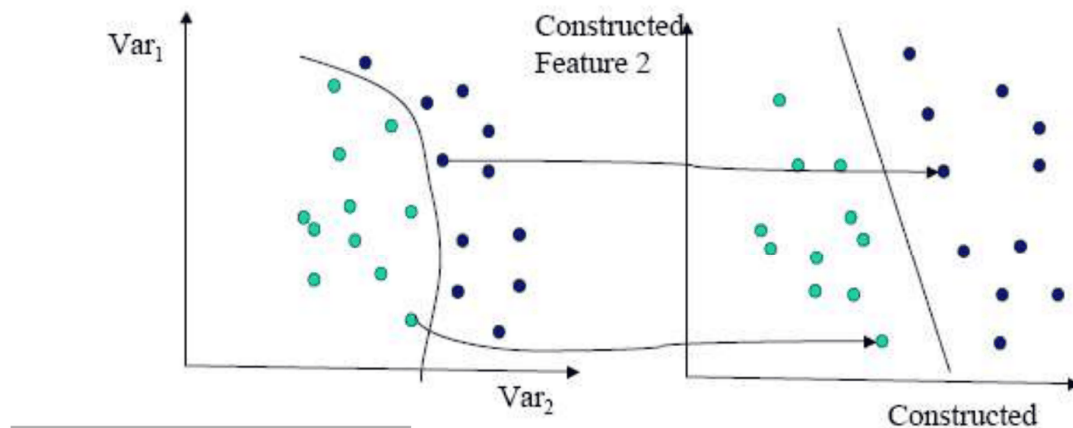


Figura 13.8: Una funzione $\Phi(\mathbf{x})$ mappa i dati in uno spazio in cui sono linearmente separabili (si ricordi la LBE dei modelli lineari).

Esempio — Da $\mathbb{R}^2$ a $\mathbb{R}^3$

Con $\Phi : \mathbb{R}^2 \rightarrow \mathbb{R}^3$, $(x_1, x_2)^T \mapsto (x_1^2, \sqrt{2}x_1x_2, x_2^2)^T$ (monomi di secondo grado), due classi separate da un'ellisse nello spazio di input diventano separabili da un **piano** nello spazio delle feature.

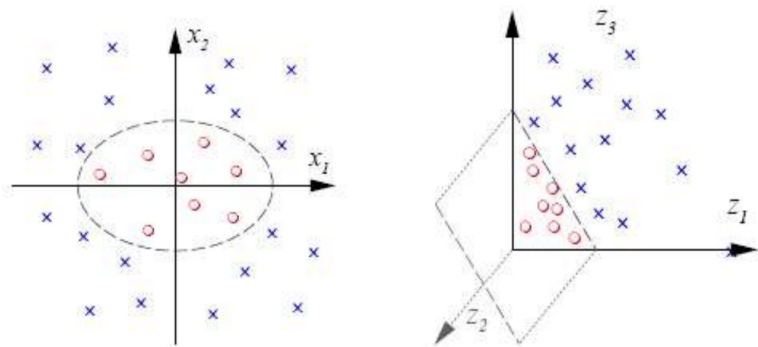


Figure: In the input space we need a quite complex decision surface, while in the second **Figura 13.9:** Nello spazio di input serve una superficie complessa; nello spazio dei monomi di secondo grado basta una funzione lineare.

La mappatura dell'esempio è costruita apposta per risolvere il problema (le feature corrispondono a ellissoidi nello spazio originale). Ma trovare la trasformazione “migliore” **non è automatico** (a meno di avere conoscenza a priori); spesso si usano trasformazioni generali. **La scelta di Φ è il punto critico delle SVM.**

13.3.2 Spazi di feature ad alta dimensione

Il procedimento è: (1) mappatura non lineare dei pattern in uno spazio delle feature ad alta dimensione (per il **teorema di Cover**, i pattern vi sono linearmente separabili con alta probabilità); (2) ricerca dell'iperpiano ottimo nello spazio delle feature.

Ma sappiamo che spazi delle feature molto grandi (espansioni in basi ampie) possono essere **computazionalmente infattibili** e portare a **overfitting** senza controllare la dimensione dello spazio e la complessità del classificatore. L'**approccio kernel** permette di gestire lo spazio delle feature **implicitamente**, in un contesto regolarizzato (dove è il **margin** a regolare la complessità, non la dimensione dello spazio di input).

Con $\Phi : \mathbb{R}^{m_0} \rightarrow \mathbb{R}^{m_1}$, il training set diventa $\{(\Phi(\mathbf{x}_i), d_i)\}$ e l'iperpiano $\mathbf{w}^T \Phi(\mathbf{x}) + b = 0$. Includendo il bias ($w_0 = b$, $\phi_0(\mathbf{x}) = 1$), la superficie di decisione $\mathbf{w}^T \Phi(\mathbf{x}) = 0$ è un'**espansione lineare in basi** in cui la dimensione dello spazio può essere molto grande e la complessità dipende dal **margin**, non dalla dimensione.

Il vettore dei pesi è $\mathbf{w} = \sum_i \alpha_i d_i \Phi(\mathbf{x}_i)$, e l'iperpiano diventa

$$\sum_{i=1}^N \alpha_i d_i \underbrace{\Phi^T(\mathbf{x}_i) \Phi(\mathbf{x})}_{\text{prodotto scalare}} = 0.$$

Il prodotto scalare è ora nello spazio delle feature, e calcolare $\Phi(\mathbf{x})$ potrebbe essere intrattabile.

13.3.3 Il kernel trick

Sotto certe condizioni **non serve calcolare** $\Phi(\mathbf{x})$, e nemmeno conoscere lo spazio delle feature! Basta una funzione che calcoli direttamente il prodotto scalare nello spazio delle feature:

Definizione — Kernel (inner product kernel)

$$k : \mathbb{R}^{m_0} \times \mathbb{R}^{m_0} \rightarrow \mathbb{R}, \quad k(\mathbf{x}_i, \mathbf{x}) = \Phi^T(\mathbf{x}_i) \Phi(\mathbf{x}).$$

Proprietà: è **simmetrica**, $k(\mathbf{x}_i, \mathbf{x}) = k(\mathbf{x}, \mathbf{x}_i)$.

Esempio — Il kernel dell'esempio

Con $\Phi(\mathbf{x}) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)^T$, per $\mathbf{x} = (x_1, x_2)^T$ e $\mathbf{y} = (y_1, y_2)^T$:

$$\Phi^T(\mathbf{x})\Phi(\mathbf{y}) = x_1^2y_1^2 + 2x_1x_2y_1y_2 + x_2^2y_2^2 = (x_1y_1 + x_2y_2)^2 = (\mathbf{x}^T\mathbf{y})^2 = k(\mathbf{x}, \mathbf{y}).$$

Molto importante: il prodotto scalare nello spazio delle feature (3D) viene calcolato **senza passare per la mappatura**, direttamente in termini dei pattern di input (2D).

13.3.4 La matrice kernel

I prodotti scalari tra le immagini dei pattern di training si organizzano in una matrice $N \times N$, la **matrice kernel** (o matrice di Gram):

$$K = \{k(\mathbf{x}_i, \mathbf{x}_j)\}_{i,j=1}^N,$$

simmetrica perché il kernel è simmetrico.

Ogni funzione kernel calcola un prodotto scalare in qualche spazio? No: la proprietà vale solo per i kernel che producono matrici kernel **semidefinite positive (teorema di Mercer)**, cioè con autovalori non negativi.

Proprietà di chiusura: se k_1 e k_2 sono kernel, lo sono anche $k_1 + k_2$, αk_1 (con $\alpha > 0$) e $k_1 k_2$.

13.3.5 SVM con kernel

Il primale nello spazio delle feature è lo stesso del soft margin con $\Phi(\mathbf{x}_i)$ al posto di $\mathbf{x}_i$ (ma $\mathbf{w}$ vive nello spazio delle feature, che può rendere il problema intrattabile). Il **duale** invece è trattabile:

Definizione — Problema duale nello spazio delle feature

Trovare $\{\alpha_i\}$ che massimizzano

$$Q(\alpha) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i,j=1}^N \alpha_i \alpha_j d_i d_j \underbrace{k(\mathbf{x}_i, \mathbf{x}_j)}_{K_{ij}}$$

soggetto a $\sum_i \alpha_i d_i = 0$ e $0 \leq \alpha_i \leq C$.

Il kernel incorpora la mappatura nello spazio delle feature. La dipendenza da N (anziché dalla dimensione dello spazio) diventa ora un **vantaggio**, perché lo spazio delle feature può avere dimensione altissima o infinita.

In sintesi — Riepilogo del procedimento

1. Si ha il training set $T = \{(\mathbf{x}_i, d_i)\}$.
2. Si scelgono il parametro C e la funzione kernel k .
3. Si calcola la matrice kernel K .
4. Si risolve il problema di ottimizzazione (programmazione quadratica) trovando $\{\alpha_i\}$.
5. Si calcola il bias b dai moltiplicatori e dalla matrice kernel.
6. Per un nuovo pattern $\mathbf{x}$: $\mathbf{w}^T \Phi(\mathbf{x}) = \sum_i \alpha_i d_i k(\mathbf{x}, \mathbf{x}_i) + b$ (senza mai calcolare $\mathbf{w}!$), e si classifica con il segno.

Sparsità: la soluzione è sparsa negli α , perché tutti i moltiplicatori dei non-support vector sono nulli.

In fase di test: $h(\mathbf{x}) = \text{sign}(\sum_i \alpha_i d_i k(\mathbf{x}, \mathbf{x}_i) + b)$. Si memorizzano gli $\mathbf{x}_i$ (i support vector) per la fase di test: somiglia al K-NN?

13.3.6 Vista “architetturale” di una SVM

Il numero di “unità nascoste” è pari al **numero di support vector**, ma non sono esattamente le stesse unità di una rete neurale: le unità nascoste di una rete hanno **parametri liberi adattivi**, qui invece i “centri” sono i support vector stessi e la funzione è fissata dal kernel. È solo una vista illustrativa: da confrontare con le reti neurali (una forma adattiva di LBE) in termini di controllo della complessità, efficienza e altre caratteristiche.

13.3.7 Esempi di kernel

- **Polinomiale:** $k(\mathbf{x}, \mathbf{x}_i) = (\mathbf{x}^T \mathbf{x}_i + 1)^p$, con p scelto dall'utente.

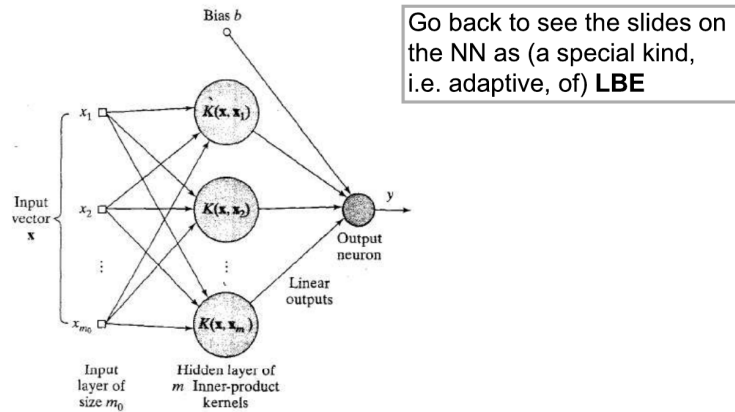


Figura 13.10: “Architettura” di una SVM: numero di unità nascoste = numero di support vector.

- **RBF** (*Radial Basis Function*, detto anche **kernel gaussiano**): $k(\mathbf{x}, \mathbf{x}_i) = e^{-\frac{1}{2\sigma^2} \|\mathbf{x} - \mathbf{x}_i\|^2}$, con σ^2 scelto dall’utente. Con kernel molto “stretti” (σ piccolo) la risposta su $\mathbf{x}_i$ è solo d_i (somiglia al 1-NN!).
- **Perceptron a due strati**: $k(\mathbf{x}, \mathbf{x}_i) = \tanh(\beta_0 \mathbf{x}^T \mathbf{x}_i + \beta_1)$, con $\beta_0 > 0$ e $\beta_1 < 0$.

Attenzione — Importante

Per il kernel polinomiale e l’RBF si ottiene **sempre** un kernel valido (prodotto scalare), mentre per il perceptron a due strati il teorema di Mercer vale solo per alcune scelte di β_0 e β_1 . Un kernel RBF corrisponde sempre a uno spazio delle feature di **dimensione infinita**.

13.4 Parte II — SVM per la regressione non lineare

13.4.1 Il problema

Si ha $d = f(\mathbf{x}) + v$, con f sconosciuta e rumore v indipendente da $\mathbf{x}$. Si stima d con un’espansione lineare di funzioni non lineari:

$$y = h(\mathbf{x}) = \mathbf{w}^T \Phi(\mathbf{x}), \quad \mathbf{w} = (b, w_1, \dots, w_{m_1})^T, \quad \Phi(\mathbf{x}) = (1, \phi_1(\mathbf{x}), \dots, \phi_{m_1}(\mathbf{x}))^T.$$

13.4.2 La loss ε -insensitive

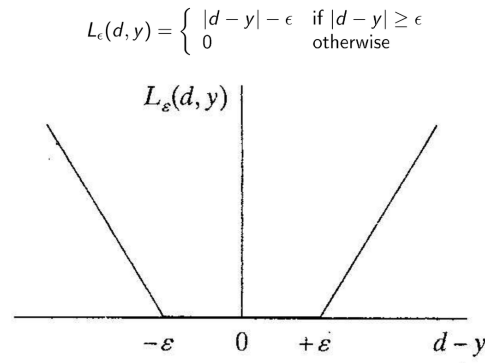
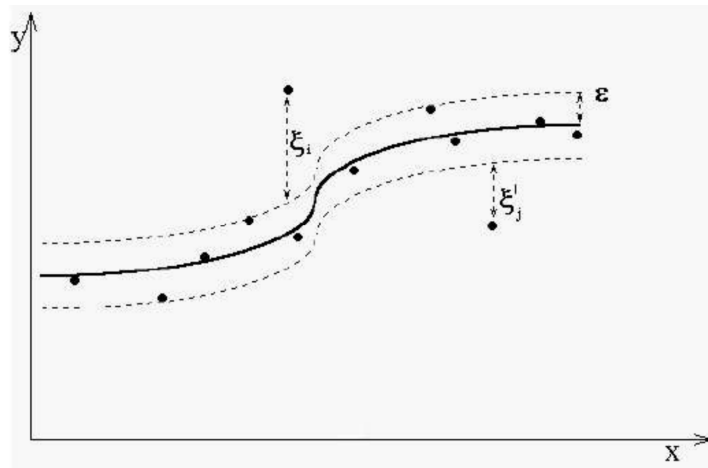
$$L_\varepsilon(d, y) = \begin{cases} |d - y| - \varepsilon & \text{se } |d - y| \geq \varepsilon \\ 0 & \text{altrimenti.} \end{cases}$$

Gli errori più piccoli di ε **non costano nulla**: si forma un **tubo** di ampiezza ε attorno alla predizione, e solo i punti fuori dal tubo contribuiscono alla loss. Di nuovo, **solo i support vector guidano la soluzione**: nella classificazione erano i punti oltre il margine, qui sono i punti fuori dal tubo.

13.4.3 Formulazione

Si introducono due insiemi di variabili slack non negative ξ_i e ξ'_i (sopra e sotto il tubo):

$$-\xi'_i - \varepsilon \leq d_i - \mathbf{w}^T \Phi(\mathbf{x}_i) \leq \varepsilon + \xi_i.$$

Figura 13.11: La loss ϵ -insensitive (oFigure: ϵ -tube and slack variables.Figura 13.12: Il tubo ϵ e le variabili slack per la regressione.**Definizione — Problema primale — SVM per la regressione**

Minimizzare

$$\Psi(\mathbf{w}, \xi, \xi') = \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum_{i=1}^N (\xi_i + \xi'_i)$$

soggetto a $d_i - \mathbf{w}^T \Phi(\mathbf{x}_i) \leq \epsilon + \xi_i$, $\mathbf{w}^T \Phi(\mathbf{x}_i) - d_i \leq \epsilon + \xi'_i$, $\xi_i \geq 0$, $\xi'_i \geq 0$.**Definizione — Problema duale — SVM per la regressione***Trovare $\{\alpha_i\}$ e $\{\alpha'_i\}$ che massimizzano

$$Q(\alpha, \alpha') = \sum_{i=1}^N d_i(\alpha_i - \alpha'_i) - \epsilon \sum_{i=1}^N (\alpha_i + \alpha'_i) - \frac{1}{2} \sum_{i,j=1}^N (\alpha_i - \alpha'_i)(\alpha_j - \alpha'_j) k(\mathbf{x}_i, \mathbf{x}_j)$$

soggetto a $\sum_i (\alpha_i - \alpha'_i) = 0$ e $0 \leq \alpha_i, \alpha'_i \leq C$.

Dalla soluzione:

$$\mathbf{w} = \sum_{i=1}^N \underbrace{(\alpha_i - \alpha'_i)}_{\gamma_i} \Phi(\mathbf{x}_i), \quad h(\mathbf{x}) = \sum_{i=1}^N \gamma_i \Phi^T(\mathbf{x}_i) \Phi(\mathbf{x}) = \sum_{i=1}^N \gamma_i k(\mathbf{x}_i, \mathbf{x}).$$

I **support vector** corrispondono ai γ_i non nulli.

In sintesi — Riepilogo per la regressione

1. Scegliere i parametri C ed ε .
2. Scegliere un kernel k .
3. Calcolare la matrice kernel K .
4. Risolvere il duale e ottenere i γ_i .
5. Calcolare il bias ottimo.
6. La funzione stimata è una combinazione lineare di prodotti scalari in uno spazio delle feature che possiamo ignorare: $h(\mathbf{x}) = \sum_i \gamma_i k(\mathbf{x}_i, \mathbf{x})$.

Domande — Cosa bisogna sapere

- La definizione di classificatore a **marginale massimo**.
- Come il margine massimo si trasforma in un problema di **programmazione quadratica**.
- Cosa la QP fa per noi (non come lo fa).
- Perché l'SVM approssima la **SRM**.
- Come si gestiscono dati **rumorosi** (non separabili): soft margin.
- Come si ottengono **confini non lineari**: kernel.
- Come i kernel permettono di lavorare con espansioni in basi di dimensione altissima senza calcolarle.
- Come funziona la SVM per la regressione (ε -insensitive loss).

Riferimenti: Haykin, *Neural Networks* (2^a ed.), cap. 6 (“necessario e sufficiente”); Burges, *A Tutorial on Support Vector Machines for Pattern Recognition*; Schölkopf e Smola, *Learning with Kernels*; Vapnik, *The Nature of Statistical Learning Theory* e *An Overview of Statistical Learning Theory* (1999).

Capitolo 14

SVM e kernel: aspetti pratici e visione critica

Questo pacco di slide completa [13 - Support Vector Machines](#) con una visione critica: pregi e difetti delle SVM, esempi di risultati, il ruolo cruciale degli iperparametri, alcuni luoghi comuni da sfatare e la generalizzazione ai **metodi kernel**.

14.1 Caratteristiche principali

In sintesi — Pregi

- **Regolarizzazione incorporata** nel problema di ottimizzazione (il margine).
- **Approssimazione della SRM** teorica (il bound sulla VC-dim degli iperpiani viene minimizzato dalla soluzione, nel caso hard margin senza kernel).
- **Problema convesso**: l'addestramento trova sempre il **minimo globale** (a differenza delle reti neurali).
- **Trasformazione implicita delle feature** tramite i kernel.

Attenzione — Difetti

- Bisogna **scegliere il kernel e i suoi parametri**.
- È un algoritmo **batch**.
- Problemi molto grandi erano computazionalmente intrattabili: oltre circa 20.000 esempi era difficile risolverli esattamente con gli approcci standard (la matrice kernel è $N \times N$). Oggi esistono molte soluzioni, anche basate sulla discesa del gradiente.

In sintesi, i vantaggi:

- un **modello lineare** (semplice, compatto) con un bound sulla complessità in termini di **margine**, che viene ottimizzato direttamente;
- un ricco insieme di **funzioni di decisione non lineari** nello spazio di input tramite i kernel: si fa ancora una separazione lineare, ma in uno spazio diverso, con un trattamento **efficiente** della LBE. Un insieme ricco non implica necessariamente una VC-dim alta, che per le SVM dipende da margine, parametri del kernel, C , ecc.;
- il **kernel trick** si estende ad altri modelli: i **metodi kernel**.

14.2 Risultati

14.2.1 La prima applicazione famosa

Il riconoscimento di cifre manoscritte (**MNIST**) fu un caso storico di successo per le SVM: circa lo 0,8% di errori nel 1998 con una SVM polinomiale di grado 9, un risultato notevole,

paragonabile a LeNet e ottenuto senza anni di sviluppo dell'architettura. L'attenzione per SVM e SLT (i cui studi risalivano agli anni '60-'70) crebbe molto: un esempio della rilevanza delle applicazioni, o dell'importanza di una buona teoria per applicazioni di successo.

14.2.2 L'esempio della miscela di gaussiane

Riprendiamo il problema delle due classi (miscela di gaussiane) già visto con i modelli lineari, il K-NN e le reti neurali.

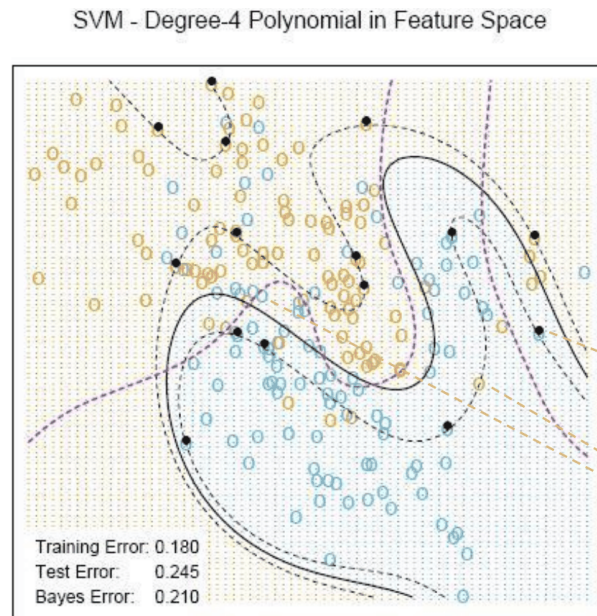


Figura 14.1: SVM con kernel polinomiale di grado 4 ($C = 1$): il 42% dei dati risulta essere support vector.

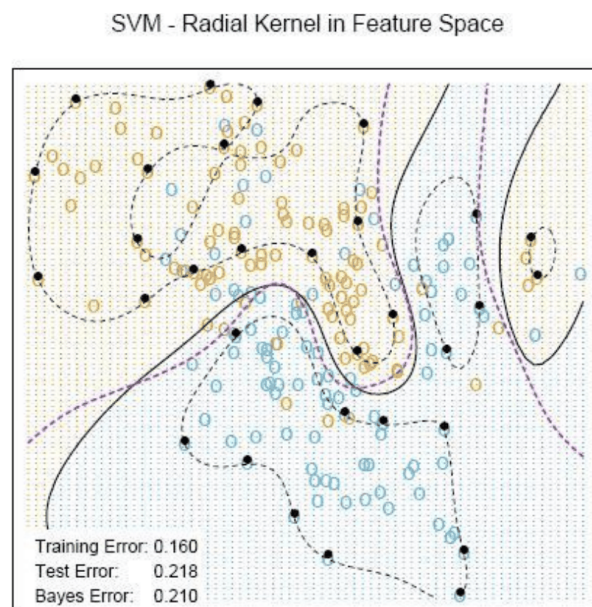


Figura 14.2: SVM con kernel RBF ($\gamma = 1/(2\sigma^2) = 1$): il 45% dei dati sono support vector.

I support vector sono i punti **sul margine** (slack = 0) o **dalla parte sbagliata** del margine (slack > 0). Il kernel radiale dà il risultato migliore, come ci si può aspettare visto che i dati provengono da miscele di gaussiane. Si confronti con la rete neurale con 10 unità e weight decay 0,02 (errore di test 0,223): risultati simili.

Attenzione — Tanti support vector

Nella teoria si assume che i support vector siano pochi, ma **non è sempre vero**: qui sono più del 40% dei dati. La sparsità della soluzione non è garantita.

14.3 Le SVM nella pratica

Nota — Nessuna garanzia

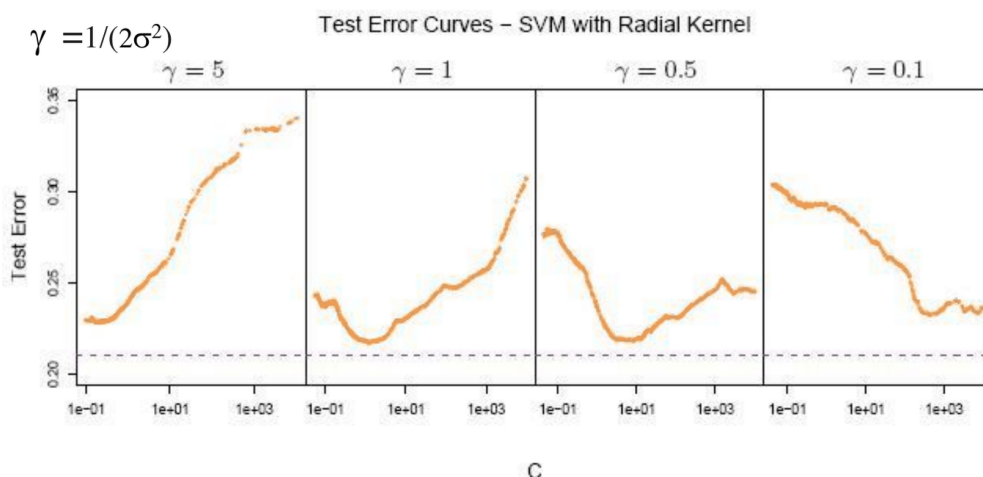
“Attualmente non esiste una teoria che garantisca che una data famiglia di SVM abbia alta accuratezza su un dato problema” (Burges, 1998).

Le belle proprietà dei classificatori hard margin **non si estendono direttamente** al soft margin e ai kernel: il parametro C e il kernel possono portare a una VC-dim anche **infinita**. Ad esempio:

- una SVM con kernel gaussiano di **larghezza σ sufficientemente piccola** può classificare correttamente un numero arbitrariamente grande di punti (come il 1-NN, solo il support vector più vicino a $\mathbf{x}$ contribuisce alla soluzione): VC-dim infinita, a meno di regolarizzare;
- all'opposto, con una larghezza **grande** tutti i support vector contribuiscono e si ottiene una sorta di “media globale”, con VC-dim bassa.

Quindi **controllando la larghezza del kernel si controlla la VC-dim**.

14.3.1 Il ruolo critico degli iperparametri



$\gamma = 5$ (small σ , narrow peaked Kernels) require lower C value (max regularization). Hence, the best value of C depends also on the kernel hyper-parameters !

Often selected using cross-validation: take care !

Figura 14.3: Errore di test in funzione di C per diversi valori di $\gamma = 1/(2\sigma^2)$.

Con $\gamma = 5$ (piccolo σ , kernel molto stretti) serve un C basso (massima regolarizzazione); con γ piccoli l'ottimo si sposta verso C più alti. **Il miglior valore di C dipende anche dagli iperparametri del kernel**: vanno selezionati **insieme** (grid search), di solito con cross-validation, con attenzione.

Per applicare una SVM bisogna quindi scegliere il **tipo di kernel** (e i suoi parametri) e il valore di C (e di ε per la regressione). Una selezione rigorosa richiederebbe una stima della complessità (VC-dim); gli iperparametri che influenzano la complessità sono quelli da usare nella model selection. In pratica si fa, ancora una volta, una **valutazione empirica accurata**: la regolazione degli iperparametri (C , γ ...) può introdurre overfitting, quindi validation set + test set, oppure double CV.

Prestazioni: spesso buone, ma il confronto con altri metodi non è sempre favorevole. Dal 2012–2013 le **reti profonde** hanno superato nettamente i record precedenti nel riconoscimento di immagini e parlato.

14.3.2 Luoghi comuni sulle SVM

Domande — Esercizio: cosa c'è di sbagliato?

1. “La SVM si può usare con i valori di default degli iperparametri.” → No: C e i parametri del kernel sono critici e vanno selezionati.
2. “Scelgo la SVM perché non va in overfitting.” → No: con soft margin e kernel la VC-dim può essere altissima; l'overfitting dipende da C e dal kernel.
3. “La SVM non ha problemi con input ad alta dimensione (risolve la maledizione della dimensionalità?).” → No: la SVM gestisce bene l'alta dimensione dello **spazio delle feature**, non dello **spazio di input**. La curse of dimensionality dell'input resta.
4. “Con la SVM si trova automaticamente il numero di unità.” → Il numero di support vector è determinato dalla soluzione, ma dipende da C e dal kernel, che vanno scelti; e non è detto che sia piccolo.

14.4 Metodi kernel

Il **kernel trick** porta ai **metodi kernel**, anche senza SVM: la “**kernelizzazione**” di approcci tradizionali. Ogni volta che in un modello compare un **prodotto scalare** o una **misura di similarità**, lo si può sostituire con un kernel, facendo operare il modello in un nuovo spazio implicito ad alta dimensione semplicemente specificando (e cambiando) il kernel.

- È un'**espansione lineare in basi** fissa e ad alta dimensione, ma molto **efficiente**.
- La scelta delle funzioni di base è **modulare**: si cambia il kernel senza cambiare la macchina di apprendimento.
- Fornisce un quadro teorico unificato per funzioni generali di tipo prodotto scalare.

14.4.1 Kernel come misura di similarità

Il kernel è legato a una misura di **similarità** tra $\mathbf{s}$ e $\mathbf{t}$; infatti induce la distanza nello spazio delle feature:

$$d_k(\mathbf{s}, \mathbf{t})^2 = k(\mathbf{s}, \mathbf{s}) - 2k(\mathbf{s}, \mathbf{t}) + k(\mathbf{t}, \mathbf{t}) = \|\Phi(\mathbf{s}) - \Phi(\mathbf{t})\|^2.$$

Oggetti simili (distanza piccola) → valore alto del kernel.

Suggerimento — L'idea dei kernel

Concentrarsi sulla rappresentazione e l'elaborazione dei **confronti tra coppie** di oggetti, anziché sulla rappresentazione di un singolo oggetto. In alcuni domini è più facile confrontare

due oggetti che definire uno spazio astratto di feature in cui rappresentarne uno.

Una SVM è caratterizzata in gran parte dalla scelta del kernel. Si confronti con le **reti neurali**, dove le funzioni di base si **adattano** ai dati (apprendendo i pesi delle unità nascoste): nelle SVM la scelta del kernel migliore per un problema è ancora un tema di ricerca.

(Esercizio: ripensare il kernel come nei metodi basati su distanze, tipo K-NN. Quando un kernel è buono? Quando raggruppa nello spazio delle feature gli oggetti della stessa classe, riflettendo la similarità rilevante per il task. Quando è cattivo? Quando la similarità che induce non ha a che fare con il task, ad esempio se tutti gli oggetti risultano ugualmente simili o ugualmente diversi.)

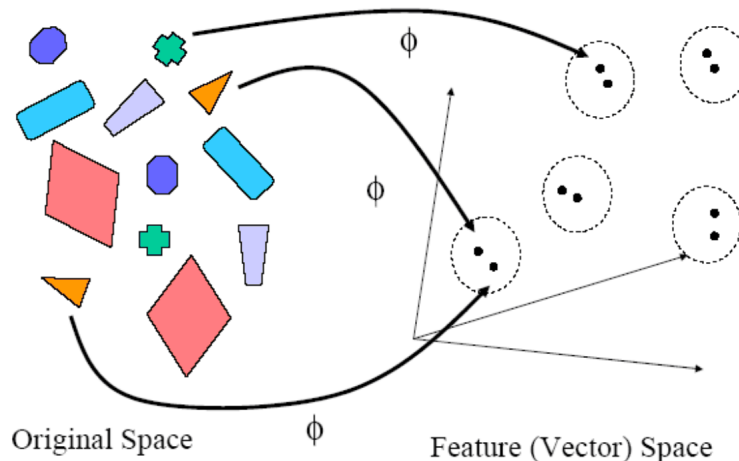


Figura 14.4: Un buon kernel per “oggetti”: la mappa ϕ porta oggetti simili vicini nello spazio delle feature.

14.4.2 Argomenti avanzati

La **progettazione di nuovi kernel** richiede di scegliere una funzione “buona” per il dominio specifico, e si può estendere a **oggetti generici** in input, non solo vettori: kernel per **stringhe**, **alberi**, **grafi**; kernel ad hoc per il dominio e il task (ad esempio kernel basati sulla *edit distance* per la bioinformatica, o su *suffix tree* per stringhe nel NLP), con attenzione all’efficienza algoritmica. I **kernel adattivi** sono un tema di ricerca.

Un vantaggio concettuale: la **conoscenza di dominio è separata dal risolutore**, perché sta tutta nel kernel.

In sintesi — Sintesi

Le SVM uniscono un modello lineare, un problema convesso con minimo globale, la regolarizzazione tramite margine e la potenza dei kernel. Ma non sono “magiche”: la scelta di kernel e iperparametri (C , σ , ε) è critica e va fatta con una validazione rigorosa; i support vector possono essere tanti; non risolvono la maledizione della dimensionalità dello spazio di input. Il kernel trick è un’idea generale (metodi kernel) che permette di lavorare con similarità tra oggetti anche strutturati.

Domande — Possibili domande d’esame

- Pregi e difetti delle SVM rispetto alle reti neurali.
- Perché la VC-dim di una SVM con kernel RBF può essere infinita? Come si controlla?

- Perché C e i parametri del kernel vanno selezionati insieme?
- La SVM risolve la maledizione della dimensionalità?
- Cosa sono i metodi kernel? Che legame c'è tra kernel e similarità?

Capitolo 15

Bias-varianza ed ensemble

La SLT ha spiegato underfitting e overfitting tramite la complessità del modello e il numero di dati. La **decomposizione bias-varianza** offre un altro punto di vista, complementare: considera il fatto che il training set è **una sola** delle possibili realizzazioni dei dati, e scompone l'errore atteso in tre componenti — **bias**, **varianza** e **rumore**. Da qui si arriva in modo naturale agli **ensemble** (bagging, boosting), che sfruttano proprio la riduzione della varianza.

15.1 L'idea

Un training set è solo una possibile realizzazione dall'universo dei dati: **training set diversi producono stime diverse**. L'errore atteso (sui vari training set) in un punto $\mathbf{x}$ si scompone in:

- **Bias**: quantifica la discrepanza tra la funzione vera e $h(\mathbf{x})$ (mediata sui dati). È alto se H è troppo piccolo (modello troppo rigido).
- **Varianza**: quantifica la variabilità della risposta del modello h per diverse realizzazioni dei dati di training. È dovuta a un'eccessiva flessibilità.
- **Rumore**: le etichette contengono un errore casuale (ad esempio, per un dato $\mathbf{x}$ ci possono essere più valori di d possibili).

Si assume lo scenario della **regressione** con target y e loss quadratica.

15.2 Analisi nella regressione

Supponiamo esempi $\langle \mathbf{x}, y \rangle$ con funzione vera $y = f(\mathbf{x}) + \varepsilon$, dove ε è rumore gaussiano con media zero e deviazione standard σ . Con la regressione lineare, dato un insieme di esempi $\langle \mathbf{x}_i, y_i \rangle$, $i = 1..l$, si adatta un'ipotesi lineare $h(\mathbf{x}) = \mathbf{w}\mathbf{x} + w_0$ minimizzando $\sum_{i=1}^l (y_i - h(\mathbf{x}_i))^2$. Due fatti:

1. per la classe di ipotesi scelta (lineari), per alcune f si avrà un **errore di predizione sistematico**;
2. a seconda del dataset, i parametri $\mathbf{w}$ trovati **saranno diversi**.

Esempio — 20 punti e 50 fit

Funzione vera $y = x + 2 \sin(1,5x) + \mathcal{N}(0; 0,2)$. Con 20 punti si adatta una retta. Ripetendo con 50 dataset diversi (ciascuno di 20 punti), si ottengono 50 rette diverse.

15.2.1 L'errore di predizione atteso

Dato un nuovo punto $\mathbf{x}$, qual è l'**errore di predizione atteso**? Si assume che i dati siano estratti in modo **i.i.d.** (indipendenti e identicamente distribuiti) da un'unica distribuzione P .

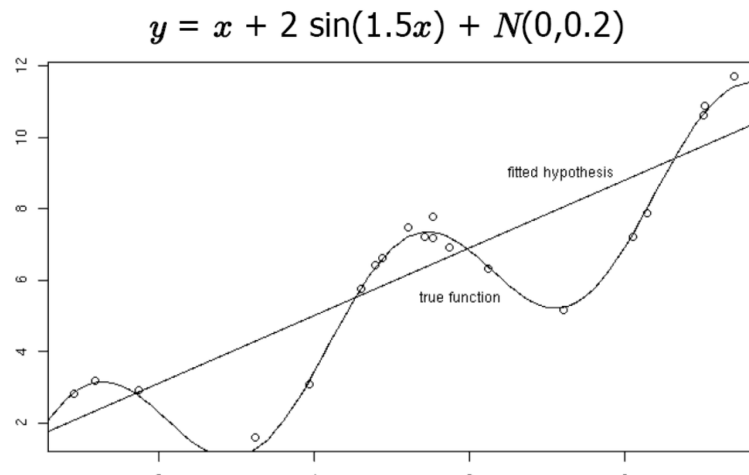


Figura 15.1: Un dataset di 20 punti, la funzione vera e l'ipotesi lineare adattata.

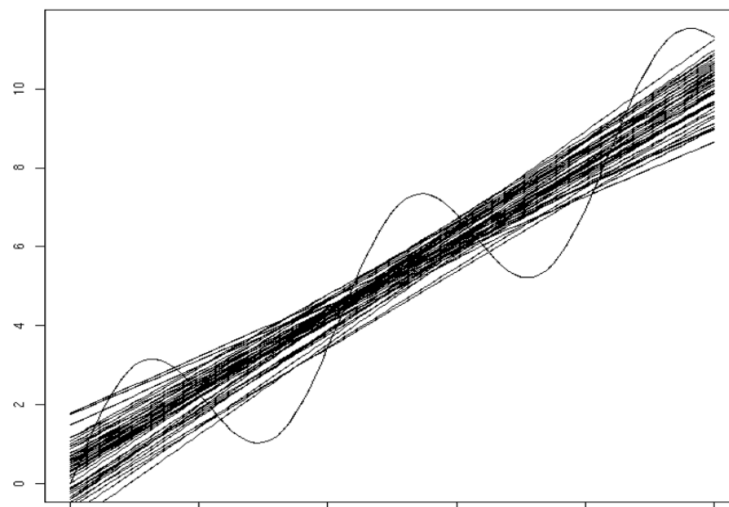


Figura 15.2: 50 fit, ciascuno su 20 punti: le rette variano da un dataset all'altro.

L'obiettivo è calcolare, per un x arbitrario,

$$E_P[(y - h(x))^2],$$

dove y è il valore di x che potrebbe comparire in un dataset, e l'aspettativa è **su tutti i training set** estratti secondo P . Attenzione: per ogni training set “estratto” c'è una h diversa (e un y diverso).

15.2.2 Richiamo di statistica

Per una variabile aleatoria discreta Z : $\bar{Z} = E_P[Z] = \sum_i z_i P(z_i)$ e $\text{Var}[Z] = E[(Z - \bar{Z})^2]$.

Teorema — Lemma della varianza

$$\text{Var}[Z] = E[Z^2] - \bar{Z}^2 \quad \Longleftrightarrow \quad E[Z^2] = \bar{Z}^2 + \text{Var}[Z].$$

Dimostrazione. $\text{Var}[Z] = \sum_i (z_i - \bar{Z})^2 P(z_i) = \sum_i z_i^2 P(z_i) - 2\bar{Z} \sum_i z_i P(z_i) + \bar{Z}^2 \sum_i P(z_i) = E[Z^2] - 2\bar{Z}^2 + \bar{Z}^2 = E[Z^2] - \bar{Z}^2.$

15.2.3 La decomposizione

Sviluppando il quadrato e usando la linearità dell'aspettativa (e il fatto che y e $h(\mathbf{x})$ sono indipendenti, perché y è un nuovo dato e h dipende dal training set):

$$E_P[(y - h(\mathbf{x}))^2] = E_P[h(\mathbf{x})^2 - 2yh(\mathbf{x}) + y^2] = E_P[h(\mathbf{x})^2] + E_P[y^2] - 2E_P[y]E_P[h(\mathbf{x})].$$

Sia $\bar{h}(\mathbf{x}) = E_P[h(\mathbf{x})]$ la **predizione media** dell'ipotesi in $\mathbf{x}$, quando h è addestrata su dati estratti da P .

- **Primo termine**, con il lemma: $E_P[h(\mathbf{x})^2] = E_P[(h(\mathbf{x}) - \bar{h}(\mathbf{x}))^2] + \bar{h}(\mathbf{x})^2$.
- Si noti che $E_P[y] = E_P[f(\mathbf{x}) + \varepsilon] = f(\mathbf{x})$ (il rumore ha media zero).
- **Secondo termine**, con il lemma: $E_P[y^2] = E_P[(y - f(\mathbf{x}))^2] + f(\mathbf{x})^2$.

Mettendo tutto insieme:

$$\begin{aligned} E_P[(y - h(\mathbf{x}))^2] &= E_P[(h(\mathbf{x}) - \bar{h}(\mathbf{x}))^2] + \underbrace{\bar{h}(\mathbf{x})^2 - 2f(\mathbf{x})\bar{h}(\mathbf{x}) + f(\mathbf{x})^2}_{(\bar{h}(\mathbf{x}) - f(\mathbf{x}))^2} + E_P[(y - f(\mathbf{x}))^2] \\ &= \underbrace{E_P[(h(\mathbf{x}) - \bar{h}(\mathbf{x}))^2]}_{\text{varianza}} + \underbrace{(\bar{h}(\mathbf{x}) - f(\mathbf{x}))^2}_{\text{bias}^2} + \underbrace{E_P[(y - f(\mathbf{x}))^2]}_{\text{rumore}^2} \\ &= \text{Var}[h(\mathbf{x})] + \text{Bias}[h(\mathbf{x})]^2 + E_P[\varepsilon^2] = \text{Var}[h(\mathbf{x})] + \text{Bias}[h(\mathbf{x})]^2 + \sigma^2. \end{aligned}$$

Teorema — Decomposizione bias-varianza

$$\text{Errore di predizione atteso} = \text{Varianza} + \text{Bias}^2 + \text{Rumore}^2.$$

15.2.4 Interpretazione delle tre componenti

- **Bias** $(\bar{h}(\mathbf{x}) - f(\mathbf{x}))^2$: la discrepanza tra la funzione vera e la predizione **media** sui diversi training set. È un **errore sistematico**, dovuto ad esempio a uno spazio H troppo piccolo o a un modello troppo rigido.
- **Varianza**: la variabilità della risposta del modello per diverse realizzazioni del training set. È dovuta a una **flessibilità eccessiva**.
- **Rumore**: anche la soluzione ottima può sbagliare (ad esempio se per un dato $\mathbf{x}$ sono possibili più valori di d). È **irriducibile**: non dipende dal modello e rappresenta una tolleranza sulla risposta (σ).

Tornando all'esempio: il **bias** è evidente nella media $\bar{h}$ delle 50 rette, che resta lontana dalla funzione vera ondulata (il modello lineare è troppo rigido); la **varianza** è la dispersione delle 50 rette attorno a $\bar{h}$; il **rumore** è la dispersione dei dati attorno alla funzione vera.

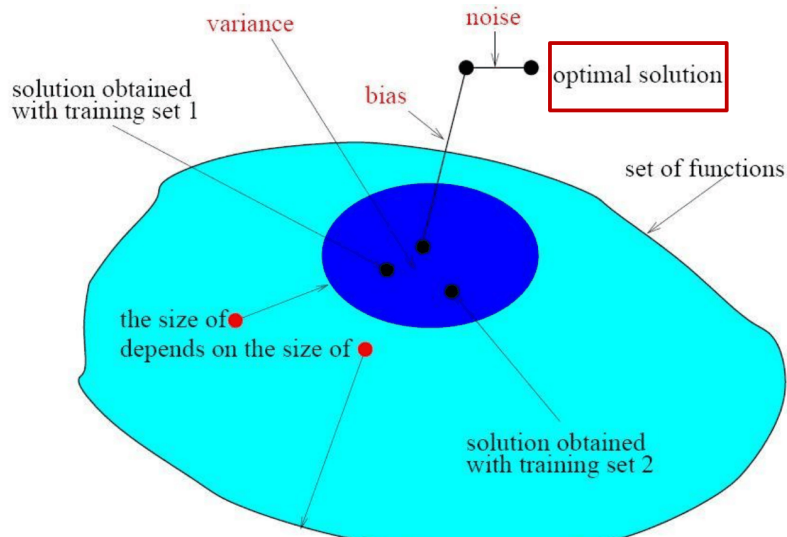


Figura 15.3: Vista grafica: training set diversi producono soluzioni diverse (varianza); la loro media dista dall'ottimo (bias); l'ottimo stesso dista dai dati per il rumore.

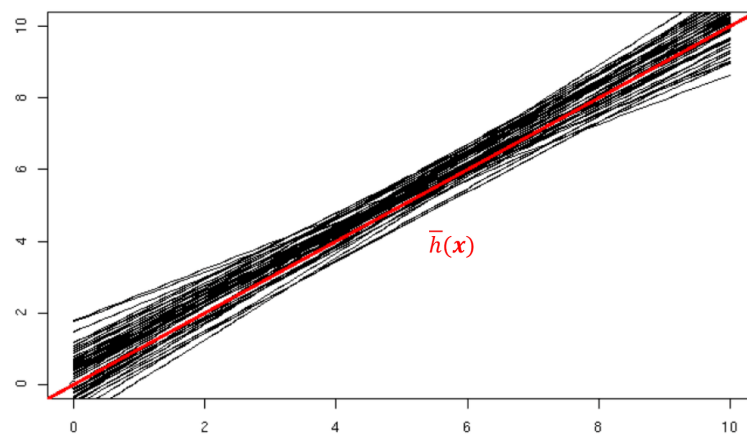
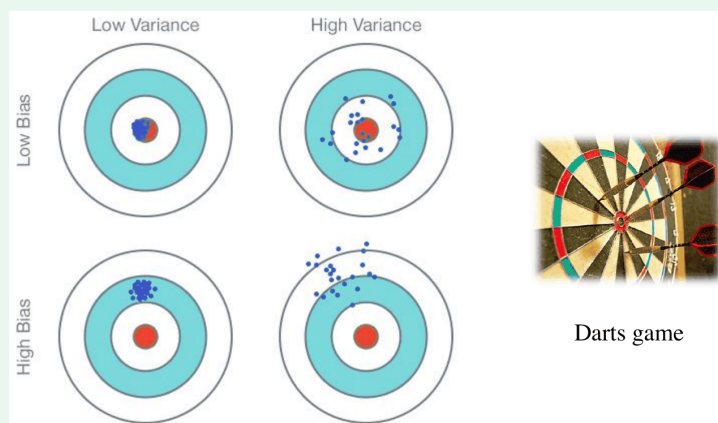


Figura 15.4: La varianza: la dispersione delle ipotesi attorno alla predizione media $\bar{h}(x)$ (in rosso).

Esempio — L'analisi delle freccette



Il centro è la funzione vera, ogni freccetta è un modello addestrato su un training set diverso. **Alto bias e bassa varianza** (in basso a sinistra) corrisponde all'**underfitting**; **basso bias e alta varianza** (in alto a destra) all'**overfitting**. L'analogia non è perfetta per il ML, ma aiuta.

15.3 Bias, varianza e regolarizzazione

Ricordiamo la loss regolarizzata:

$$\text{Loss}(\mathbf{w}) = \underbrace{\sum_p (d_p - o(\mathbf{x}_p))^2}_{\text{termine sui dati}} + \underbrace{\lambda \|\mathbf{w}\|^2}_{\text{penalità}}.$$

Variando λ si ottengono soluzioni meno regolarizzate (complesse, λ basso) o più regolarizzate (meno complesse, λ alto). (L'intercetta è tipicamente esclusa dalla penalità.)

Esempio di Bishop: 25 dataset estratti dalla senoide, modello (una rete RBF) addestrato su ciascuno con diversi valori di λ .

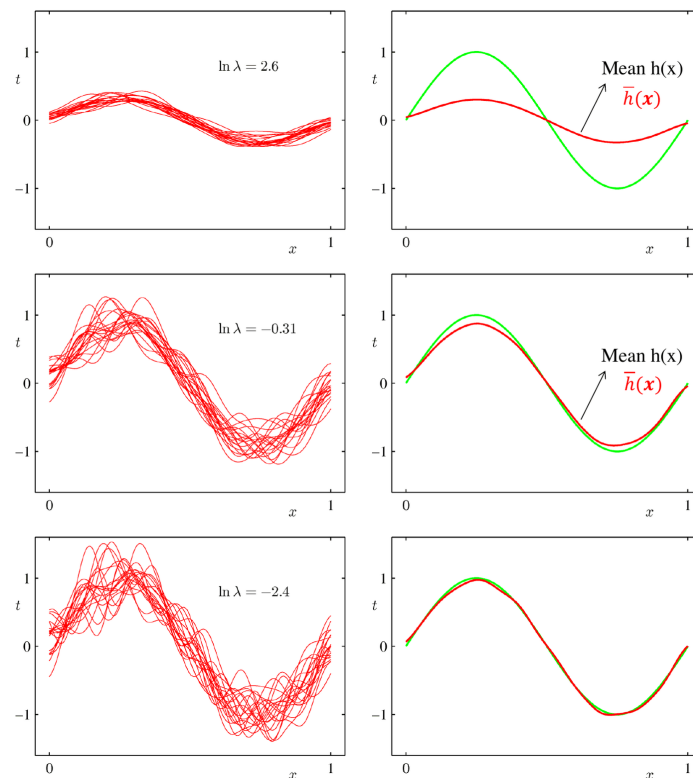


Figura 15.5: Effetto di λ su bias e varianza: $\ln \lambda = 2,6$ (in alto), $-0,31$ (al centro), $-2,4$ (in basso). A sinistra le 25 ipotesi, a destra la loro media $\bar{h}(x)$ (rossa) e la funzione vera (verde).

- **λ alto: alto bias, bassa varianza.** La penalità domina, il modello dipende poco dai dati ed è “troppo rigido”: le curve si somigliano ma la loro media è lontana dalla senoide.
- **λ medio:** bias più basso, varianza più alta. Diminuendo λ si dipende di più dal particolare training set.
- **λ molto basso: basso bias, alta varianza.** l'errore di test per un singolo training set può essere alto (per la componente di varianza), ma anche se le curve sono molto diverse tra loro, **in media** sono corrette (grafico a destra).

Un modello **sovra-regolarizzato** (grande λ) ha bias alto; uno **sotto-regolarizzato** (piccolo λ) ha varianza alta. Di nuovo, **il compromesso**: il minimo di $\text{bias}^2 + \text{varianza}$ coincide circa con il minimo dell'errore di test.

La stessa figura vista nella validazione ([09 - Validazione \(parte 1\) - model selection e assessment](#)) mostra, al crescere della complessità, errori di training e test su 100 training set: un

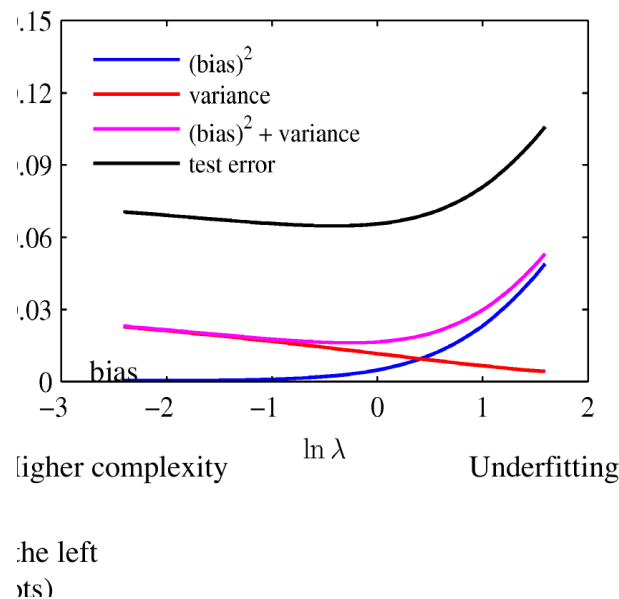


Figura 15.6: Il compromesso bias-varianza al variare di $\ln \lambda$. Attenzione: qui la complessità è più alta a **sinistra** (al contrario dei grafici usuali).

modello complesso ha **bias basso e varianza alta**, quindi rischio di overfitting, e a seconda del training set si può essere fortunati o sfortunati.

Suggerimento — Il collegamento con la SLT

Bias e varianza sono un'altra faccia del compromesso tra adattamento ai dati e complessità: il bias corrisponde all'incapacità del modello di rappresentare la funzione (alto R_{emp} , underfitting), la varianza alla sensibilità ai dati specifici (alta VC-confidence, overfitting).

15.4 Ensemble learning

L'idea degli **ensemble** è sfruttare **più modelli** combinandone le risposte.

Nel caso più semplice (**schema a voto**):

- **Regressione:** comitato a media semplice, $o(\mathbf{x}) = \frac{1}{K} \sum_{i=1}^K h_i(\mathbf{x})$.
- **Classificazione:** voto su molti classificatori (purché diano risposte **diversificate**), oppure si classifica dopo aver mediato le uscite continue.

In alternativa (**stacking**) il combinatore è a sua volta un modello di ML.

Teorema — Il comitato non è peggiore della media dei suoi membri

Per una loss convessa come l'errore quadratico, l'errore del comitato è minore o uguale alla media degli errori dei singoli modelli:

$$\text{loss}(E_i[h_i]) \leq E_i[\text{loss}(h_i)].$$

(È la disuguaglianza di Jensen.) Esempio con target $t = 5$ e due modelli che predicono 2 e 4: la media degli errori è $\frac{(5-2)^2 + (5-4)^2}{2} = 5$, mentre l'errore del comitato ($o = 3$) è $(5-3)^2 = 4$. Con predizioni 6 e 4: media degli errori $\frac{1+1}{2} = 1$, errore del comitato $(5-5)^2 = 0$.

15.4.1 Bagging (bootstrap aggregating)

- Si addestrano K classificatori su **sottoinsiemi diversi** del training set, differenziati con il **bootstrap** (ricampionamento con reinserimento).
- Regressione: si usa la media; classificazione: voto dei K classificatori.

Il legame con bias-varianza è diretto (si ricordi il caso “ λ basso”): modelli ad **alta varianza** e **basso bias** possono funzionare bene **in media**. In altri termini, **la media delle h riduce la varianza** senza aumentare il bias. Inoltre il bagging tende ad aumentare il **margin** di un classificatore: la media di K curve di separazione tende a stare “nel mezzo” (*in medio stat virtus*).

15.4.2 Boosting (es. AdaBoost)

Se i modelli commettono gli **stessi errori**, combinarli non serve. Il boosting li differenzia di proposito:

1. addestra i classificatori in sequenza concentrandosi sugli **errori**: il secondo dà più peso alle istanze sbagliate dal primo (ad esempio sono più probabili nel suo training set), e così via;
2. combina i risultati con un **voto pesato**, dando più peso ai classificatori con errore più basso.

Se non lo si ferma, il boosting impara a classificare correttamente **tutte** le istanze di training usando **weak learner** (classificatori appena migliori del caso, con errore $< 1/2$ nel binario), costruendo in modo incrementale modelli complessi. Può migliorare molto le prestazioni di learner deboli; riduce il bias senza aumentare la varianza (resiste all’overfitting) e, come il bagging, tende a produrre curve di separazione “nel mezzo” (margin massimo). Però **soffre con dati rumorosi** (che vengono pesati sempre di più), e non risolve di per sé il problema underfitting/overfitting.

15.5 Feature selection (cenni)

Trovare il sottoinsieme delle feature più informative per il problema porta grandi benefici: **riduzione della dimensionalità** (effetti sulla VC-dim delle reti, sulla curse of dimensionality...), filtraggio di informazione irrilevante e rumore (con possibili forti miglioramenti delle prestazioni), **interpretabilità** (trovare descrittori utili). Ma:

- è **computazionalmente difficile** (numero enorme di sottoinsiemi possibili, e ogni valutazione richiede di riaddestrare);
- si usa tipicamente una **ricerca euristica** (greedy, algoritmi genetici o altre tecniche di ottimizzazione), includendo o meno il learner;
- i risultati sono particolarmente sensibili al metodo di validazione: attenzione al **feature subset selection bias**, cioè all’errore di fare la selezione sull’intero dataset (vedi le slide sulla cross-validation).

Altri argomenti collegati: PAC learning; confronto tra principi induttivi (SRM, MDL, inferenza bayesiana, regolarizzazione) rispetto al controllo della complessità.

In sintesi — Sintesi

L'errore atteso (sui training set) si scompone in **varianza + bias² + rumore**. Modelli rigidi (o molto regolarizzati) hanno alto bias e bassa varianza (underfitting); modelli flessibili (poco regolarizzati) basso bias e alta varianza (overfitting); il rumore è irriducibile. Gli **ensemble** sfruttano la decomposizione: il **bagging** media modelli ad alta varianza riducendola; il **boosting** combina learner deboli concentrandosi sugli errori.

Domande — Possibili domande d'esame

- Derivare la decomposizione bias-varianza per la loss quadratica.
- Cosa rappresentano bias, varianza e rumore? Quale componente è irriducibile?
- Come variano bias e varianza con il parametro di regolarizzazione λ ?
- Perché un comitato non è peggiore della media dei suoi membri?
- Differenza tra bagging e boosting; perché il bagging riduce la varianza?

Reti neurali convoluzionali (CNN)

Le reti neurali hanno un'enorme quantità di applicazioni di successo, grazie alle prestazioni e al fatto che si configurano facilmente come primo approccio per apprendere funzioni arbitrarie di forma ignota, senza assumere un modello statistico a priori. I task **sub-simbolici** (dati sensoriali, controllo motorio, elaborazione visiva) sono il loro cavallo di battaglia.

Questa lezione presenta un esempio storico e paradigmatico: le **reti convoluzionali** (CNN, *ConvNet*, *LeNet*). Mostra la **flessibilità nella progettazione** delle reti, come si cambia il progetto per trattare un'applicazione specifica, e come si realizza una rete feedforward **non completamente connessa** per un problema preciso. Le CNN sono anche un'istanza storica di rete **profonda**, e fanno da ponte verso il deep learning.

16.1 Il problema: riconoscere caratteri

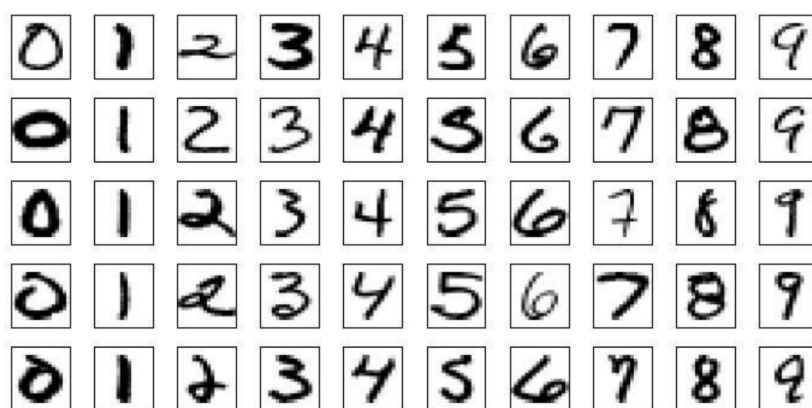


Figura 16.1: Esempi dal dataset ZIP code: immagini normalizzate 16×16 a 8 bit in scala di grigi (Le Cun et al., 1990). Il benchmark MNIST è “la drosfila del ML”.

Un primo approccio è una rete standard con **256 input** (uno per pixel, 16×16): si ottiene un errore del 5–20%, con molti errori dovuti alla **mancanza di invarianza** a rotazioni, traslazioni, ecc. Una rete completamente connessa tratta ogni pixel come un input indipendente: non “sa” che pixel vicini sono correlati né che uno stesso tratto può comparire in posizioni diverse.

Suggerimento — L'idea di base

Sfruttare l'**architettura** per includere **conoscenza a priori** nella rete: - **connessioni locali** (campi recettivi locali): ogni unità vede solo una piccola zona dell'immagine, ed estrae **feature locali**; - **condivisione dei pesi** (*weight sharing*) tra unità diverse: la **stessa operazione** viene applicata a parti diverse dell'input, perché le feature di un carattere possono comparire ovunque nell'immagine (indipendentemente da posizione, piccole rotazioni...). Si riduce il numero di parametri liberi pur mantenendo una connettività complessa.

16.2 La convoluzione

Il nome viene dall'**operatore di convoluzione**: una media pesata di una funzione f , pesata da un'altra funzione g che scorre nel tempo:

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau) g(t - \tau) d\tau.$$

L'immagine di g è limitata (non nulla solo su un intervallo): g agisce come una **finestra scorrevole**.

16.2.1 Convoluzione 1D: una unità su uno stream

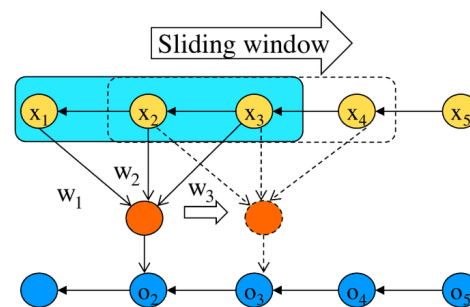
Un'unità con tre pesi che scorre su una sequenza di input:

$$out_t = \sum_{i=1}^3 w_i x_{t+i-2}.$$

$$out_t = \sum_{i=1}^3 w_i x_{t+i-2}$$

Exercise: compute o_2 and o_3 according to the equation above (start with $t=2$)

Weights are tuned as usual by learning:
An adaptive filter



Weight sharing
for the two instances of units
More units operate as a “filter bank”

Figura 16.2: Convoluzione 1D: la stessa unità (con gli stessi pesi) scorre sull'input.

(Esercizio: con $t = 2$, $o_2 = w_1x_1 + w_2x_2 + w_3x_3$; con $t = 3$, $o_3 = w_1x_2 + w_2x_3 + w_3x_4$.) I pesi sono **condivisi** tra le istanze dell'unità e vengono appresi come al solito: l'unità è un **filtro adattivo**. Più unità diverse formano un **banco di filtri**. Questa rete si chiama anche *Time-Delay Neural Network*.

16.2.2 Convoluzione 2D

Su un'immagine si usa un **kernel 2D**, ad esempio 3×3 (il **campo recettivo locale** dell'unità), che si sposta di un pixel alla volta (**stride 1**). Il **padding** gestisce i bordi. Il kernel scorre sull'immagine e produce una **feature map** (le feature estratte dal filtro).

In termini di reti neurali: si percorre l'immagine con **lo stesso neurone**, che usa connessioni locali.

Nota — Forma usata nelle librerie

La convoluzione 2D di un'immagine I con un kernel K è $S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(m, n) K(i - m, j - n)$. Molte librerie per CNN implementano in realtà la **cross-**

- **One kernel** (one local receptive field), 1 pixel stride (stride 1 for downsizing)
- **Padding** is used to treat the border

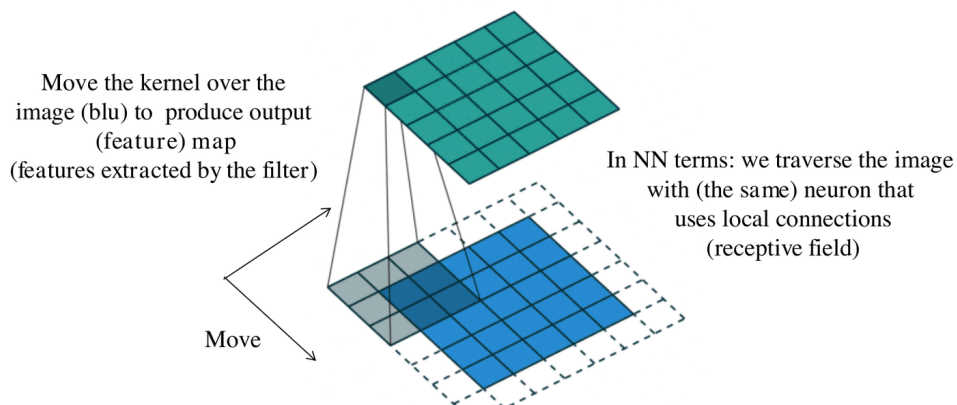


Figura 16.3: Convoluzione 2D: il kernel scorre sull'immagine e produce la feature map.

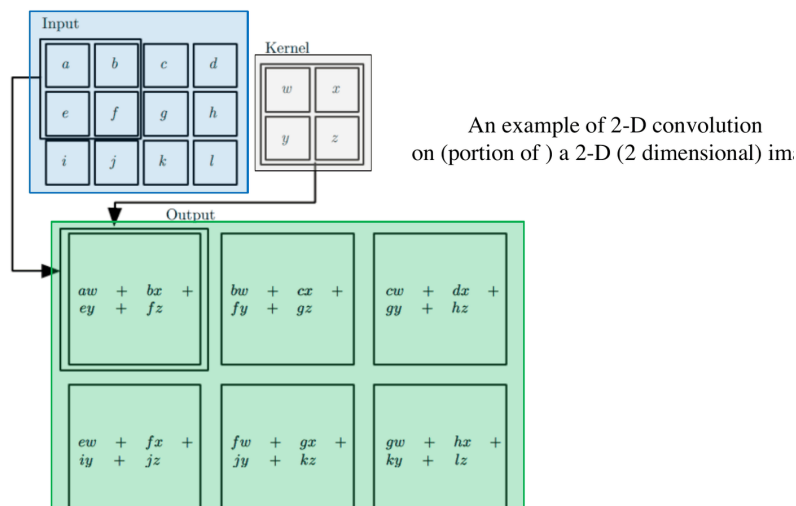


Figura 16.4: Esempio di convoluzione 2D di un input 3×4 con un kernel 2×2.

correlazione: $S(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n)$ (per i dettagli: *Deep Learning book*, sez. 9.1; non richiesto all'orale).

16.2.3 Stride maggiore di 1

Con **stride** > 1 il kernel “salta” pixel (ad esempio due alla volta): è un **sotto-campionamento**, e produce una feature map ridotta.

16.3 Una rete di “filtri”

I pesi delle unità sono un **filtro addestrato** a rilevare certe feature o pattern nell'immagine. Il progetto dell'architettura prevede:

- **campo recettivo piccolo e locale:** grazie a questo vincolo, i filtri appresi rispondono al massimo a pattern **spazialmente locali** (come nella corteccia visiva degli animali);
- **lo stesso filtro applicato su tutta l'immagine:** le feature vengono rilevate **indipendentemente dalla posizione**, ottenendo l'**invarianza per traslazione**;

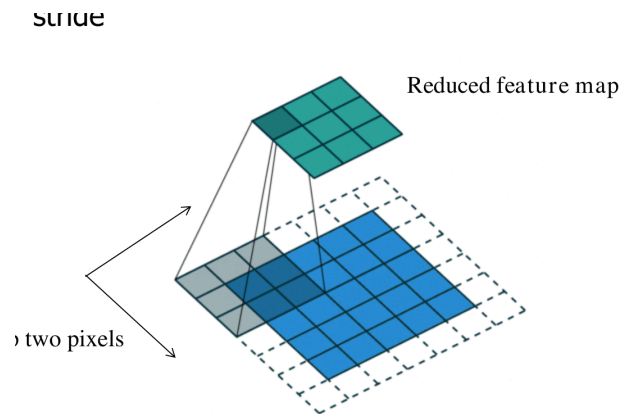


Figura 16.5: Convoluzione con stride 2: la feature map risultante è ridotta.

- **filtri appresi:** i filtri che negli algoritmi tradizionali di elaborazione di immagini erano progettati a mano vengono **appresi**. L'indipendenza dalla conoscenza a priori e dallo sforzo umano nel progettare le feature è un grande vantaggio;
- **molti strati impilati:** le unità delle feature map rappresentano aree via via più grandi dell'immagine originale (assemblando aree delle mappe precedenti).

16.4 Pooling

Il **pooling** riduce la feature map:

- con sotto-campionamento (stride > 1 , visto sopra);
- con una **media** (semplice o pesata);
- con il **max pooling** (il più comune).

In pratica, invece di produrre un valore per ogni pixel di input, se ne produce uno per un insieme (rettangolare) di pixel, prendendo la media o il massimo delle uscite.

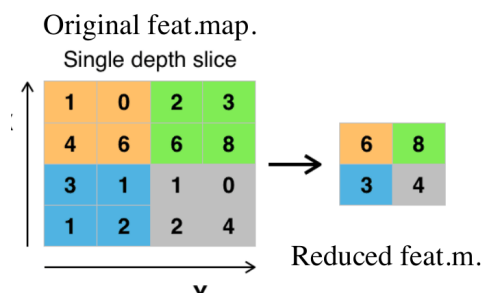


Figura 16.6: Max pooling con filtro 2x2 e stride 2.

Il pooling aiuta anche a rendere la rappresentazione **approssimativamente invariante a piccole traslazioni** dell'input: il massimo (o la media) cambia poco se l'input si sposta di poco, quindi un'uscita alta resta alta per una versione leggermente traslata.

16.5 La CNN nel complesso

Una CNN sfrutta la condivisione dei pesi per realizzare una finestra scorrevole di campi locali su un segnale, estesa alle immagini 2D e ripetuta su molti strati (feature map), alternata a operazioni di pooling. I quattro ingredienti:

1. **connessioni locali;**
2. **pesi condivisi;**
3. **pooling;**
4. **molti strati.**

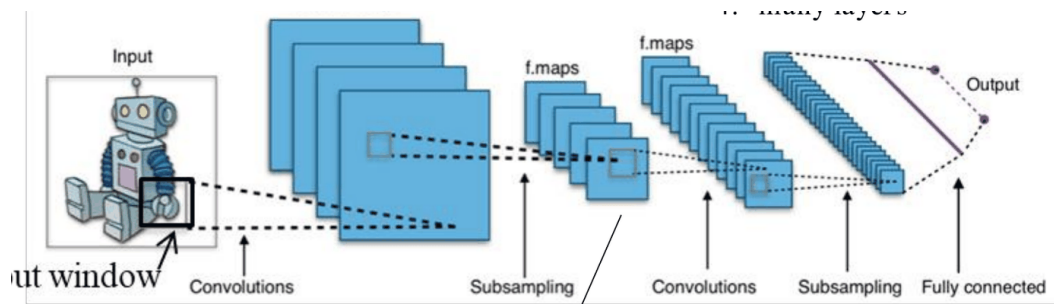


Figura 16.7: Una CNN completa: convoluzioni e sotto-campionamenti alternati, seguiti da strati completamente connessi. Il numero di filtri (feature map) può crescere negli strati più alti.

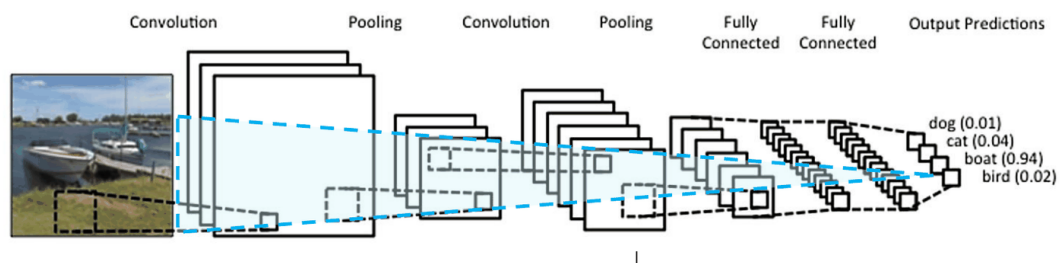


Figura 16.8: Il “cono” dall’immagine alle mappe finali: il campo recettivo delle unità finali è molto più ampio di quello delle unità dei primi strati, perché sono collegate indirettamente a gran parte dell’immagine.

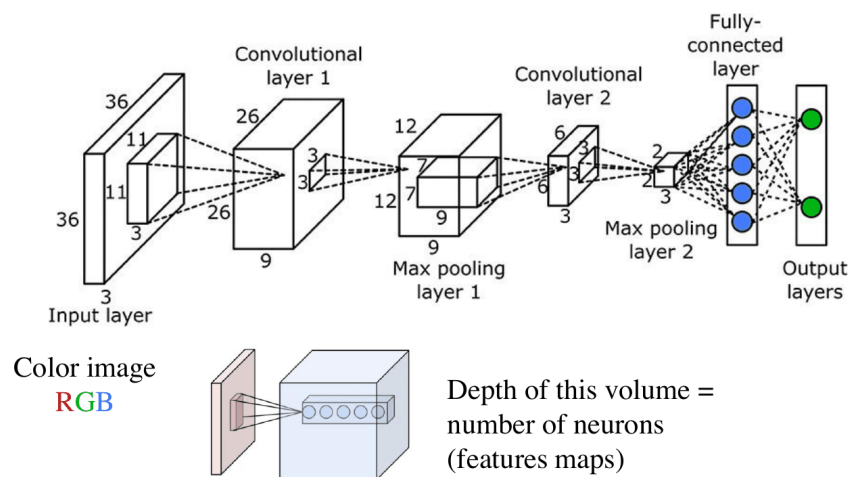


Figura 16.9: Un esempio con le dimensioni; per un’immagine a colori (RGB) la profondità del volume è il numero di feature map.

In sintesi — Vantaggi delle CNN

- **Connessioni locali e pesi condivisi:** rilevano motivi/pattern locali nelle immagini, in modo **invariante alla traslazione** (alla posizione del pattern); **riducono il numero di parametri liberi** pur mantenendo le connessioni, il che è una forma di **regolarizzazione**.

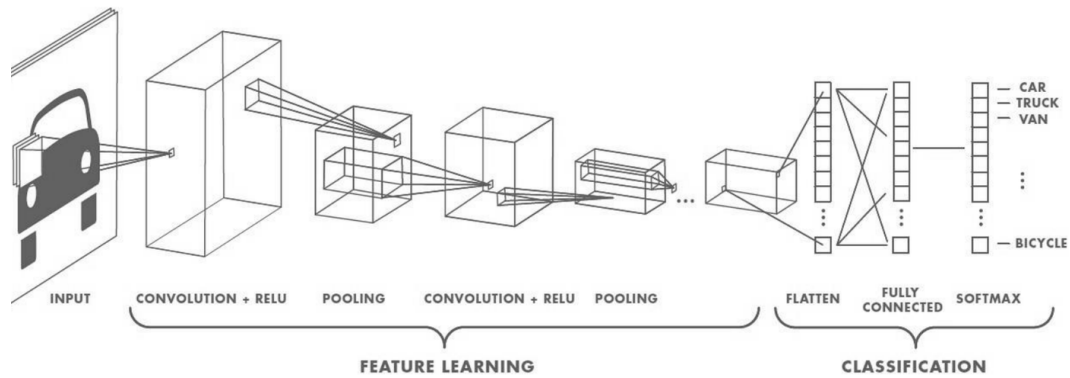


Figura 16.10: Un'architettura tipo AlexNet: una parte di

- **Pooling:** riduce la dimensione della rappresentazione a ogni strato (una **piramide** di strati) e aiuta l'invarianza a piccoli spostamenti e distorsioni.
- **Molti strati:** le operazioni 1 e 2 applicate a tutte le feature map producono un'**astrazione progressiva** delle feature, permettendo di **comporre primitive** (feature di basso livello, apprese una volta sola) in tutte le combinazioni possibili negli strati successivi.
- Sono un'istanza storica di **rete profonda**, e sono state studiate analogie con il sistema visivo (neuroscienze).

16.6 Come si usano

L'addestramento avviene tipicamente con la **backpropagation**, con le euristiche note e alcune specializzazioni (per la condivisione dei pesi, il pooling...). Visto l'uso tipico di reti enormi e grandi quantità di dati, molti iperparametri vengono fissati **per esperienza** o su suggerimento di esperti, perché sarebbe troppo costoso fare cross-validation su grandi spazi di iperparametri. Addestramento, effetti di regolarizzazione e buon comportamento di queste reti enormi saranno discussi nel quadro del deep learning.

16.7 Risultati storici

16.7.1 Le architetture di LeCun sul dataset ZIP code

Rete	Connessioni	Pesi
Net-1	2570	2570
Net-2	3214	3214
Net-3	1226	1226
Net-4	2266	1132
Net-5	5194	1060

Con i pesi condivisi il numero di **connessioni** può crescere mentre il numero di **pesi** (parametri liberi) resta basso.

Net-5 raggiunge un errore dell'**1,6%** con 5194 connessioni e soli 1060 pesi. Oggi si arriva allo 0,8% (con reti e SVM) e anche meno.

16.7.2 MNIST

Il dataset completo ha 60.000 immagini di training e 10.000 di test. I progressi (1998–2012) sono documentati sul sito di LeCun. Con il deep learning (MLP o CNN con molti strati, pre-addestramento non supervisionato strato per strato) si è arrivati allo 0,39% (2006) e 0,35% (2011): il migliore nel 2012 è lo 0,23% (comitato di reti convoluzionali). 0,21% nel 2016.

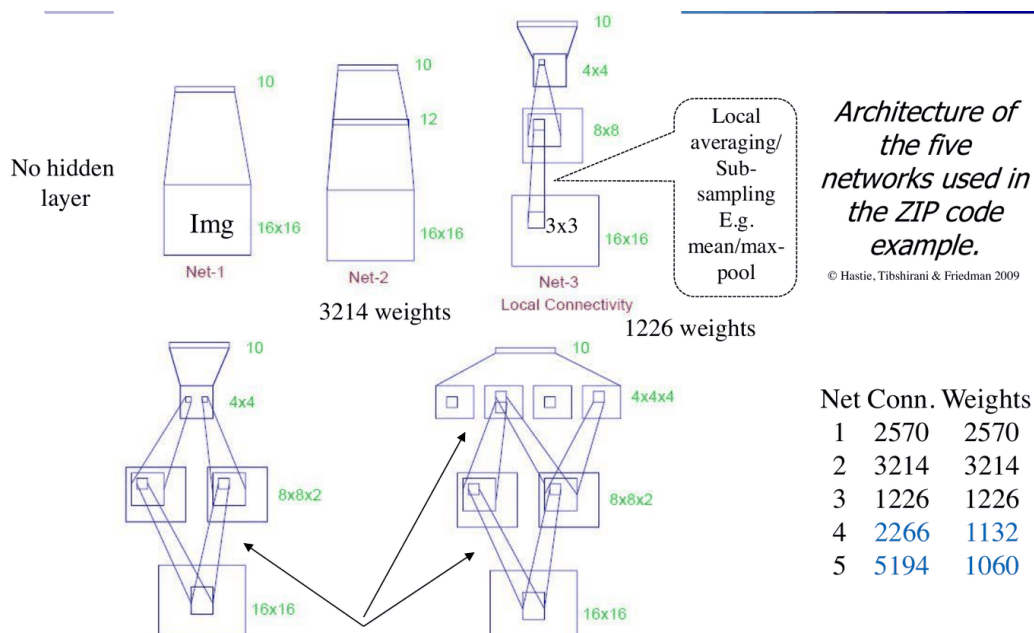


Figura 16.11: Le cinque reti usate da LeCun sul problema ZIP code: da una rete senza strati nascosti (Net-1) a reti con connessioni locali (Net-3) e pesi condivisi (Net-4, Net-5).

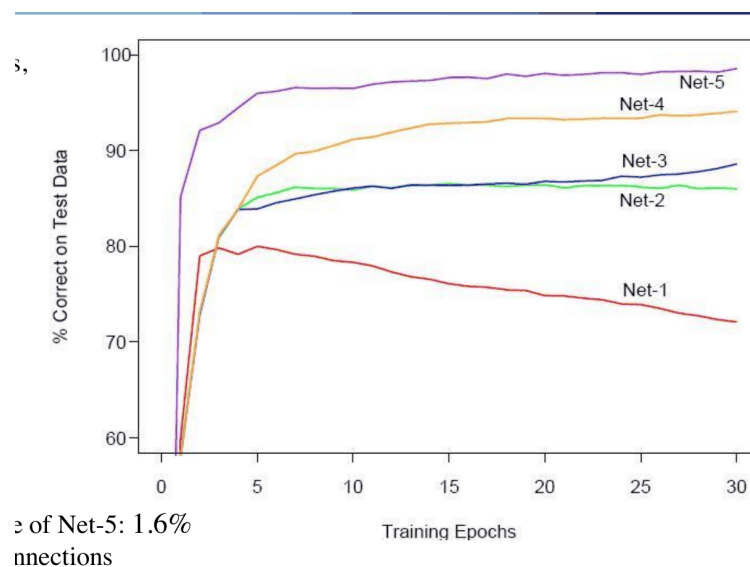


Figura 16.12: Curve di prestazione sul test per le cinque reti (Le Cun, 1989). L'errore di training è zero per tutte, su un insieme ridotto di circa 300 immagini.

Nota — Anche un MLP profondo funziona?

Un articolo del 2010 (*Neural Computation*) mostra che la “buona vecchia” backpropagation on-line su un semplice MLP ottiene lo 0,35% su MNIST: bastano **molti strati**, **molti neuroni** per strato, **molte immagini deformate** per il training e **GPU** per accelerare. La rete 784-2500-2000-1500-1000-500-10 ha 12 milioni di pesi e si addestra in 2 ore. Il controllo della complessità è ottenuto deformando le immagini per avere molti più esempi (*data augmentation*).

16.7.3 Riconoscimento facciale e ImageNet

- **DeepFace** (CVPR 2014): reti convoluzionali a 9 strati addestrate su 4 milioni di immagini di più di 4000 persone; riconosce se due foto mostrano la stessa persona con accuratezza del 97,25% (umani: 97,53%).
- **AlexNet** (Krizhevsky, Sutskever, Hinton, NIPS 2012): una CNN profonda addestrata su 1,3 milioni di immagini ad alta risoluzione di ImageNet (ILSVRC-2010) in 1000 classi. Errori top-1 e top-5 del 39,7% e 18,9% (dal precedente ~26%), molto meglio dello stato dell'arte. La rete ha **60 milioni di parametri** e 500.000 neuroni: cinque strati convoluzionali (alcuni seguiti da max pooling) e due strati completamente connessi con softmax finale a 1000 classi. Per velocizzare l'addestramento: neuroni **non saturanti** (ReLU) e un'implementazione GPU molto efficiente. Per ridurre l'overfitting negli strati densi: un nuovo metodo di regolarizzazione, il **dropout**.

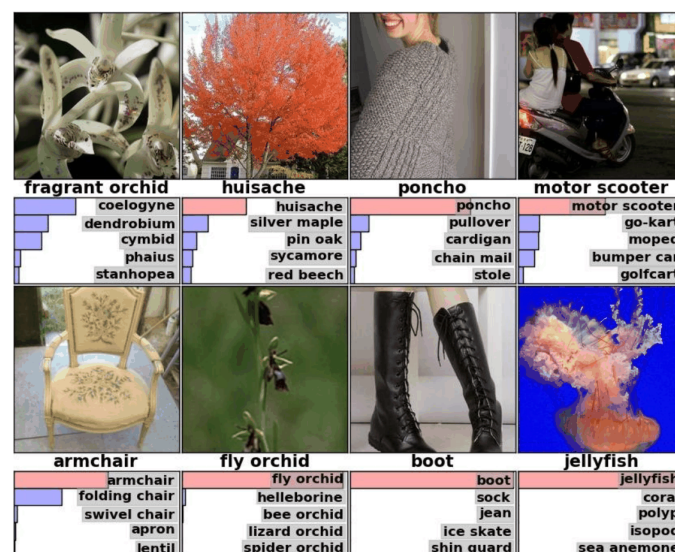


Figura 16.13: Predizioni top-5 di AlexNet su immagini casuali del test set.

16.8 CNN moderne e GPU

Esistono molte varianti e architetture specializzate per le immagini (corso *Intelligent Systems for Pattern Recognition*), con grande impatto su Pattern Recognition e Computer Vision; le basi neuroscientifiche sono trattate nel corso di *Computational Neuroscience*. Si possono anche riusare le feature di modelli **già addestrati** su grandi benchmark (AlexNet, GoogLeNet, VGG, ResNet...): sono disponibili molte CNN pre-addestrate per classificazione, segmentazione, riconoscimento facciale, rilevamento di testo.

Le implementazioni efficienti sfruttano le **GPU** tramite la rappresentazione a **tensori**. La moltiplicazione di matrici (cioè i prodotti scalari) è un'operazione lineare che le GPU eseguono in modo molto efficiente, parallelizzando le somme e i prodotti indipendenti. Molte operazioni di kernel si possono scrivere come un'unica moltiplicazione tra matrici a blocchi; la notazione a **tensori** (array multidimensionali con l'operazione *tensordot*, ad esempio in TensorFlow o NumPy) permette di farlo senza costruire a mano matrici a blocchi complesse.

Nota — Oltre le CNN: le Capsule Network

Le **Capsule Network** (Sabour, Frosst, Hinton, 2017) usano sotto-reti specializzate: una “capsula” è un gruppo di neuroni che produce sia un parametro che indica se un’entità è presente, sia un vettore di **parametri di posa** (posizione, orientamento) rispetto a una versione canonica. Al posto del max pooling usano un meccanismo di **routing** (l’idea che il cervello instrada l’informazione visiva di basso livello verso la capsula più adatta). Ottengono ottimi risultati su MNIST, riconoscendo molto meglio di una CNN cifre sovrapposte.

Domande — Possibili domande d’esame

- Quali sono le idee chiave delle CNN (connessioni locali, pesi condivisi, pooling, molti strati) e cosa comporta ciascuna?
- Cos’è la convoluzione in una rete neurale? Cosa sono kernel, stride, padding, feature map?
- A cosa serve il pooling? Come funziona il max pooling?
- Perché la condivisione dei pesi è una forma di regolarizzazione?
- Come cresce il campo recettivo con la profondità?

Capitolo 17

Deep learning

Questa lezione introduce il **deep learning** (DL) dal punto di vista del corso: non i dettagli delle architetture per le applicazioni (che si vedranno nei corsi successivi del curriculum AI, come Pattern Recognition e NLP), ma una **visione d'insieme** delle reti con molti strati, i concetti che le rendono efficaci (**servono davvero molti strati?**, **representation learning**, **rappresentazioni distribuite**) e le principali **tecniche** di addestramento, molte delle quali utili anche nel caso “base”.

17.1 Che cos'è il deep learning

Nota — LeCun, Bengio, Hinton, *Nature* (2015)

“Il deep learning permette a modelli computazionali composti da più strati di elaborazione di apprendere rappresentazioni dei dati con più livelli di astrazione. Questi metodi hanno migliorato drasticamente lo stato dell'arte nel riconoscimento del parlato, nel riconoscimento visivo di oggetti, nel rilevamento di oggetti e in molti altri domini.”

17.1.1 Un po' di storia

Dal 2010 circa il deep learning ha avuto un forte impatto sull'industria, per tre fattori:

- **grandi dataset** disponibili (milioni di immagini);
- **potenza di calcolo** (le GPU sono il fattore abilitante);
- **nuove idee** per facilitare l'addestramento.

L'apprendimento con architetture profonde non è più “infattibile”. I nuovi risultati allo stato dell'arte nel riconoscimento del parlato, nella traduzione automatica (Google Neural Translator dal 2016, DeepL) e nel riconoscimento di immagini hanno attratto moltissimi ricercatori. In passato ci si aspettava un miglioramento cambiando la scala di modelli e dati, ma non necessariamente un tale salto di prestazioni. Tutte le grandi aziende informatiche basano oggi le loro strategie di ricerca e sviluppo su questi approcci.

17.1.2 Il quadro generale

Il DL è un **quadro generale** che include modelli diversi:

- **reti neurali profonde** (MLP con molti strati);
- **reti convoluzionali** (vincenti in computer vision);
- *deep belief network* e approcci generativi;
- reti **ricorrenti** e **ricorsive**;
- ...

Si contrappone ai **modelli “shallow”** (superficiali), ad esempio reti con meno di 3 strati. Gli aspetti comuni ai vari approcci sono:

- **più strati** di unità di elaborazione **non lineari**;
- apprendimento (supervisionato o non supervisionato) di **rappresentazioni delle feature** in ogni strato, con gli strati che formano una **gerarchia** da feature di basso livello a feature di alto livello (diversi livelli di astrazione);
- quindi, una **rappresentazione gerarchica, sparsa e distribuita** dei dati.

(Le differenze dipendono dai modelli specifici: max pooling nelle CNN, addestramento basato sull'energia nelle RBM, ecc.)

17.1.3 Gli ingredienti concettuali: astrazione gerarchica

Si ricordi lo XOR risolto con due strati (06 - Reti neurali (parte 1) - dal neurone al MLP): lo strato nascosto crea una **ri-rappresentazione interna** che rende facile il compito allo strato di uscita, e questa composizione di operazioni intermedie si può estendere su molti livelli di astrazione.

- **Feature learning** non supervisionato o semi-supervisionato ed estrazione gerarchica di feature, **invece del feature engineering** (feature estratte a mano, per esperienza, come la texture nelle immagini): si passa al concetto di **apprendere rappresentazioni dei dati**.
- **Livelli di astrazione crescenti** attraverso gli strati. Un'immagine si può rappresentare come vettore di intensità dei pixel, oppure in modo più astratto come insieme di bordi, regioni di forma particolare, ecc.: **i bordi formano motivi, i motivi si assemblano in parti, le parti formano oggetti...** su molti livelli.
- Funziona al meglio quando l'input ha una qualche forma di **struttura** (spaziale, temporale, ...).

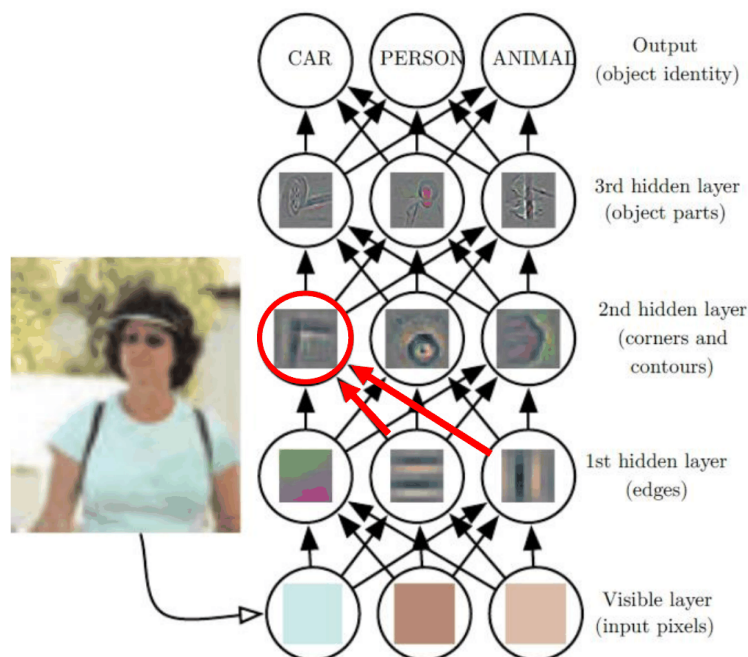


Figura 17.1: Dai pixel all'identità dell'oggetto: una serie di strati nascosti estrae feature sempre più astratte (bordi → angoli e contorni → parti di oggetti).

Il deep learning risolve la difficoltà di una mappatura complicata (pixel → identità) **scomponendola in una serie di mappature semplici annidate**, ciascuna descritta da uno strato

diverso. La nuova rappresentazione **semplifica la classificazione** nell'ultimo strato. Ci sono analogie con il sistema visivo biologico.

Nelle reti queste feature astratte **si apprendono dagli esempi**: con la backpropagation, l'errore sullo strato di uscita (ad esempio per distinguere auto e persone) si propaga tramite i delta attraverso gli strati, e le unità si specializzano per fornire feature di alto livello utili a quello scopo (la ruota per l'auto, la mano per la persona...). Il legame tra le feature sviluppate automaticamente e la realtà è un aspetto affascinante; e sorprendentemente i primi filtri estraggono linee orizzontali o verticali, proprio come farebbe un semplice filtro convolutivo lineare.

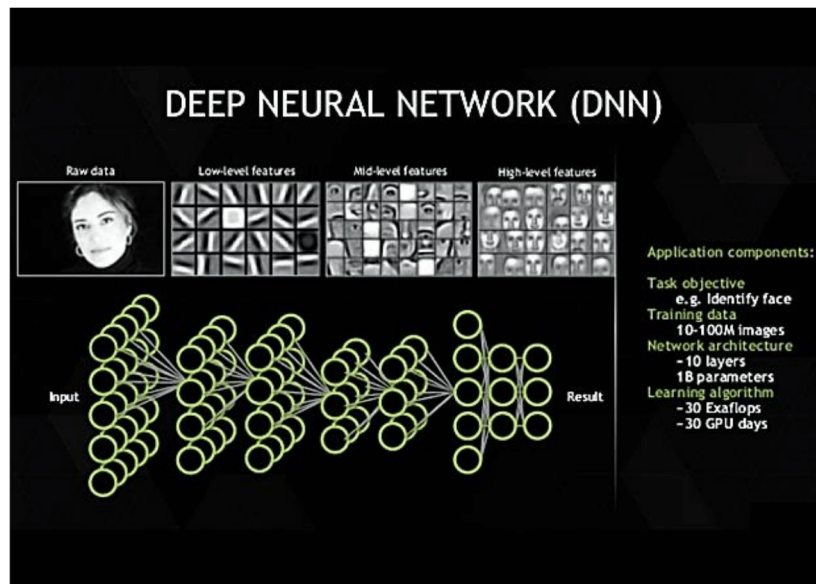


Figura 17.2: Un altro esempio: feature di basso, medio e alto livello in una rete profonda per volti.

17.1.4 Combinare per generalizzare

Le feature astratte si possono **riusare** ai livelli superiori (es. genere, occhiali...) e **combinare** per generalizzare a casi mai visti in addestramento: se la rete ha visto donne senza occhiali e uomini con occhiali, può rappresentare facilmente anche una donna con gli occhiali.

Operations on vector representations learned
model and then used to generate new images.

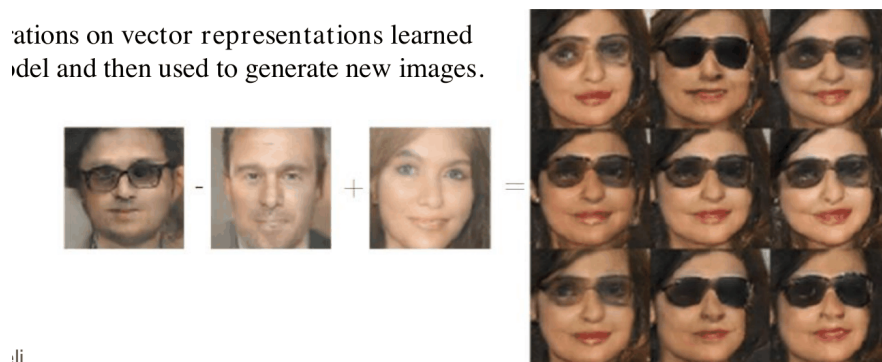


Figura 17.3: Operazioni sulle rappresentazioni vettoriali apprese dal modello e usate per generare nuove immagini: “uomo con occhiali – uomo + donna = donna con occhiali”.

Lo stesso accade nel linguaggio: nello spazio vettoriale sviluppato dagli strati interni di un modello profondo,

- $\text{Rep}(\text{re}) - \text{Rep}(\text{maschio}) + \text{Rep}(\text{femmina}) \approx \text{Rep}(\text{regina})$: la differenza tra “re” e “maschio” cattura il concetto di monarchia;

- $\text{Rep}(\text{Parigi}) - \text{Rep}(\text{Francia}) + \text{Rep}(\text{Polonia}) \approx \text{Rep}(\text{Varsavia})$: la differenza cattura il concetto di capitale.

17.2 Servono davvero molti strati?

17.2.1 Un esempio dai circuiti logici

L'idea generale (**no-flattening**): quando una funzione si rappresenta in modo compatto con un'architettura profonda, potrebbe richiedere un'architettura molto grande per essere rappresentata con una insufficientemente profonda.

Un circuito logico a **due strati** può rappresentare **qualsiasi** funzione booleana, scritta come somma di prodotti (forma normale disgiuntiva): porte AND nel primo strato (con eventuale negazione degli input) e una porta OR nel secondo. Ma **con circuiti di profondità due, la maggior parte delle funzioni booleane richiede un numero esponenziale di porte** (rispetto alla dimensione dell'input).

Esempio — La funzione di parità

La **parità dispari** estende lo XOR a N input: restituisce 1 se e solo se il numero di 1 tra gli N bit è dispari. (Si assume di disporre anche dei NOT; con 2 input bastano 3 porte.)

Con 2 strati bisogna elencare tutte le configurazioni positive: per 3 input, $x_1x_2x_3 + x_1\bar{x}_2\bar{x}_3 + \bar{x}_1x_2\bar{x}_3 + \bar{x}_1\bar{x}_2x_3$ (1 se l'input è 111, 100, 010 o 001), cioè 4 AND + 1 OR = 5 porte. Per 4 input servono 9 porte, per 8 input **129 porte**, per N input $2^N/2 + 1 = 2^{N-1} + 1$ porte: un numero **esponenziale**.

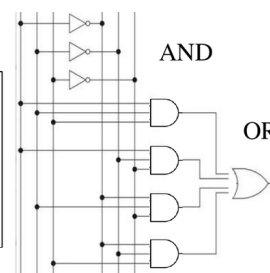
Con $\log N$ strati si usa un **albero binario completo** di XOR: un albero con N foglie ha $N - 1$ nodi interni, ognuno uno XOR da 3 porte AND/OR, per un totale di $3(N - 1)$ porte, un numero **polinomiale**. Per 8 input bastano $3 \times 7 = 21$ porte contro le 129 della soluzione a 2 strati.

- For 3 inputs: 4 AND + 1 OR: **5 gates**

$$x_1x_2x_3 + x_1\bar{x}_2\bar{x}_3 + \bar{x}_1x_2\bar{x}_3 + \bar{x}_1\bar{x}_2x_3$$

1 iff inputs are 111 or 100 or 010 or 001

⇒ All the combinations of the positive patterns



- For 4 inputs: 8 AND + 1 OR: **9 gates**

Figura 17.4: Parità con 2 strati: un numero esponenziale di porte.

Questo però **non vale per tutte le classi di funzioni**: caratterizzare le classi che traggono un vantaggio (esponenziale) da una struttura a più strati è un problema di ricerca aperto.

17.2.2 Il caso delle reti neurali

Ricordiamo che il **teorema di approssimazione universale** mostra che uno strato nascosto basta in generale, ma non garantisce che basti un numero **piccolo** di unità. Esistono però risultati di **no-flattening** (sull'efficienza, non sull'espressività):

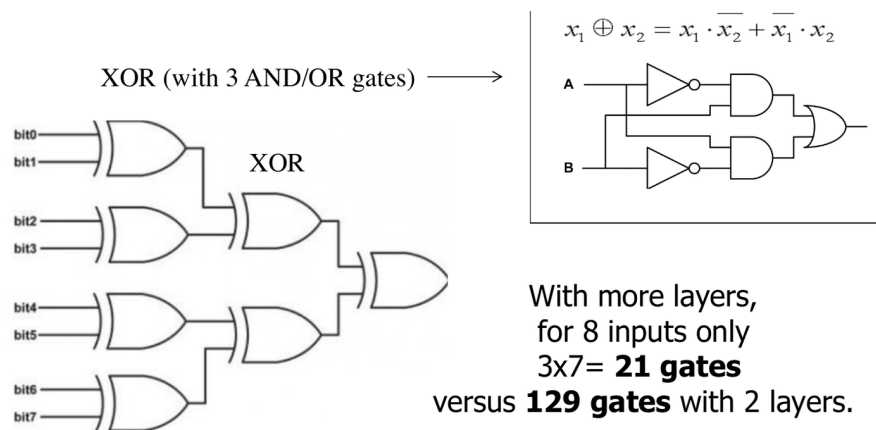


Figura 17.5: Parità con $\log N$ strati: un albero di XOR con un numero polinomiale di porte.

1. **reti shallow:** ci sono casi in cui l'implementazione con un solo strato nascosto richiede un numero **esponenziale** di unità (rispetto alla dimensione n dell'input) o di pesi non nulli. È facile vederlo nel caso binario: le funzioni booleane su $\{0, 1\}^n$ sono 2^{2^n} , e nel caso peggiore può servire un'unità nascosta per ogni configurazione di input da distinguere (come nell'esempio della parità). Non è solo un problema di dimensione della rete: significa anche che diventa **difficile apprendere con pochi esempi** (ed è questo il punto più rilevante per il ML);
2. **no-flattening delle reti profonde:** esistono famiglie di funzioni approssimabili in modo efficiente con profondità maggiore di d , ma che richiedono un modello molto più grande (fino a un guadagno esponenziale) se la profondità è al più d . Esempi per porte logiche (1986), reti con attivazioni standard (1990–1994), ReLU (2014)...

Resta la domanda: è facile addestrare un MLP con molti strati? (Vedi le tecniche più avanti.)

Nota — Analisi teorica (ricerca aperta)

I **teoremi di no-flattening** individuano funzioni composizionali che una rete profonda implementa bene e che non si possono implementare con la stessa efficienza “appiattendolo” la rete (ad esempio passando da un numero lineare a uno esponenziale di unità). Ma non c'è garanzia che un dato task abbia questa proprietà: un elenco completo dei teoremi di no-flattening direbbe esattamente quando le reti profonde sono più efficienti di quelle superficiali.

17.2.3 Perché: la composizionalità

Le reti profonde possono sfruttare la **composizionalità delle rappresentazioni interne**: c'è un guadagno esponenziale di potere rappresentativo perché i concetti semplici rappresentati in uno strato vengono usati come **primitive** dallo strato successivo per rappresentare concetti più complessi, evitando la rappresentazione (e l'apprendimento) esplicita e combinatoria delle feature.

Ad esempio, si devono imparare le singole parole o sotto-pattern, ma non direttamente tutte le loro possibili combinazioni: si può imparare senza vedere tutte le configurazioni. E forse la natura stessa (non solo immagini e linguaggio) ha una struttura gerarchica (fenomeni osservati a scale diverse nello spazio e nel tempo).

Suggerimento — Meno è meglio!

Non bisogna vedere le reti profonde come modelli **enormi** rispetto a piccoli modelli superficiali (solo perché nelle applicazioni si vedono istanze gigantesche: profondo non significa grande). Si possono vedere come modelli **compatti** rispetto a modelli superficiali potenzialmente molto più grandi per lo stesso task (anche esponenzialmente). Quindi sono “qualcosa di meno” e più efficienti. E non solo per ragioni computazionali: **meno unità e meno pesi aiutano l'apprendimento** su task complessi, permettendo di generalizzare bene con **meno esempi** (cioè rendendolo fattibile). Una “piccola rete profonda” da 50.000 unità può sostituire una “enorme rete superficiale” da un milione di unità.

17.2.4 Il bias induttivo

Scegliere un modello profondo codifica una **convinzione molto generale**: che la funzione da apprendere sia una **composizione di funzioni più semplici** (o di fattori di variazione annidati). Se il task corrisponde a questo bias, la forma profonda è adatta, e la generalizzazione risulta migliore proprio grazie ai molti strati.

Task tipicamente gerarchici: la struttura delle **immagini** (composizione di parti grafiche), la struttura del **linguaggio** (testo e parlato), forse la musica; nuovi campi si stanno scoprendo. Ma non necessariamente tutti i dati e i task! È comunque un bias molto più **generale** di vincoli ad hoc per task specifici (come le ϕ scelte a mano nelle LBE o le feature ingegnerizzate). Ma, di nuovo, nulla è magico senza assunzioni.

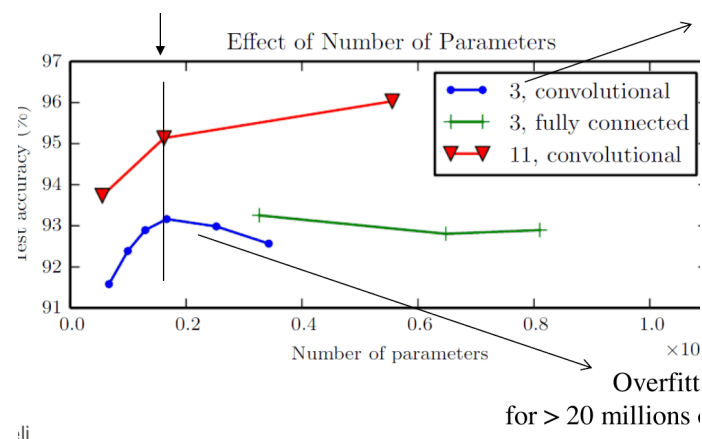


Figura 17.6: Risultati empirici (SVHN, ICLR 2014): a parità di numero di parametri, le reti più profonde generalizzano meglio.

Quando il task corrisponde al bias, i modelli più profondi tendono a funzionare meglio: la generalizzazione è migliore di un modello con meno strati e **lo stesso numero di unità/pesi**.

17.2.5 Maledizione della dimensionalità e bias

Altri argomenti a favore dei modelli profondi riguardano la **curse of dimensionality** e il *manifold learning* (la distribuzione delle immagini e dei testi reali sembra concentrata su una varietà che occupa solo una piccola frazione del volume di tutte le possibili immagini o testi casuali).

- Molti learner si basano su **approssimazioni locali** (un prior di *smoothness* locale): K-NN, kernel locali, alberi di decisione... Ma spesso questo non basta: servono esempi per

generalizzare nell'intorno di ogni regione, e in alta dimensione servono moltissimi esempi per coprire tutto lo spazio. Per distinguere $O(k)$ regioni, tutti questi metodi richiedono $O(k)$ esempi.

- Per lavorare con $O(k)$ esempi su un numero **esponenziale** (in k) di regioni, cioè per generalizzare **non localmente**, servono ipotesi aggiuntive sulla distribuzione che genera i dati (un bias induttivo).
- Si potrebbero fare ipotesi forti e specifiche per ogni task (perdendo generalità). Il deep learning sceglie invece un bias **generale**, legato alla **composizione di funzioni**: si assume che i dati siano generati dalla composizione di fattori o feature, potenzialmente su più livelli di una gerarchia. Quando l'ipotesi è vera, si ottiene un guadagno potenzialmente esponenziale tra numero di esempi e numero di regioni distinguibili, si generalizza non localmente, e si impara con meno esempi.

17.2.6 Questioni pratiche

Quanti strati? Quante unità? Non c'è ancora una risposta generale (è un problema di model selection). In generale le reti profonde usano spesso meno unità per strato, quindi meno parametri e meno dati per generalizzare bene; ma molti strati possono essere **più difficili da ottimizzare**. Gli iperparametri di questi modelli molto grandi possono essere troppi: l'**esperienza** fa la differenza nel fissare molti aspetti architetturali.

17.3 Representation learning

Definizione — Representation learning

“Il representation learning è un insieme di metodi che permette a una macchina di ricevere dati grezzi e scoprire automaticamente le rappresentazioni necessarie per il rilevamento o la classificazione.” (LeCun, Bengio, Hinton, 2015)

I metodi di deep learning sono metodi di representation learning con **più livelli** di rappresentazione (rappresentazioni espresse in termini di altre più semplici). Ma il concetto è più astratto e vale per molti modelli (ad esempio le reti neurali in generale). Si noti che si parla di apprendere la rappresentazione soprattutto per **dati grezzi** (immagini, stringhe di testo), non quando si fornisce già un piccolo insieme di feature.

17.3.1 Idee di base

- Molti compiti di elaborazione dell'informazione sono facili o difficilissimi **a seconda di come l'informazione è rappresentata** (vale in tutta l'informatica).
- Nel ML, **una buona rappresentazione è quella che rende più facile il compito di apprendimento successivo**.
- Progettare le feature a mano è difficile: interi decenni di lavoro per alcune comunità (linguaggio, immagini). Equivale a ingegnerizzare a mano le $\phi(\mathbf{x})$ della LBE.
- L'apprendimento supervisionato di un MLP porta a una **rappresentazione automatica** in ogni strato nascosto, con proprietà che rendono più facile il compito dello strato di uscita (lo XOR!). Cioè $\phi(\mathbf{x}, \mathbf{w})$ viene appresa trovando il miglior $\mathbf{w}$.
- Nelle CNN si usa l'immagine grezza come input e il modello impara a estrarre le feature salienti per il task, senza pattern o feature specificati in anticipo.

Come ottenere o sfruttare le rappresentazioni nascoste (oltre all'addestramento completo di MLP e CNN)?

- **Ottenerle** (caso storico): con l'apprendimento **semi-supervisionato** si impara una rappresentazione dai dati non etichettati e la si usa per task supervisionati → **pre-training**.
- **Sfruttarle**: la rappresentazione appresa si usa per altri task → **transfer learning**.

17.3.2 Pre-training

Il *greedy layer-wise unsupervised pretraining* (circa 2006) fu il primo approccio che rese possibile addestrare reti supervisionate profonde (a parte CNN e RNN). Si usa l'apprendimento **non supervisionato** per catturare la forma della distribuzione dell'input, **strato per strato**, e si inizializza così la rete, rendendo più facile l'addestramento complessivo (anziché fare subito un addestramento *end-to-end* con la discesa del gradiente, che per i modelli profondi era difficile).

Perché aiuta? Ogni strato è ottimizzato indipendentemente in modo non supervisionato, costituendo un pre-addestramento per il **fine-tuning** finale. Funziona sia come **buona strategia di inizializzazione** sia come **regolarizzazione**: non nel senso di un modello più semplice, ma di scoprire feature che catturano le regolarità dei dati. Se le funzioni vere sono complicate e modellate dalle regolarità della distribuzione degli input, l'apprendimento non supervisionato può essere un regolarizzatore più appropriato. Riduce anche la **varianza** del processo di stima (i modelli pre-addestrati tendono a finire in una regione più piccola).

Autoencoder

Definizione — Autoencoder

Un **autoencoder** è una rete neurale addestrata a **copiare il proprio input in uscita**. Internamente ha uno strato nascosto **h** che descrive un **codice** per rappresentare l'input. Si compone di un **encoder** $h = f(x)$ e di un **decoder** che produce la ricostruzione $r = g(h)$.

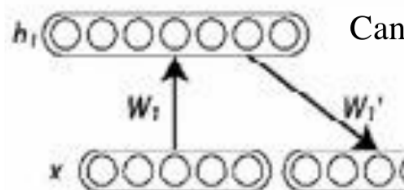


Figura 17.7: Un autoencoder: encoder (W_1) e decoder (W_1').

- **Undercomplete**: lo strato nascosto è **più piccolo** dell'input, e il vincolo architetturale forza la rete a catturare le feature più salienti. (Con decoder lineare e loss quadratica si ottiene la PCA.)
- **Overcomplete**: lo strato nascosto è **più grande** dell'input, ma si usa una regolarizzazione che impone sparsità, robustezza al rumore o altre proprietà, oltre alla banale capacità di copiare.
- È apprendimento **non supervisionato**.

Esempio d'uso: **denoising autoencoder** e memorie auto-associative. Si addestra su immagini cercando una buona rappresentazione (*“una buona rappresentazione è quella che si può ottenere in modo robusto da un input corrotto e che è utile per recuperare l'input pulito”*), poi si recupera l'immagine originale presentandone una versione rumorosa e ripetendo le fasi di codifica e decodifica.

L'algoritmo di pre-training (Bengio)

In sintesi — Pre-training layer-wise

1. Addestrare il primo strato come **autoassociatore** per minimizzare l'errore di ricostruzione dell'input grezzo (non supervisionato: bastano esempi non etichettati).
2. Usare le uscite delle unità nascoste come input per un altro strato, anch'esso addestrato come autoassociatore.
3. Ripetere fino al numero di strati desiderato.
4. Usare l'uscita dell'ultimo strato nascosto come input per uno strato supervisionato, inizializzato (casualmente o con addestramento supervisionato, tenendo fisso il resto).
5. **Fine-tuning** di tutti i parametri rispetto al criterio supervisionato.

Si spera che l'inizializzazione non supervisionata abbia portato i parametri in una regione da cui la discesa locale raggiunge un buon ottimo locale.

Gli autoencoder si usano come RBM (*Restricted Boltzmann Machines*, modelli basati sull'energia, nelle *deep belief network* di Hinton) o come autoencoder (denoising) impilati (Bengio, 2007).

Nota — Serve ancora il pre-training?

Il pre-training ha permesso di **far partire** il deep learning e dà ancora miglioramenti in alcuni task (soprattutto NLP, dove permette di sfruttare grandi corpora di testo per apprendere rappresentazioni distribuite delle parole). Ma è difficile da gestire (iperparametri divisi in due fasi) e oggi *“il pre-training non supervisionato è largamente abbandonato”* (DL book, cap. 15): le tecniche moderne bastano per un buon addestramento **end-to-end** da zero con la backpropagation.

Nota — Encoder e decoder oggi

- Gli **autoencoder variazionali** usano un quadro bayesiano: lo spazio latente è una miscela di distribuzioni invece di un vettore fisso.
- La **traduzione automatica neurale** si realizza con architetture encoder-decoder in cui l'uscita non coincide con l'input ma è in un'altra lingua.
- I **transformer** (come GPT, *Generative Pre-trained Transformer*) sono reti encoder/-decoder con un meccanismo di **attenzione**. Si può usare solo l'encoder (BERT, per vari task NLP) o solo il decoder, a scopo generativo (la serie GPT per la generazione autoregressiva di testo).

17.3.3 Transfer learning

Si usa la rappresentazione scoperta da un modello per **migliorare un altro modello**, assumendo che esistano feature utili in contesti o task diversi, corrispondenti a fattori sottostanti comuni. Forme diverse (sono veri sotto-campi del ML, non solo terminologia):

- dall'autoencoder al classificatore (come sopra);
- **multi-task learning**: stessi input, target diversi (una regolarizzazione implicita);
- **domain adaptation**: dominio di input diverso ma feature condivise (es. analisi del sentiment su recensioni di video e di smartphone);

- riuso di **modelli pre-addestrati** su dataset più grandi.

Esempio — Transfer learning con AlexNet

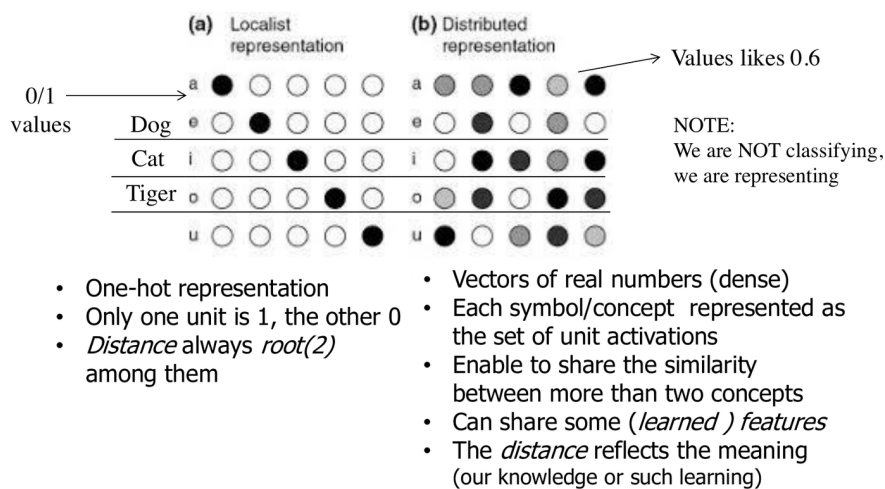
AlexNet è una CNN addestrata su circa 1,2 milioni di immagini ImageNet in 1000 categorie: ha appreso **rappresentazioni ricche** per un'ampia gamma di immagini, con primitive di basso livello condivise tra immagini diverse. Ha 5 strati convoluzionali e 3 completamente connessi. Per un nuovo task si **sostituiscono gli ultimi 3 strati** e si addestrano solo quelli con poche immagini nuove, oppure si fa il **fine-tuning** di tutta la rete. Molte CNN pre-addestrate (VGG, ResNet, GoogLeNet...) sono disponibili nelle librerie.

17.4 Rappresentazioni distribuite

Definizione — Rappresentazione distribuita

“In una rappresentazione distribuita gli elementi (le feature) non sono mutuamente esclusivi e le loro molte configurazioni corrispondono alle variazioni osservate nei dati.” (LeCun, Bengio, Hinton, 2015)

I metodi di deep learning sfruttano rappresentazioni distribuite su più livelli, ma il concetto vale per molti modelli (reti neurali in generale, sulle unità nascoste; modelli grafici probabilistici, con più variabili latenti).



❖ Richer and smooth representation

• Shared attributes allow generalization across different concepts

Figura 17.8: Rappresentazione simbolica (one-hot) e distribuita.

Simbolica (one-hot)	Distribuita
una sola unità vale 1, le altre 0	vettori di numeri reali (densi)
distanza sempre $\sqrt{2}$ tra concetti diversi	ogni concetto è l'insieme delle attivazioni di tutte le unità
	la similarità si può condividere tra più di due concetti
	si possono condividere feature (apprese)
	la distanza riflette il significato

Attenzione: qui **non si classifica**, si **rappresenta**. Le rappresentazioni distribuite sono più ricche e lisce, e gli attributi condivisi permettono di generalizzare tra concetti diversi.

Input o rappresentazione interna? Il discorso è generale, ma l'apprendimento agisce sulla rappresentazione **interna**. Una rappresentazione distribuita in **input** si usa solo se la conoscenza di dominio permette di fissarla (raro e difficile; fissarla arbitrariamente può danneggiare il task). In genere si usa comunque un input one-hot (non informato) e si lascia che il modello sviluppi **internamente** la rappresentazione distribuita necessaria.

17.4.1 Contare la differenza

Quattro concetti: auto blu, bici blu, auto rossa, bici rossa. Non servono 4 neuroni: ne bastano 2 (2^2 concetti), uno per blu/rosso e uno per auto/bici. E il neurone che descrive il “rosso” impara il rosso da immagini sia di auto sia di bici, non solo da una specifica categoria.

In generale, con n feature a k valori una rappresentazione distribuita descrive k^n concetti diversi (ad esempio 2^n configurazioni binarie), contro gli n di una rappresentazione simbolica one-hot. Si può assegnare un codice unico a un numero **esponenziale** di regioni: i modelli con rappresentazioni distribuite (come le reti con unità nascoste) distinguono esponenzialmente più regioni di quelli non distribuiti.

Modelli basati su rappresentazioni **non distribuite**: K-NN (uno o pochi prototipi per input, senza condividere informazione con tutto il training set), kernel RBF (si riducono al K-NN, come visto per le SVM), miscele di gaussiane, alberi di decisione (si attiva una sola foglia).

17.4.2 Attributi condivisi: separare i concetti

The same example again (but for the meaning)

Distribute representation			Localist representation			
	Car (Bike)	Red (Blue)	Red Car	1	0	0
			Red Bike	0	1	0
			Blu Car	0	0	1
			Blu Bike	0	0	0

Sharing a column (feature)
we *can* disentangle the concept of
“redness”
(shared among different objects)

Figura 17.9: Condividendo una colonna (feature) si separa (

Separando i concetti colore e tipo, si può imparare la distinzione auto/bici o il colore **senza dover vedere esempi di tutte le combinazioni**. Questa **separabilità statistica** rende facile generalizzare a configurazioni mai viste: avendo visto gli altri tre casi, si classifica facilmente la bici blu (ad esempio “non mi piace” perché non è rossa, se il modello ha imparato che mi piacciono gli oggetti rossi); con la rappresentazione localista la bici blu sarebbe del tutto nuova.

Con le **parole**: invece di un simbolo one-hot per parola (dimensione pari al vocabolario), una rappresentazione distribuita in cui “cane” e “gatto” condividono feature permette al modello di:

1. **trattarli in modo simile**: le frasi con “gatto” informano le predizioni per le frasi con “cane” e viceversa, trasferendo informazione da ogni frase di training a un numero esponenziale di frasi semanticamente affini (ad esempio “entrambi hanno la coda” → animale, non umano);
2. **imparare automaticamente che sono simili**, avvicinandone le rappresentazioni perché compaiono in contesti simili: è il **word embedding** (ad esempio *word2vec*). Nello spazio

one-hot ogni parola è a distanza $\sqrt{2}$ da tutte le altre; nello spazio appreso parole con significato simile diventano vicine.

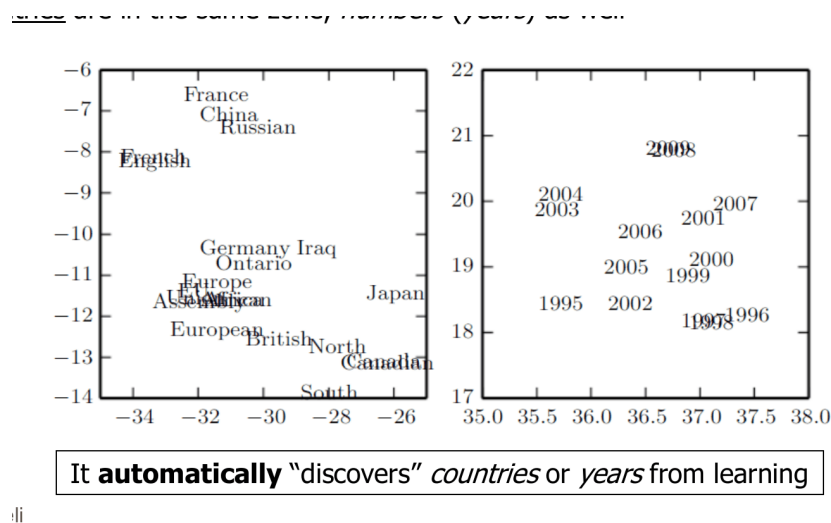


Figura 17.10: Word embedding: parole semanticamente simili (paesi, anni) finiscono vicine. Il modello "scopre" automaticamente paesi e anni dall'apprendimento.

Suggerimento — Una considerazione sui risultati

Il modello impara non solo la grammatica (non sorprende) ma rappresentazioni **semanticamente significative**, e lo fa automaticamente, solo leggendo testo. Perché la nostra semantica coincide con ciò che la rete impara? Probabilmente il linguaggio (e le immagini) hanno intrinsecamente una struttura **gerarchica** e la rete profonda ha il bias induttivo giusto per apprenderla in modo distribuito (le immagini reali sono una frazione minuscola delle immagini di rumore bianco). È ancora un campo aperto.

Il dibattito sulle rappresentazioni distribuite si estende a quello tra paradigmi **logico-simbolici** e **neurali** per la cognizione: i successi nella traduzione mettono in dubbio la necessità di approcci simbolici per capire le frasi.

Attenzione — Interpretabilità

Il rovescio della medaglia: una rappresentazione distribuita è **meno facile da interpretare** di una localista.

17.4.3 Rappresentazioni distribuite profonde

Il DL sfrutta rappresentazioni distribuite **attraverso molti strati**, ottenute componendo livelli di astrazione o una gerarchia di feature riusate. La composizionalità porta un ulteriore guadagno esponenziale di efficienza statistica, oltre a quello delle rappresentazioni distribuite: **due vantaggi potenzialmente esponenziali** rispetto ai modelli non distribuiti e superficiali. Nel complesso, i modelli di DL imparano una rappresentazione distribuita dei dati trovando (separando) i **fattori causali condivisi** che li generano, su diversi livelli di astrazione.

Esempi applicativi: la combinazione di CNN e RNN per generare **didascalie di immagini** (con un meccanismo di attenzione che si concentra sulla parte dell'immagine relativa a ogni parola generata); la classificazione dei **tumori della pelle** con una rete profonda addestrata su 130.000 casi, con competenza paragonabile a quella dei dermatologi (*Nature* 2017).

17.5 Altri spunti e risultati recenti

- **Smoothness tramite la struttura:** a parità di parametri, le reti più profonde impongono più smoothness di quelle superficiali (ogni strato lavora sulla superficie già liscia prodotta dal precedente), e sembrano apprendere meglio a parità di neuroni totali: vincoli di smoothness **impliciti**, anziché espliciti.
- **Il puzzle del non-overfitting:** reti enormi, sovra-parametrizzate, capaci di azzerare l'errore di training anche su etichette **casuali**, eppure generalizzano. Una spiegazione: la discesa del gradiente impone una forma di **regolarizzazione implicita**, con convergenza verso la soluzione a **marginе massimo** (Poggio et al., 2017–2019). Varianti di SGD con batch/weight normalization massimizzano il margine normalizzato, e perfino la discesa del gradiente standard massimizza il margine L^2 senza normalizzazione o regolarizzazione esplicita.
- **Double descent:** nelle CNN, ResNet e transformer, al crescere della dimensione del modello (o dei dati, o del tempo di addestramento) le prestazioni prima migliorano, poi peggiorano (la classica U) e poi **migliorano di nuovo** oltre la soglia di interpolazione (dove l'errore di training diventa quasi zero). L'effetto è spesso evitato con una regolarizzazione accurata; non è ancora pienamente compreso ed è un'importante direzione di ricerca. (Non necessariamente riguarda i piccoli modelli del progetto.)

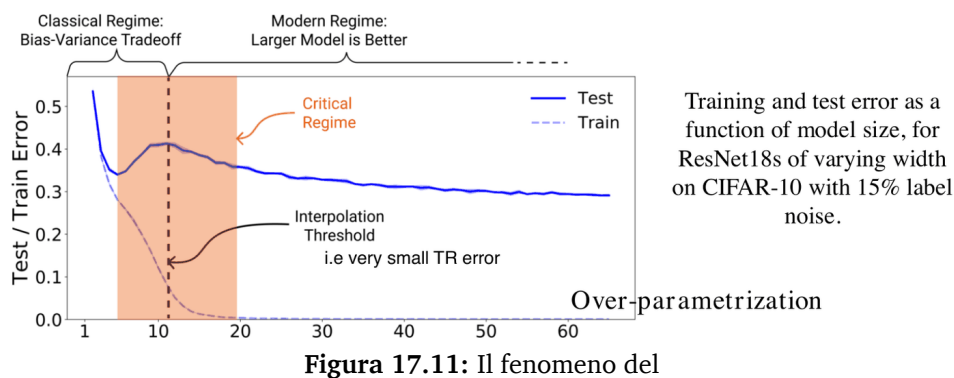
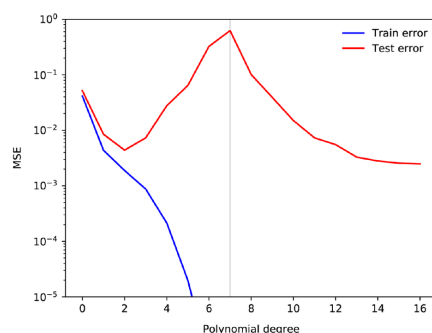


Figura 17.11: Il fenomeno del

If we look at the [median over 250 runs] of the MSEs, we get the pretty "double descent"!



Credits: François Fleuret

heli

111

Figura 17.12: Un esempio giocattolo con regressione polinomiale regolarizzata: anche qui compare il double descent. Superato l'errore di training nullo, all'aumentare del grado il fit è guidato solo dal termine di penalità e il polinomio diventa sempre più "lineare a tratti".

- **Ridondanza:** nei modelli profondi c'è una ridondanza significativa nei parametri; dati pochi pesi per feature si possono predire gli altri, fino a oltre il 95% dei pesi senza perdita di accuratezza (Denil et al., 2013). Si possono anche **comprimere** i parametri (pruning,

quantizzazione, codifica di Huffman) senza perdite (Han et al., 2015): sembra che reti piccole bastino!

- **Lottery ticket hypothesis** (Frankle e Carbin, 2018): perché allora addestriamo reti grandi? Perché le buone prestazioni dipendono da una **inizializzazione fortunata** di una o più sotto-reti, e le reti grandi ne contengono esponenzialmente di più. Riaddestrando la rete potata con la stessa inizializzazione si ottengono prestazioni simili.
- Il ruolo della **casualità** (reti inizializzate casualmente e reti a pesi casuali) riceve sempre più attenzione (vedi [18 - Reti neurali randomizzate](#)).

17.6 Tecniche

Molti strati possono essere difficili da ottimizzare, quindi l'enfasi è sui metodi per migliorare la **discesa del gradiente** (anche contro il *vanishing gradient*), la **regolarizzazione** (reti grandi) e lo **sfruttamento dei dati** (semi-supervisionato, reinforcement learning, multi-task, apprendimento avversario...). A questo si aggiungono dataset più grandi, infrastrutture software pubbliche e miglioramenti hardware.

In sintesi — Tecniche chiave per il DL

- In origine: approcci di **pre-training** (ancora usati in domini come l'NLP per i word embedding).
- Oggi (nelle librerie più diffuse), in gran parte già note:
 - **SGD con momentum** (con decadimento di η , mini-batch) o **Adam**;
 - attivazioni **ReLU** nelle unità nascoste;
 - loss di **massima verosimiglianza** (cross-entropy) con **softmax** in uscita (anche per evitare saturazione e gradienti piccoli);
 - regolarizzazione: early stopping, weight decay, **dropout**, **batch normalization**.

17.6.1 Problemi del gradiente

Cosa succede alla norma del gradiente quando lo si retropropaga attraverso molti strati?

- se i pesi sono **piccoli**, i gradienti **si riducono esponenzialmente** (*vanishing gradient*), anche per via della derivata delle sigmoidi;
- se i pesi sono **grandi**, i gradienti **crescono esponenzialmente** (*exploding gradient*).

Il *vanishing gradient*

Le attivazioni tradizionali come la tanh hanno gradienti in $(-1, 1)$, e la backpropagation calcola i gradienti con la regola della catena ripetuta attraverso gli strati. In una rete a d strati si moltiplicano d di questi numeri piccoli per calcolare i gradienti degli strati vicini all'input: il gradiente (il delta) **decresce esponenzialmente con d** e gli strati vicini all'input si addestrano lentissimamente.

Nota — Schizzo formale

Se $\mathbf{h}_l = F_l(\mathbf{h}_{l-1})$ e $F = F_M \circ \dots \circ F_1$, allora

$$\frac{\partial F}{\partial \mathbf{w}_l} = \frac{\partial F_M}{\partial \mathbf{h}_{M-1}} \dots \frac{\partial F_1}{\partial \mathbf{w}_l}, \quad \left| \frac{\partial F}{\partial \mathbf{w}_l} \right| \approx \rho^{M-l+1} \text{ se } \left\| \frac{\partial F_i}{\partial \mathbf{h}_i} \right\| \approx \rho.$$

Con $\rho < 1$ il gradiente svanisce, con $\rho > 1$ esplode.

Gli approcci per affrontarlo vanno da R-prop alle connessioni *shortcut*, fino alle reti randomizzate, e alle ReLU.

Gradient clipping

La moltiplicazione ripetuta attraverso gli strati può introdurre **scogliere** (*cliff*) nella funzione di costo: derivate altissime che “catapultano” i pesi lontanissimo. Il **clipping** limita la norma del gradiente mantenendone la direzione:

$$\text{se } \|\mathbf{g}\| > v \text{ allora } \mathbf{g} \leftarrow \frac{v \mathbf{g}}{\|\mathbf{g}\|},$$

con v soglia sulla norma.

if $\|\mathbf{g}\| > v$ then $\mathbf{g} = v \mathbf{g} / \|\mathbf{g}\|$, $\mathbf{g}$: gradient and v : norm threshold
we in the gradient direction, but bounding the weight update

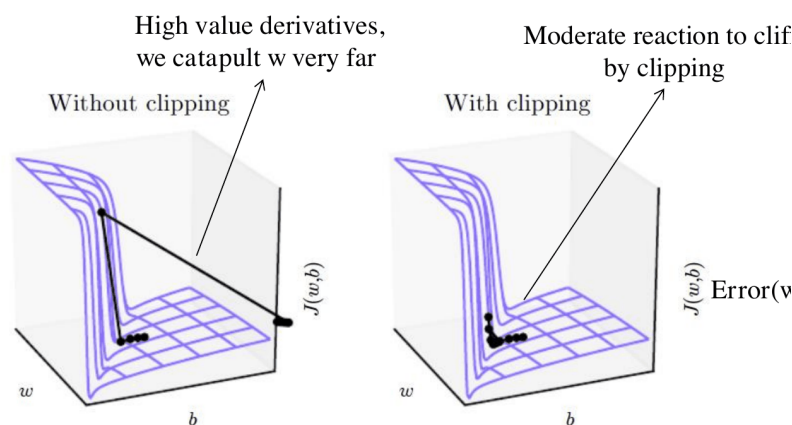


Figura 17.13: Senza clipping (sinistra) e con clipping (destra) di fronte a una scogliera della funzione di costo.

17.6.2 ReLU

Definizione — ReLU (*Rectified Linear Unit*)

$$f(x) = \max(0, x) = \begin{cases} 0 & x < 0 \\ x & x \geq 0. \end{cases}$$

Perché usarla nel DL (dal 2009)?

- **Propagazione del gradiente** migliore attraverso molti strati: evita la saturazione delle sigmoidi, perché $f'(x) = 1$ per ogni net positivo.
- **Attivazione sparsa**: in una rete inizializzata casualmente circa il 50% delle unità nascoste è attivo (uscita non nulla).
- **Calcolo efficiente**: solo confronti, somme e prodotti.
- Ha ancora un'ispirazione neuroscientifica, e permette un addestramento più veloce ed efficace delle reti profonde.

La ReLU **non è differenziabile in 0**, ma nelle librerie si assume di solito derivata 0 (sinistra) o 1 (destra): l'approssimazione è accettabile e sicura, perché per le approssimazioni numeriche

è improbabile valutare esattamente $f(0)$. (Un approccio più rigoroso usa l'ottimizzazione non differenziabile per funzioni lineari a tratti.)

Un trucco: iniziare con **net positivi**, ad esempio con bias piccoli e positivi (0,1), così che le ReLU siano inizialmente attive per la maggior parte degli input e lascino passare le derivate.

Varianti: **ELU** (*Exponential Linear Unit*), $f(x) = \alpha(e^x - 1)$ per $x \leq 0$ e x per $x > 0$; **Leaky ReLU**, $f(x) = 0,01x$ per $x < 0$ e x per $x \geq 0$. Imparano anche sulla parte sinistra (i neuroni non si “spengono”) e hanno attivazione media più vicina a zero. Molte varianti hanno prestazioni comparabili; in generale evitano alcuni difetti mantenendo la parte lineare, che rende il modello più facile da ottimizzare.

17.6.3 Batch normalization

La **batch normalization** normalizza ogni mini-batch calcolandone le statistiche (media e varianza) per ogni strato: si normalizza la matrice [esempi del batch $\times$ attivazioni delle unità] a media zero e varianza unitaria, e la trasformazione è inclusa nella backpropagation. Come normalizzare l'input è pratica standard, così la BN mantiene normalizzati i dati che fluiscono tra gli strati intermedi. Effetti: **regolarizzazione** (per il rumore introdotto dal mini-batch) e **apprendimento più rapido** con accuratezza più alta (rilevante perché ha permesso di addestrare efficacemente CNN e modelli profondi con sigmoidi). (Dettagli non richiesti all'esame, a meno di usarla.)

17.6.4 Dropout

Il **dropout** seleziona casualmente un sottoinsieme della rete durante l'addestramento. Si spiega con due concetti già noti: **ensemble** (un bagging implicito di sotto-reti diverse, ma computazionalmente economico) e **regolarizzazione**.

Il bagging addestra n modelli su sottoinsiemi diversi del training set e ne media le uscite: modelli ad alta varianza e basso bias funzionano bene in media. Il dropout approssima e sfrutta questo processo con un numero **esponenziale** di sotto-reti, pur avendo a test time **una sola rete**, e massimizza la diversità dell'ensemble evitando co-adattamenti complessi sui dati di training.

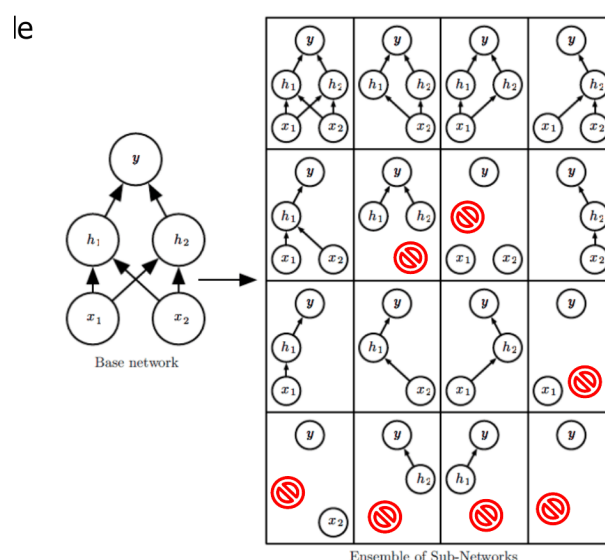


Figura 17.14: Il dropout addestra l'ensemble di tutte le sotto-reti ottenibili rimuovendo unità non di uscita dalla rete base. Alcune sotto-reti non funzionano, ma in reti grandi è improbabile che non ci sia alcun percorso dall'input all'uscita.

In sintesi — Algoritmo del dropout

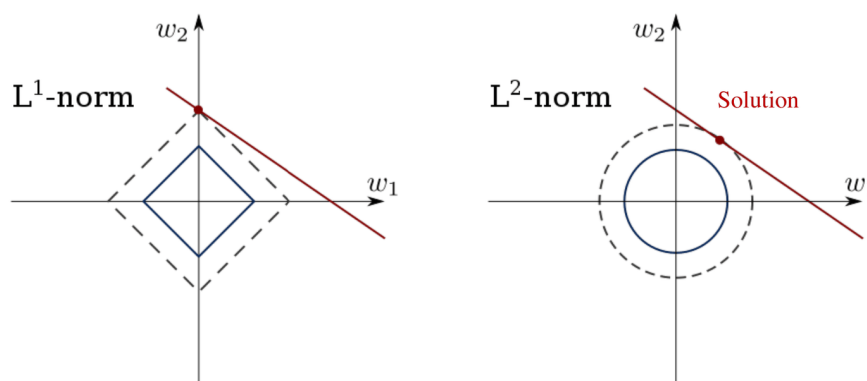
- Ogni volta che si carica un esempio in un mini-batch, si campiona una diversa **maschera binaria** da applicare a tutte le unità di input e nascoste, indipendentemente per ogni unità. La probabilità di includere un'unità è un iperparametro (tipicamente 0,8 per l'input e 0,5 per le unità nascoste).
- Ogni unità viene moltiplicata per la sua maschera (zero equivale a rimuoverla, *dropout*): si ottiene una sotto-rete.
- Si eseguono forward, backpropagation e aggiornamento come al solito, addestrando un sottoinsieme di unità alla volta.
- Le unità rimosse vengono poi reinserite con i loro pesi originali.

Solo una piccola frazione delle sotto-reti viene addestrata (ciascuna per un solo passo), ma la **condivisione dei parametri** fa sì che anche le altre arrivino a buoni parametri: si gestisce un numero esponenziale di modelli con memoria trattabile. Per approssimare la predizione dell'ensemble a test time si usa la *weight scaling inference rule*: si moltiplicano i pesi per la probabilità di inclusione (tipicamente 0,5), così che l'uscita attesa di ogni unità sia la stessa dell'addestramento.

Effetto regolarizzante: evita di addestrare tutte le unità su tutti i dati riducendone le interazioni (e diversificandole, come serve ai comitati); riduce la varianza come il bagging; inserisce **rumore strutturato** (ad esempio riconoscere un volto senza l'unità che rileva il naso). Soprattutto, regolarizza ogni unità a essere non solo una buona feature, ma **una feature buona in molti contesti** (sotto-reti diverse). Per la regressione lineare equivale al weight decay L^2 con un λ diverso per ogni input. Si può usare con qualsiasi modello a rappresentazione distribuita addestrato con SGD.

17.6.5 Regolarizzazione L1

Usare la norma L^1 (somma dei valori assoluti) al posto della L^2 nella penalità (per i modelli lineari è il **LASSO**) favorisce l'**eliminazione** di feature (pesi esattamente a zero), mentre la L^2 di solito riduce solo la grandezza dei pesi: si ottengono modelli più semplici. La soluzione richiede tecniche di ottimizzazione non differenziabile (sub-gradiente, approssimazioni lisce della norma L^1 ...).



A. Micheli

149

Figura 17.15: Perché L1 produce soluzioni sparse: le curve di livello della norma L1 hanno “spigoli” sugli assi, dove una coordinata è nulla.

17.6.6 Apprendimento avversario

Le **Generative Adversarial Network** (GAN) sono due reti in competizione: una **genera** candidati, l'altra li **valuta** (distinguendo i veri dai generati). Sono usate ad esempio per generare immagini fotorealistiche di persone che non esistono.

17.7 Applicazioni, sfide e conclusioni

Il DL ha successo nell'**NLP** (riconoscimento del parlato, traduzione, sentiment analysis, question answering...), nella **computer vision** (immagini e video: riconoscimento di oggetti, applicazioni mediche, sintesi di immagini...) con prestazioni mai raggiunte prima, e nel gioco del **Go**. Se ne parla più a fondo nei corsi di NLP e ISPR.

Sfide aperte: analisi e quadro teorici; progettazione ottima dell'architettura (quanti strati? che unità? la cross-validation può essere troppo costosa); efficienza per i big data; elaborazione di stream (on-line); dati strutturati (sequenze, alberi, grafi).

Anche l'hardware è cambiato: TPU di Google (inizialmente solo per l'inferenza), GPU Nvidia, NPU nei chip per smartphone (Huawei Kirin 970), il *Neural Engine* di Apple (dal 2017).

In sintesi — Sintesi

- I progressi tecnici e la disponibilità di dati hanno reso possibile apprendere con reti profonde e molto grandi, con risultati allo stato dell'arte soprattutto dove la **composizionalità** delle feature è rilevante (visione, linguaggio).
- Due vantaggi potenzialmente esponenziali: la **profondità** (composizione di funzioni, no-flattening) e le **rappresentazioni distribuite** (attributi condivisi).
- Il concetto più generale emerso è il **representation learning**.
- Tecniche: ReLU, cross-entropy con softmax, SGD con momentum/Adam, dropout, batch normalization, gradient clipping.
- Il ML rende relativamente più facile sviluppare sistemi di AI (e software in generale) sofisticati per problemi reali.

Domande — Possibili domande d'esame

- Cosa si intende per deep learning? Quali sono gli aspetti comuni ai vari approcci?
- Servono davvero molti strati? Discutere l'esempio della parità e i risultati di no-flattening.
- Qual è il bias induttivo dei modelli profondi?
- Cos'è il representation learning? Pre-training e transfer learning.
- Cos'è un autoencoder?
- Rappresentazioni simboliche contro distribuite: vantaggi delle seconde.
- Cos'è il vanishing gradient? Come si affronta (ReLU, clipping...)?
- Descrivere il dropout e spiegarne l'effetto regolarizzante.

Capitolo 18

Reti neurali randomizzate

Dopo il deep learning, l'altro paradigma recente presentato nel corso è quello delle **reti neurali a pesi casuali** ("deep learning ed *extreme learning*", scherza il professore). L'idea sembra paradossale: e se lo strato nascosto non venisse addestrato affatto? La lezione mostra come la casualità sia una componente utile del ML e come le reti randomizzate si ricolleghino a concetti già visti: espansione in basi, teorema di Cover, regolarizzazione.

18.1 La casualità nel Machine Learning

Un approccio **randomizzato** usa un certo grado di casualità come parte della sua strategia costruttiva. La casualità può migliorare le prestazioni predittive e alleviare le difficoltà delle metodologie classiche, ed è presente a diversi livelli:

- **elaborazione dei dati**: suddivisione dei dati (hold-out, K-fold CV, leave-one-out), generazione di dati (bootstrap, boosting, tecniche di bilanciamento);
- **apprendimento**: ordine delle osservazioni (shuffling per il mini-batch), inizializzazione casuale dei pesi delle reti;
- **selezione degli iperparametri** (inclusa la ricerca casuale);
- **ensemble**: differenziazione casuale tra modelli (inizializzazioni diverse, campionamento dei training set in bagging e boosting);
- **nel cuore del modello**.

18.1.1 Modelli intrinsecamente casuali

- I neuroni stocastici delle **Restricted Boltzmann Machine**.
- Gli **Extremely Randomized Trees** e le **Random Forest**: un ensemble di alberi di decisione randomizzati, ciascuno costruito su campioni estratti casualmente dal training set e con una scelta casuale delle variabili di input su cui dividere i nodi. Uniscono i vantaggi degli ensemble e della casualità.
- Il **dropout** come regolarizzazione (anche se qui la casualità non è necessaria di per sé).

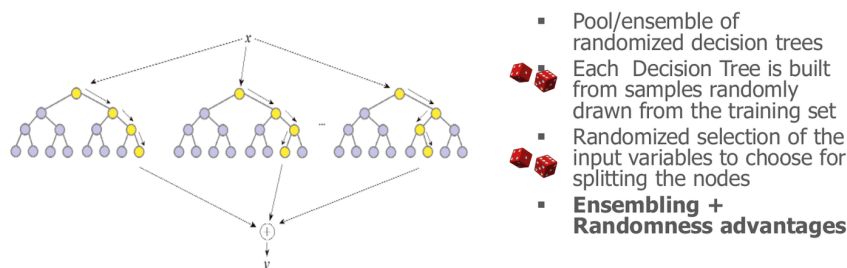


Figura 18.1: Random Forest: un ensemble di alberi di decisione randomizzati.

18.2 Reti neurali a pesi casuali

Domande — La domanda

È possibile sfruttare le architetture MLP (o ricorrenti) **senza lunghi cicli di addestramento** degli strati nascosti?

Esiste una lunga tradizione di reti randomizzate, a pesi casuali, a proiezione casuale: ad esempio la **Random Vector Functional Link** (RVFL), fin dalle origini. Di recente sono state rese popolari con vari nomi, tra cui **ELM** (*Extreme Learning Machine*).

Definizione — Idea di base

- Una rete con uno (o più) **strati nascosti connessi casualmente**: i pesi vengono **fissati dopo l'inizializzazione casuale** e non vengono più addestrati.
- Si addestrano **solo i pesi di uscita** (un modello lineare: con la pseudoinversa o la ridge regression).

Si superano così i problemi degli algoritmi di addestramento complessi e computazionalmente onerosi tradizionalmente usati per le reti neurali.

Le radici sono nel lavoro pionieristico sul **Perceptron** di Rosenblatt, in cui un'area di "proiezione" connessa casualmente alla retina alimentava un'area di "associazione", e solo le risposte finali venivano apprese. Altri lavori pionieristici avevano sottolineato l'importanza dell'apprendimento nello strato di uscita rispetto a quello nascosto.

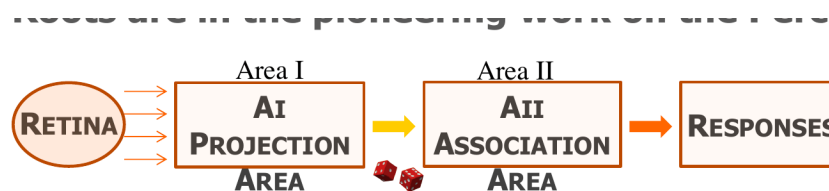


Figura 18.2: Il Perceptron originale: le connessioni dalla retina all'area di proiezione erano casuali.

Le reti randomizzate sono oggi una linea di ricerca in crescita, particolarmente adatta quando l'**efficienza** è fondamentale (l'addestramento è limitato all'ultimo strato).

18.2.1 Struttura generale

- **Strato nascosto (non addestrato)**: immerge in modo **non lineare** l'input in uno **spazio delle feature ad alta dimensione**, dove il problema ha più probabilità di essere risolvibile linearmente (è una **LBE!**). Ha una base teorica nel teorema di Cover.
- **Readout (addestrato)**: combina le feature dello spazio nascosto per calcolare l'uscita; tipicamente è un **modello lineare**.

Teorema — Teorema di Cover (1965)

“Un problema complesso di classificazione di pattern, proiettato in modo non lineare in uno spazio ad alta dimensione, ha più probabilità di essere linearmente separabile che in uno spazio a bassa dimensione, purché lo spazio non sia densamente popolato.”

Ad esempio, con una mappatura deterministica: dati l esempi, li si solleva sui vertici di un simpleso $(l - 1)$ -dimensionale (il “triangolo” generalizzato). Ora **ogni** partizione binaria

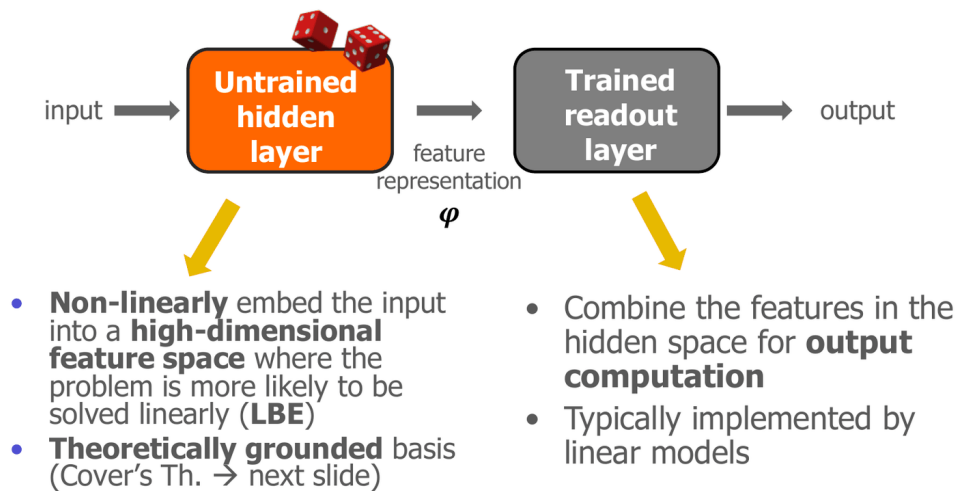


Figura 18.3: Struttura generale: uno strato nascosto non addestrato (casuale) e uno strato di readout addestrato.

dei campioni è linearmente separabile.

È esattamente la stessa proprietà sfruttata dai **kernel** nelle SVM.

18.2.2 Reti feedforward randomizzate

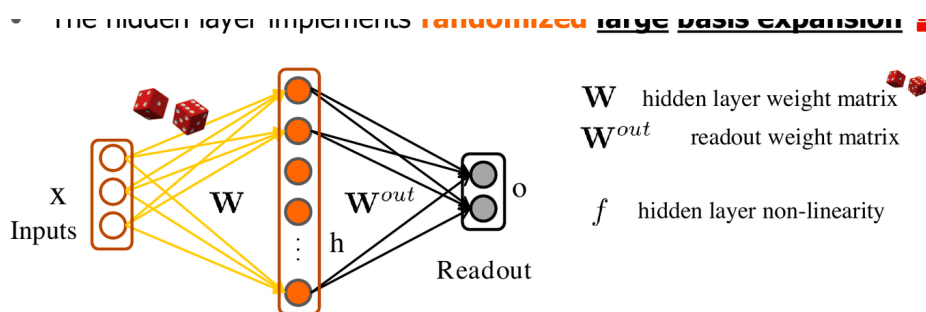


Figura 18.4: Rete feedforward randomizzata: W (casuale, fissa) e W^{out} (addestrata).

La relazione input-output è implementata da uno strato nascosto non addestrato, che realizza una **grande espansione in basi casuale**.

In sintesi — Costruzione, apprendimento e uso

Costruzione: riempire W con valori casuali (ad esempio rumore gaussiano), usando **molte unità**.

Apprendimento del readout: dati l esempi $(\mathbf{x}_p, \mathbf{d}_p)$, come per i modelli lineari si minimizza l'errore quadratico con il metodo diretto. Con $H = f(WX)$ (le attivazioni nascoste di tutti i pattern):

$$W^{out} = H^+ \mathbf{d},$$

spesso con **regolarizzazione** L^2 (utile per l'alto numero di unità):

$$W^{out} = (H^T H + \lambda I)^{-1} H^T \mathbf{d},$$

oppure con L^1 .

Uso: si calcolano le attivazioni nascoste $\mathbf{h}$ come per i normali neuroni e poi le uscite

$$\mathbf{o}(\mathbf{x}) = W^{out} f(W\mathbf{x}),$$

applicando eventualmente una soglia (o una funzione non lineare) per la classificazione.

I principali modelli di riferimento sono la **RVFL** (*Random Vector Functional-Link*), la **ELM** (*Extreme Learning Machine*) e le **reti RBF con centri casuali**.

Suggerimento — Perché funziona

Si confronti con le reti neurali “normali”: lì lo strato nascosto è un’espansione in basi **adattiva** $\phi_j(\mathbf{x}, \mathbf{w})$ i cui pesi vengono appresi; qui è un’espansione in basi **casuale** ma molto **ampia**. Se le basi sono abbastanza numerose, per il teorema di Cover il problema diventa (quasi) linearmente separabile nello spazio nascosto, e basta un modello lineare in uscita, addestrabile in un solo passo con i minimi quadrati. Il costo di addestramento è quello di una regressione lineare regolarizzata.

18.2.3 Pro e contro

- Il grande insieme di unità nascoste ha un ruolo interessante: la proiezione espande in modo non lineare la dimensione dell’input, rendendo i dati più separabili.
- Un numero elevato di unità può fornire un’espansione in basi **sufficiente** per il task (con basi teoriche, inclusi teoremi di approssimazione universale).
- **Estremamente efficienti**.
- Esistono molte varianti: apprendimento incrementale aggiungendo neuroni (come il Cascade Correlation), pruning, ecc., spesso legate a ricerche passate con nomi diversi.
- Modelli “mirati”, con meno unità adattive addestrate sul task specifico, sembrano comunque **più eleganti**: c’è un dibattito tra approcci a **feature casuali** e a **feature apprese**, e vale la pena conoscerli entrambi.
- Si estendono alle reti ricorrenti: le **Echo State Network** (ESN), vedi [20 - Reti neurali ricorrenti \(RNN\)](#).

18.2.4 Dove sono utili

- **Big Data**, grazie all’efficienza estrema.
- Implementazioni minuscole per sistemi **embedded** e distribuiti, su dispositivi con risorse limitate (reti di sensori, IoT, edge computing).
- **Deep learning**, dove gli algoritmi di addestramento sono particolarmente onerosi: le reti randomizzate alleviano il carico computazionale (reti profonde randomizzate come pile di moduli RVFL/ELM, CNN randomizzate).
- **Reti ricorrenti randomizzate**: dove l’addestramento è ancora più oneroso che per gli MLP, e con solide basi teoriche sul bias introdotto dalla dinamica del sistema (stabilità, apprendibilità).

In sintesi — Conclusioni

La casualità non è un nemico: è un’amica, se la si controlla. (Vale anche in altri campi dell’informatica, come gli algoritmi randomizzati, e in matematica.) Si noti come ritornino sempre gli stessi concetti — teorema di Cover, LBE, regolarizzazione, rappresentazione delle feature: il corso è ormai “maturo”.

Nota — FAQ: si possono usare le ELM nel progetto?

Sì, ma come **parte aggiuntiva** rispetto all'implementazione standard di un MLP addestrato completamente con backpropagation e SGD. Sono facili da implementare come sotto-parte di una rete feedforward.

Domande — Possibili domande d'esame

- A quali livelli compare la casualità nel ML?
- Descrivere una rete feedforward randomizzata: costruzione, addestramento del readout, uso.
- Enunciare il teorema di Cover e spiegarne il legame con le reti randomizzate e con i kernel.
- Pro e contro delle feature casuali rispetto alle feature apprese.

Apprendimento non supervisionato: K-means e SOM

Questa lezione è un'introduzione leggera all'**apprendimento non supervisionato** con le reti neurali. Il fuoco è sul **clustering** visto come **quantizzazione vettoriale**: si parte dal classico **K-means** e si arriva a un modello neurale storico, le **Self-Organizing Map (SOM)** di Kohonen, che aggiungono al clustering la proprietà di preservare la topologia e quindi di visualizzare i dati.

19.1 Apprendimento non supervisionato

Nell'apprendimento **non supervisionato non c'è insegnante**: il training set contiene solo dati **non etichettati** (x). I task principali:

- **Clustering**: trovare raggruppamenti naturali nei dati.
- **Riduzione di dimensionalità**, visualizzazione, pre-elaborazione: i dati multidimensionali vengono proiettati in uno spazio a dimensione minore (PCA, Multi-Dimensional Scaling, Independent Component Analysis...).
- Modellazione della densità dei dati (non trattata nel corso).

A cosa serve? Esempi: organizzare le forme di vita in una tassonomia (cosa sono i mammiferi?); i neonati imparano a riconoscere (a “raggruppare”) i volti familiari prima di capire il linguaggio. Soprattutto, i **dati non etichettati costano molto meno**: ad esempio 10 milioni di frame di YouTube hanno permesso risultati allo stato dell'arte nel riconoscimento di oggetti in 22.000 categorie.

19.1.1 Clustering

Definizione — Clustering

Partizionare i dati in **cluster** (sottoinsiemi di dati “simili”): i pattern di un cluster sono più simili tra loro che a un pattern di un altro cluster. Ogni cluster è rappresentato da un **centroide** (detto anche prototipo, vettore di riferimento, centro del cluster, *codevector*; l'insieme dei centroidi è il **codebook**).

Il clustering è stato affrontato in moltissimi contesti e discipline, con migliaia di algoritmi diversi: riflette la sua utilità come passo dell'analisi esplorativa dei dati. Qui si adotta la prospettiva della **quantizzazione vettoriale** per il **clustering partizionale**.

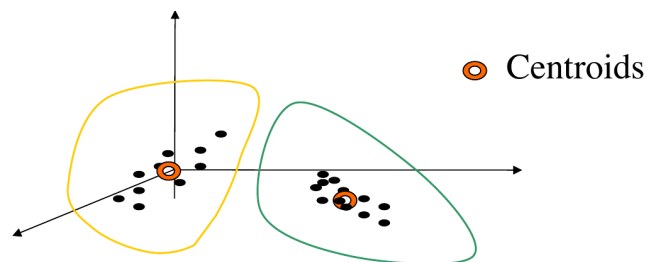


Figura 19.1: Clustering: partizione dei dati in gruppi rappresentati da centroidi.

Attenzione — La metrica domina

Come per il K-NN, è la **distanza assunta** a dire quali esempi sono simili, e l'algoritmo decide i raggruppamenti in base a quella metrica. La distanza euclidea è popolare ma è **solo una scelta**, e la scelta può avere implicazioni profonde (ad esempio si possono pesare di meno gli outlier usando 1– una gaussiana centrata sul prototipo).

19.2 Quantizzazione vettoriale

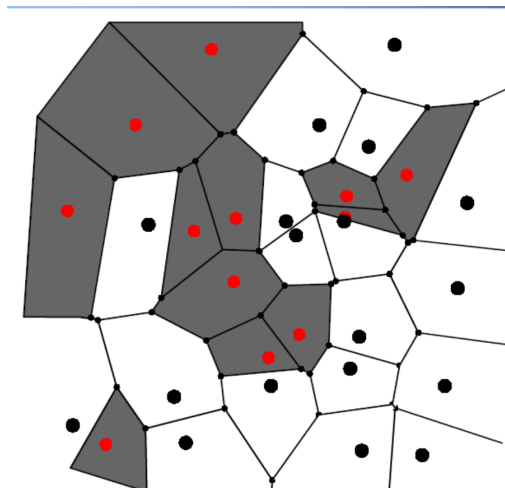
Definizione — Vector Quantization (VQ)

Le tecniche di quantizzazione vettoriale codificano una varietà di dati $V \subseteq \mathbb{R}^n$ usando solo un insieme finito $\mathbf{w} = (\mathbf{w}_1, \dots, \mathbf{w}_K)$ di vettori di riferimento (*codebook*) $\mathbf{w}_i \in \mathbb{R}^n$. Un vettore $\mathbf{x} \in V$ è descritto dal vettore di riferimento **vincente** (*best matching*) $\mathbf{w}_{i^*(\mathbf{x})}$, quello con distorsione $d(\mathbf{x}, \mathbf{w}_{i^*(\mathbf{x})})$ minima.

Questo divide la varietà in sottoregioni, i **poliedri di Voronoi**:

$$V_i = \{\mathbf{x} \in V : \|\mathbf{x} - \mathbf{w}_i\| \leq \|\mathbf{x} - \mathbf{w}_j\| \forall j\},$$

e ogni $\mathbf{x}$ è descritto dal vettore di riferimento corrispondente.



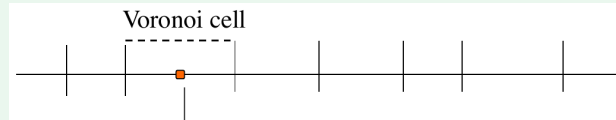
In general Hard to be co

Figura 19.2: Diagramma di Voronoi: ogni cella contiene i punti più vicini al suo centro che a qualsiasi altro.

Il nostro problema è il contrario di quello del K-NN: **dato il dataset, trovare i centri** che minimizzano la loss. In generale è difficile: trovare il codebook ottimo è **NP-completo**.

Esempio — Quantizzazione in 1D

Nell'elaborazione digitale dei segnali, la **quantizzazione** approssima un intervallo continuo di valori (o un insieme molto grande di valori discreti) con un insieme relativamente piccolo di simboli discreti, commettendo un errore di quantizzazione (distorzione). È ciò che succede alla musica nei dispositivi digitali.



VQ e clustering. Il clustering ha l'obiettivo più generale di trovare raggruppamenti **interessanti o utili**, dove “interessante” è spesso definito implicitamente dalla procedura stessa e non necessariamente dall'errore di quantizzazione minimo. Tuttavia la VQ fornisce un quadro utile, e i suoi algoritmi sono spesso usati per il clustering.

19.2.1 L'errore di quantizzazione

L'obiettivo è partizionare in modo ottimo una distribuzione sconosciuta nello spazio degli input in regioni approssimate da prototipi: un insieme di quantizzatori $\mathbf{x} \mapsto c(\mathbf{x}) = \mathbf{w}_{i^*(\mathbf{x})}$, da uno spazio continuo a uno discreto. Si considera la **distorzione quadratica** $d(\mathbf{x}, c(\mathbf{x})) = \|\mathbf{x} - c(\mathbf{x})\|^2$; il suo valore medio sulla distribuzione degli input è la distorsione (o errore di ricostruzione, o di quantizzazione) attesa. **Questa è la nostra loss:**

$$E = \int f(d(\mathbf{x}, \mathbf{w}_{i^*(\mathbf{x})})) p(\mathbf{x}) d\mathbf{x} = \int \|\mathbf{x} - \mathbf{w}_{i^*(\mathbf{x})}\|^2 p(\mathbf{x}) d\mathbf{x},$$

dove $p(\mathbf{x})$ è la distribuzione degli input. In versione discreta, su l dati e K prototipi:

$$E = \sum_{i=1}^l \sum_{j=1}^K \|\mathbf{x}_i - \mathbf{w}_j\|^2 \delta_{winner}(i, j), \quad \delta_{winner}(i, j) = \begin{cases} 1 & \text{se } j \text{ è il vincitore per } \mathbf{x}_i \\ 0 & \text{altrimenti} \end{cases}$$

(δ_{winner} è la funzione caratteristica del campo recettivo di $\mathbf{w}_j$).

Minimizzare E rispetto a $\mathbf{w}$ è il problema della VQ. Attenzione: l'integrando **non è differenziabile con continuità**, perché il vincitore $\mathbf{w}_{i^*(\mathbf{x})}$ cambia in modo discreto al variare di $\mathbf{x}$; si calcolano però le derivate **localmente**, fissando $\mathbf{x}$ in una cella di Voronoi.

19.3 K-means

19.3.1 Versione on-line

Derivando E rispetto ai $\mathbf{w}_j$ si ottiene la regola di apprendimento della VQ, cioè il noto algoritmo **K-means** di Lloyd e MacQueen (versione on-line): per ogni $\mathbf{x}_i$,

$$\Delta \mathbf{w}_{i^*} = \eta \delta_{winner}(i, i^*)(\mathbf{x}_i - \mathbf{w}_{i^*}).$$

(La derivata di $\|\mathbf{x}_i - \mathbf{w}_j\|^2$ rispetto a $\mathbf{w}_j$ è $-2(\mathbf{x}_i - \mathbf{w}_j)$; muovendosi nel verso opposto al gradiente si avvicina $\mathbf{w}_j$ a $\mathbf{x}_i$.)

- Si modifica **solo il prototipo vincitore**, spostandolo verso $\mathbf{x}$.
- È una discesa del gradiente stocastica → **minimi locali**; il metodo dipende dall'**inizializzazione**.
- Il **numero di cluster** va fornito in anticipo.

19.3.2 Versione batch

In sintesi — Algoritmo K-means batch (Linde-Buzo-Gray, o Lloyd generalizzato)

1. Scegliere K centri: K pattern estratti a caso, o K punti casuali nell'ipervolume che contiene i pattern.
2. Assegnare ogni pattern al centro più vicino (il **vincitore**):

$$i^*(\mathbf{x}) = \arg \min_i \|\mathbf{x} - \mathbf{w}_i\|^2, \quad \|\mathbf{x} - \mathbf{w}_i\|^2 = \sum_{j=1}^n (x_j - w_{ij})^2.$$

3. Ricalcolare i centri come **media** (centroide geometrico) dei membri del cluster:

$$\mathbf{w}_i = \frac{1}{|\text{cluster}_i|} \sum_{\mathbf{x}_j \in \text{cluster}_i} \mathbf{x}_j.$$

4. Se il criterio di convergenza non è soddisfatto (nessuna o minima riassegnazione dei pattern, minima diminuzione dell'errore quadratico), tornare al passo 2.

(Esercizio: dalla versione on-line a quella batch. Sommando — o mediando — sui pattern di un cluster il delta on-line con $\eta = 1$, si ottiene $\mathbf{w}_{new} = \mathbf{w}_{old} + \frac{1}{|C|} \sum_{\mathbf{x} \in C} (\mathbf{x} - \mathbf{w}_{old}) = \frac{1}{|C|} \sum_{\mathbf{x} \in C} \mathbf{x}$: proprio la media.)

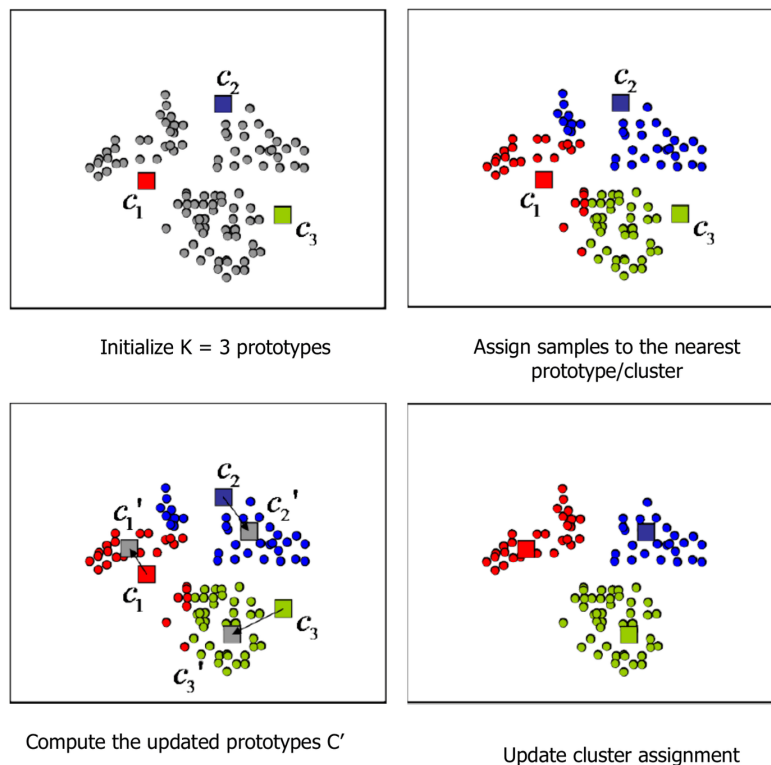


Figura 19.3: Esempio di K-means con $K = 3$: inizializzazione, assegnazione, aggiornamento dei prototipi, nuova assegnazione.

19.3.3 Pregi e difetti

- È l'algoritmo più semplice e usato con un criterio d'errore quadratico; è popolare perché facile da implementare e in generale efficiente.
- Il numero K di cluster va fornito.
- I minimi locali di E rendono il metodo dipendente dall'inizializzazione, con scelte solo sub-ottime dei prototipi.
- Funziona molto bene per cluster **compatti e ipersferici**; con cluster di altre forme (ad esempio due "spirali" intrecciate) fallisce.
- **Nessuna proprietà di visualizzazione**: non permette di proiettare i dati in uno spazio a dimensione minore, e l'indicizzazione dei $\mathbf{w}$ è arbitraria, quindi la mappatura non è ordinata.

19.3.4 Soft-max

Per evitare i minimi locali si introduce una regola di adattamento **soft-max**: non si modifica solo il vincitore, ma **tutti** i centri, in base alla loro **vicinanza** a $\mathbf{x}$, con un passo che decresce con la distanza $d(\mathbf{x}, \mathbf{w}_i)$ (ad esempio il *maximum-entropy clustering* con distanze gaussiane). Un'istanza di questa strategia nelle reti neurali sono le **SOM di Kohonen**, dove però la vicinanza tra i vettori di riferimento è definita su una **mappa ordinata**, cioè una griglia neurale su cui sono disposti.

19.4 Self-Organizing Map

Le **Self-Organizing Map** (SOM), o **mappe di Kohonen** (1981), sono reti di N neuroni disposti su una **griglia regolare a bassa dimensione** (di solito 2D, un *lattice*).

- Ogni unità è identificata dalle sue **coordinate** sulla mappa.
- Ogni unità riceve **lo stesso input** $\mathbf{x}$.
- Ogni unità ha un **peso** $\mathbf{w}$, con $\dim(\mathbf{w}) = \dim(\mathbf{x}) = n$.

19.4.1 Obiettivo

La SOM impara una mappa dallo spazio degli input a un reticolo di unità neurali che **preserva la topologia**:

- unità vicine sulla mappa rispondono a pattern di input simili;
- punti vicini nello spazio di input vengono mappati sulla stessa unità o su unità vicine;
- la mappa di output (2D) ha proprietà di **visualizzazione**.

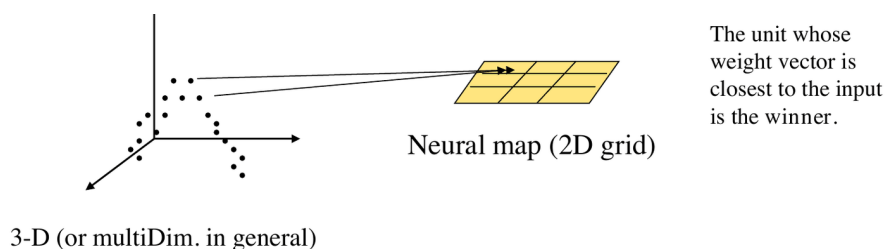


Figura 19.4: Una SOM mappa lo spazio di input (qui 3D) su una griglia neurale 2D.

Usi della SOM:

- **clustering**: i cluster si identificano sulla mappa (si trovano rapidamente strutture nei dati);
- **pattern recognition e VQ**: l'input viene trasformato nel vettore dei pesi del neurone più vicino (il vettore del codebook);
- **compressione**: l'input viene trasformato nell'indice (codice) dell'unità vincitrice;
- **proiezione ed esplorazione**: la distribuzione dei dati multidimensionali si visualizza in uno spazio a dimensione minore (la mappa 2D).

19.4.2 Apprendimento competitivo e ispirazione biologica

L'apprendimento competitivo è un processo adattivo in cui i neuroni diventano gradualmente sensibili (specializzati) a categorie o insiemi di input diversi: i neuroni **competono** per un dato, e il vincitore apprende di più.

La natura sfrutta questo meccanismo: nella **corteccia somatosensoriale** (e motoria) c'è una mappa **topologicamente ordinata** del corpo, l'**homunculus**. Neuroni adiacenti rappresentano sorgenti di attivazione vicine nel corpo, e le parti più sensibili occupano aree più ampie della corteccia. In una mappa che preserva la topologia, unità fisicamente vicine rispondono a classi di input a loro volta vicine.



Figura 19.5: L'homunculus: una mappa topologicamente ordinata del corpo nella corteccia.

19.4.3 L'algoritmo

In sintesi — Algoritmo SOM

- Inizializzare casualmente i pesi della mappa.
- Estrarre un input $\mathbf{x}$.
- **Fase competitiva**: il vincitore è l'unità con $\mathbf{w}$ più simile a $\mathbf{x}$.
- **Fase cooperativa**: aggiornare i pesi delle unità che hanno relazioni topologiche **sulla mappa** con il vincitore (una forma di soft-max).
- Continuare fino a convergenza (nessun cambiamento).

Fattore chiave: i vicini **sulla mappa** (mentre il soft-max standard usa la vicinanza nello spazio dei dati).

Fase competitiva

L'input $\mathbf{x}$ viene confrontato con i pesi di tutte le unità con la distanza euclidea. La SOM standard adotta una strategia **winner-take-all** (come il K-means):

$$i^*(\mathbf{x}) = \arg \min_i \|\mathbf{x} - \mathbf{w}_i\|,$$

dove i è un indice sulla mappa (ad esempio le coordinate 2D).

Fase cooperativa

I pesi del vincitore **e delle unità del suo vicinato** vengono avvicinati all'input (apprendimento hebbiano). All'iterazione t :

$$\mathbf{w}_i(t+1) = \mathbf{w}_i(t) + \eta(t) h_{i,i^*(\mathbf{x})}(t) [\mathbf{x} - \mathbf{w}_i(t)],$$

dove h è la **funzione di vicinato** (*neighborhood kernel*), che decresce monotonamente all'aumentare della distanza **sulla mappa** tra l'unità i e il vincitore. Ad esempio una gaussiana:

$$h_{i,i^*}(t) = \exp\left(-\frac{\|\mathbf{r}_i - \mathbf{r}_{i^*}\|^2}{2\sigma^2(t)}\right),$$

con $\mathbf{r}_i$ le coordinate dell'unità i sulla griglia e $\sigma(t)$ la larghezza.

Il **learning rate** η e il **raggio del vicinato** σ diminuiscono con le iterazioni: il vicinato è **ampio all'inizio** (ordinamento globale della mappa) e poi si restringe lentamente durante l'apprendimento (convergenza, raffinamento locale). In pratica la forma del vicinato si sceglie tra forme geometriche standard sulla griglia (rettangolare, esagonale...) che includono un insieme finito di unità.

19.4.4 L'emergere dell'ordine topologico

La regola della fase cooperativa è **fondamentale** per la formazione di mappe topograficamente ordinate. I pesi non vengono modificati indipendentemente, ma **per sottoinsiemi topologicamente legati**: a ogni passo si seleziona il vicinato del vincitore sulla griglia, e tutte le unità del sottoinsieme ricevono aggiornamenti simili. Così l'informazione topologica viene trasmessa alla mappa: il vincitore e i suoi vicini sulla griglia ricevono aggiornamenti simili e, dopo l'apprendimento, rispondono a input simili. Anche se sembra naturale, una dimostrazione formale dell'ordinamento è difficile.

Suggerimento — K-means contro SOM

- **K-means**: aggiorna solo il vincitore; i prototipi non hanno alcun ordine; non c'è visualizzazione.
- **SOM**: aggiorna il vincitore **e i suoi vicini sulla griglia**, con un vicinato che si restringe nel tempo; i prototipi sono disposti su una mappa ordinata che **preserva la topologia**, permettendo la visualizzazione; il vicinato ampio iniziale aiuta anche a evitare i minimi locali. (Con vicinato di raggio zero, la SOM si riduce al K-means on-line.)

Durante l'addestramento le unità “vanno a rappresentare” i cluster dei dati.

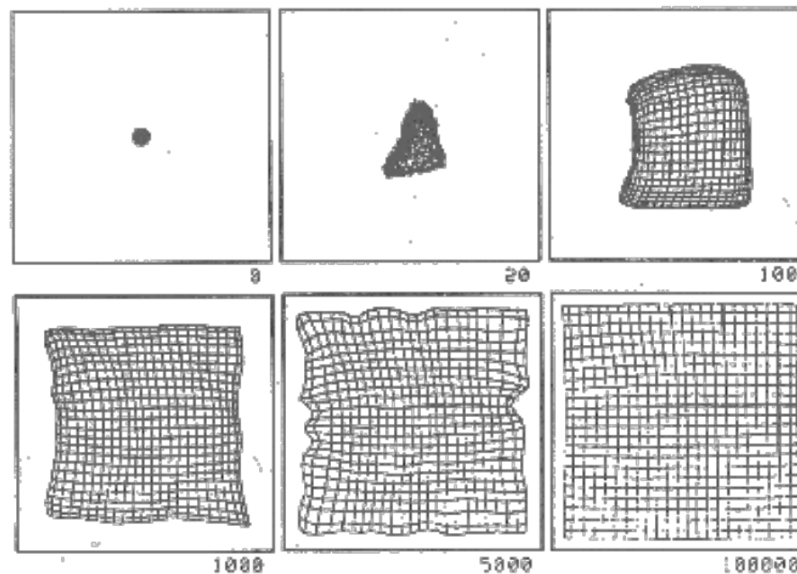


Fig. 3.4. Reference vectors during the ordering process, square array. The numbers at lower right-hand corner indicate learning cycles

Figura 19.6: Addestramento di una SOM su dati uniformi in 2D: i vettori di riferimento (collegati secondo la griglia) si distendono progressivamente sulla distribuzione.

19.4.5 Visualizzazione

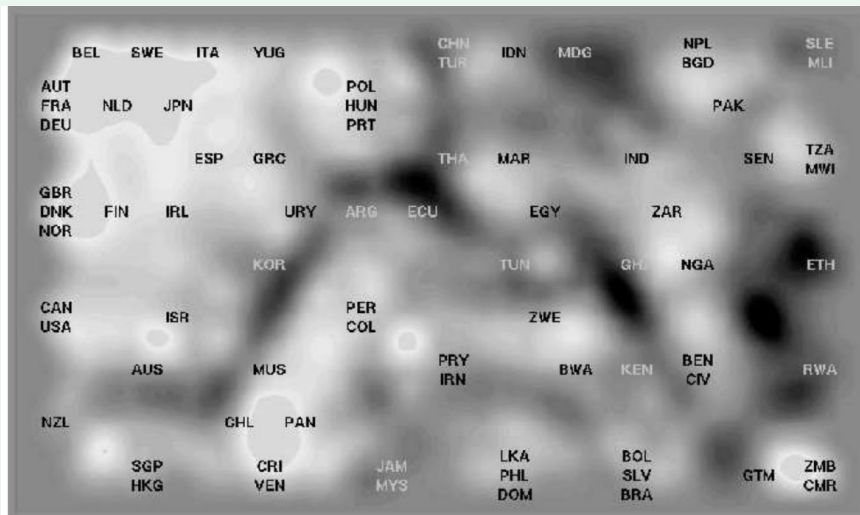
La mappa si può usare per visualizzare diverse caratteristiche della SOM e dei dati:

- la **densità** dei vettori di riferimento (colore proporzionale al numero di “hit”);
- le **distanze** tra i prototipi di unità vicine (**U-matrix**): colori scuri indicano grandi distanze tra unità adiacenti, colori chiari piccole distanze;
- le mappe di Sammon (con linee)...

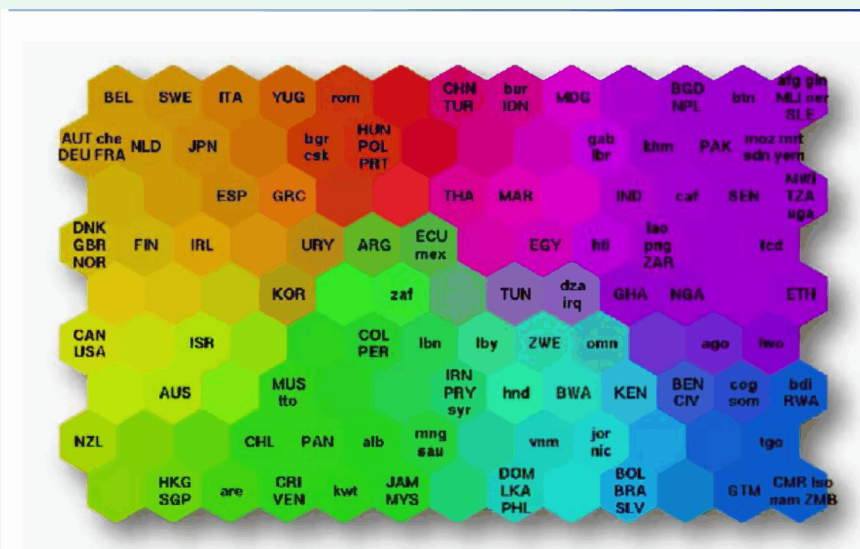
Questo rende la SOM facilmente **interpretabile**.

Esempio — Welfare e povertà nel mondo (Kaski e Kohonen, 1995)

39 indicatori di benessere per 77 paesi (World Development Report 1992). Sulla mappa 13×9, il grigio tra due unità indica la distanza (scalata e smussata) tra i loro vettori di riferimento: le zone chiare sono cluster di paesi simili, le zone scure i confini tra cluster. Non serve alcuna ipotesi a priori sulla forma dei cluster.



A smoothed distance matrix of 12 X-Y SOM lattice cells (illustrating the clustering structure)



19.5 Valutare un clustering (cenni)

Come si valuta il risultato di un clustering? Cosa distingue un risultato buono da uno scadente?

- La valutazione è spesso **sogettiva**: esistono pochi “gold standard”, tranne in sotto-domini ben definiti.
- Misure **oggettive**, come l'errore di quantizzazione (che però dipende dal numero di centroidi).
- **Non** si usa banalmente l'etichetta di un task di classificazione sottostante (la *purity*, con entropia, informazione mutua...): se si hanno le etichette, tanto vale usare un classificatore.
- Qualità del modello/clustering contro qualità dei risultati (dipendente dal dominio).

Altri argomenti dell'apprendimento non supervisionato: clustering gerarchico (dendrogrammi), analisi delle componenti principali (e di curve e superfici principali), regole di associazione, ICA, Multidimensional Scaling, t-SNE, Generative Topographic Mapping, reti ART.

Domande — Possibili domande d'esame

- Definire clustering e quantizzazione vettoriale. Cos'è l'errore di quantizzazione?
- Derivare la regola on-line del K-means e passare alla versione batch.
- Pregi e difetti del K-means.
- Descrivere la SOM: architettura, fase competitiva, fase cooperativa, funzione di vicinato.
- Perché la SOM preserva la topologia? Differenze tra K-means e SOM.
- Come si visualizza una SOM (U-matrix)?

Capitolo 20

Reti neurali ricorrenti (RNN)

Questa è un'introduzione leggera alle **reti neurali ricorrenti** (RNN), che saranno approfondite nei corsi ISPR e CNS. Finora le reti erano **feedforward** (direzione input → output) e lavoravano su **vettori**. Le RNN aggiungono **connessioni di feedback** (cicli) nella topologia: la presenza di auto-connessioni dà alla rete **proprietà dinamiche** e una **memoria** (uno stato) delle computazioni passate. Questo estende la capacità rappresentativa del modello all'elaborazione di **sequenze** (e di dati strutturati). Le RNN sono anche biologicamente plausibili: le reti neurali biologiche sono ricorrenti.

20.1 Perché i dati sequenziali

Servono modelli per sequenze quando l'uscita dipende dalla **storia degli input** (ad esempio nel tempo: modelli dinamici), o quando il dominio è fatto di **sequenze di lunghezza variabile**:

- processi dinamici, elaborazione di segnali (filtri, controllo), robotica;
- **linguaggio**: riconoscimento del parlato, NLP, linguaggi formali, information retrieval;
- visione e ragionamento su eventi temporali;
- **serie temporali**: previsioni finanziarie, elaborazione di segnali;
- genomica e proteomica (bioinformatica): ad esempio predire la struttura di una proteina dalla sequenza di amminoacidi;
- riconoscimento di attività umane da dati di sensori.

Le RNN (e gli approcci collegati) sono state il riferimento per l'elaborazione di sequenze, con risultati allo stato dell'arte nel **riconoscimento del parlato** e nell'**elaborazione del testo** (traduzione automatica...), ma anche per applicazioni “divertenti” come la composizione musicale e la generazione di testo e parlato.

20.2 Dai dati piatti ai dati strutturati

Finora i dati erano **piatti** (vettori). I dati **strutturati** sono sequenze, alberi, grafi, dati multi-relazionali: stringhe, proteine, serie temporali, piccole molecole, reti.

20.2.1 Il nuovo dominio: sequenze e trasduzioni

- **Dati**: sequenze discrete di **vettori** (con un ordine seriale, ad esempio il tempo), di lunghezza variabile. Ogni elemento $\mathbf{l}_t$ è un vettore (es. $\mathbf{l}_t = [1, 0, 1, 0, 7]$).
- **Task**: una **trasduzione** nel dominio sequenziale, cioè una mappatura *sequenza di input* → *valore o sequenza di output*.

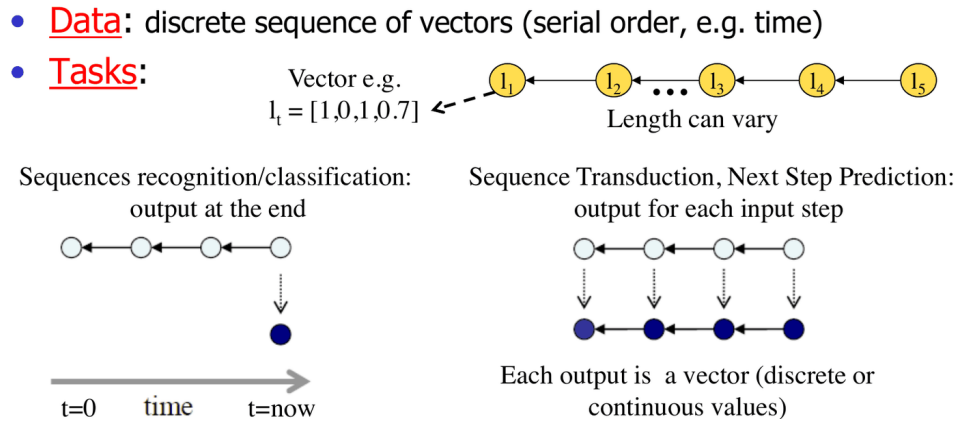
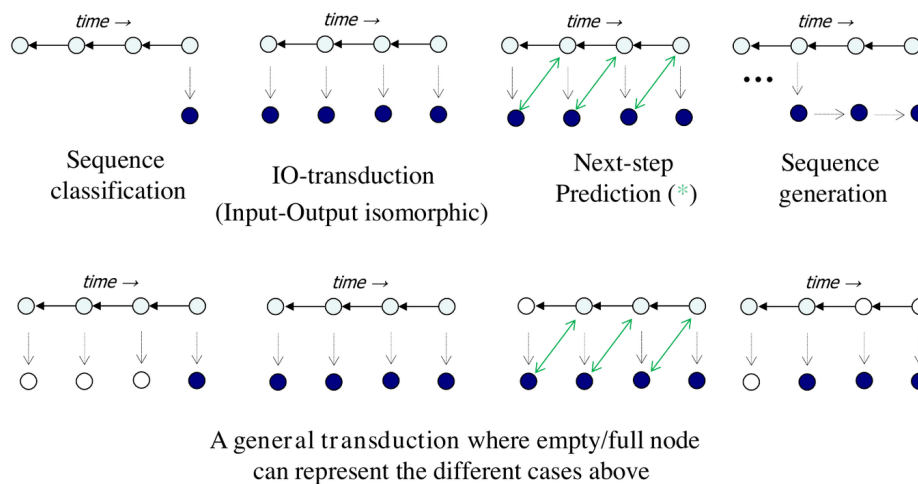


Figura 20.1: Riconoscimento/classificazione di sequenze (uscita alla fine) e trasduzione di sequenze (uscita a ogni passo).



*Autoregressive: next-step prediction with just 1 sequence

Figura 20.2: Tipi di trasduzione: classificazione di sequenze, trasduzione IO isomorfa (un'uscita per ogni input), predizione del passo successivo, generazione di sequenze. La predizione autoregressiva usa una sola sequenza.

20.3 La memoria nelle reti neurali

L'obiettivo è che l'uscita dipenda dagli input **precedenti**.

20.3.1 Approccio 1: memoria finita (IDNN)

Il primo approccio è "spaziale", a **memoria finita**: le **Input Delay Neural Network** (nella classe delle *Time-Delay NN*, già viste all'inizio delle CNN). I ritardi sono nelle connessioni e fanno da memoria dell'input: un **registro a scorrimento** di dimensione finita, una **finestra scorrevole** sulla sequenza. (Le CNN hanno esteso questa idea alle immagini 2D.)

- La dimensione della finestra va **nota in anticipo**.
- Il numero di pesi **dipende dalla dimensione del buffer**.
- Se il buffer contiene abbastanza informazione, si risolve il problema con un normale addestramento di MLP (ad esempio NETtalk, 1988).

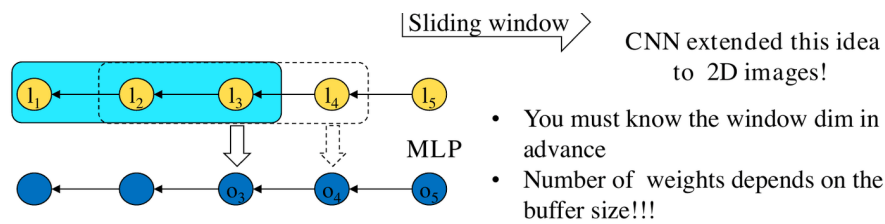


Figura 20.3: IDNN: una finestra scorrevole di dimensione fissa sulla sequenza, elaborata da un MLP

20.3.2 Approccio 2: unità ricorrenti

Un'**unità neurale ricorrente** ha una connessione di feedback (un *self-loop*) e usa sia l'input corrente sia l'informazione di **stato**:

$$x(t) = \tau(\mathbf{l}(t), x(t-1)) = f(\mathbf{w}^T \mathbf{l}(t) + \hat{w} x(t-1) + \theta), \quad x(0) = 0,$$

dove $\mathbf{l}(t)$ è l'input (etichetta) al tempo t , f è l'attivazione (sigmoidale), $\hat{w}$ è il **peso ricorrente**, θ il bias, e $x(t-1)$ è lo stato al passo precedente, fornito da un ritardo unitario q^{-1} . La parte nuova rispetto a un neurone normale è il termine $\hat{w} x(t-1)$.

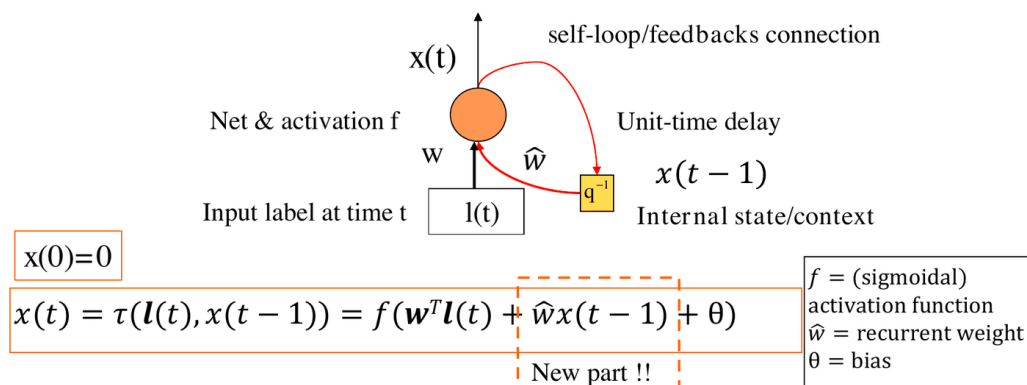


Figura 20.4: Un'unità ricorrente: il self-loop con ritardo unitario porta lo stato precedente come input aggiuntivo.

- Lo **stato** riassume l'informazione passata (il **contesto**): cambia valore seguendo il flusso degli input.
- La **codifica del passato è adattiva**, perché dipende dai pesi liberi del modello.

Esempio — Esercizio: contare gli “1”

Realizzare una RNN con una sola unità (trovando i valori di w e $\hat{w}$) che restituisca il numero di “1” ricevuti finora in una sequenza di 0 e 1.

Soluzione: unità lineare con $w = 1$, $\hat{w} = 1$, $\theta = 0$, cioè $x(t) = l(t) + x(t-1)$. Con l'input 1, 0, 1, 1, 0, 1 lo stato vale 1, 1, 2, 3, 3, 4: **lo stato accumula la somma degli “1”** ricevuti. La RNN riassume l'informazione della sotto-sequenza passata, cosa che una IDNN **non può fare** per sequenze di lunghezza variabile (la sua memoria è limitata alla finestra).

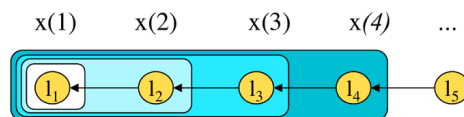
20.3.3 Sistemi a transizione di stato

In generale, un modello ricorrente è un **sistema a transizione di stato**:

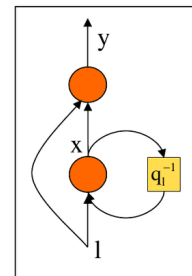
$$\begin{cases} \mathbf{x}(t) = \tau(\mathbf{x}(t-1), \mathbf{l}(t)) \\ \mathbf{y}(t) = g(\mathbf{x}(t), \mathbf{l}(t)) \end{cases} \quad \mathbf{x}(0) = \mathbf{0}.$$

Given $\mathbf{x}(0) = \mathbf{0}$

$$\begin{cases} \mathbf{x}(t) = \tau(\mathbf{x}(t-1), \mathbf{l}(t)) \\ \mathbf{y}(t) = g(\mathbf{x}(t), \mathbf{l}(t)) \end{cases}$$



State formation over time



Graphical model

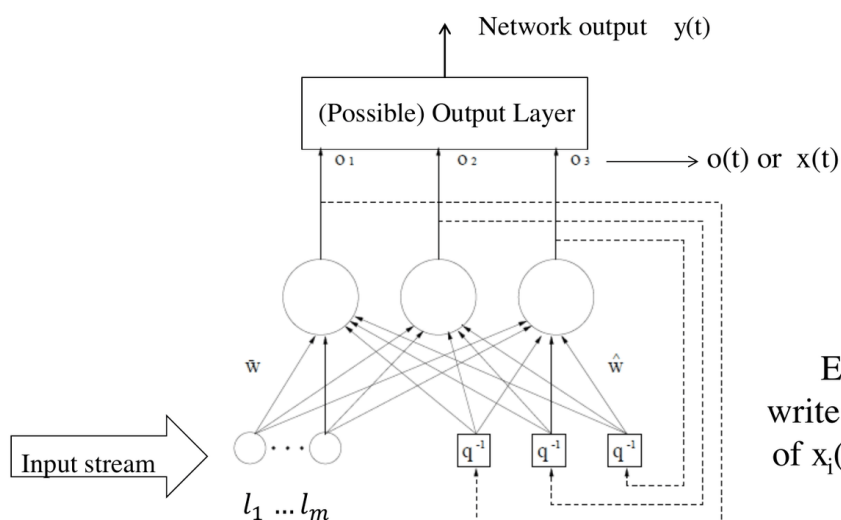
E.g. also HMM

Figura 20.5: Sistema a transizione di stato: lo stato al tempo t codifica (è un

- τ è la **funzione di transizione di stato** (*next-state function*), realizzata da una rete neurale.
- Lo stato riassume gli input passati: è una **codifica** (embedding) delle sotto-sequenze.
- Lo stato ha valori **continui** (reali) (a differenza, ad esempio, degli automi o degli HMM, che hanno stati discreti).
- $\mathbf{x}(t)$ può essere un insieme di stati, per comporre una rete di unità ricorrenti.

20.3.4 Una rete ricorrente (Simple RNN)

Con molte unità ricorrenti nascoste si ottiene la **Simple RNN** di Elman: ogni unità nascosta riceve l'input e gli stati precedenti di **tutte** le unità nascoste (attraverso i ritardi), e uno stato di uscita (eventuale) produce $\mathbf{y}(t)$.



Exercise:
write the equation
of $\mathbf{x}_i(t)$ (and then
of $\mathbf{x}(t)$)

Figura 20.6: Una Simple RNN (Elman) con più unità ricorrenti nascoste.

Esempio — Esercizio: le equazioni

Per l'unità nascosta i :

$$x_i(t) = f \left(\sum_{j=1}^m w_{ij} l_j(t) + \sum_k \hat{w}_{ik} x_k(t-1) + \theta_i \right),$$

e in forma vettoriale $\mathbf{x}(t) = f(W\mathbf{l}(t) + \hat{W}\mathbf{x}(t-1) + \theta)$.

20.3.5 Proprietà

Anche la semplice RNN è già molto potente:

- è un **approssimatore universale** di sistemi dinamici non lineari;
- è **Turing-equivalente** (può simulare qualunque automa);
- è un sistema dinamico non lineare (non autonomo), studiabile con la teoria dei sistemi dinamici (caos, frattali...).

I modelli ricorrenti si basano su tre assunzioni:

- **Causalità**: l'uscita al tempo t_0 dipende solo dagli input ai tempi $t \leq t_0$. È necessaria e sufficiente per avere uno stato interno.
- **Stazionarietà**: invarianza nel tempo (dopo l'addestramento): la funzione di transizione τ è **la stessa** in ogni istante, indipendentemente dalla lunghezza delle sequenze. È ciò che permette di elaborare dati di lunghezza variabile con un modello di dimensione fissa.
- **Adattività**: le funzioni di transizione sono realizzate da reti neurali con pesi liberi, quindi apprese dai dati.

20.4 Apprendimento: l'unfolding

Si **srotola** (*unfolding*) nel tempo lo stesso modello lungo la sequenza di input: si costruisce un MLP feedforward (la **rete di codifica**, *encoding network*) equivalente alla RNN sui k passi della sequenza, con **una replica del modello per ogni passo**. Per la stazionarietà, le repliche **condividono i pesi**.

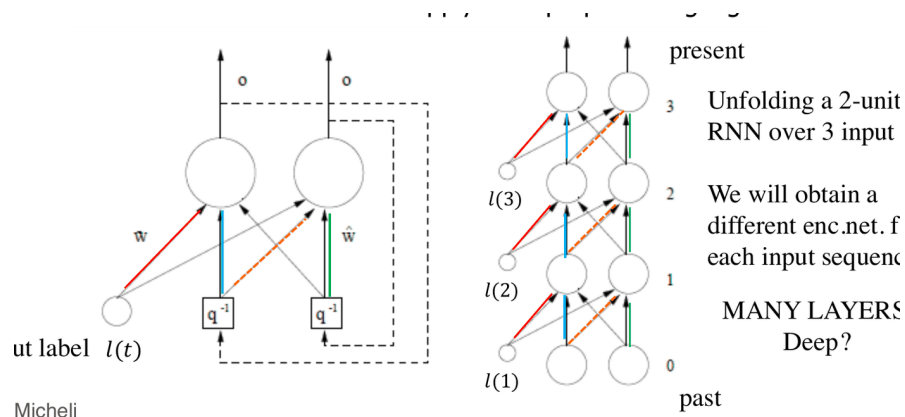


Figura 20.7: Unfolding di una RNN a 2 unità su 3 passi di input: si ottiene una rete feed-forward a molti strati, diversa per ogni sequenza di input.

Suggerimento — Il punto chiave

Sulla rete di codifica srotolata si può applicare la **backpropagation**! Si ottiene una rete diversa per ogni sequenza di input, con **molti strati** (tanti quanti i passi): è di fatto una rete **profonda**, con pesi condivisi tra gli strati.

Gli algoritmi di apprendimento supervisionato per RNN devono tenere conto di tutte le transizioni di codifica sviluppate dal modello a ogni passo:

- **BPTT** (*Back-Propagation Through Time*);
- **RTRL** (*Real-Time Recurrent Learning*).

Calcolano in modo diverso lo stesso gradiente degli errori di uscita sulla rete srotolata equivalente.

Attenzione — Dipendenze a lungo termine

Poiché la rete srotolata è molto profonda, il **vanishing gradient** rende difficile apprendere **dipendenze a lungo termine** (quando l'uscita dipende da input molto lontani nel passato).

20.4.1 Modelli avanzati

È un campo di ricerca in rapida crescita, con molte varianti:

- **LSTM** (*Long Short-Term Memory*): cercano di risolvere il vanishing gradient con **unità “gate”** capaci di selezionare il flusso del passato e del gradiente. Sono nate per le dipendenze **lunghe**: non sono adatte a serie brevi, quindi, nonostante la popolarità, non vanno usate acriticamente;
- **GRU** (*Gated Recurrent Units*): una LSTM semplificata;
- **BRNN** (RNN bidirezionali): considerano sia il contesto a sinistra sia quello a destra;
- spesso si fondono reti profonde e RNN;
- contro il vanishing gradient: ottimizzatori Hessian-free, tecniche di pre-training...

Sono incluse nelle principali librerie (PyTorch, Keras, Matlab...). Alcuni dati sulla loro diffusione: nel 2015 le LSTM erano usate in 2 miliardi di telefoni Android e 1 miliardo di iPhone (Siri); nel 2016 quasi il 30% della potenza di calcolo per l'inferenza dei datacenter di Google era usato per le LSTM; nel 2017 erano in Alexa e in Facebook (30 miliardi di traduzioni a settimana). Un esempio curioso: generare la versione **manoscritta** di un testo predicendo un punto (x, y) della traiettoria della penna alla volta (Graves, 2013).

20.5 Approcci collegati

- SOM e reti non supervisionate per domini sequenziali.
- **Hidden Markov Model** (modellano la distribuzione di probabilità delle transizioni di stato).
- **Transformer**.
- **RNN randomizzate**: il **Reservoir Computing** (ESN, *Liquid State Machine*).
- Modelli basati su distanze (string matching), kernel per stringhe, inferenza grammaticale, ILP.

20.5.1 Transformer e attenzione

Un **transformer** (2017) è un'architettura di deep learning basata sul meccanismo di **attenzione multi-head** in parallelo. L'attenzione permette al modello di **accedere a qualunque punto precedente** della sequenza; encoder e/o decoder più reti feedforward completano l'architettura per scopi diversi.

Lo **strato di attenzione** pesa tutti gli stati precedenti secondo una misura di **rilevanza appresa**, fornendo potenzialmente informazione anche su token molto lontani. Si è rivelato particolarmente utile nella traduzione, dove un contesto lontano può essere essenziale per il significato di una parola. Usato prima per migliorare le RNN, poi anche da solo (*"Attention is all you need"*). Tutti i token vengono elaborati **simultaneamente**, calcolando pesi tra le loro rappresentazioni (embedding) in strati successivi: sono semplicemente unità a **prodotto scalare scalato** tra vettori *query* e vettori delle parole, che cercano le correlazioni più alte tra le parole di una frase (ad esempio in "see that girl", "that" viene associato a "girl"). Si applica anche a *patch* di immagini invece che a parole (*vision transformer*).

20.5.2 Reti ricorrenti randomizzate: le Echo State Network

Possiamo sfruttare direttamente questa macchina a stati per codificare le sequenze, e poi usare l'embedding risultante per imparare la mappatura di uscita? Sì, **anche con reti ricorrenti connesse casualmente con pesi fissi** (non addestrati)! È la classe del **Reservoir Computing** (RC): Echo State Network, *liquid state machine*, *fractal machine* (si veda [18 - Reti neurali randomizzate](#)).

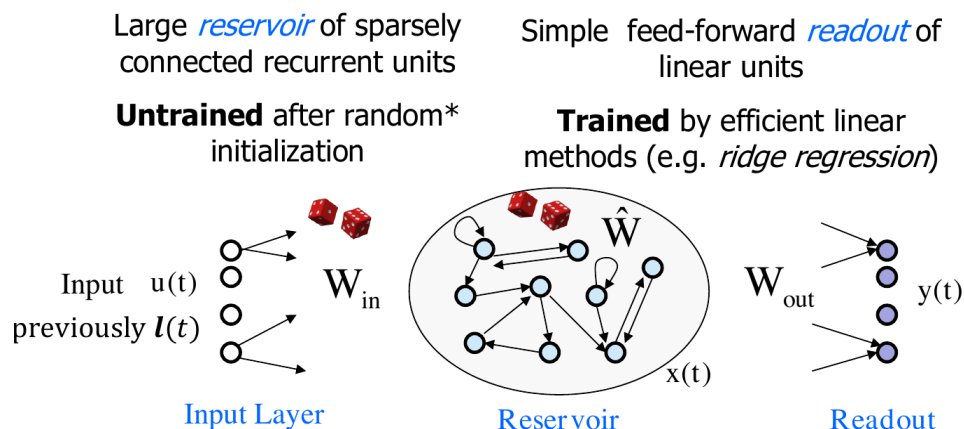


Figura 20.8: Una Echo State Network: reservoir casuale non addestrato e readout lineare addestrato.

Definizione — Echo State Network (ESN)

Una ESN è composta da: - un grande **reservoir** di unità ricorrenti connesse in modo sparso, **non addestrate** dopo un'inizializzazione casuale; - un semplice **readout** feedforward di unità **lineari**, addestrato con metodi lineari efficienti (es. **ridge regression**).

Echo State Property: la funzione di transizione di stato deve essere **contrattiva** (si limita il raggio spettrale dei pesi del reservoir, cioè si garantisce la stabilità del sistema dinamico). Così lo stato ricorrente dipende asintoticamente (come un'eco) **solo dalla storia degli input**, e non dalle condizioni iniziali dello stato.

Le ESN sono **molto efficienti** (nessun addestramento delle unità ricorrenti) e risolvono bene i task sotto certe condizioni.

Organizzazione markoviana dello spazio degli stati

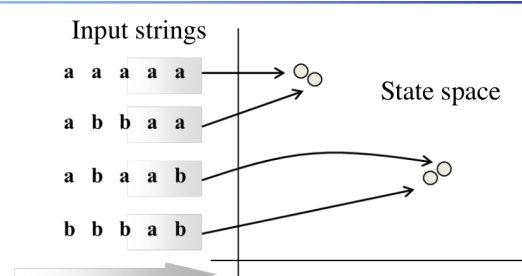


Figura 20.9: Il reservoir proietta le sequenze nello spazio degli stati in modo “markoviano”: sequenze con lo stesso suffisso finiscono vicine.

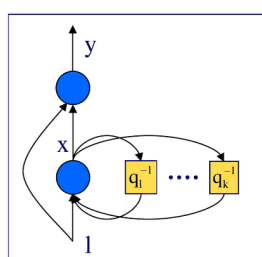
Il reservoir agisce come una **proiezione** (embedding) delle sequenze di input nel vettore degli stati $\mathbf{x}(t)$, e riesce intrinsecamente a **discriminare le sequenze in base al suffisso**, senza addestrare i pesi ricorrenti: gli stati sono tanto più vicini quanto più è lungo il **suffisso comune** (gli input recenti contano di più, per la contrattività). Questo rende facile apprendere task **markoviani** (anche per le RNN addestrate, ma le ESN sono molto più efficienti), ed è un **bias architetturale** che vale anche per i modelli ricorrenti con apprendimento (Gallicchio, Micheli, 2011).

Reservoir Computing profondo. Si possono impilare più reservoir (**DeepESN**, con molti risultati del gruppo CIML di Pisa), unendo la potenza delle RNN profonde all’efficienza delle ESN. L’organizzazione a strati mostra vantaggi in termini di **scale temporali multiple** e di **ricchezza delle dinamiche**: nell’analisi in frequenza, gli strati più alti si concentrano sulle frequenze più basse (scale temporali più ampie), formando una rappresentazione temporale gerarchica.

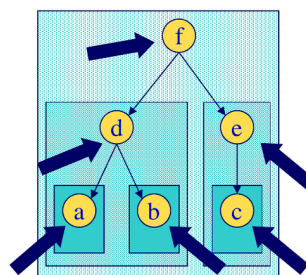
20.6 Verso i domini strutturati

Si può estendere l’approccio ricorrente agli **alberi**? Sì: le **reti neurali ricorsive** (RecNN). La codifica procede **dal basso verso l’alto** (dalle foglie alla radice), e lo stato di ogni vertice dipende dal suo label e dagli stati dei **figli** (estendendo la funzione di transizione a più stati).

- Extending states for children of vertices of the tree



Graphical model for trees



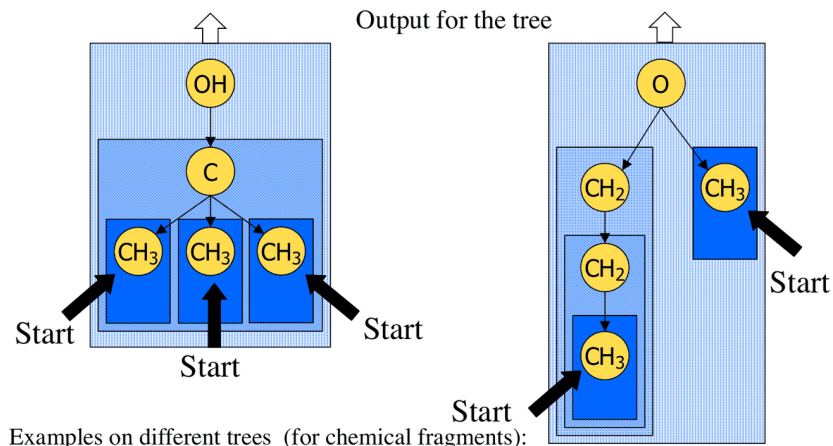
Unfolding the encoding process through structure

If we change the tree structure, the encoding change accordingly ! See next slide

Figura 20.10: Modello grafico per alberi e unfolding della codifica lungo la struttura: se la struttura dell’albero cambia, la codifica cambia di conseguenza.

In sintesi — Sintesi

Le RNN aggiungono **stati** e **feedback** alle reti, diventando **sistemi a transizione di stato** capaci di elaborare sequenze di lunghezza variabile. Si basano su causalità, stazionarietà e



Examples on different trees (for chemical fragments):

Figura 20.11: Una rete ricorsiva applicata ad alberi diversi (frammenti chimici): le unità sono le stesse per tutti i vertici di un albero e per tutti gli alberi del dataset (condivisione dei pesi).

adattività; si addestrano **srotolandole** nel tempo e applicando la backpropagation (BPTT, RTRL), ma soffrono di **vanishing gradient** sulle dipendenze lunghe (da cui LSTM e GRU). Alternative e sviluppi: transformer (attenzione), Echo State Network (reservoir casuale + readout lineare, con un bias markoviano), reti ricorsive per alberi e grafi.

Domande — Possibili domande d'esame

- Perché servono modelli per dati sequenziali? Cos'è una trasduzione?
- Differenza tra IDNN e RNN; perché una IDNN non può contare gli "1" in sequenze arbitrarie?
- Scrivere l'equazione di un'unità ricorrente e di una Simple RNN.
- Quali sono le assunzioni di causalità, stazionarietà e adattività?
- Cos'è l'unfolding e come permette di usare la backpropagation?
- Cos'è una Echo State Network e cos'è la Echo State Property?
- Come si estendono le RNN agli alberi?

Apprendimento su dati strutturati e grafi

L'ultima lezione del corso è un'introduzione avanzata all'**apprendimento in domini strutturati** (SD, *Structured Domains*) e in particolare all'**apprendimento su grafi** con le **Deep Graph Networks** (DGN). È anche una panoramica delle ricerche del gruppo CIML di Pisa, che ha contribuito in modo pionieristico a questo campo. Le domande guida: si può fare deep learning sui grafi in modo **efficiente** (senza un addestramento completamente *end-to-end*)? La **profondità** nelle DGN serve, ma causa problemi: perché? Qual è il rapporto tra profondità dei modelli e difficoltà di addestramento?

21.1 Perché dati strutturati e grafi

Perché i dati hanno relazioni. Esempi di grafi e reti:

- astrazioni di immagini e reti per la comprensione di scene;
- **social network**, **knowledge graph**;
- analisi sintattica del linguaggio (alberi di parsing);
- reti di trasporto (ad esempio la previsione del traffico su Google Maps, di DeepMind);
- **piccole molecole** (drug design), reti biologiche (proteine);
- termini logici, modellazione di reti cerebrali.

EXAMPLES: GRAPHS & NETWORKS

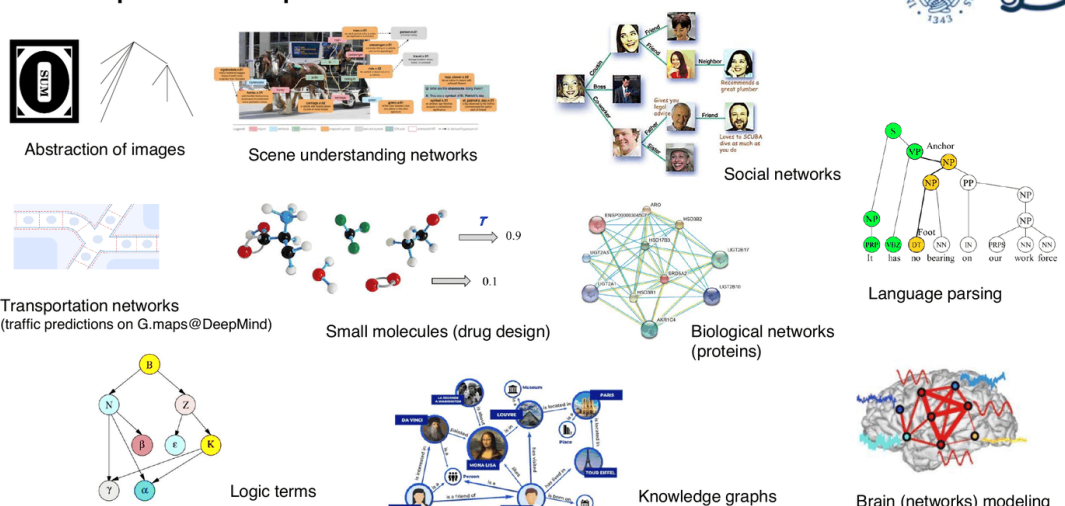


Figura 21.1: Esempi di grafi e reti in domini diversi.

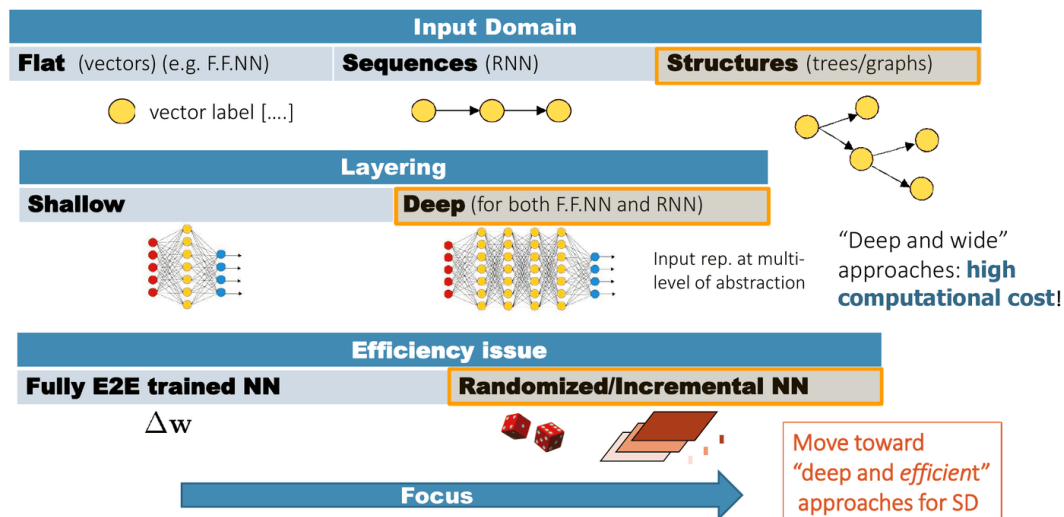


Figura 21.2: Lo scenario: dal dominio di input piatto a quello strutturato, da modelli superficiali a profondi, dall'addestramento end-to-end a quello randomizzato o incrementale. L'obiettivo è ottenere approcci "profondi ed efficienti" per i dati strutturati.

21.1.1 Lo scenario

Il corso ha seguito questa progressione: dai **vettori** (reti feedforward) alle **sequenze** (RNN) alle **strutture** (alberi, grafi). Gli approcci profondi e "larghi" (*deep and wide*) rappresentano l'input su più livelli di astrazione ma hanno un alto costo computazionale; le reti **randomizzate** o **incrementali** offrono l'efficienza.

21.1.2 Un po' di storia

Le reti neurali per dati strutturati hanno una storia in gran parte **italiana**:

- dalla fine degli anni '90, le **reti neurali ricorsive** per alberi e grafi aciclici orientati (Sperduti, Starita, Frasconi, Gori, Hammer, Micheli, Bianchini, Scarselli, Hagenbuchner...): reti ricorsive (1997–2000), Cascade Correlation per strutture, SOM per strutture (2003–2005), Hidden Tree Markov Model bottom-up (2012), Tree ESN e reservoir computing profondo per alberi (2010–2018);
- l'estensione ai **grafi generali**, pionieristica tra Pisa e Siena (2005–2009), con due approcci:
 - ricorsivi:** GNN (Scarselli et al., 2009) e GraphESN (Gallicchio, Micheli, 2010);
 - a strati / convoluzionali:** NN4G (Micheli, 2009) in versione costruttiva, poi seguito come reti convoluzionali per grafi (GCN, ecc.).

Dal 2015 è un'area in **esplosione** a livello mondiale, con decine di articoli in tutte le principali conferenze e applicazioni reali, sotto nomi diversi: *Graph Representation Learning*, *learning on graphs*, *deep learning for graphs*, *Graph Neural Networks* (GNN), *Graph Convolutional Networks* (GCN), e il quadro delle **Deep Graph Networks** (DGN; Bacciu, Errica, Micheli, Podda, 2020).

21.2 Deep Graph Networks

21.2.1 I grafi considerati

Si considerano **grafi etichettati**: ogni **nodo** (vertice) v ha un'**etichetta vettoriale** $l(v)$ (es. $[1, 0, 1, 0, 7]$), ogni **arco** (link) può avere un'etichetta (es. una posizione) e può essere orientato; possono esserci **cicli**. A è la **matrice di adiacenza** del grafo.

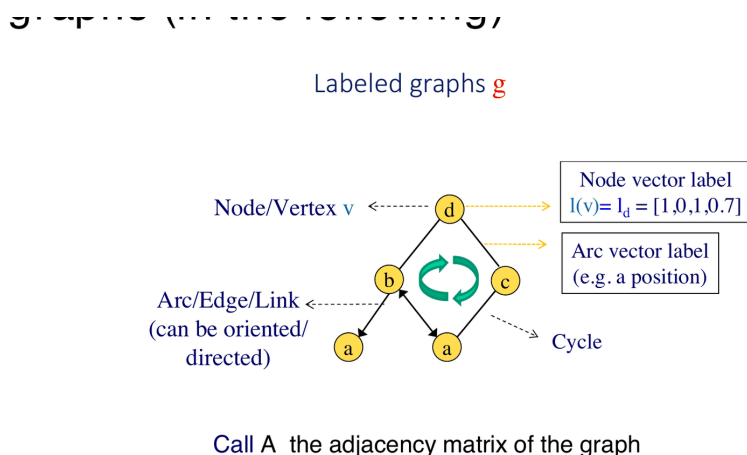


Figura 21.3: Un grafo etichettato: etichette vettoriali su nodi e archi, possibili cicli.

21.2.2 Il problema della rappresentazione

Non esisteva un modo **sistematico** (valido per qualunque task) per estrarre feature o metriche di relazione tra esempi strutturati. È un'istanza del **representation learning**, estesa ai domini strutturati.

- Le rappresentazioni basate su **feature** sono incomplete o fortemente dipendenti dal task (ad esempio gli indici topologici in chimica).
- Le rappresentazioni con **matrici di adiacenza** o altre rappresentazioni di dimensione fissa hanno problemi: sovra-dimensionamento o incompletezza (spreco con il padding o perdita di informazione), **allineamento** tra grafi diversi (quale nodo di un grafo corrisponde a quale dell'altro?), ordine topologico (che rende difficile la generalizzazione).

Suggerimento — Muovere i modelli verso i dati

“La capacità di trattare la natura propria dei dati di input è la chiave per un'applicazione di successo delle metodologie di ML.” Invece di **portare i dati ai modelli** (trasformare grafi in vettori o alberi in sequenze, con problemi di allineamento e perdita di informazione), si **portano i modelli ai dati**.

Esempio — Molecole e trasduzioni

Nel **QSPR/QSAR** si vuole correlare la struttura chimica delle molecole con le loro proprietà: Proprietà/Attività = $T(\text{Struttura})$, un valore di proprietà (regressione) o “tossico sì/no” (classificazione). Le molecole **non sono vettori**: sono rappresentate più naturalmente da strutture di dimensione variabile. Si può predire direttamente dalle strutture?

21.2.3 Trasduzioni su grafi

L'obiettivo è apprendere una **trasduzione** T da un dominio strutturato a uno spazio discreto o continuo, a partire da esempi ($\text{grafo}_i, \text{target}_i$) con grafi di dimensione variabile.

A transduction on graphs can be either

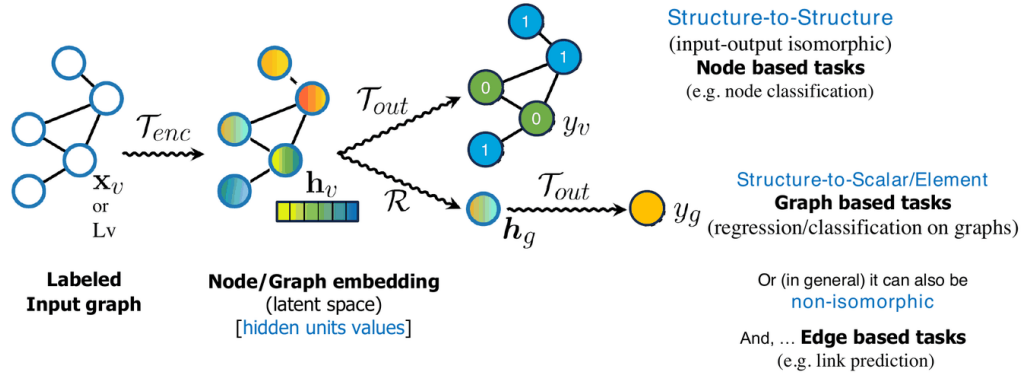


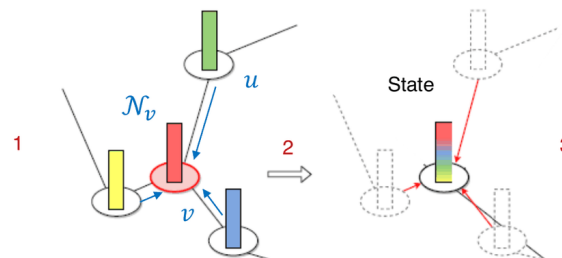
Figura 21.4: Trasduzioni su grafi: il grafo viene codificato in un embedding dei nodi, da cui si ottengono uscite per nodo o per l'intero grafo.

- **Struttura** → **struttura** (isomorfa input-output): compiti **a livello di nodo** (es. classificazione dei nodi).
- **Struttura** → **scalare/elemento**: compiti **a livello di grafo** (regressione o classificazione di grafi).
- In generale anche non isomorfa; e compiti **sugli archi** (es. *link prediction*).

21.2.4 Il message passing

Come funziona una DGN? Il concetto astratto è il **message passing**: i nodi “parlano” tra loro, scambiandosi messaggi per informarsi a vicenda e diventare consapevoli della struttura circostante.

1. Ogni nodo calcola un **messaggio**, funzione della propria etichetta, dei vicini $\mathcal{N}_v$ e degli archi tra loro, e lo invia ai vicini.
2. I nodi **aggregano** i messaggi ricevuti con una funzione **invariante per permutazione** (non importa l'ordine in cui arrivano).
3. Ogni nodo **aggiorna** i propri attributi (lo **stato**) combinando i messaggi con i parametri liberi W .



Nodes “talk” each other to inform about themselves and to be aware of the surrounding structure. h_v

Figura 21.5: Il message passing: messaggi dai vicini, aggregazione, aggiornamento dello stato.

Lo stato non dipende solo dal vicinato locale: **iterando** il message passing, l'informazione si **diffonde** da ogni nodo a tutti gli altri. Infine si può emettere un'uscita **per ogni nodo**, oppure **per l'intero grafo** tramite un'aggregazione invariante per permutazione delle rappresentazioni dei nodi, detta **global pooling** (ad esempio somma o media).

21.2.5 Il legame con le CNN

Sia le CNN sia le reti convoluzionali per grafi visitano ogni nodo dei dati, ma:

- nella **CNN** il kernel convoluzionale si applica su una **griglia regolare 2D**: si fa una media pesata dei pixel nella finestra, e i vicini di un pixel sono **ordinati** e in **numero fisso**;
- nei **grafi** la CNN non si applica direttamente: i vicini di un vertice sono **non ordinati** e in **numero variabile**. Si limita il campo recettivo ai **vicini**, si usa la **condivisione dei pesi** in modo invariante per permutazione (astruendo da qualunque ordinamento dei nodi), e si estende il vicinato locale tramite la **stratificazione**.

CNN and GNN both visit each node of the input data, but ...

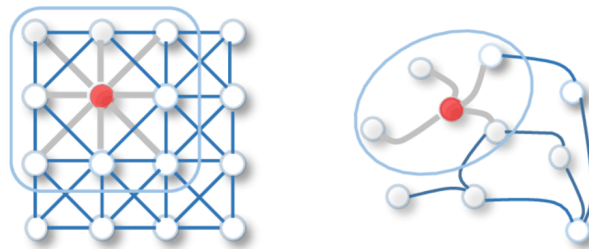


Figura 21.6: Convoluzione su griglia (CNN) e su grafo.

21.2.6 La formula generale

Il message passing si itera sugli strati (o iterazioni) l :

$$\mathbf{h}_v^{(l)} = \text{AGG}_{W^{(l)}}(\mathbf{L}_v, \mathbf{h}_v^{(l-1)}, \{\mathbf{h}_u^{(l-1)} : u \in \mathcal{N}(v)\}), \quad l = 1, \dots, L,$$

dove **AGG** aggrega (propaga) i messaggi dai vicini u con un operatore **invariante per permutazione** sull'insieme dei vicini (es. una somma) e combinazioni delle attivazioni delle unità, W sono i parametri liberi, $\mathbf{L}_v$ l'etichetta del nodo, e $\mathcal{N}$ è dato dalla matrice di adiacenza A . Applicare questo calcolo a ogni nodo corrisponde a una **visita parallela e non ordinata** del grafo a ogni iterazione.

Ci sono molte varianti:

- **GCN** (Kipf, Welling, 2017): $\mathbf{h}^{(l)} = \text{ReLU}(\hat{A} \mathbf{h}^{(l-1)} W^{(l)})$, con $\hat{A}$ la matrice di adiacenza normalizzata per grado;
- **GraphConv** (Morris et al., 2019): $\mathbf{h}^{(l)} = f(W_1^{(l)} \mathbf{h}^{(l-1)} + W_2^{(l)} A \mathbf{h}^{(l-1)})$;
- **GAT** (Veličković et al., 2018): aggregazione basata sull'**attenzione**.

(Molte sono disponibili in *PyTorch Geometric*.)

Suggerimento — Perché è rilevante per il deep learning

Le DGN sono **intrinsecamente profonde**: il processo non è locale. Iterando sugli strati, le rappresentazioni latenti dei nodi includono progressivamente l'informazione **contestuale**

di nodi sempre più lontani (diffusione).

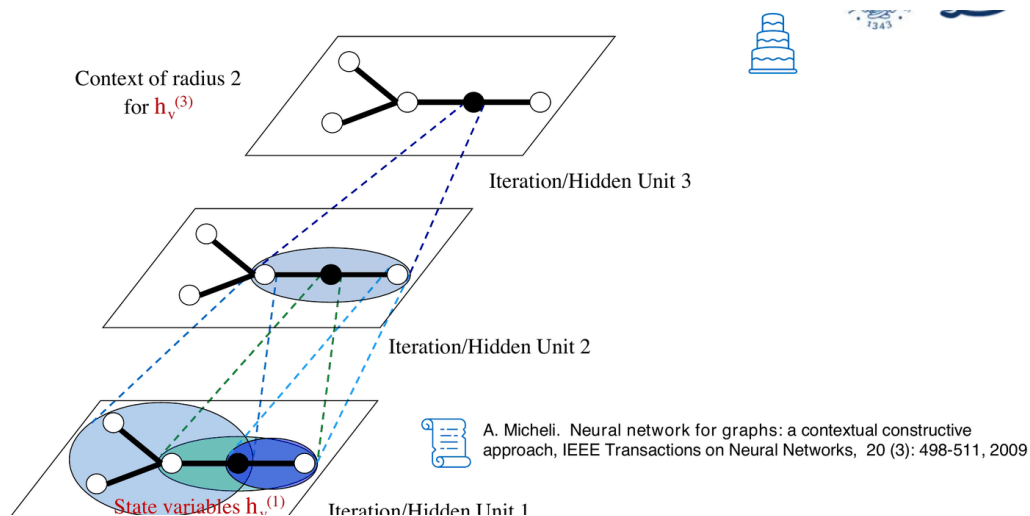


Figura 21.7: Evoluzione del contesto (composizionale) attraverso le iterazioni: al terzo strato lo stato del nodo ha un contesto di raggio 2.

21.2.7 NN4G e GCN: iterazioni come strati

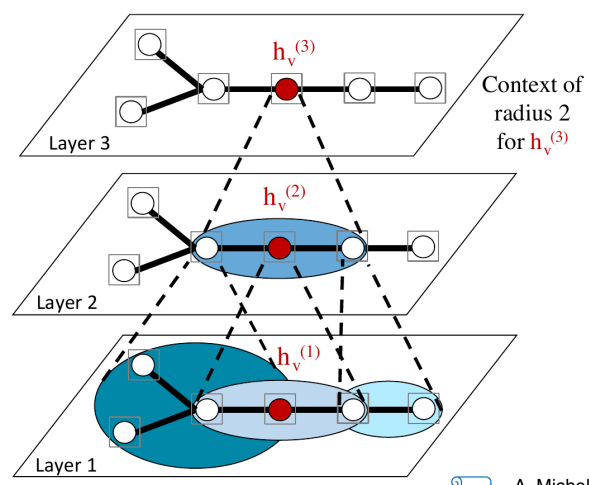


Figura 21.8: GCN e NN4G: ogni iterazione del message passing è un nuovo strato.

Nelle reti convoluzionali per grafi (GCN, NN4G), **ogni iterazione del message passing è un nuovo strato**, e per ogni nodo il contesto si **compone** attraverso gli strati:

- il **campo recettivo** si estende incrementalmente all'aumentare degli strati;
- la **profondità è funzionale alla crescita del contesto**;
- addestrando il modello si impara **come rappresentare il grafo**: end-to-end per le GCN generali, **strato per strato** (in modo incrementale) per NN4G, con numero di strati automatico e senza problemi di vanishing gradient.

NN4G (Micheli, 2009) calcola una variabile di stato per ogni strato e ogni vertice:

$$h_v^{(1)} = f\left(\sum_{j=0}^{|L_v|} \bar{w}_{1j} L_j(v)\right), \quad h_v^{(l)} = f\left(\underbrace{\sum_{j=0}^{|L_v|} \bar{w}_{lj} L_j(v)}_{\text{etichetta di } v} + \underbrace{\sum_{j=1}^{l-1} \hat{w}_{lj} \sum_{u \in \mathcal{N}(v)} h_u^{(j)}}_{\text{contesto dai vicini, da tutti gli strati precedenti}}\right), \quad l = 2, \dots, N.$$

Non è ricorsivo: ogni strato usa gli stati degli strati **precedenti**. Le unità si aggiungono una alla volta (in stile Cascade Correlation), e le uscite dei vari strati vengono combinate per produrre il risultato sul grafo.

21.2.8 GNN e GraphESN: approccio ricorsivo

Negli approcci **ricorsivi** (GNN, GraphESN) il processo di diffusione è simile, ma l è il numero di **iterazioni sullo stesso strato** (o strati con pesi condivisi). È un **sistema dinamico**: tipicamente si richiede la **convergenza a un punto fisso**, usato come embedding del grafo, e per questo si vincola la dinamica del message passing a essere **contrattiva**.

- **GNN** (Scarselli et al., 2009): con addestramento dell'embedding.
- **GraphESN** (Gallicchio, Micheli, 2010): estende le Echo State Network: la funzione di message passing **non è addestrata** (è un reservoir casuale), ma si controllano direttamente i parametri del sistema dinamico.

$$\mathbf{h}_v^{(l)} = \tanh\left(\sum_{j=0}^{|L_v|} \bar{w}_{lj} L_j(v) + \sum_{u \in \mathcal{N}(v)} \hat{W} \mathbf{h}_u^{(l-1)}\right),$$

iterata fino a convergenza; l'uscita per un grafo si ottiene con un global pooling X e un **readout lineare** addestrato con pseudoinversa o ridge regression: $y(g) = W_{out} X(\mathbf{h}(g))$.

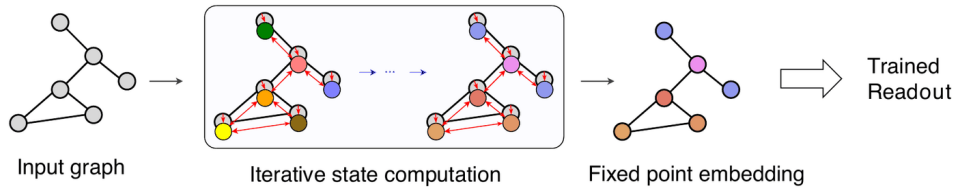


Figura 21.9: GraphESN: calcolo iterativo degli stati fino a un embedding a punto fisso, poi readout addestrato.

21.3 Problemi aperti e alcune proposte

21.3.1 Efficienza: oltre la backpropagation end-to-end

- **GraphESN per la classificazione di nodi:** su grafi molto grandi (es. Twitch-gamers, 168.000 nodi) ottiene un'accuratezza pari o superiore alla maggior parte dei modelli addestrati end-to-end, con un'accelerazione fino a 10 volte, e l'addestramento scala facilmente oltre i 100.000 nodi.
- **NN4G+** (progettazione automatica dell'architettura): la rete si costruisce **incrementalmente**, con una doppia espansione sinergica. Si addestra **un'unità alla volta** (niente backpropagation end-to-end attraverso gli strati); uno strato viene espanso finché l'accuratezza di validazione migliora, e si aggiungono nuovi strati finché migliora. Le architetture

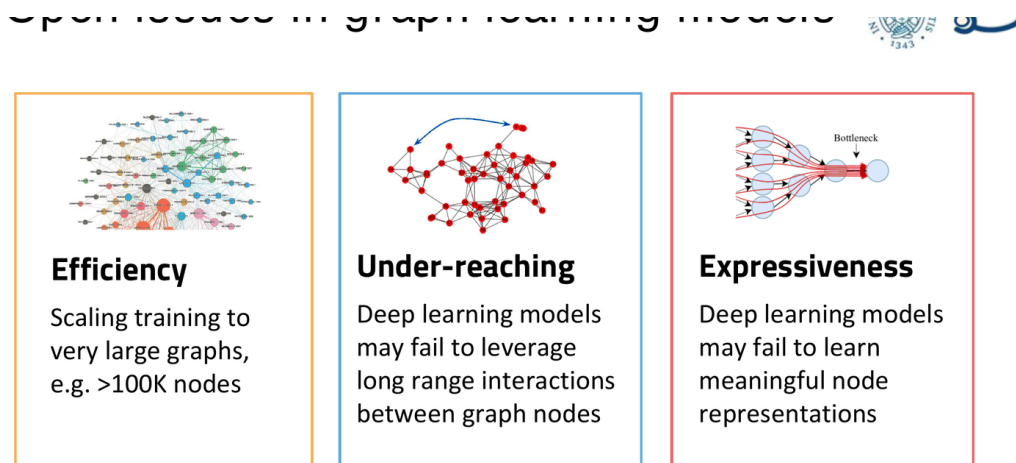


Figura 21.10: Problemi aperti nei modelli di apprendimento su grafi: efficienza, under-reaching, espressività.

ottenute sono adattate al task, costantemente più profonde e snelle rispetto a una ricerca esplicita dell'architettura, e vengono costruite in minuti invece che ore (oltre 10 volte più veloce, senza perdita di accuratezza).

21.3.2 I problemi della profondità: gli “over-”

La profondità serve, ma il message passing ha dei bias e dei problemi:

- **Over-smoothing** (Li et al., 2018): l'accuratezza tipicamente cala oltre 4–5 strati; le rappresentazioni dei nodi **collassano** negli strati profondi. Il message passing agisce come una **diffusione**, e le rappresentazioni di alto livello dei nodi diventano tutte simili.
- **Over-squashing** (Alon, Yahav, 2021): è un **collo di bottiglia**. Il campo recettivo cresce **esponenzialmente** con la profondità, ma tutta questa informazione va compressa in un embedding di dimensione fissa per nodo. La **sensibilità** della rappresentazione di un nodo alle feature di input di nodi lontani, misurata dallo Jacobiano $\partial \mathbf{h}_v^{(L)} / \partial \mathbf{x}_u$, decresce esponenzialmente con il numero di strati.
- Per l'addestramento end-to-end, la **difficoltà di retropropagare il gradiente** attraverso molti strati di message passing.

L'interazione tra questi fenomeni è un problema di ricerca aperto. Gli effetti:

1. **Under-reaching**: gli over-* impediscono di avere il campo recettivo necessario per apprendere interazioni tra nodi lontani.
2. **Eterofilia** nella classificazione dei nodi. Nei grafi **omofili** i nodi dello stesso vicinato condividono per lo più la stessa classe; nei grafi **eterofili** i nodi della stessa classe sono generalmente lontani, e le predizioni basate soprattutto sui vicini immediati possono essere fuorvianti. Sono task potenzialmente difficili per il message passing standard, per via degli over-* e del bias verso grafi “omogenei”.

Strumenti per l'analisi

- **Modelli che separano il bias del message passing dai problemi dell'addestramento end-to-end**: NN4G (incrementale, senza propagare il gradiente attraverso gli strati) e GraphESN (nessun addestramento dell'embedding, controllo diretto della norma spettrale)

- Heterophilic graphs: nodes belonging to the same class are generally further apart (predictions that rely chiefly on a node's immediate neighbours can be misleading)

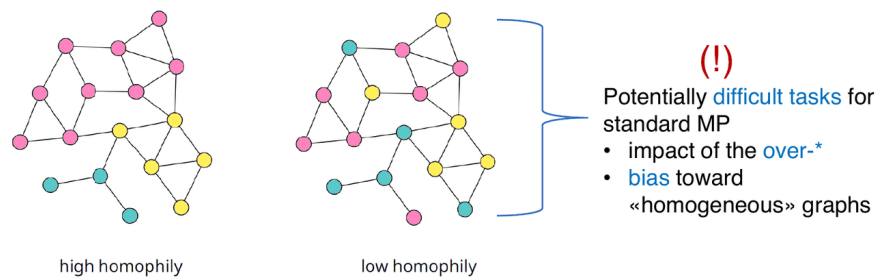


Figura 21.11: Grafi ad alta e bassa omofilia: i secondi sono difficili per il message passing standard.

$\|W\|$, cioè della costante di Lipschitz della funzione di message passing ricorsiva). Poiché

$$\left\| \frac{\partial \mathbf{h}_v^{(L)}}{\partial \mathbf{x}_u} \right\| \leq \prod_{l=1}^L \|W^{(l)}\| \cdot (A^L)_{u,v},$$

imporre $\|W\| > 1$ impedisce che la sensibilità rispetto all'input svanisca esponenzialmente (over-squashing).

- **Analisi spettrale** degli strati. La “frequenza” su un grafo è la variabilità del segnale (etichette, embedding) tra nodi vicini. **l'energia di Dirichlet** $E(\mathbf{h}) = \sum_v \sum_{u \in \mathcal{N}_v} \|\mathbf{h}_v - \mathbf{h}_u\|^2$ misura quanto il segnale varia sul grafo (visione spaziale) e, tramite la trasformata di Fourier su grafo (ottenuta dalla decomposizione del Laplaciano $L = D - A$), come la “massa” del segnale si distribuisce sullo spettro delle frequenze: un segnale a bassa frequenza (liscio) ha bassa energia. I grafi eterofili hanno segnali ad **alta frequenza**, e i task eterofili richiedono di **preservare le alte frequenze**.

Sul dataset **Chameleon** (una rete di pagine Wikipedia, 2277 nodi, 31.400 archi, omofilia 0,23), una “prova da sforzo” per l'eterofilia, NN4G e GraphESN **preservano meglio l'energia** attraverso gli strati, e mentre l'accuratezza degli altri modelli cala all'aumentare del message passing, la loro no. In un esperimento con 12 strati: GraphESN 73,7%, NN4G 68,3%, GCN 57,8%, GAT 55,0%, GraphConv 33,8%.

21.3.3 Estensioni in corso

- **Grafi dinamici/temporali**: relazioni o valori dei nodi che evolvono nel tempo (diffusione di infezioni, contenuti sui social, segnali sui nodi). Le **Dynamic Graph Echo State Network** realizzano una convoluzione spazio-temporale con un reservoir multistrato, efficaci quanto le reti temporali per grafi addestrate completamente, ma con un'accelerazione di 100 volte nell'addestramento e 10 nell'inferenza.
- **Generazione di grafi**: apprendere la distribuzione $p(G)$ di un insieme di grafi per campionarne di nuovi (es. generazione di molecole basata su frammenti), utile per equità, robustezza, privacy.
- **Spiegabilità (XAI)** per le reti su grafi: ad esempio identificare gli anelli benzenici che determinano una classificazione, o i frammenti strutturali noti (come l'accettore di Michael) che influenzano l'attivazione del pathway P53 nei dati Tox21.
- **Network quantification**: predire la **prevalenza** delle classi tra i nodi non etichettati invece di classificarli singolarmente (voto, ricerche di mercato, epidemiologia).
- **Bioinformatica**: predire proprietà dinamiche (robustezza, sensitività) di pathway biochimici e reti di interazione proteina-proteina rappresentati come grafi.

21.4 Altri approcci: i kernel per strutture

I **metodi kernel** si estendono facilmente a grafi di tipi diversi (sequenze, alberi, grafi orientati o no, ciclici o aciclici), perché basta definire un prodotto scalare $k(x, x') = \langle \phi(x), \phi(x') \rangle$, cioè una **similarità** tra dati di qualsiasi tipo, con un embedding implicito in uno spazio euclideo.

- **Kernel marginalizzati**: la similarità tra due grafi si basa sui cammini di etichette comuni ottenuti con random walk; il kernel è il prodotto scalare dei vettori di conteggio, mediato su tutti i possibili cammini. Il costo scala tipicamente in modo quadratico con la dimensione del grafo.
- **Kernel di convoluzione**: si decompongono gli oggetti in **sotto-strutture** e si definisce il kernel combinando kernel tra le sotto-strutture.

Convolution Kernels

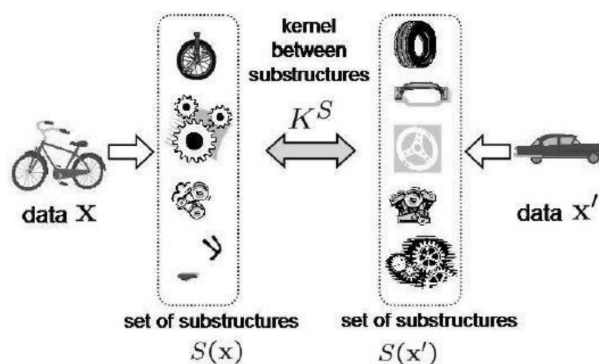


Figura 21.12: Kernel di convoluzione: la similarità tra oggetti si calcola a partire dalla similarità tra le loro sotto-strutture.

Attenzione — Limiti dei kernel per strutture

- **Efficienza**: il costo è critico, per via della scomposizione in sotto-strutture.
- **Adattività**: sono metodi guidati dalla conoscenza, con una misura di similarità **fissata prima dell'apprendimento**: la codifica nello spazio delle feature non è appresa. Il vantaggio è poter sfruttare la conoscenza a priori per un problema specifico.
- Per i domini strutturati **non si conosce l'universalità**: la scelta del kernel è critica, e calcolare un kernel **completo** per grafi è difficile almeno quanto il problema dell'**isomorfismo di grafi**. (Si studiano kernel adattivi.)

In sintesi — Conclusioni

- Si può fare deep learning su dati complessi **in modo efficiente**, senza un addestramento completamente end-to-end: modelli come GraphESN e NN4G costruiscono DGN accurate in modo efficiente e permettono di studiare il bias del message passing.
- La **profondità** nelle DGN è necessaria (fa crescere il contesto), ma può causare problemi (over-smoothing, over-squashing, difficoltà di addestramento); andare oltre l'end-to-end può aiutare sia per l'efficienza sia per gli over-*
- Il ML per dati strutturati e grafi è una realtà concreta, ancora in sviluppo, che offre soluzioni nuove dove le **relazioni** contano. Se i dati hanno relazioni (seriali, gerarchiche, reti), **non conviene usare una rappresentazione piatta**. È anche una grande opportunità per trattare in modo uniforme tipi di dati diversi.

Domande — Possibili domande d'esame

- Perché trattare i dati strutturati direttamente, invece di trasformarli in vettori?
- Quali tipi di trasduzione si possono definire sui grafi?
- Descrivere il message passing e la formula generale delle DGN.
- Differenza tra approcci a strati (GCN, NN4G) e ricorsivi (GNN, GraphESN).
- Cosa sono over-smoothing e over-squashing? Cos'è l'eterofilia?
- Pro e contro dei kernel per strutture.